

TUS
Technological University of the Shannon: Midlands Midwest
Ollscoil Teicneolaíochta na Sionainne: Lár Tíre Iarthar Láir

Artificial Intelligence to Enable a Self-optimising of a Flexible Robotic Framework for Adaptive Pick-and-Place Tasks Using Hybrid 3D Vision Data

---

Eber Lawrence Souza Gouveia

Supervisors:
Dr. Declan M. Devine
Dr. John G. Lyons

PhD MP Eng by Research

Materials Research Institute

Faculty of Engineering and Informatics

Submitted to the Technological University of the Shannon: Midlands
2026

Declaration

I certify that this PhD dissertation which I now submit for examination, is entirely my own work and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of my work. The dissertation was prepared according to the regulations for postgraduate study of the Technological University of the Shannon and has not been submitted in whole or part for an award in any other Institute or University.

The work reported on in this dissertation conforms to the principles and requirements of the Institute's guidelines for ethics in research.

Signed: _____ Student No.: A00289880

Date: ___ /___ /2026

Acknowledgments

I would first like to express my sincere gratitude to my supervisors, Dr. Declan Devine and Dr. John Lyons, for the guidance, support and encouragement provided throughout this PhD. Their advice, feedback and experience were fundamental to the development of this research and to my progress throughout the programme.

I would also like to thank Technological University of the Shannon, Midlands Midwest (TUS), particularly the Faculty of Engineering, the Software Research Institute (SRI) and the Materials Research Institute (MRI), for providing the facilities, resources and research environment required to carry out this work. This publication emanated from research conducted with the support of Science Foundation Ireland (SFI) co-funded by the European Regional Development Fund, the Technological University of the Shannon President's Doctoral Scholarship, and Johnson & Johnson. I am grateful for that support, without which this research would not have been possible.

My thanks also go to the colleagues, researchers, technical staff and collaborators who contributed to different stages of this work, whether through practical assistance in the laboratory, technical support, or discussions that helped shape the direction of the research.

I would also like to thank the friends who made these years more enjoyable and who were always available for a conversation when one was needed. In particular, I want to thank Eduardo, who was my closest companion throughout this journey. We spent countless hours talking, not only about research and technology, but about the present and the future, about the problems we were both facing and the experiences we shared along the way. Those conversations kept me grounded and made the difficult stretches far easier to get through. I am also very grateful to Guilherme and Yulia, whose company over the past years gave me some of the happiest moments of this period. Travelling around Ireland with them, and simply spending time together, was a reminder that there is life outside the thesis.

A very special thank you goes to my parents, Eber and Sandra, and to my whole family, for their continuous support and encouragement throughout my education. My father passed away during the first year of this PhD, and it is impossible to write these words without thinking of him. He believed in me long before I had any reason to believe in myself, and everything I have achieved, including this thesis, rests on the support and the example he gave me throughout my entire life. I carry his memory with me, and I know how proud he would have been to see this finished. To my mother, thank you for your strength and everything you have given me, then and now.

Finally, I would like to thank my wife, Adrielle, for her patience, understanding and constant support throughout this journey. Building a life far from home, in another country, is not easy, and she is the person who made all of it easier. She kept me standing through the hardest moments of my life, and her encouragement during both the difficult and the rewarding parts of this process made everything easier and more meaningful. I am deeply grateful for everything she has done and for being by my side from beginning to end.

Abstract

Industrial pick-and-place systems are highly reliable when objects, tools and workspaces are structured in advance. However, they become difficult to reconfigure when parts arrive in random positions and orientations or when new object types must be introduced. Approaches based entirely on Artificial Intelligence (AI) can improve flexibility and adaptability, but may introduce challenges related to data requirements, validation, explainability and predictable execution in industrial environments. This thesis addresses this gap by developing a modular, semi-autonomous robotic pick-and-place system that combines user-guided configuration, hybrid dataset generation, 3D point-cloud perception, template-based pose estimation and Robot Operating System (ROS)-based execution.

The proposed system operates within a structured manufacturing cell in which the robot, camera, conveyor and workspace are known, while the object flow and target parts can change. A key contribution is a hybrid point-cloud dataset generation process that combines computer-aided design (CAD) models, physics-based object arrangement and real background scans to generate task-specific perception data. This supports adaptability by allowing new object types to be introduced with limited expert intervention.

The research is evaluated through two connected experimental stages. The first establishes the physical and digital robotic-cell infrastructure and validates the ROS-based perception, planning and pick-and-place pipeline using simpler object geometries. The second extends this foundation to complex injection-moulded parts using learning-based instance segmentation and template-based registration. Offline experiments evaluate synthetic and hybrid training conditions on generated and real scanner data, followed by registration and end-to-end perception evaluation. The selected perception pipeline is subsequently integrated with the physical robotic system for online pick-and-place validation.

The results show that incorporating measured background information substantially improves transfer to real scanner observations. PointGroup mIoU increased from 39.95% with fully synthetic training to 81.14% with hybrid training, while mAP50 increased from 49.28% to 73.92%. DCP followed by ICP achieved 97.84% registration success using reference object instances. The complete offline perception pipeline achieved 74.38% end-to-end success, while registration remained successful for 99.79% of usable PointGroup instances, indicating that the principal remaining perception limitation was associated with incomplete single-view observations. During 120 online trials, the system achieved 100% target identification, 99.17% correct classification, 94.17% registration acceptance and 71.67% complete pick-and-place success.

Overall, the thesis demonstrates that adaptable industrial pick-and-place can be achieved through a hybrid architecture that combines data-driven perception with object-specific geometric knowledge, modular software integration and planning-based validation, while retaining the controlled execution and reliability expected in industrial environments.

Table of Contents

Chapter 1: INTRODUCTION 1
1.1 Background and Context 2
1.2 Problem Statement and Research Motivation 2
1.3 Research Aim, Questions and Objectives 4
1.4 Scope of the Research 5
1.5 Contributions 6
1.5.1 Peer Reviewed Publications 6
1.6 Thesis Structure 7
Chapter 2: LITERATURE REVIEW 9
2.1 Introduction 10
2.2 Industrial Robotic Pick-and-Place Systems 10
2.2.1 Conventional industrial automation and structured environments 11
2.2.2 Online/offline programming and reconfiguration limitations 12
2.2.3 Industrial pick-and-place with random object flows 13
2.3 Reliability, Flexibility and Adaptability in Manufacturing Cells 15
2.3.1 Defining reliability, flexibility and adaptability 16
2.3.2 Flexible versus adaptable pick-and-place systems 16
2.3.3 The need for bounded autonomy in manufacturing 17
2.4 Sensing and Perception for Random Industrial Object Flows 18
2.4.1 3D sensing and point cloud preprocessing 20
2.4.2 Semantic and Instance Segmentation in 3D Point Clouds 23
2.5 Template-Based Pose Estimation and Point Cloud Registration 24
2.5.1 Classical registration methods 25
2.5.2 Deep learning-based registration 26
2.5.3 Hybrid registration pipelines 27
2.5.4 CAD/template-constrained industrial pose estimation 28
2.6 Grasp Planning and Motion Execution Under Industrial Constraints 28
2.6.1 Industrial gripper types and end-effector selection 29
2.6.2 Grasp generation for industrial grippers 31
2.6.3 Motion feasibility and pick-to-place execution 31
2.7 Hybrid Dataset Generation and Sim-to-Real Transfer 32
2.7.1 Synthetic data for robotic perception 33

2.7.2 Physics-based and hybrid point-cloud generation....33
2.7.3 Sim-to-real transfer and dataset reuse....34
2.8 ROS-Based Modular Robotic Systems and Digital Twins....35
2.8.1 ROS and ROS-Industrial for manufacturing robotics....37
2.8.2 Digital twins, motion planning, and virtual commissioning....38
2.9 User-Guided Configuration and Human Involvement in Robotic Systems....39
2.9.1 Reducing expert dependence in robot programming....39
2.9.2 User-guided configuration and bounded autonomy....40
2.10 Hybrid Robotic Systems: Combining AI with Industrial Reliability....40
2.10.1 Limits of purely traditional and purely AI-based systems....41
2.10.2 Hybrid architectures for bounded autonomy....41
2.11 Research Gaps and Positioning of This Thesis....41
2.11.1 Positioning of the proposed system and mapping to SRQs....42
Chapter 3: FOUNDATIONAL ROBOTIC CELL DEVELOPMENT AND VALIDATION....44
3.1 Introduction....45
3.2 Aims and Objectives....45
3.3 Robotic Cell Development....46
3.3.1 Physical Robotic Cell....48
3.3.2 Digital Counterpart....50
3.3.3 ROS and ROS-Industrial Integration....51
3.3.4 MoveIt-Based Motion Planning....54
3.4 Vision-Based Point Cloud Processing....54
3.4.1 Point Cloud Acquisition....55
3.4.2 Background Removal and Preprocessing....55
3.4.3 DBSCAN-Based Object Clustering....57
3.4.4 PCA-Based Centroid and Orientation Estimation....58
3.4.5 Object Shape Matching and Sorting....59
3.5 ROS Computer-Vision Package and Pick-and-Place Execution Pipeline....61
3.5.1 ROS Workspace Organisation....62
3.5.2 Computer-Vision Package Architecture....64
3.5.3 System Initialisation and Object Detection....65
3.5.4 Point-Cloud Processing Outputs and ROS Messages....66
3.5.5 Detected-Object TF Generation....66
3.5.6 Target Object Matching and Slot Allocation....67
3.5.7 Calibration and Coordinate Consistency....67
3.5.8 Robot Control and Pick-and-Place Execution....68

3.5.9 Execution Sequence Summary 69
3.6 Experimental Protocol 69
3.6.1 Experimental Setup 69
3.6.2 Test Objects and Placement Tray 70
3.6.3 Orientation Estimation Test 71
3.6.4 Pick-and-Place Success Test 72
3.7 Evaluation Metrics 72
3.7.1 Orientation Error 72
3.7.2 Pick-and-Place Success and Failure Rate 73
3.7.3 Statistical Analysis 74
3.8 Results and Discussion 75
3.8.1 Orientation Estimation Accuracy 75
3.8.2 Error Distribution Across Object Orientations 76
3.8.3 Pick-and-Place Success and Failure Analysis 77
3.8.4 Overall Discussion and Transition to the Next Study 78
3.9 Conclusion 82
Chapter 4: EXTENSION TO COMPLEX INDUSTRIAL PARTS 84
4.1 Introduction 85
4.2 Aims and Objectives 86
4.3 Relationship with Chapter 3 Experiment 87
4.3.1 System Adaptations 88
4.4 Hybrid Dataset Generation Methodology 90
4.4.1 Template Object Preparation 91
4.4.2 Physics-Based Object Dropping in MuJoCo 92
4.4.3 Synthetic Foreground Point-Cloud Generation 93
4.4.4 Background Integration 93
4.4.5 Lighting, Noise, and Self-Occlusion Simulation 94
4.4.6 Semantic and Instance Ground Truth Generation 95
4.5 Dataset Preparation for Perception Models 96
4.5.1 Scene-Level Dataset for Instance Segmentation 97
4.5.2 Object-Level Dataset for Registration 97
4.6 Instance Segmentation Methodology 98
4.6.1 Segmentation Model Selection 99
4.6.2 Model Input Preparation 100
4.6.3 Training Procedure 100
4.6.4 Instance Segments and Semantic Class Predictions 101

4.7 Registration Methodology 102
4.7.1 Template-to-Scan Registration Problem 102
4.7.2 Registration Method Selection 103
4.7.3 Coarse-to-Fine Registration Configurations 103
4.7.4 Training and Inference Procedure 104
4.7.5 Estimated Transformation Output 105
4.8 ROS-Based System Implementation 106
4.8.1 Workspace and Package Organisation 106
4.8.2 Package Responsibilities and Common Interfaces 109
4.8.3 Data Flow and System Initialisation 110
4.9 User-Guided Configuration Workflow 112
4.9.1 Loading Target Object CAD Models 113
4.9.2 Defining Placement Slots 114
4.9.3 Generating and Applying Task Configurations 115
4.10 Target Industrial Use Case 116
4.10.1 Selected Injection-Moulded Assemblies 117
4.11 Offline Validation Protocol 118
4.11.1 Experimental Datasets and Training Conditions 120
4.11.2 Structured Real-Scan Validation Dataset 122
4.11.3 Segmentation Evaluation 125
4.11.4 Registration Evaluation 127
4.11.5 End-to-End Perception Evaluation 129
4.11.6 Evaluation Metrics 130
4.12 Online Validation Protocol 135
4.12.1 Integrated Online Pick-and-Place Pipeline 137
4.12.2 Pose-Conditioned Grasp and Motion Execution 139
4.12.3 Experimental Procedure 141
4.12.4 Evaluation Metrics 142
4.13 Results and Discussion 144
4.13.1 Model Training 145
4.13.2 Offline 148
4.13.3 Online 158
4.13.4 Overall Discussion 163
4.14 Chapter Summary 166
Chapter 5: Conclusions 167
5.1 Introduction 168

5.2 Research Question Synthesis 168
5.3 Research Contributions and Industrial Significance 171
5.4 Limitations and Future Work 173
5.5 Final Conclusions 174
References 176
APPENDICES 188
A. Photoneo PhoXi Scanner S Calibration Sheet 188
B. Two-Finger Electronic Gripper Development 191
B.1 Design Requirements 192
B.2 Mechanical Design 192
B.3 Materials and Manufactured Components 194
B.4 Actuation and Sensing System 196
B.5 Embedded Electronic Hardware 197
B.6 Firmware Architecture 198
B.7 Communication Architecture 198
B.8 ROS Software Architecture 199
B.9 Digital Model and MoveIt Integration 202
B.10 Use in the Chapter 4 Online Experiment 204
B.11 Limitations and Future Improvements 205

List of Figures

Figure 2.1: Global trends in industrial robot deployment. (a) Annual installations of industrial robots worldwide from 2014 to 2024. (b) Operational stock of industrial robots worldwide over the same period. The figure highlights both the sustained demand for new industrial robots and the continuing growth of the installed base, supporting the view that industrial robotic automation is a mature and expanding manufacturing technology. Source adapted from World Robotics 2025 [22]. 11
Figure 2.2: General perception–reasoning–action loop in robotic systems, showing how sensor data are converted into an environment representation and used by AI/ML methods to support planning and execution. Adapted from [41]. 19
Figure 2.3: Shape-acquisition taxonomy, showing contact and non-contact methods for acquiring 3D information. This thesis focuses on non-contact optical sensing, particularly the active approaches highlighted by the dashed red box, which are suitable for close-range industrial pick-and-place. Adapted from [55]. 20
Figure 2.4: Example of point-cloud preprocessing stages applied to a sample pipe-fitting scan supplied by Photoneo with PhoXi Control software [61]. (a) Original point cloud. (b) ROI-cropped point cloud. (c) Background-reduced point cloud. (d) Outlier-filtered point cloud. (e) Downsampled or voxelised point cloud used as the segmentation-ready input for robotic perception and registration. The representations shown in (b)-(e) were generated through preprocessing performed in this work. 21
Figure 2.5: Effect of preprocessing intensity on point-cloud quality for the scanned pipe-fitting object. (a) Original point cloud with 58934 points. (b) Under-processed point cloud with 34669 points, where noise and background artefacts remain. (c) Balanced preprocessing with 3458 points, where artefacts are reduced while object geometry is preserved. (d) Over-processed point cloud with 269 points, where useful geometric details are removed. The processed examples were generated in this work. 22
Figure 2.6: Timeline of representative 3D point-cloud semantic and instance segmentation methods, including fully supervised and weakly supervised approaches. The progression from PointNet-based methods to instance-level and transformer-based architectures illustrates the increasing focus on object-level scene understanding in 3D perception. Adapted from [54]. 23
Figure 2.7: Example of hybrid point-cloud registration combining a learning-based method with classical refinement. The figure shows the input misalignment, the result of ICP alone, the result of DCP alone, and the result of DCP followed by ICP. Adapted from [80]. 27
Figure 2.8: Taxonomy of robotic grippers based on five common classification criteria, including number of fingers, actuation method, gripping mode, mechanism type and gripping method. Adapted from [92]. 29
Figure 2.9: Representative industrial gripper types. (a) Parallel gripper, SCHUNK EGU. (b) Three-finger adaptive gripper, Robotiq 3-Finger Adaptive Robot Gripper. (c) Angular pneumatic gripper, GIMATIC PB series. (d) Vacuum gripper, UFactory xArm Vacuum Gripper. (e) Soft gripper, OnRobot Soft Gripper. (f) Magnetic gripper, SMC MHM-X6400. Product images adapted from manufacturer documentations. 30
Figure 2.10: Overview of ROS as a modular robotics framework. 36
Figure 2.11: Overview of the ROS-Industrial ecosystem, showing its role in connecting ROS/ROS 2 with industrial robot hardware interfaces, 3D sensors, calibration tools, communication bridges, developer tools, applications and human interfaces. Adapted from [113]. 37
Figure 3.1: Physical and digital layers of the robotic cell used in this experiment. (a) Physical robotic cell including the ABB IRB 1200-5/09 robot, PhoXi 3D scanner, conveyor belt, gripper and placing table. (b) Digital counterpart of the cell in the ROS/RViz environment, showing the robot model, scanner structure, conveyor region, detected objects and predefined placement area. 48
Figure 3.2: ABB IRB 1200-5/09 robotic arm used in the experimental cell. (a) Physical appearance of the robot. (b) Workspace envelope of the robot, showing its approximate reach and vertical operating range. 49
Figure 3.3: PhoXi 3D Scanner S operating characteristics. (a) Manufacturer-specified field of view and working-volume dimensions. (b) Sample pipe-fitting point cloud supplied by Photoneo with PhoXi Control [61], illustrating the dense 3D surface information produced by the scanner. 49
Figure 3.4: The ROS workspace structure used to connect the external computer with the ABB IRC5 controller. The external computer hosted the ROS workspace, including the computer-vision package, robotic-cell description, socket communication client, RViz and MoveIt, while the IRC5 controller handled low-level robot execution through a RAPID program. 53
Figure 3.5: Autonomous pick-and-place process flowchart. The system initialises all components, activates the conveyor belt, detects objects using retroreflective sensors, stops the conveyor, triggers the 3D

scanner, processes the point cloud, generates robot targets for the detected objects, executes pick-and-place actions and repeats the sequence until no objects remain in the scanning region. 62
Figure 3.6: Simplified ROS workspace structure used in this experiment, showing the main ABB robot packages and the configuration files and scripts within the computer-vision package. 63
Figure 3.7: Pick-and-place experimental setup. Objects moved along the conveyor through the scanner region for 3D scanning, then were picked by the robot and placed into the corresponding tray slots. 70
Figure 3.8: Violin plot of the orientation error for the small, medium and large rectangular objects. 76
Figure 3.9: Absolute orientation error for each predefined object angle from 0 to 165 degrees in increments of 15 degrees. 76
Figure 3.10: Pick-and-place success and failure rates for the small, medium and large rectangular objects. Each object type was evaluated over 720 executions. Failure rates increased from 3.75% for the small object to 5.83% for the medium object and 7.78% for the large object, corresponding to success rates of 96.25%, 94.17% and 92.22%, respectively. Across all 2160 executions, the overall pick-and-place success rate was 94.21%. 77
Figure 4.1: Overview of the second experimental stage. (a) Revised robotic cell and physical data sources, (b) hybrid point-cloud dataset generation, and (c) perception-method development, evaluation and inference for semantic and instance segmentation, and template-to-scan registration. 85
Figure 4.2: Digital robotic-cell configurations used in the (a) first and (b) second experiments. 89
Figure 4.3: Placement Config tab of the user-guided configuration interface, showing the 3D task viewer, slot-definition table and controls used to define and apply object placement poses. 112
Figure 4.4: User-guided placement-configuration workflow. (a) Loading the task geometry, (b) defining parent-relative placement slots, and (c) visualising the applied configuration through ROS TF frames and mesh markers in RViz. 115
Figure 4.5: CAD representations of the three injection-moulded assemblies used during the second experimental stage. (a) AT1 with four dissimilar connected products. (b) AT2 with two identical rectangular plate-like products. (c) AT3 with two identical square plate-like products. 118
Figure 4.6: Overview of the offline validation protocol. PointGroup models trained using fully synthetic and hybrid datasets are evaluated across generated and real-scanner scenes. Registration is assessed independently using five configurations, followed by end-to-end evaluation of the selected Hybrid PointGroup followed by DCP and ICP pipeline. 119
Figure 4.7: Structure of the real-scanner validation dataset. The three resting configurations are each sampled at 24 angular conditions between 0 ◦ and 345 ◦ using 15 ◦ increments. Every scene contains one instance of each of the three target assembly types, resulting in 72 scenes and 216 reference object observations. 123
Figure 4.8: Example of the structured real-scan reference data. (a) Original measured point cloud. (b) Semantic and instance reference annotation. (c) Canonical CAD meshes transformed using the stored reference 6-DoF poses and shown within the measured scene. 125
Figure 4.9: Overview of the integrated robotic cell used during the online validation. The physical cell is shown from the front, the conveyor workspace is shown from above, and the corresponding RViz scene presents the digital representation used during planning and execution. The views highlight the main experimental components, including the PhoXi scanner, ABB robot with the custom gripper, conveyor, placing table and placement tray. 137
Figure 4.10: Flowchart of the integrated online pick-and-place sequence. After sensor-triggered point-cloud acquisition, the perception stage performs instance segmentation, template registration and registration validation. Valid objects proceed to grasp and placement target generation, motion planning and physical pick-and-place execution, while rejected objects are excluded from manipulation. 138
Figure 4.11: Representative stages of one online pick-and-place trial. (a) shows the scene after scan acquisition, including the physical view and the corresponding raw point cloud in RViz. (b) shows the perception result, where the detected object instances are separated and displayed in different colours. (c) shows the robot after placing object AT3 in its assigned slot, together with the corresponding digital scene in RViz. 139
Figure 4.12: Placement tray used during the online experiment. (a) shows the empty tray containing the object-specific slots for AT1, AT2 and AT3, while (b) shows the three assemblies positioned within their corresponding destinations. Each slot provided approximately 2 mm of clearance around the corresponding object. 142
Figure 4.13: PointGroup training and validation behaviour using the fully synthetic scene dataset for random seeds 11, 22 and 33. The figure shows the total optimisation objective and semantic-loss component over 150 epochs. The proposal-scoring component is introduced after the initial 30-epoch

preparation stage. Negative values of the total objective result from the directional point-offset component of the PointGroup loss formulation. 145
Figure 4.14: PointGroup training and validation behaviour using the hybrid scene dataset for random seeds 11, 22 and 33. The figure shows the total optimisation objective and semantic-loss component over 150 epochs. The proposal-scoring component is introduced after the initial 30-epoch preparation stage. Negative values of the total objective result from the directional point-offset component of the PointGroup loss formulation. 146
Figure 4.15: Training and validation behaviour of the adapted DCP model using the combined object-level registration dataset for random seeds 11, 22 and 33. The curves show the NLL correspondence loss over 150 epochs. 147
Figure 4.16: Example real-scanner scene used to compare synthetic-trained and hybrid-trained PointGroup predictions. (a) shows the original scanner observation from resting configuration 1 at a rotation of 210°. (b) shows the corresponding reference semantic and instance annotation. (c) shows the prediction produced by the PointGroup model trained on the fully synthetic scene dataset. (d) shows the prediction produced by the PointGroup model trained on the hybrid scene dataset. 150
Figure 4.17: Effect of resting configuration on the geometry available to the segmentation model. Representative real-scanner observations are shown for RC1, RC2 and RC3, with the number of visible target points indicated for each example. Changes in resting configuration and orientation alter the amount of object surface visible from the fixed scanner viewpoint and can produce substantially more partial observations. 151
Figure 4.18: Registration success rate for the five configurations evaluated using the 216 reference object observations. DCP and DCP + ICP show the mean across models trained using seeds 11, 22 and 33, with error bars representing the standard deviation between the three runs. The remaining configurations were evaluated using a single fixed implementation. 153
Figure 4.19: Progression through the selected end-to-end offline perception pipeline across the three seed-paired runs. Of the 648 checkpoint-object evaluations, PointGroup produced 483 usable instances, of which 482 were successfully registered using DCP followed by ICP. The remaining 165 evaluations did not produce a usable instance, while only one registration failed after a usable instance had been recovered. 157
Figure 4.20: Performance of the principal stages during the 120 online pick-and-place trials. The figure shows target identification, semantic classification, registration acceptance and successful physical placement. 159
Figure 4.21: Representative example of the online perception process. (a) Physical arrangement of the moulded assemblies on the conveyor, (b) corresponding raw point-cloud observation displayed in RViz, and (c) PointGroup perception output showing the detected object instances coloured according to their predicted classes, with AT1 shown in green, AT2 in blue and AT3 in orange. 161
Figure 4.22: Successful physical placement for AT1, AT2 and AT3 under the sprue-top and sprue-sideways resting conditions. Each object and resting-condition combination contains 20 trials. 161

List of Tables

Table 3.1: Equipment, computing hardware and software used in the foundational robotic cell. 47
Table 3.2: Summary of the main YAML configuration files used by the computer-vision package and their functions. 64
Table 3.3: Summary of the main scripts and nodes used by the package. 65
Table 3.4: Summary the object and tray-slot dimensions used in the experiment. 71
Table 3.5: Summary of the orientation-test protocol. 71
Table 3.6: Mean orientation error and error dispersion obtained for the small, medium and large objects. 75
Table 3.7: Number of executions, failures and success rates for each object type. 78
Table 4.1: Composition of the generated datasets used during offline validation. 121
Table 4.2: Model training and evaluation conditions used during offline validation. 122
Table 4.3: Composition of the structured real-scanner validation dataset. 123
Table 4.4: PointGroup segmentation evaluation conditions. 126
Table 4.5: Registration configurations used during offline validation. 128
Table 4.6: PointGroup segmentation performance across the generated and real evaluation datasets. Results are reported as mean ± standard deviation across seeds 11, 22 and 33. 148
Table 4.7: Registration success and ADD-S performance on the 216 reference object observations. DCP and DCP + ICP results are reported as mean ± standard deviation across seeds 11, 22 and 33. 152
Table 4.8: Translation and rotation errors for the evaluated registration configurations. DCP and DCP + ICP results are reported as mean ± standard deviation across seeds 11, 22 and 33. 154
Table 4.9: End-to-end perception performance for the selected Hybrid PointGroup followed by DCP and ICP configuration. Each run contains the same 216 reference objects from the 72 real-scanner scenes. 156
Table 4.10: Terminal outcomes obtained during the 120 online pick-and-place trials. 160

List of Acronyms

6-DoF	Six Degrees of Freedom
ABB	ASEA Brown Boveri
ADD	Average Distance of Model Points
ADD-S	Average Distance of Model Points for Symmetric Objects
AI	Artificial Intelligence
AP	Average Precision
AP25 / AP50	Average Precision at IoU thresholds of 0.25 / 0.50
AT	Assembly Type
BOP	Benchmark for 6D Object Pose Estimation
CAD	Computer-Aided Design
CHOMP	Covariant Hamiltonian Optimization for Motion Planning
CPU	Central Processing Unit
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DC	Direct Current
DCP	Deep Closest Point
DDS	Data Distribution Service
DHCP	Dynamic Host Configuration Protocol
DGCNN	Dynamic Graph Convolutional Neural Network
FN	False Negative
FP	False Positive
FPFH	Fast Point Feature Histograms
FSR	Force-Sensitive Resistor
GeDi	General and Distinctive 3D Local Descriptors
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
I/O	Input/Output
ICP	Iterative Closest Point
IoC	Intersection over Coverage
IoT	Internet of Things
IoU	Intersection over Union
IP	Internet Protocol
KD-tree	k-Dimensional Tree
LiDAR	Light Detection and Ranging
LLM	Large Language Model
mAP	Mean Average Precision
mDNS	Multicast Domain Name System
mIoU	Mean Intersection over Union
ML	Machine Learning
MuJoCo	Multi-Joint dynamics with Contact
NLL	Negative Log-Likelihood
ODE	Open Dynamics Engine
OMPL	Open Motion Planning Library
PCA	Principal Component Analysis
PETG	Polyethylene Terephthalate Glycol-modified

PETG-CF	Carbon-Fibre-Reinforced PETG
PLA	Polylactic Acid
PLA-CF	Carbon-Fibre-Reinforced PLA
PLC	Programmable Logic Controller
PLY	Polygon File Format
QoS	Quality of Service
RANSAC	Random Sample Consensus
RAPID	Robotics Application Programming Interactive Dialogue
RC	Resting Configuration
REST	Representational State Transfer
RDS	Robotics & Drives Services Ltd
RGB	Red, Green and Blue
RGB-D	Red, Green, Blue and Depth
ROI	Region of Interest
ROS	Robot Operating System
RQ	Research Question
RRT	Rapidly-exploring Random Tree
RWS	Robot Web Services
SRQ	Sub-Research Question
STL	Stereolithography File Format
STOMP	Stochastic Trajectory Optimization for Motion Planning
SVD	Singular Value Decomposition
TCP/IP	Transmission Control Protocol / Internet Protocol
TF	Transform
TP	True Positive
UDP	User Datagram Protocol
URDF	Unified Robot Description Format
Xacro	XML Macros
YAML	YAML Ain't Markup Language

Chapter 1: INTRODUCTION

1.1 Background and Context

Industrial automation has transformed manufacturing processes since the appearance of the first industrial robots during the Third Industrial Revolution. This period, beginning in the 1960s, was marked by the introduction of electronics, computers and programmable automation into manufacturing. In the manufacturing sector, labour-intensive tasks, traditionally executed by workers such as welding, assembly, and material handling, became automated through the utilisation of robotic arms. An early example of this shift was the Unimate robot, one of the first to be introduced into manufacturing in the 1960s [1]. Developed by George Devol and Joseph Engelberger, the Unimate robot was initially deployed in General Motors' assembly lines to handle repetitive tasks, such as moving die-cast metal components and welding car bodies. By automating these processes, Unimate not only improved production efficiency but also ensured worker safety by eliminating exposure to hazardous tasks. Building on this earlier phase of automation, the Fourth Industrial Revolution, beginning in the 2010s and often associated with Industry 4.0, introduced new capabilities through the increasing integration of connected systems, advanced sensors, computer vision, data-driven methods and artificial intelligence (AI) into manufacturing. These innovations have allowed robots to operate with greater awareness and intelligence, enabling more complex applications such as dynamic object handling and adaptive manufacturing [2].

Computer vision and AI have become particularly critical in modern robotic systems. It enables robots to perceive and interpret their environment, while AI algorithms allow them to make intelligent decisions and adapt to changes. When integrated with robotic arms, these technologies can potentially create intelligent pick-and-place systems capable of handling objects in diverse and dynamic conditions, such as identifying and grasping objects on a fast-moving conveyor belt, sorting items with varying shapes and sizes, or picking fragile items without causing damage [3], [4], [5]. As industrial processes become more complex, automation systems that integrate flexibility and adaptability are increasingly required, resulting in more autonomy in pick-and-place manufacturing operations. Although flexibility and adaptability are closely linked, they contribute in different ways to enhancing system autonomy. In this thesis, and in line with manufacturing flexibility and reconfigurable manufacturing literature, flexibility is understood as the ability of a system to respond to variation during operation, such as changes in the position, orientation or size of objects it is designed to handle [6], [7], [8]. It implies the capacity to operate effectively within predefined parameters, ensuring consistent performance in dynamic but familiar scenarios. On the other hand, adaptability is related to reconfigurability and goes a step further by referring to the ability of a system to be prepared for new object types, placement requirements or task variants without extensive reprogramming or manual intervention [9], [10]. While flexibility ensures reliable operation in varying conditions within a known context, adaptability allows the system to extend its functionality to accommodate new tasks or object types, making it more autonomous and versatile.

1.2 Problem Statement and Research Motivation

Despite advances in sensing, computer vision and AI, industrial robots in manufacturing still present significant limitations. Since their emergence during the Third Industrial Revolution, industrial robots have evolved from limited programmable machines into more capable systems with improved control, accuracy and speed [11],

[12]. However, there is still room to better integrate modern approaches, such as AI, to improve flexibility and adaptability in manufacturing processes. Even with these potential benefits, AI adoption in manufacturing is limited by concerns such as “black-box” behaviour, data quality, reliability and validation [4], [13]. As a result, many industrial robot applications continue to rely on preprogrammed behaviours developed through online or offline programming methods [14], [15]. While this approach has proven highly reliable and efficient in structured environments, it struggles in unstructured scenarios, such as when objects arrive on a conveyor in random positions and orientations. Many industrial applications continue to reduce uncertainty by structuring the manufacturing line through fixtures, feeders, predefined locations or controlled part presentation [16], [17]. This approach becomes a bottleneck when development time is considered, since traditional robot deployment often depends on programming methods that require specialist knowledge and substantial programming effort [14], [15]. This process requires highly skilled programmers and is both time-consuming and costly. The resulting lack of flexibility and time-consuming development process limit their usability in applications where object flow is unpredictable, such as in custom manufacturing. This limitation is particularly evident in pick-and-place operations involving random object flow, including bin-picking, feeder-based picking and conveyor-based handling. These problems have been widely studied in industrial robotics because they require object localisation, pose estimation and grasp planning to operate reliably under clutter, occlusion and variable object presentation [3], [17], [18]. Most existing systems focus on flexibility, enabling robots to automatically adjust their pick positions based on the detection of objects. For instance, recent approaches employ advanced vision systems and sensor-based feedback to enhance precision in dynamic environments [19], [20], [21]. Although these systems increase flexibility, they commonly lack adaptability. This limitation highlights the need for further research to develop systems that integrate flexibility and adaptability, ensuring robust performance across diverse scenarios and object types.

In response to these challenges, advanced AI solutions such as reinforcement learning have emerged, demonstrating the potential to enable systems to learn and adapt dynamically. However, despite their potential, learning-based and end-to-end AI approaches remain difficult to deploy in industrial settings when safety, robustness, reliability, data quality, explainability and validation cannot be guaranteed [5], [13]. Furthermore, their unpredictability is a significant barrier, as industrial applications require consistent and reliable behaviour to ensure operational safety and efficiency. These limitations highlight the necessity for a balanced approach that combines the reliability of traditional systems with the flexibility and adaptability of modern AI methods.

One potential solution lies in designing hybrid systems that integrate elements of both traditional and AI-driven approaches to meet industrial demands. Such a system would highlight the strengths of both paradigms, ensuring the robustness and safety demanded by industrial environments while also incorporating the capacity to adapt to new object types and conditions. A system that can be reconfigured for new object types through limited high-level user input, rather than extensive manual reprogramming, could reduce reconfiguration effort and production downtime. This type of system would also expand the applicability of robotics in unstructured environments, such as injection-moulded part handling dropped directly onto a conveyor belt, where objects arrive in varying orientations and positions, logistics operations, where parcels of different sizes

and shapes are sorted on a conveyor, or food processing, where irregularly shaped items like fruits or baked goods must be handled dynamically. By achieving this balance between flexibility, adaptability, and reliability, such a system could bridge the gap between the rigidity of traditional automation and the complexity of fully autonomous robotics.

1.3 Research Aim, Questions and Objectives

The primary aim of this research is to design, implement and experimentally validate a modular, semi-autonomous robotic pick-and-place system for industrial parts arriving in random positions and orientations. The system is designed to combine flexibility in random object flows, adaptability to new object types and reliable execution in a structured manufacturing cell.

The proposed system combines a user-guided configuration interface, hybrid dataset generation, 3D point-cloud perception, template-based pose estimation and robotic execution using the Robot Operating System (ROS). The user defines the target parts and placement locations through the interface, providing the object models and task information needed by the rest of the pipeline. This information is then used to automatically generate hybrid point-cloud datasets and configure the perception and registration stages for the selected objects, reducing the need for manually collected training data or direct robot reprogramming. A physical robotic cell and its digital counterpart were developed to support implementation, testing and validation. The digital layer represents the robot, gripper, conveyor, camera volume and cell structure, while module communication, motion planning, collision checking and visualisation are supported through ROS, MoveIt and RViz.

To achieve this aim, the research is guided by the following central research question:

How can a semi-autonomous robotic pick-and-place system be designed to combine flexibility in random object flows, adaptability to new object types and reliable execution in an industrial manufacturing cell?

This central question is addressed through the following sub-research questions and objectives.

SRQ1. Modular system architecture
How can a ROS-based robotic-cell architecture integrate the physical cell, digital counterpart, perception, planning and execution modules for adaptable pick-and-place?
Objective: Develop a modular ROS-based architecture that supports communication between the physical cell, digital counterpart, user interface, perception, registration, motion planning and robotic execution modules, and reuse this foundation across both experimental stages.

SRQ2. System adaptability through user-guided configuration
How can a robotic pick-and-place system be reconfigured for new object types without requiring extensive manual reprogramming or expert intervention?
Objective: Develop a user-guided configuration workflow that allows target parts, computer-aided design (CAD) models and placement locations to be defined with limited expert input and use this information to trigger task-specific hybrid dataset generation and perception configuration.

SRQ3. Flexibility in random object flows
How can a robotic system detect, localise, classify and pick objects arriving in random positions and orientations, from simple geometric parts to complex industrial shapes?
Objective: Develop and evaluate perception and pick-planning pipelines for random object flows, beginning with classical geometric methods for simple rectangular objects and extending to deep-learning-based instance segmentation and registration for complex industrial parts. The evaluation includes object localisation, semantic class matching and assignment to available placement slots.

SRQ4. Reliability under changing object and scene conditions
How can the system maintain reliable performance across different object types and scene conditions while limiting dependence on unconstrained AI models that are difficult to validate in industrial settings?
Objective: Evaluate how data-driven perception can be combined with object-specific geometric knowledge, template-based localisation, collision checking and motion-planning validation to support reliable pick-and-place execution.

SRQ5. Balance between autonomy and user input
What level of user input is needed to support adaptable pick-and-place while keeping perception, planning and execution sufficiently automated?
Objective: Define a bounded-autonomy workflow in which the user provides high-level task information, such as target objects, CAD models, placement slots and initial parameters, while the system automates dataset generation, perception, pose estimation, object-to-slot assignment, motion validation and execution within predefined limits.

1.4 Scope of the Research

This research is developed and evaluated through two connected experimental stages. The first experiment establishes the foundation of the robotic cell by developing the physical and digital layers, the ROS-based architecture, the MoveIt planning interface and the basic pick-and-place execution pipeline. This experiment validates the integration between the virtual and physical cell using simpler object geometries and constrained top-down picking.

The second experiment extends this foundation to complex industrial parts and adaptive perception by introducing automatic hybrid dataset generation from user-defined object models. It is divided into offline and online validation. Offline, PointGroup models trained under fully synthetic and hybrid scene-generation conditions are evaluated on generated and structured real-scanner data. Registration methods are then evaluated independently using reference object instances, followed by an end-to-end evaluation in which predicted PointGroup instances are passed to the selected DCP and ICP registration configuration. Online, a proof-of-concept robotic experiment is conducted using the developed physical and digital cell, where the system detects parts in random poses, estimates their location and supports pick-and-place execution through the ROS-based planning and control pipeline.

The scope of this thesis is not to develop a general-purpose robot that can manipulate any unknown object in any environment. Instead, the focus is on a structured manufacturing cell where the robot, camera, conveyor and workspace are known, but

the object flow and target parts can change. Together, these experiments investigate whether the proposed approach can reduce the effort required to introduce new parts while preserving the structured validation, modular integration and reliable execution expected in industrial automation.

1.5 Contributions

This research delivers a set of experimentally validated contributions to the development of adaptable and reliable robotic pick-and-place systems for industrial manufacturing. Through the design, implementation and evaluation of a modular semi-autonomous robotic cell, the following primary research contributions are made:

C1. Modular architecture for adaptable industrial pick-and-place
A modular semi-autonomous robotic-cell architecture was developed to integrate user configuration, dataset generation, perception, registration, planning and execution for industrial pick-and-place.

C2. Hybrid point-cloud dataset generation for target industrial parts
A hybrid dataset generation method was developed to combine physics-based object arrangement, object CAD models and real background scans, supporting task-specific training data for point-cloud perception.

C3. Perception and registration validation using hybrid and real data
Instance segmentation and template-based registration methods were evaluated using hybrid datasets and real scans of injection-moulded assemblies, providing evidence of how hybrid training data transfer to real sensor observations.

C4. ROS-based physical and digital robotic-cell implementation
A physical robotic cell and its digital counterpart were implemented using ROS, MoveIt and RViz, enabling motion planning, collision checking, visualisation and pick-and-place execution.

C5. Experimental validation across two connected stages
The proposed approach was validated first through a foundational robotic-cell experiment and then through an extended experiment involving hybrid perception, complex industrial parts and online robotic execution.

1.5.1 Peer Reviewed Publications

1. **Smart Autonomous Part Displacement System based on Point Cloud Segmentation**. ELS Gouveia, R Srivastava, M Singh, S Lyons, E Armstrong, DM Devine ICINCO, 549-554.
2. **Implementing a Vision-Based ROS Package for Reliable Part Localization and Displacement from Conveyor Belts**.
   EL Gouveia, JG Lyons, DM Devine. Journal of Manufacturing and Materials Processing 8 (5), 218.
3. **Hybrid Synthetic–Real Point-Cloud Generation for Sim-to-Real Industrial Robotic Perception.** *(Under review)*
   EL Gouveia, JG Lyons, DM Devine
4. **Bounded Autonomy for Reconfigurable Industrial Pick-and-Place with AI Learning-Based 3D Perception**. *(Under review)*
   EL Gouveia, JG Lyons, DM Devine
5. **SMA-based haptic gloves usage in the smart factory concept: XR use case**.
   R Srivastava, V Kuts, EL Souza Gouveia, N Murray, D Devine, E O'Connell. ASME international mechanical engineering congress and exposition.

6. **Unity and ROS as a digital and communication layer for digital twin application: Case study of robotic arm in a smart manufacturing cell**.
   M Singh, J Kapukotuwa, ELS Gouveia, E Fuenmayor, Y Qiao, N Murry, D Devine
   Sensors 24 (17), 5680.
7. **Comparative Study of Digital Twin Developed in Unity and Gazebo**.
   M Singh, J Kapukotuwa, ELS Gouveia, E Fuenmayor, Y Qiao, N Murray, D Devine
   Electronics 14 (2), 276.

1.6 Thesis Structure

The thesis is organised into the following chapters, each contributing to the design, implementation and experimental validation of a modular semi-autonomous robotic system for adaptable industrial pick-and-place.

Chapter 1: Introduction. This chapter introduces the research topic and presents the background and motivation for developing an adaptable robotic pick-and-place system for industrial manufacturing. It defines the problem context, research aim, research questions, scope and main contributions of the thesis.

Chapter 2: Literature Review. This chapter reviews the literature relevant to industrial pick-and-place, flexible and adaptable manufacturing cells, 3D sensing and perception, point-cloud segmentation, pose estimation, grasp planning, hybrid dataset generation, ROS-based integration, digital representations and human involvement in robotic systems. It identifies the main research gaps and positions the thesis at the intersection of adaptable perception, modular robotic integration and reliable industrial execution.

Chapter 3: Foundational Robotic Cell Development and Validation. This chapter presents the design, implementation and validation of the foundational robotic cell used in the thesis. It describes the physical setup, digital cell representation, ROS-based software architecture, robot communication, motion planning and first computer-vision pipeline. The chapter evaluates the system using rectangular objects in random positions and orientations, validating the integration between sensing, perception, target generation and physical pick-and-place execution.

Chapter 4: Extension to Complex Industrial Parts. This chapter extends the robotic cell developed in *Chapter 3* towards more complex industrial parts and a more adaptable perception workflow. It presents the user-guided configuration process, hybrid point-cloud dataset generation, learning-based instance segmentation and template-based registration methods used to support new target objects. The chapter evaluates these methods using complex injection-moulded assemblies and demonstrates how the validated robotic infrastructure can be reused with an updated perception pipeline.

Chapter 5: Conclusions and Future Work. This chapter summarises the main findings of the thesis and discusses how the research questions were addressed through the two experimental stages. It highlights the main contributions, including the modular robotic-cell architecture, hybrid dataset generation approach, perception and registration validation, and physical pick-and-place implementation. The chapter also discusses the limitations of the work and outlines future research directions for improving adaptability, perception robustness, cycle time and industrial deployment.

References and Appendices. The references section lists all cited works used throughout the thesis. The appendices contain supplementary material, including additional

technical details, experimental data, implementation notes and supporting documentation.

Chapter 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the main concepts and technologies that support the development of an adaptable and reliable robotic pick-and-place system for industrial applications. It begins by discussing conventional industrial pick-and-place, including structured robotic cells, online and offline programming, and the limitations that appear when objects arrive in random positions or when new products must be introduced frequently.

The chapter then defines the key concepts used throughout this thesis: reliability, flexibility and adaptability. These definitions provide the basis for reviewing sensing and perception for random object flows, including 3D sensing, point-cloud preprocessing, semantic and instance segmentation, template-based pose estimation and point-cloud registration.

The review then moves from perception to robotic action by examining grasp planning, motion feasibility and pick-to-place execution under industrial constraints. It also discusses hybrid dataset generation and sim-to-real transfer, with emphasis on generating task-relevant data for perception models when new objects are introduced.

Finally, the chapter reviews ROS-based modular robotic systems, digital counterparts, user-guided configuration and hybrid robotic architectures. These topics are brought together in the final section, where the main research gaps are identified and the proposed system is positioned in relation to the research questions of this thesis.

2.2 Industrial Robotic Pick-and-Place Systems

Industrial robotic pick-and-place remains one of the most established and economically important forms of manufacturing automation. According to the International Federation of Robotics, an industrial robot is an automatically controlled, reprogrammable, multipurpose manipulator with at least three axes for use in industrial automation. The *World Robotics 2025* statistics reported 542 thousand industrial robot installations in 2024 and an operational stock of 4.6 million units worldwide [22]. Figure 2.1 illustrates these trends, showing both the sustained annual demand for industrial robots and the continued growth of the global installed base. This scale of this deployment has two important implications. First, robotic handling is no longer an emerging capability in factory settings. Second, it suggests that the core research challenge has shifted from demonstrating the feasibility of robot-based handling to enabling such systems to deal with increasing product variability while maintaining industrial repeatability [23], [24].

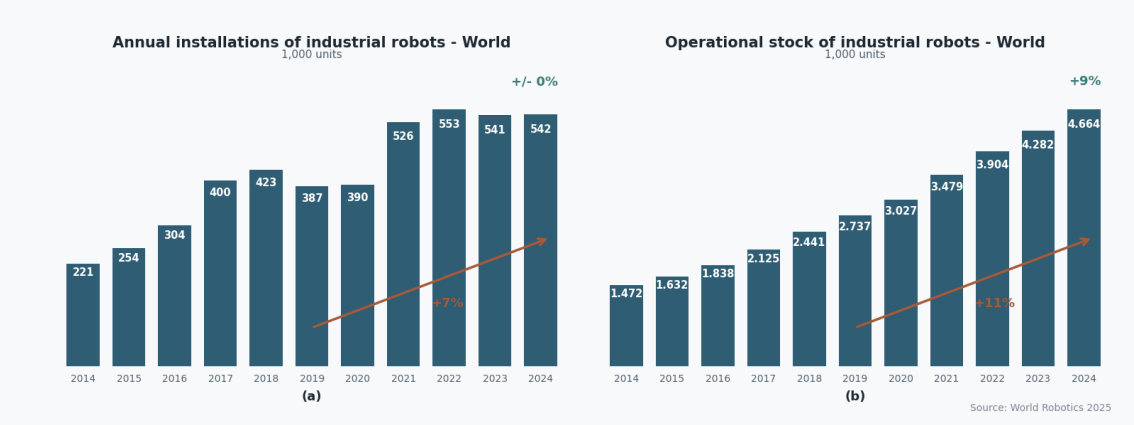


Figure 2.1: Global trends in industrial robot deployment. (a) Annual installations of industrial robots worldwide from 2014 to 2024. (b) Operational stock of industrial robots worldwide over the same period. The figure highlights both the sustained demand for new industrial robots and the continuing growth of the installed base, supporting the view that industrial robotic automation is a mature and expanding manufacturing technology. Source adapted from World Robotics 2025 [22].

2.2.1 Conventional industrial automation and structured environments

The conventional industrial pick-and-place cell is fundamentally a structured system. Parts are commonly presented at known locations in pallets, trays, nests, or fixtures. Robot bases and end-effectors are fixed, approach directions are defined in advance, and destination poses are predetermined. This design philosophy reduces uncertainty at every stage of the cycle, including calibration, motion planning, collision checking, and safety validation. *Tipary et al.* describe this assumption explicitly, noting that traditional robot cells generally presume parts to be available at known locations in a structured way, from which they can be grasped and transferred to predefined target poses [18].

This structured logic aligns closely with factory priorities. Ojer et al. discuss robotic bin-picking in industrial applications, where a robot must identify, localise, grasp, and remove parts that are randomly arranged in a bin or tray. They distinguish this type of industrial handling from laboratory manipulation tasks by observing that industrial practitioners usually work with known parts, custom-designed end-effectors, and controlled contexts. They also highlight that the major constraints in factory deployment are picking rate and cycle time [25]. In other words, industrial handling problems are rarely defined by open-world uncertainty alone. They are defined by the need to deliver precision, cycle efficiency, and operational stability under tightly bounded conditions. Structured robotic cells persist not because they are technologically outdated, but because they offer a reliable way to meet these requirements [18].

The literature, however, shows that the advantages of structure often come at the cost of reduced versatility. When parts are no longer available in precisely arranged fixtures, manufacturers frequently attempt to restore order before robotic handling begins. *Tipary et al.* note that when components are accessible only in semi- or unstructured ways, such as in trays or bins in bulk, the first industrial response is often to structure them prior to further processing [18]. They further argue that traditional part-specific feeders are frequently unsuitable for flexible robotic cells because part variety is high and floor space is limited. *Giannoccaro et al.* reach a similar conclusion in their study of feeding systems, which are mechanisms used to orient, separate, or distribute randomly arranged parts before robotic picking. They argue that the handling

of randomly arranged objects is becoming increasingly relevant in industrial plants, although practical solutions still rely heavily on mechanisms that meter, spread, or redistribute parts so that vision and picking remain manageable within production-time constraints [17].

This literature shows that conventional automation should not be interpreted simply as an outdated alternative to modern robotic systems. Rather, it represents a carefully engineered response to industrial demands for consistency and reliability. However, the more structured the robotic cell, the smaller the degree of uncertainty that must be handled by perception and decision-making. This explains why deterministic pick-and-place from trays, pallets, and precisely located fixtures remains the industrial benchmark against which more flexible systems are evaluated. For this research, this establishes the baseline from which flexibility must be measured. The aim is not to reject structured automation, but to retain its reliability while reducing the dependence on fixed object presentation, part-specific tooling, and extensive manual reconfiguration.

2.2.2 Online/offline programming and reconfiguration limitations

The literature on industrial robot programming helps explain both the success and the limitations of conventional industrial robot deployment. The review by *Pan et al.* provides a clear interpretation of how industrial robot tasks are programmed in practice, identifying online programming and offline programming as two main categories of industrial robot programming [15]. In online programming, a skilled operator uses a teach pendant to guide the robot through the desired poses, while in offline programming, the complete robot cell is modelled in 3D, and programmes are generated and tested virtually before being downloaded to the controller. The appeal of online programming lies in its simplicity. It is intuitive, requires relatively low initial investment, and has long been acceptable for uncomplicated processes and simple geometries. Yet *Pan et al.* also highlight that programme quality depends heavily on operator skill and that changes become difficult once the programme has been created [14], [15].

The cost of this programming paradigm becomes particularly visible when products change frequently. *Pan et al.* report the example of robotic arc welding for a large vehicle hull, where manual programming took more than eight months although the welding cycle itself lasted only 16 hours [15]. The absolute values in that example are application-specific, but the imbalance it illustrates is widely recognisable in industrial robotics: programming overhead may exceed productive machine time, especially when a task must be commissioned, tuned and amended for a new part or layout. This is one reason why high-mix, low-volume manufacturing has historically presented difficulties for conventional robot deployment despite the reliability of robot hardware itself [15], [24], [26].

Offline programming addresses several of these limitations, but it does not eliminate the reconfiguration problem. *Pan et al.* characterise offline programming as more reliable than teach-pendant programming and more flexible with respect to product design changes because it is based on a full 3D model of the robotic cell and allows reach-and-range testing, motion refinement and programme validation before deployment [15]. At the same time, they emphasise that process accuracy usually demands additional calibration, and that introducing an offline programming system often results in substantial modelling effort, capital investment and long delivery times.

More recent works reach a similar conclusion, noting that traditional online and offline programming methods still struggle to balance intuitiveness, safety and efficiency in complex or fast-changing manufacturing tasks [26], [27]. The engineering effort is therefore displaced rather than removed. Instead of relying primarily on a robot operator on the manufacturing floor, the process relies on careful digital modelling and engineering effort in advance.

Later work makes this limitation even more explicit in the context of flexible robotic cells. *Tipary et al.* argue that, under conditions of mass personalisation and increasingly flexible manufacturing, conventional robot automation of workpiece manipulation may seem solved only as long as the task remains sufficiently structured. They note that on-site programming methods are intuitive and easy to use, but often not applicable in flexible scenarios, while offline programming still requires careful modelling and planning. Their response is a digital-twin-based development methodology. In this context, a digital twin refers to a virtual representation of the robotic cell that can be used to model, test, validate, and reconfigure robot operations before changes are implemented in the physical system. This methodology is intended to speed commissioning and reconfiguration and to reduce the amount of work needed in the physical cell. This represents an important shift in emphasis: reconfiguration is no longer treated purely as a programming problem, but as a defining systems challenge for modern pick-and-place robotic cells [16], [28]. This line of thinking is expanded by *Asif et al.*, whose Reconfigurable and Responsive Robot Manufacturing architecture is explicitly designed to adapt to unpredictable product variants and demand changes. Their framework integrates automated programme generation to reduce reliance on human input and emphasises modular, reconfigurable components to reduce adaptation cycles. Even though such architectures are not limited to classical online/offline programming, they address the same industrial problem: the dominant obstacle is not robot motion itself, but the effort required to produce, validate, and maintain robot behaviour when products, layouts, or process requirements change [29], [30].

The literature therefore suggests that programming methods should be understood historically. Online programming was well suited to highly structured cells with stable, repetitive tasks. Offline programming improved safety, productivity and pre-deployment validation in more complex cells, but its benefits came with calibration and modelling overheads [14], [15]. Contemporary reconfigurable manufacturing research emerges because neither approach, in isolation, resolves the central difficulty of rapid, reliable changeover in modern production environments [16], [29], [30]. This motivates the need for a system in which reconfiguration is partially transferred from manual robot programming to a modular software pipeline, where user-defined object and placement information can be converted into perception, planning, and execution resources with minimal intervention.

2.2.3 Industrial pick-and-place with random object flows

The limitations of conventional automation become most visible when parts are no longer presented in known poses. In many industrial contexts, components arrive in bulk, on feeder surfaces, in trays with uncertain pose, or on conveyors with random spacing and orientation. Under these conditions, pick-and-place can no longer be reduced to replaying a predefined and validated motion sequence. *Tipary et al.* describe this shift clearly. Whereas traditional cells assume structured part presentation, semi- or unstructured access requires sensory localisation and introduces additional planning

complexity if reliable, collision-free operation is to be maintained [18]. In their work on highly constrained industrial environments, they go so far as to reformulate a bin-picking problem as a semi-structured pick-and-place problem to achieve reliable factory operation, separating offline preparation from online planning so that computation and safety can remain tractable.

Earlier vision-guided handling studies show how this problem first entered the industrial robotics literature. *Tsarouchi et al.* proposed a camera-based method for calculating part-feature coordinates in a world reference frame and used it to identify multiple randomly placed objects of complicated geometry in a consumer-product manufacturing setting. In a later study, the same group described a generic method for recognising 3D randomly placed objects for robotic handling by combining a 2D vision system with CAD files to generate 3D coordinates. However, they also emphasised that image-based systems still faced open issues with respect to simplicity, low cost and limited maintenance [19]. These studies were important because they demonstrated that random object presentation was already being addressed as an industrial problem, while also making clear that deterministic robot programming alone could no longer be sufficient once the presentation of parts became variable.

More recent survey work confirms that random-flow manipulation remains challenging and clarifies why. The review by *Cordeiro et al.* on visual perception for robotic bin-picking shows that the field must be analysed through three tightly coupled stages: object localisation, pose estimation and grasp prediction. They emphasise that industrial picking increasingly involves low-texture or complex-shaped objects in unstructured and cluttered environments, often with occlusion and dynamic workspaces [3]. This matters because it shifts the literature away from a narrow concern with object detection accuracy and towards an integrated understanding of how scene complexity affects the whole manipulation chain. In random industrial flows, localisation quality, pose certainty and grasp generation are interdependent rather than separable.

Industrial case studies also show that random pick-and-place is limited not only by perception, but also by cycle-time requirements. *Giannoccaro et al.* show that, although bin picking is becoming increasingly central to industrial object handling, only a limited number of plug-and-play industrial solutions exist, and random picking remains more a research topic than a fully standardised technology. Their work is revealing because it treats the uncontrolled object pose problem not only as a recognition issue, but also as a workflow problem. By using a multi-component feeder with two working surfaces, they hide the viewing time of the vision system within the overall handling cycle, regulate the number of objects delivered to the scan surface and improve the distribution of pickable parts. The result is an average handling time of roughly 1.5 seconds per piece for randomly arranged aluminium components [17]. The underlying message is that industrial viability depends on eliminating dead time and orchestrating perception and handling in parallel, not only on improving recognition rates.

Ojer et al. provide a complementary and highly relevant perspective by contrasting academic and industrial bin-picking scenarios directly. They note that academic research often targets unknown daily-life objects in uncontrolled environments and is typically validated at low levels of applicability for industrial deployment, whereas industrial systems usually manipulate known parts in more controlled settings, but under much stricter requirements for cycle time and picking rate.

They further argue that deep learning is now common in bin-picking perception, yet data requirements and computational cost can extend the execution cycle unless the overall workflow is redesigned. Their contribution is therefore significant not only for perception, but for problem framing: industrial random-flow pick-and-place is best understood as a whole-system optimisation problem involving localisation, grasp calculation, collision-free motion and cycle-time management simultaneously [25].

The same conclusion emerges from conveyor-based systems. *Ozer et al.* describe a real-time robotic stacking line in which visually classified objects move randomly on a conveyor and are picked and placed through tightly synchronised communication between vision software, programmable logic controller (PLC) logic and the robot controller. After environmental and mechanical refinement, the system reportedly operated over 9600 packages across five days with sustained accuracy and cycle times of 10 to 11 seconds [20]. Although conveyor sorting differs from dense 3D bin-picking, it highlights a similar systems problem. Once material flow becomes variable, successful operation depends on coordinating perception, actuation, transport logic, and industrial communication, rather than improving any single module in isolation.

Taken together, the literature on random industrial object flows shows a transition from fixed object presentation to perception-guided manipulation under unstructured object flow conditions. What distinguishes factory applications from laboratory demonstrations is not simply that the parts are more difficult to perceive. The central issue is that object-flow variability must be managed while preserving placement accuracy, collision safety, and repeatable behaviour. In this thesis, the emphasis is not on achieving the shortest possible cycle time, but on improving the adaptability of the robotic cell when new objects, layouts, or presentation conditions are introduced. This explains why random-flow pick-and-place is relevant to the present research. It exposes the limits of conventional programming and motivates the development of reconfigurable, perception-driven architectures that can support flexible operation. The proposed system targets random object flows by combining user-defined object information with computer-vision-based localisation and modular reconfiguration, so that new parts can be detected, selected, and placed without rebuilding the entire robotic cell.

2.3 Reliability, Flexibility and Adaptability in Manufacturing Cells

Before reviewing the perception, registration, planning, and software methods used in adaptable pick-and-place systems, it is necessary to clarify the criteria used to evaluate them in this thesis. The previous section showed that conventional industrial robotic cells achieve high reliability by reducing uncertainty through structured layouts, fixed part presentation, and extensive programming. However, random object flows and frequent product changes introduce two different types of variation: variation during execution and variation between tasks. For this reason, this thesis distinguishes between reliability, flexibility, and adaptability as three related but distinct requirements for industrial robotic cells.

In manufacturing robotics, the ability to deal with variation is often discussed through the concept of flexibility. However, flexibility is a broad term and is not used consistently across the literature. Some studies use it to describe variation during operation, such as changes in product mix, routing, part position, or process sequence,

while others use it more generally to describe the capacity of the system to respond to change [6], [7], [8]. This broad use becomes problematic in industrial pick-and-place, where handling objects in random positions is not the same as preparing the cell for a new object or task.

To separate these ideas, this thesis distinguishes between flexibility and adaptability. The distinction is informed by manufacturing flexibility literature and by work on reconfigurable manufacturing systems, where changing system functionality or configuration in response to new requirements is treated as a specific capability rather than simply another form of flexibility [9], [10]. This distinction is important because many pick-and-place systems can respond to pose variation during execution, but still require expert intervention when a new product, placement layout, or handling task is introduced.

Reliability provides the third and fundamental requirement. A system that is flexible or adaptable is not necessarily suitable for industrial deployment unless it can perform repeatedly, safely, and predictably For this reason, recent work on ROS-based manipulation, human-in-the-loop (HITL) robot learning and low-code robot programming is relevant to this thesis. These approaches aim to increase system autonomy while still preserving human input, validation procedures and reliable robot control [31], [32], [33].

2.3.1 Defining reliability, flexibility and adaptability

For this thesis, the distinction between flexibility, adaptability and reliability is based on the type of challenge the robotic cell must address. Flexibility concerns variation during execution. A flexible pick-and-place system is expected to handle changes in object position, orientation, spacing, clutter, object count or arrival sequence, while still working within a known task. In this case, the object type, placement logic and task objective are already defined, but the scene conditions vary from cycle to cycle.

Adaptability concerns change between tasks. It refers to the ability of the robotic cell to be prepared for new object types, placement requirements or handling tasks with limited expert intervention. This interpretation is close to the concept of reconfigurability in manufacturing systems, where system functionality or capacity is modified in response to new requirements [9]. However, this thesis uses adaptability as a broader and more practical term. It focuses mainly on software-based and user-guided change, such as adding object models, defining placement slots, generating perception resources, and reusing the same architecture for related tasks.

Reliability concerns dependable execution. A system that is flexible or adaptable is only useful in an industrial context if it can perform its task repeatedly, safely and predictably. In the proposed system, reliability depends on perception accuracy, calibration, feasible motion planning, collision avoidance, placement accuracy and understandable failure behaviour. It is therefore not limited to the robot controller, but also depends on the perception pipeline, user-defined inputs and integration between software modules.

2.3.2 Flexible versus adaptable pick-and-place systems

Many pick-and-place systems in the literature are flexible because they can handle random object positions, clutter, or uncertain object orientation. For example, perception-based picking pipelines typically combine sensing, object localisation, pose

estimation, and grasp prediction to cope with variation during execution [3]. Industrial examples such as vision-guided feeding cells also show that high flexibility can be achieved when the object family, feeder design, and handling objective are already known.

A recurring limitation in the literature is that many systems described as flexible still externalise adaptability to the engineer. Reprogramming, retraining, feeder redesign, or interface-specific tuning often remain outside the reported performance envelope. This gap is visible in end-user programming studies. *Halim et al.* note that robotic programming still requires considerable time, effort, and special training, especially in small-batch contexts and across heterogeneous platforms. Recent reviews of robotic low-code software similarly argue that low-code systems are needed because conventional programming workflows remain a major obstacle to wider deployment [16], [29], [33].

2.3.3 The need for bounded autonomy in manufacturing

Recent reviews make clear that AI is becoming indispensable in manufacturing robotics, especially for perception, planning under uncertainty, and handling complex task variation. *Gao et al.* survey AI applications across planning, optimisation, quality assurance, maintenance, automated assembly, and disassembly [4]. *Liu et al.* show that robot learning now spans imitation learning, policy gradients, actor-critic methods, model-based learning, and applications in grasping, assembly, process control, and industrial human-robot collaboration [34]. Intelligent methods are expanding the feasible range of manufacturing automation.

However, the same literature also makes clear why unbounded autonomy is still difficult to justify in industrial cells. *Ahangar et al.* identify several barriers to industrial AI adoption, including the black-box nature of many AI models, where internal decision-making processes are difficult for users to interpret or verify, as well as data quality issues, reliability concerns, regulatory uncertainty, cybersecurity, and workforce adaptation [13]. *Chen et al.* add that HITL robot learning remains in its infancy in human-centric smart manufacturing, despite its promise for improving performance through human prior knowledge [32]. Taken together, these findings imply that the industrial problem is not whether to use autonomy, but how to limit and structure it so that operators, engineers, and plant managers can still understand, verify, and recover from system decisions.

This is where the notion of bounded autonomy becomes beneficial. The term is used here as a synthesis of several literature streams rather than as a universally standardised label. In this synthesis, bounded autonomy denotes a design in which autonomous modules are allowed to handle high-dimensional perception, candidate generation, or dynamic function allocation, but only within explicit technical and organisational bounds. Those bounds include confidence thresholds, verification steps, human approval where needed, auditability, fallback routines, and low-level controller safety constraints. *Bernabei et al.* argue that a complete transition from static to adaptive automation is not expected to be suitable in all manufacturing settings. In high accident-risk processes, a very high level of automation with minimal human-machine interaction may be preferable [35]. In collaborative settings, adaptive transformations may be valuable, but only if feasibility and benefits are carefully demonstrated. *Rieth et al.* reach a similar conclusion in a different domain. They show that adaptable automation can

improve perceived autonomy and satisfaction without reducing performance, provided that the operator remains in control of task delegation [36].

The human-centred programming literature converges on a similar design logic. *Halim et al.* show that no-code multimodal programming can dramatically reduce teaching time and effort compared with conventional practice [37]. *Wang et al.* argue that mixed-reality low-code programming should elevate the operator from low-level trajectory drawing to high-level task guidance, with the system generating refined trajectories [33]. *Dong et al.* push the idea further by embedding Large Language Model (LLM) reasoning into an industrial robotic low-code system but notably retain human verification as part of the multi-agent pipeline for intent recognition and parameter extraction [38]. Official ROS-Industrial guidance articulates the same principle from the industrial software side, namely combining ROS high-level functionality with the low-level reliability and safety of industrial robot controllers rather than replacing existing controller technologies altogether. This convergence supports a strong literature-based case for bounded autonomy as the most credible route for manufacturing cells that need more intelligence without surrendering reliability.

2.4 Sensing and Perception for Random Industrial Object Flows

In robotic systems, sensing and perception are closely related but not equivalent. Sensing refers to the acquisition of raw information from the physical environment through devices such as cameras or depth sensors. Perception refers to the interpretation of this sensor data into information that the robot can use for decision-making and action, such as object location, object class, surface geometry, pose or free space. This distinction is reflected in robotics literature, where sensor measurements are treated as the input to higher-level estimation, mapping, recognition and planning processes [39], [40]. In industrial pick-and-place, sensing therefore provides the raw 3D data, while perception converts that data into task-relevant outputs such as segmented objects, pose estimates and graspable regions.

Perception is the main technical bottleneck in robotic pick-and-place under random industrial object flow. In structured cells, the robot, conveyor, and camera volume are fixed, but the parts are not. Objects arrive with unpredictable poses, contact between parts, partial overlap, and sometimes strong self-occlusion. Reviews focused on bin-picking and industrial point-cloud learning show that successful perception depends on a sequence of connected stages. First, sensing acquires raw data from the environment. Preprocessing then cleans, filters, crops, or downsamples the point cloud. Segmentation separates objects from the background and from each other. Pose estimation or refinement then estimates the position and orientation of each object more precisely [3], [34]. These stages are linked because object localisation, pose estimation, and grasp planning inherit the errors made earlier in the pipeline. Figure 2.2 illustrates this general perception-reasoning-action loop. Sensor data from the physical environment are converted into an environment representation, which is then used by AI, machine learning (ML) or planning methods to support robotic action. In the context of this thesis, the figure is useful because it shows where the perception pipeline sits within the broader robotic system. Sensing and environment representation provide the information on which localisation, pose estimation, grasp planning, and motion execution depend.

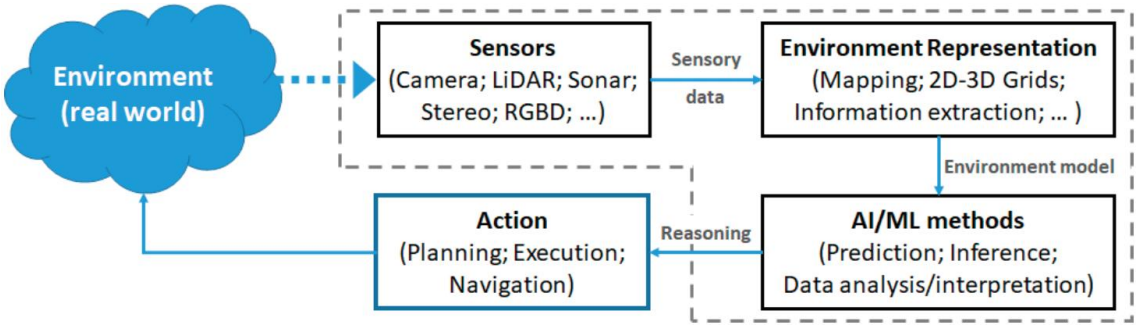


Figure 2.2: General perception–reasoning–action loop in robotic systems, showing how sensor data are converted into an environment representation and used by AI/ML methods to support planning and execution. Adapted from [41].

For conveyor pick-and-place, sensor choice should be based on the task requirements rather than general capability. Short-range industrial robotic cells usually benefit from dense close-range 3D sensing, especially structured light, active stereo, or short-range time-of-flight. Light Detection and Ranging (LiDAR) remains valuable where the sensing envelope is larger or ambient-light robustness is dominant, but it is less naturally aligned with dense close-range part localisation. Across sensor families, the hardest practical artefacts are consistent in the literature: reflective or dark surfaces, missing depth, edge artefacts, multipath or flare, occlusion, and density variation across the field of view. These effects directly affect segmentation quality and bias registration if not addressed early [42], [43], [44].

Preprocessing should be understood as a core part of the perception pipeline, not only as a cleanup stage. In industrial point-cloud pipelines, downsampling, outlier filtering, background masking, normal estimation, density checks, and voxelisation determine whether the following segmentation model receives a geometrically consistent but computationally manageable scene. Organised point clouds, which preserve the sensor's image-like layout, are particularly useful in manufacturing because they allow connectivity-based filtering and efficient removal of floating artefacts that standard outdoor point-cloud filters often miss [45].

For segmentation, recent work can be grouped into point-based, voxel-based, and projection-based methods. Point-based methods process raw point sets directly. Examples include PointNet, which learns global features from unordered point clouds, and PointNet++, which extends this approach by learning hierarchical local features at multiple spatial scales [46], [47]. Dynamic Graph Convolutional Neural Network (DGCNN) further develops point-based processing by dynamically constructing local neighbourhood graphs to capture geometric relationships between points [48]. Voxel-based methods convert the point cloud into a 3D grid or sparse voxel representation, allowing convolutional processing while improving scalability for larger scenes. Examples include sparse convolutional networks and Minkowski networks [49], [50]. Projection-based methods convert 3D data into a 2D representation, such as a depth, range, or spherical image, and then apply efficient image-based networks. This approach is common in LiDAR segmentation, as seen in SqueezeSeg and related range-view methods [51].

For the type of close-range industrial pick-and-place considered in this thesis, the most relevant methods are those that preserve enough geometric detail for later pose estimation and registration. Projection-based methods can be efficient when the sensor naturally provides organised depth images, but they may lose fine 3D detail when

the data are projected and reconstructed. Point-based methods preserve geometry more directly, while voxel-based methods can provide a useful compromise between geometric structure and computational efficiency. In cluttered industrial scenes with touching objects, instance segmentation remains harder than semantic segmentation. For this reason, segmentation quality should be judged not only using standard metrics such as Mean Intersection over Union (mIoU) and Average Precision (AP), which respectively evaluate region overlap and detection quality, but also using practical criteria such as instance separation, false merges, background rejection, latency, and confidence calibration [52], [53], [54]. The specific evaluation metrics used in this thesis are detailed in the methodology chapter.

The review therefore focuses on sensor families, preprocessing choices, and segmentation method classes that are most relevant to dense close-range 3D perception in conveyor pick-and-place. The specific sensor and implementation choices used in this research are discussed later in the methodology chapter.

2.4.1 3D sensing and point cloud preprocessing

Industrial random-flow pick-and-place requires 3D perception that is accurate, repeatable, and fast enough to support robotic action before the object leaves the working area. Several shape-acquisition methods can be used to obtain 3D information, ranging from contact-based approaches to non-contact optical and non-optical techniques [55]. However, robotic pick-and-place normally favours non-contact optical sensing because the object must be measured without disturbing its position on the conveyor. Figure 2.3 summarises this broader taxonomy and highlights the active optical methods most relevant to this research, including time-of-flight, structured light and active stereoscopy.

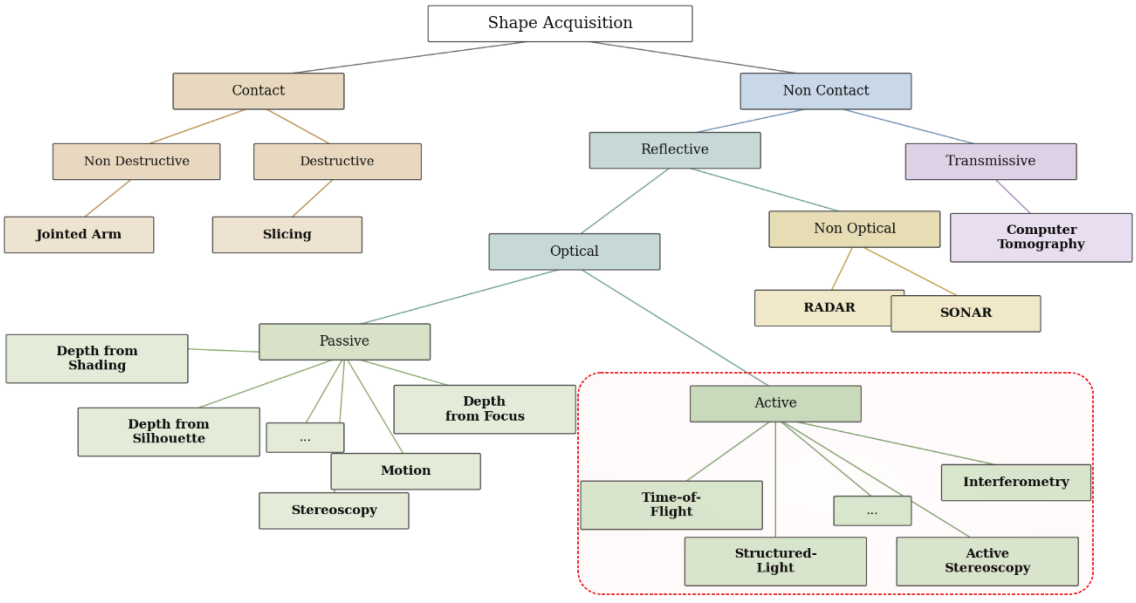


Figure 2.3: Shape-acquisition taxonomy, showing contact and non-contact methods for acquiring 3D information. This thesis focuses on non-contact optical sensing, particularly the active approaches highlighted by the dashed red box, which are suitable for close-range industrial pick-and-place. Adapted from [55].

For close-range conveyor-based pick-and-place, dense optical 3D sensing is particularly relevant because the robot must recover object shape, position and orientation from parts that may be randomly placed, partially occluded or in contact with one another [42], [43], [56]. Structured-light systems are well suited to close-range

industrial perception because they can provide dense and geometrically detailed point clouds. However, their performance depends on both the acquisition mode and the properties of the scanned parts. Reflective, dark, transparent, or highly occluded surfaces can produce missing depth, boundary noise, or distorted local geometry [42], [57]. Moreover, sequential structured-light scanners project one or more patterns during the scan, so the scene often needs to remain static while the patterns are captured [58]. In conveyor applications, this can require a stop-and-scan strategy, triggering, or a scan time short enough that object motion is negligible. This limitation is sensor-specific rather than inherent to all structured-light systems, since snapshot or parallel structured-light cameras have been developed specifically for dynamic scenes [59], [60].

In this thesis, the sensing stage is treated as a static scan of the working area because the priority is accurate and repeatable object localisation and pose estimation rather than minimum cycle time. The aim is not to identify a single best sensor for all applications, but to use a sensing approach that supports accurate point-cloud processing for industrial pick-and-place. Since the experimental system relies on dense close-range 3D scans, the perception pipeline must preserve the geometric features needed for segmentation, registration and later grasp planning. This is especially important for injection-moulded parts, where edges, cavities and repeated geometric features can strongly affect both object localisation and pose estimation. To illustrate the effect of common preprocessing stages, a sample pipe-fitting point cloud supplied by Photoneo with the PhoXi Control software was used. The original point cloud was processed in this work to generate the ROI-cropped, background-reduced, outlier-filtered and downsampled representations shown in Figure 2.4.

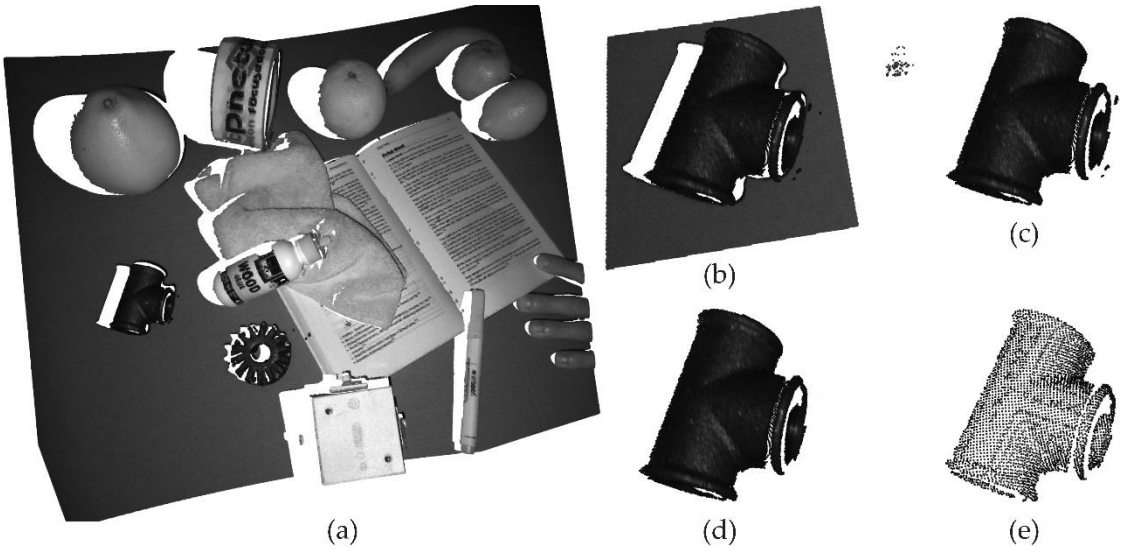


Figure 2.4: Example of point-cloud preprocessing stages applied to a sample pipe-fitting scan supplied by Photoneo with PhoXi Control software [61]. (a) Original point cloud. (b) ROI-cropped point cloud. (c) Background-reduced point cloud. (d) Outlier-filtered point cloud. (e) Downsampled or voxelised point cloud used as the segmentation-ready input for robotic perception and registration. The representations shown in (b)-(e) were generated through preprocessing performed in this work.

Preprocessing converts the raw scan into a point cloud suitable for segmentation, registration and motion planning. As illustrated in Figure 2.4, this process typically begins with a raw scene point cloud and progressively reduces irrelevant or unstable data through region-of-interest (ROI) cropping, background removal and outlier filtering before applying downsampling, voxelisation or normal estimation. In this example, a scanned pipe-fitting object is used to show how preprocessing isolates the

object of interest from the surrounding scene while preserving the geometric information required for later perception tasks.

These steps are especially important in classical geometric pipelines, where object isolation depends mainly on fixed thresholds, workspace limits and hand-designed filters. Outlier filtering is particularly important in manufacturing scenes because floating points can affect part segmentation, obstacle avoidance and gripping attempts [45]. Cropping and background removal reduce the influence of static cell elements such as the conveyor, fixtures or surrounding structure, while downsampling lowers computational cost. Normal estimation can then provide local surface information for pose refinement and registration, as shown in pose estimation pipelines that combine depth and normal features after instance segmentation [62]. Density-aware filtering is also important for registration when missing depth or reflective surfaces distort local geometry [63].

However, preprocessing should not be treated as a fixed recipe. As illustrated in Figure 2.5, the same pipe-fitting point cloud can produce very different results depending on the selected filtering parameters. Insufficient preprocessing can preserve background artefacts and isolated noisy points, which may lead to false object boundaries or unstable registration. Balanced preprocessing reduces these artefacts while preserving the main object geometry needed for segmentation and pose estimation. Excessive preprocessing, however, can remove thin edges, small gaps or surface details that help distinguish touching parts or support accurate alignment.

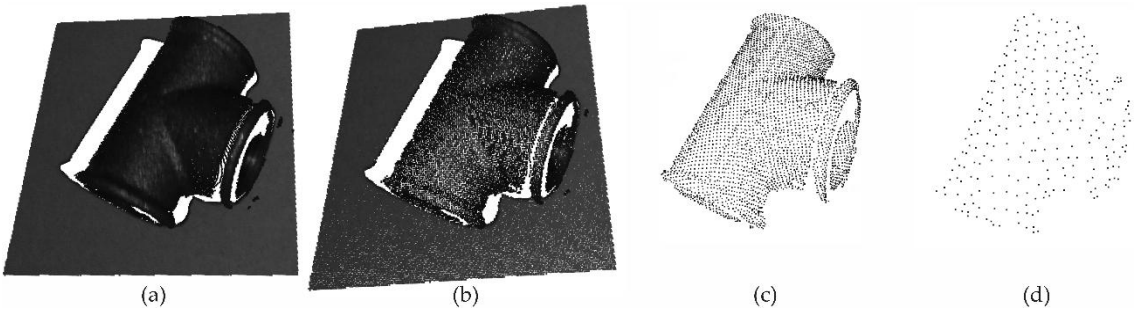


Figure 2.5: Effect of preprocessing intensity on point-cloud quality for the scanned pipe-fitting object. (a) Original point cloud with 58934 points. (b) Under-processed point cloud with 34669 points, where noise and background artefacts remain. (c) Balanced preprocessing with 3458 points, where artefacts are reduced while object geometry is preserved. (d) Over-processed point cloud with 269 points, where useful geometric details are removed. The processed examples were generated in this work.

Learning-based segmentation can reduce some dependence on manually tuned preprocessing by learning to separate object instances from background and clutter, as shown in point-cloud instance segmentation methods such as PointGroup, where semantic prediction and instance grouping are learned directly from 3D scene data [64]. This does not remove the need for preprocessing, but it can shift part of the dependency from hand-coded filtering to data-driven scene understanding. For this reason, preprocessing must balance geometric consistency, computational efficiency and robustness to scene change. This connects directly to semantic and instance segmentation, where missing depth, background artefacts, uneven density or poor boundary preservation can cause false merges, false splits or incorrect object masks [65].

2.4.2 Semantic and Instance Segmentation in 3D Point Clouds

Figure 2.6 summarises the development of representative 3D point-cloud segmentation methods over time. It shows the progression from early point-based semantic segmentation networks, such as PointNet-based methods, towards instance-level methods and more recent transformer-based approaches. In this context, transformer-based approaches refer to methods that use attention mechanisms to model relationships between points or regions in the point cloud, allowing the network to capture wider spatial context [66]. The timeline is useful here because it shows that semantic and instance segmentation have developed as related but distinct research directions, both of which are relevant to robotic perception.

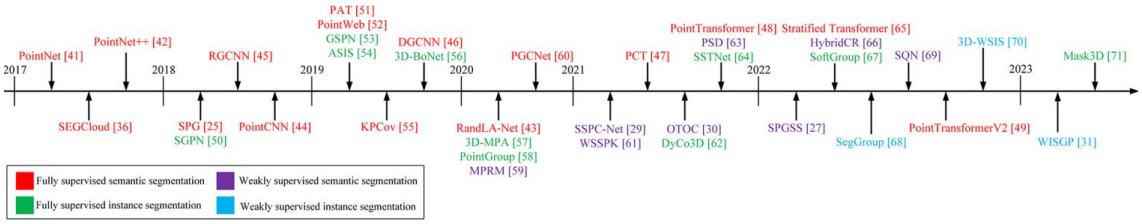


Figure 2.6: Timeline of representative 3D point-cloud semantic and instance segmentation methods, including fully supervised and weakly supervised approaches. The progression from PointNet-based methods to instance-level and transformer-based architectures illustrates the increasing focus on object-level scene understanding in 3D perception. Adapted from [54].

The preprocessing stage described in Section 2.4.1, 3D sensing and point cloud preprocessing, prepares the point cloud for scene understanding. In random industrial object flow, this scene understanding may involve classification, semantic segmentation and instance segmentation. Classification assigns a label to an object, crop or scene, making it useful when the object has already been isolated. Semantic segmentation extends this idea to the point level by assigning a class label to each point, which allows the system to distinguish target parts, background, conveyor surfaces and non-target objects. Instance segmentation goes further by separating individual object occurrences. This distinction is essential in conveyor pick-and-place because several objects may belong to the same semantic class but still need to be picked, registered and placed as separate physical parts [52], [64].

For robotic action, instance segmentation is often more critical than semantic segmentation alone. A semantic mask, meaning the set of points predicted as belonging to a particular class, may indicate where the target class is located, but it does not necessarily tell the robot how many individual parts are present or where the limits of each object instance are. In cluttered conveyor scenes, touching or overlapping objects can produce false merges, where multiple parts are treated as one instance, or false splits, where one part is divided into several fragments. Both errors are problematic because they propagate directly to registration, pose estimation and grasp planning [52], [64], [65].

These segmentation approaches differ in how they represent and process 3D data, but their relevance to industrial pick-and-place depends on the quality of the object-level output they produce. For this thesis, the key requirement is not only to classify points correctly, but to produce instance masks that preserve the geometry needed for registration and grasp planning. This means that segmentation must separate touching parts, reject conveyor and background points, and retain edges, cavities and local features that support pose estimation. Therefore, the choice of segmentation

method is closely linked to the sensing and preprocessing choices discussed in Section 2.4.1.

PointGroup is a relevant example for this thesis because it treats 3D instance segmentation as a combination of semantic prediction and instance grouping. Instead of relying only on hand-coded background removal or fixed geometric thresholds, the model learns to group points that belong to the same object instance [64]. This is important for adaptable pick-and-place systems because it can reduce dependence on manually tuned preprocessing when the background, object arrangement or scene structure changes. However, the output is still only useful for robotic manipulation if the predicted instance contains the geometry required for registration and grasp planning.

Evaluation also needs to be interpreted from an industrial perspective. Standard metrics such as mIoU and AP, are useful for comparing segmentation models, but they do not fully describe whether the output is suitable for robotic manipulation. For pick-and-place, an instance mask should also be judged by whether it rejects background points, avoids false merges and false splits, preserves registration-relevant geometry, and can be produced within the required processing time [52], [54]. In this sense, the best segmentation result is not necessarily the one with the highest benchmark score, but the one that produces stable, registration-ready object instances.

Benchmark selection reinforces this issue. Many 3D segmentation methods are evaluated on indoor or autonomous-driving datasets, which are useful for comparing method families but do not fully represent conveyor-based industrial picking. Manufacturing scenes often involve close-range sensing, repeated rigid parts, contact between identical objects, reflective or with low texture materials and strict requirements for downstream pose estimation. More industrially focused datasets and benchmarks, including bin-picking datasets and industrial benchmarks designed for industrial object detection and 6D pose estimation, are important for evaluating whether segmentation methods transfer to realistic manipulation tasks [56].

Overall, semantic and instance segmentation form the bridge between raw point-cloud perception and object-level robotic manipulation. In this thesis, segmentation is treated not as an isolated classification problem, but as the stage that produces object-level point clouds for registration, grasp planning and motion execution. The following sections examine how segmented object point clouds can be aligned with CAD or template models to estimate object pose and support reliable pick-and-place execution.

2.5 Template-Based Pose Estimation and Point Cloud Registration

Robotic pick-and-place requires not only object detection, but also an estimate of the position and orientation of each object in the working area. Pose estimation and point-cloud registration address this requirement by aligning observed 3D data with a known object representation. In template-based pose estimation, the known representation is typically obtained from CAD data, an STL model, or a reference scan. The registration process estimates the rigid transformation between the observed object and the template, producing a pose that can support grasp planning, placement definition and robot motion execution [67], [68], [69].

This step is particularly important in industrial pick-and-place because the object class may already be known, but its pose is not. Segmentation can identify and

isolate a candidate object instance, but registration is needed to estimate how that instance is arranged in 3D space. In this study, this is relevant because the target parts are known industrial objects, and the perception system is expected to use object-specific geometric information to support reliable localisation. The literature on point-cloud registration therefore provides the basis for understanding how segmented object crops can be aligned with templates, CAD models or previously generated object point clouds [68], [69], [70].

Point-cloud registration methods can broadly be divided into classical, learning-based and hybrid approaches. Classical methods rely on geometric features, correspondences and optimisation. Deep learning-based methods learn features or correspondences directly from data. Hybrid pipelines combine both, often using learned methods for coarse alignment and classical optimisation for refinement. This structure is useful for industrial robotics because each approach has different strengths. Classical methods are interpretable and well established, deep models can improve robustness to noise and partial overlap, and hybrid methods can combine global and local alignment [71], [72]. The following subsections review these three categories and their relevance to industrial pick-and-place.

2.5.1 Classical registration methods

Classical registration methods estimate the transformation between two point clouds using geometric correspondences and iterative optimisation. The most widely known method is Iterative Closest Point (ICP), introduced by Besl and McKay, which repeatedly assigns closest-point correspondences and updates the rigid transformation until convergence [67]. ICP and its variants remain widely used because they are simple, efficient and effective when the initial alignment is close to the correct solution. Point-to-plane ICP and related variants improve convergence by using surface normals, making them especially useful when local surface orientation is reliable [73], [74].

However, ICP is a local optimisation method. It can converge to an incorrect solution when the initial pose is poor, when the object is symmetric, when there is partial overlap, or when the scene contains missing or noisy depth [75], [76]. These limitations are common in industrial pick-and-place, where objects may be partially occluded, touching other parts, or captured from a single viewpoint. For this reason, classical pipelines often combine ICP with a global initialisation step [75], [77].

Global registration methods address this limitation by estimating a coarse alignment before local refinement. A common strategy is to compute local geometric descriptors, which describe the local surface shape around selected points, match candidate correspondences between the observed object and the template, and use Random Sample Consensus (RANSAC), an iterative method for rejecting incorrect matches, to estimate an initial transformation [78]. In point-cloud registration, Fast Point Feature Histograms (FPFH) are often used as local descriptors and combined with sample-consensus alignment to provide an initial pose that can later be refined with ICP [77]. Other global methods, such as Fast Global Registration, optimise over candidate correspondences more directly and can align partially overlapping surfaces without requiring an initial pose close to the solution [75]. These methods improve robustness compared with ICP alone, but their performance still depends on descriptor quality, sufficient overlap, reliable normals and the distinctiveness of the object geometry.

The main limitation of classical registration is not that it is obsolete, but that it is sensitive to the quality of the segmented object crop and the initial alignment. In industrial scenes with repeated geometry, reflective surfaces, missing depth or similar-looking parts, hand-designed descriptors may produce ambiguous correspondences. This motivates the use of learning-based registration methods that attempt to learn stronger features and more robust correspondences from data.

2.5.2 Deep learning-based registration

Deep learning-based registration emerged to address some of the limitations of hand-crafted features and local optimisation. Early learning-based approaches built on the success of PointNet-style point-cloud representation. PointNetLK, for example, combined PointNet features with a Lucas-Kanade-style alignment procedure, showing that learned global point-cloud features could be used to estimate rigid transformations without relying entirely on manually designed local descriptors [79]. In a pick-and-place context, this means that a segmented object point cloud can be aligned with its template more directly, reducing the dependence on hand-crafted geometric features before the pose is passed to grasp planning. Deep Closest Point (DCP) then introduced an end-to-end registration framework based on learned point embeddings, attention-based matching and a differentiable singular value decomposition (SVD) layer to estimate the rigid transformation [80]. Although later models have improved robustness in more difficult settings, DCP remains important because it provides a clear and widely used formulation of registration as learned correspondence estimation followed by transformation recovery.

More directly relevant to real object-pose estimation, Dang et al. investigated limitations that arise when learning-based registration methods such as DCP are transferred from controlled synthetic conditions to real point-cloud data. They identified two principal issues: sensitivity of SVD-based training to large rotational variation and differences between the feature distributions of the source and target point clouds. To address these issues, they proposed Match Normalization together with direct negative log-likelihood (NLL) supervision of point correspondences rather than pose supervision through the SVD stage [81]. These adaptations are particularly relevant when a complete object template must be registered against a partial sensor observation.

Later approaches focused more explicitly on partial overlap, outliers and weak correspondences. RPM-Net introduced differentiable soft matching and annealing to improve registration under noise and partial visibility [82]. PREDATOR addressed low-overlap registration by learning overlap-attention features and identifying points likely to lie in overlapping regions [83]. GeoTransformer further advanced this direction by using geometric transformer features that encode pairwise distances and angular relationships, improving correspondence quality in difficult registration settings [84]. For industrial pick-and-place, these developments are relevant because real object scans are often incomplete, noisy or partially occluded by other parts, the conveyor or the camera viewpoint. A registration method is useful only if it can recover a sufficiently accurate object pose from imperfect segmented point clouds and provide an output that remains suitable for subsequent grasp planning and motion validation.

The progression of this literature shows a shift from learning global point-cloud representations, to learning correspondences, to learning overlap-aware and geometry-aware matching. This is relevant to industrial pick-and-place because real scans often

contain partial views, missing depth, occlusion and clutter. However, in this work, learned registration is not treated as an isolated objective. Its effectiveness depends on the wider perception pipeline, including segmentation quality, template generation, data preparation and geometric validation. This perspective is important for industrial deployment. Learning-based registration models can improve robustness, but they depend on representative training data and may generalise poorly when object geometry, sensor noise or scene conditions differ from the training domain. Their outputs may also be harder to interpret than classical optimisation. Therefore, for industrial use, learned registration is most useful when supported by suitable data generation, validation and geometric refinement rather than used as a fully independent black-box estimator.

2.5.3 Hybrid registration pipelines

Hybrid registration pipelines combine the strengths of learning-based and classical methods. A common strategy is to use a global or learned method to estimate a coarse alignment, followed by ICP or a similar local optimisation method for fine alignment. This structure is practical because the coarse stage helps avoid poor initialisation, while the local stage improves geometric accuracy once the model is close to the correct pose.

This combination is particularly relevant for industrial pick-and-place. Deep learning can improve the robustness of feature extraction, correspondence estimation or initial pose prediction, especially when objects are partially visible or noisy. Classical methods can then refine the pose using explicit geometric distances between the model and the observed point cloud. In this sense, deep learning provides a stronger starting point, while ICP provides precise local alignment. The literature on global registration, learned correspondence estimation and ICP refinement therefore supports a layered view of registration, rather than a strict replacement of classical methods by deep models. Figure 2.7 illustrates this idea using results reported for DCP, where the output of the learned registration model is further improved by ICP refinement.

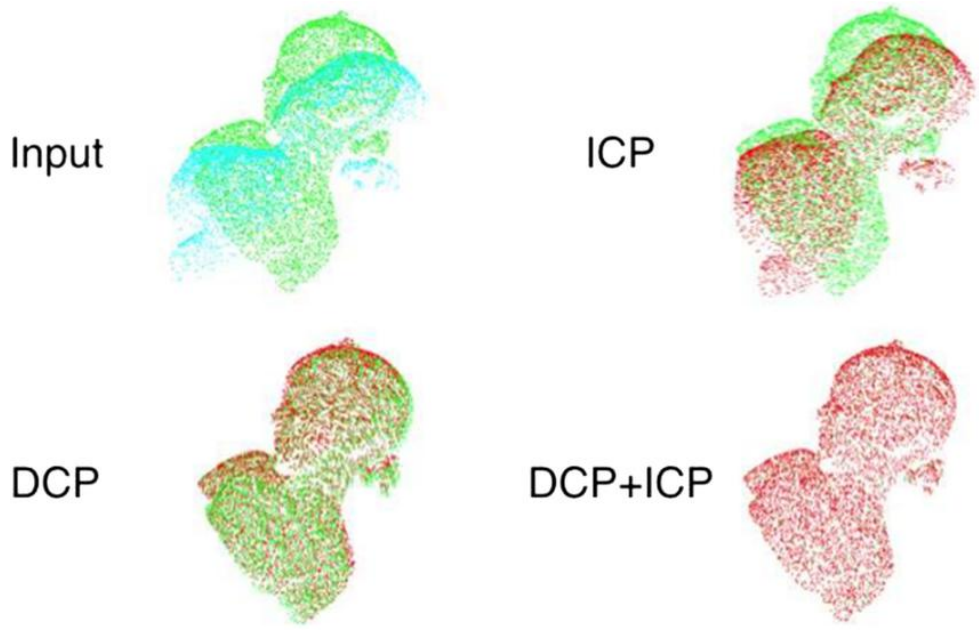


Figure 2.7: Example of hybrid point-cloud registration combining a learning-based method with classical refinement. The figure shows the input misalignment, the result of ICP alone, the result of DCP alone, and the result of DCP followed by ICP. Adapted from [80].

Hybrid pipelines are also compatible with bounded autonomy. The learned component can propose a pose or correspondence set, but the result can still be checked through geometric residuals, overlap scores, collision constraints or task-level validation. This is important in manufacturing because registration is not only a perception output. It becomes the basis for grasp selection and robot motion. A pose that is visually plausible but geometrically inconsistent may still lead to a failed pick. Hybrid registration therefore provides a useful compromise between adaptability and reliability.

2.5.4 CAD/template-constrained industrial pose estimation

Industrial pick-and-place often differs from open-world object perception because the target objects are known in advance. In many manufacturing applications, CAD models, STL files or reference scans are available before deployment. This makes template-based pose estimation especially relevant. Instead of estimating pose for arbitrary unknown objects, the system can align a known geometric model with a segmented scene instance. This constrains the problem and makes it more suitable for reliable robotic execution.

Classical industrial approaches such as Point Pair Features are built around this idea: a model is created from the known object geometry and then matched locally in the scene to detect object pose [70]. This model-based view is also central to 6D object pose estimation research using red-green-blue and depth (RGB-D) benchmarks, such as LINEMOD and the Benchmark for 6D Object Pose Estimation (BOP). These benchmarks evaluate how accurately methods estimate the 3D position and 3D orientation of known objects under occlusion, clutter and sensor noise [68], [85]. These benchmarks are relevant because they emphasise the same core problem faced in industrial picking: estimating the pose of known rigid objects from imperfect observations.

For this work, CAD/template-constrained pose estimation provides the conceptual bridge between segmentation and robotic action. Segmentation produces an object-level crop. Registration aligns this crop with a known object model. The resulting pose can then inform grasp planning, placement validation and motion execution. This approach is well aligned with adaptable industrial pick-and-place because new object types can be introduced by providing object-specific geometric information, while the same general perception and registration pipeline can be reused. In this sense, template-based registration supports the thesis goal of reducing manual reprogramming while preserving the reliability expected in industrial robotic cells.

2.6 Grasp Planning and Motion Execution Under Industrial Constraints

The previous sections discussed how industrial parts can be detected, segmented and registered in the 3D space. However, object localisation alone does not complete a pick-and-place task. The robot must also determine how the object can be grasped, whether the grasp pose is reachable, and whether the part can be moved safely to its destination. Grasp planning and motion execution therefore form the connection between sensing/perception and physical action.

Robotic grasping is a broad research area, ranging from analytical grasp synthesis to data-driven grasp prediction. Classical work established concepts such as force closure, friction cones and grasp quality metrics, while later approaches used depth images or point clouds to generate grasp candidates directly from sensor data [86], [87], [88], [89]. Learning-based grasping has become especially relevant for flexible

and high-mix production, where objects may vary and manual grasp specification becomes costly. *Xie et al.* explicitly connect learning-based grasping with high-mix, low-volume manufacturing, while also noting that AI-enabled grasping still faces challenges that limit deployment in such settings [5].

In industrial pick-and-place, the main concern is not only whether a grasp is geometrically possible, but whether it is executable in the complete cell. A grasp candidate must be compatible with the gripper, the object geometry, the approach direction, the robot workspace and the surrounding obstacles. Motion-planning frameworks such as MoveIt reflect this systems view, since planning requests are evaluated with kinematic constraints and collision checking, including self-collisions and attached objects [90], [91]. *Fresnillo et al.* also show that industrial users often need advanced manipulation functions beyond basic planning, such as end-effector speed control, tool changing and synchronisation [31]. This supports the idea that grasp planning and motion execution should be evaluated as a complete sequence, not as isolated pose generation.

2.6.1 Industrial gripper types and end-effector selection

Robotic grippers can be classified in several ways, depending on the aspect being emphasised. As shown in Figure 2.8, common taxonomic criteria include the number of fingers, the actuation method, the gripping mode, the mechanism used and the gripping method. This broader taxonomy is useful because it shows that gripper selection is not determined by a single property, but by a combination of mechanical design, actuation principle and object interaction strategy. In industrial pick-and-place, however, only a subset of these categories is typically relevant, since the main objective is not dexterous manipulation in general, but reliable and repeatable handling under application-specific constraints.

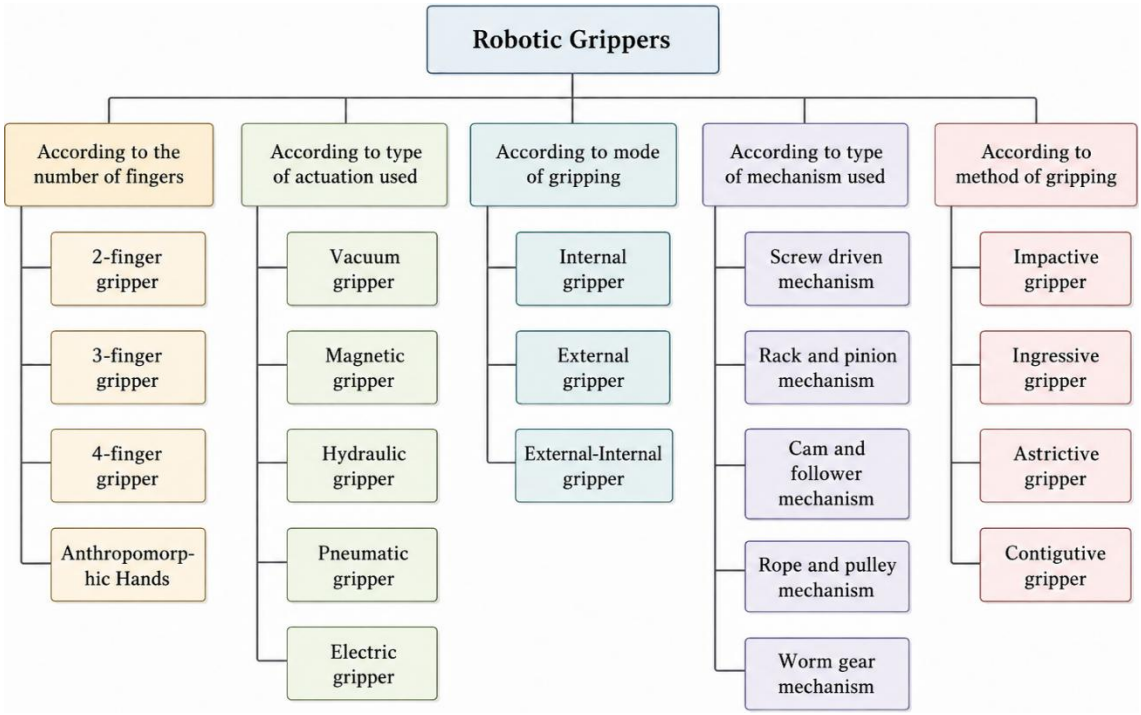


Figure 2.8: Taxonomy of robotic grippers based on five common classification criteria, including number of fingers, actuation method, gripping mode, mechanism type and gripping method. Adapted from [92].

Within this broader space, industrial pick-and-place uses a range of gripper types, each suited to different object properties and task requirements. End-effector selection is influenced by object geometry, size, material, surface condition, fragility, payload, cycle-time requirements and operating environment, since the gripper is the direct mechanical interface between the robot and the workpiece [92], [93], [94]. Common industrial examples include parallel grippers, multi-finger adaptive grippers, angular grippers, vacuum grippers, soft grippers and magnetic grippers, as illustrated in Figure 2.9. These designs differ in how they generate contact with the object. Finger-based grippers apply mechanical contact through jaws or fingers, vacuum grippers rely on suction, soft grippers use compliant contact surfaces and magnetic grippers are restricted to ferromagnetic materials.

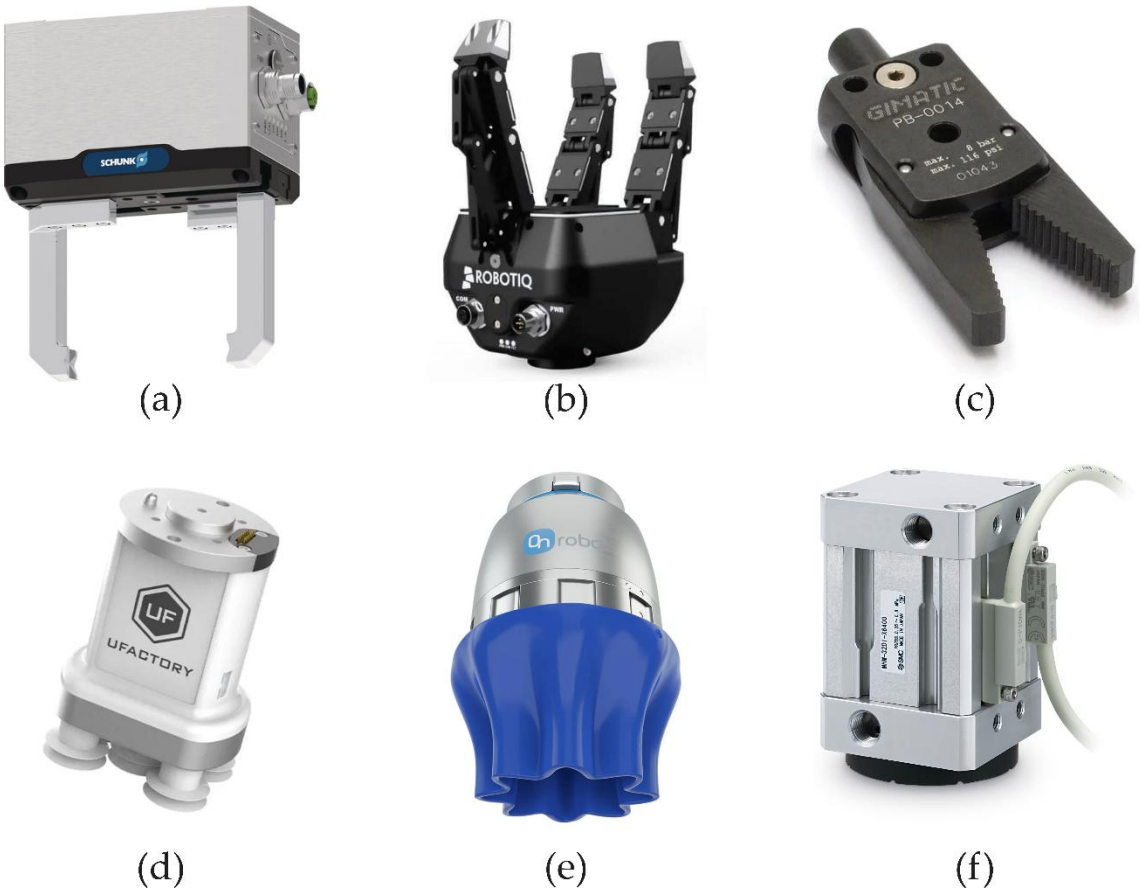


Figure 2.9: Representative industrial gripper types. (a) Parallel gripper, SCHUNK EGU. (b) Three-finger adaptive gripper, Robotiq 3-Finger Adaptive Robot Gripper. (c) Angular pneumatic gripper, GIMATIC PB series. (d) Vacuum gripper, UFactory xArm Vacuum Gripper. (e) Soft gripper, OnRobot Soft Gripper. (f) Magnetic gripper, SMC MHM-X6400. Product images adapted from manufacturer documentations.

For the system developed in this work, the most relevant category is the two-finger parallel gripper. Parallel grippers are widely used because they provide simple and repeatable opposing-jaw motion, making them suitable for rigid parts and predictable grasp planning [95]. This type of gripper reduces the grasping problem to a constrained set of possible actions, but it also imposes clear requirements. The fingers must fit around the object, the contact regions must provide enough stability to lift the part, and the approach path must be collision-free. Since the proposed system uses this type of gripper, the following subsection focuses on how two-finger parallel gripping affects grasp generation.

2.6.2 Grasp generation for industrial grippers

For two-finger parallel grippers, grasp generation usually involves selecting an approach direction, a closing direction and suitable contact regions. The approach direction defines how the gripper moves towards the object, for example from above in a top-down grasp or from the side in a lateral grasp. The closing direction defines how the fingers move towards the object and determines where the opposing contacts are formed. This constrained formulation makes grasp planning more manageable than dexterous multi-finger manipulation, but the grasp is only useful if it can be executed safely in the robotic cell.

Classical grasp generation evaluates candidate grasps using object geometry and contact assumptions. For two-finger grippers, antipodal grasps are particularly important because they rely on opposing contact forces acting on the object. Analytical methods can evaluate such candidates through force-closure or grasp-quality measures, but they often require accurate geometry, friction assumptions and contact modelling [86], [87]. In cluttered industrial scenes, these assumptions may be difficult to satisfy because the visible point cloud may be partial, noisy or occluded.

Data-driven methods simplify this process by learning to predict grasp candidates from depth or point-cloud observations. Methods such as Grasp Pose Detection and Dex-Net showed that grasp candidates can be generated and ranked from 3D perception data, reducing the need for fully hand-designed grasp rules [89], [96]. However, learned grasp proposals still require validation in the context of the actual robot and robotic cell. A grasp predicted from local geometry may still fail if the gripper collides with the conveyor, if the robot cannot reach the approach pose, or if the object cannot be transferred safely to the placement area.

For industrial applications, the grasping problem can often be simplified by using prior knowledge. If the object is known, candidate grasps can be defined from CAD models, reference scans or object templates and then transformed into the scene using the estimated pose. Known object geometry is also useful beyond grasp generation, because the full object model can be used for collision reasoning even when only part of the object is visible in the scan. The system can also restrict the search to grasp types that are compatible with the gripper and task, such as top-down or side grasps. This approach is less general than unconstrained 3D grasp generation, where candidate grasps may be sampled from many approach directions around the object. However, it is often more suitable for reliable manufacturing automation because the objective is repeatable execution of known tasks rather than arbitrary object manipulation.

2.6.3 Motion feasibility and pick-to-place execution

As discussed in the previous subsection, a grasp candidate must be evaluated in the context of the complete robotic cell. A valid grasp candidate does not automatically imply a valid robot action. The robot must be able to reach the grasp pose, find a feasible joint configuration and move without collision. This requires inverse kinematics, reachability analysis, collision checking and trajectory generation. Motion-planning literature provides the foundation for this stage, including sampling-based planners such as Probabilistic Roadmaps, Rapidly-exploring Random Trees (RRT) and RRT-Connect, as well as optimisation-based planners such as Covariant Hamiltonian Optimization for Motion Planning (CHOMP) [97], [98], [99], [100]. These methods are

relevant because they provide the computational basis for testing whether the robot can move between required task poses while satisfying kinematic and collision constraints.

For pick-and-place, feasibility must be checked across the full sequence rather than at a single pose. A typical operation includes an approach pose, grasp pose, retreat motion, transfer path, pre-place pose and final placement pose. Each stage can fail for different reasons. The robot may reach the object but collide during retreat, find a valid grasp but fail to carry the object through the workspace, or reach the destination but violate placement constraints. Therefore, grasp selection and motion planning should be treated as coupled problems.

This sequence-level view is important for industrial reliability. Perception may propose an object pose, and grasp generation may propose a contact configuration, but the final decision depends on whether the complete pick-to-place motion can be executed safely and repeatedly. In this sense, grasp planning narrows the space of possible actions, while motion feasibility determines which of those actions can be used in the real cell. This supports the broader argument of the thesis: reliable adaptable pick-and-place requires a pipeline in which perception, pose estimation, grasp generation and motion validation remain connected, rather than optimised independently.

2.7 Hybrid Dataset Generation and Sim-to-Real Transfer

The previous sections showed that learning-based segmentation, registration and grasping can improve the flexibility of robotic perception, but they also depend strongly on training data. In industrial pick-and-place, this creates a practical difficulty: real point-cloud datasets are expensive to collect, time-consuming to annotate and often limited to a small number of products or cell layouts. This is especially problematic when the system is expected to adapt to new object types with limited manual effort.

Synthetic data generation is an important research direction for robotic perception. By using CAD models, simulated scenes and automatic annotation, synthetic datasets can provide large quantities of labelled data for tasks such as object detection, semantic segmentation, instance segmentation and pose estimation. However, the main limitation is the sim-to-real gap, which refers to the mismatch between simulated data and real sensor observations. In point-cloud perception, this mismatch may appear as differences in noise, point density, missing surfaces, occlusion, object placement, lighting conditions or background geometry. As a result, models trained on synthetic data may learn patterns that do not transfer well to real sensor data, real object arrangements or real industrial backgrounds. Recent work on industrial point-cloud segmentation shows that synthetic point clouds can support real-world perception, but only when the generated data are processed to better match the conditions of real scans [101].

For this thesis, the key issue is not only whether synthetic data can be generated, but whether it can be generated in a way that preserves the physical and sensing conditions of the target task. This motivates hybrid dataset generation, where synthetic objects and annotations are combined with elements of the real cell, such as camera viewpoint, background geometry or sensor-specific point-cloud structure. This approach is relevant because it aims to keep the annotation advantages of simulation while reducing the gap between generated and real observations.

2.7.1 Synthetic data for robotic perception

In real industrial environments, collecting diverse examples of object poses, occlusions, clutter, lighting variation and part arrangements can be difficult and disruptive. In this context, synthetic data is widely used in robotic perception because it can reduce the dependence on manually collected and annotated real data [102]. Manual annotation of 3D data is also costly, especially for dense point clouds and instance-level labels. Simulation can address this by producing labelled scenes automatically, including object classes, instance masks, depth maps, 6D poses and segmentation labels [68].

This advantage has made synthetic data important in perception benchmarks and robotic learning. Datasets such as the BOP benchmark and related synthetic pose-estimation resources show that CAD models and rendered scenes can support training and evaluation for known-object pose estimation [68]. In industrial point-cloud perception, *Wu et al.* show that synthetic point cloud data can be used for sim-to-real transfer in an autonomous disassembly task, where collecting and annotating real-world industrial point clouds is difficult or impractical [101]. Their work is especially relevant because it addresses point-cloud segmentation directly, rather than only RGB image simulation.

The main weakness of synthetic data is that it may not reproduce the structure of real sensor observations. Synthetic scenes may contain complete object surfaces, idealised geometry, unrealistic object poses, clean backgrounds or noise patterns that differ from the real scanner. If the training data do not reflect real acquisition conditions, a model may perform well in simulation but fail on real scans. This is the sim-to-real gap [101], [102], [103]. It can be reduced through domain randomisation, domain adaptation, realistic rendering, sensor-noise modelling and hybrid data generation, but it cannot be ignored. For industrial robotics, the goal is not simply to generate more data, but to generate data that is relevant to the real cell and task.

2.7.2 Physics-based and hybrid point-cloud generation

One way to reduce the sim-to-real gap is to make synthetic scenes physically plausible. Real point clouds are acquired from objects that obey physical constraints: parts rest on surfaces, collide with each other, settle under gravity and occupy stable configurations. If a synthetic dataset contains unrealistic floating objects, interpenetrating meshes or impossible poses, the model may learn patterns that do not exist in real deployment. Physics-based scene generation addresses this by using simulation to create object arrangements that are more consistent with real industrial scenes.

Physics engines are widely used in robotics because they can simulate gravity, contact, collision and object settling. Engines such as MuJoCo (Multi-Joint dynamics with Contact), Bullet, Gazebo with the Open Dynamics Engine (ODE), PyBullet and NVIDIA Isaac Sim have been used across robotics, reinforcement learning and simulation-based data generation [104], [105], [106], [107], [108]. MuJoCo is particularly relevant for robotics because it was designed for fast and accurate simulation of articulated systems and contact dynamics, and its current documentation describes it as a physics engine for robotics, biomechanics, graphics and ML research [104]. For dataset generation, the important point is that physics simulation can reduce physically invalid object configurations, such as floating objects, interpenetrating meshes, unstable poses or

object placements outside the intended workspace, before synthetic point clouds are produced.

A second issue is visibility. Real 3D scanners do not observe complete objects. They observe only the surfaces visible from the camera perspective. In conveyor scenes, parts can occlude themselves, hide nearby objects, and cover parts of the conveyor. Therefore, synthetic point clouds generated from full CAD meshes may be too complete compared with real scans. Visibility filtering methods such as Hidden Point Removal address this problem by determining which points are visible from a given viewpoint without requiring full surface reconstruction [109]. This makes synthetic point clouds closer to single-view sensor observations.

A third issue is background and sensor context. In fixed industrial cells, the camera often observes a stable background, such as the conveyor surface, fixtures, walls or other cell structures. If synthetic objects are trained only on clean simulated backgrounds, the model may not learn how to separate parts from the real environment. Hybrid point-cloud generation addresses this by combining simulated object arrangements with real background scans. This preserves the controllability and annotation advantages of simulation while introducing scene structure from the real robotic cell. More generally, this follows the same principle as sim-to-real work in robotic perception, where generated data should include the variations and artefacts that the model will encounter during deployment [101], [110].

Lighting, material appearance and sensor artefacts also affect perception, especially for optical 3D sensors and RGB-D pipelines. Domain randomisation and realistic rendering literature show that varying visual conditions can improve robustness by preventing the model from overfitting to a single synthetic appearance. For point clouds, similar variation can include noise, density changes, missing depth, occlusion patterns and viewpoint constraints. In industrial perception, the most useful synthetic data are therefore not the most visually ideal, but the most representative of the sensing and physical conditions under which the robot must operate.

For the type of industrial pick-and-place considered in this work, physics-based and hybrid generation are especially relevant because they address two major sources of the sim-to-real gap: unrealistic object arrangements and unrealistic sensor context. The literature therefore supports moving beyond clean synthetic CAD scenes toward generated point clouds that include physically valid object poses, single-view visibility and real or realistic background structure.

2.7.3 Sim-to-real transfer and dataset reuse

Building on the previous discussion of synthetic and hybrid point-cloud generation, sim-to-real transfer concerns whether models trained with synthetic or simulated data can perform reliably on real sensor observations. In robotic perception, this transfer is difficult because the model may encounter different noise patterns, lighting conditions, material properties, object arrangements or background geometry at deployment. Point-cloud sim-to-real is especially challenging because the gap is not only visual. It also includes missing depth, density variation, occlusion, viewpoint-dependent visibility and scanner-specific artefacts [101], [111].

The literature proposes several strategies to reduce this mismatch. Domain randomisation exposes the model to many variations during training so that real data

appears as one possible variation of the simulated training distribution [103]. Domain adaptation attempts to align synthetic and real feature distributions and has been applied to point-cloud segmentation in industrial sim-to-real settings [101]. Sensor-noise modelling adds artefacts that resemble the real scanner, which is particularly relevant for point clouds because synthetic and real data may differ in local density, missing points and noise patterns [111]. Physics-based simulation improves the probability of object poses by modelling gravity, collision, contact and object settling, while hybrid data generation introduces real background or scene structure into the synthetic training set [101]. These strategies are complementary rather than mutually exclusive.

Dataset reuse is also important for adaptable robotic systems. A well-designed synthetic or hybrid dataset can support more than one perception task. The same generated scene may provide instance labels for segmentation, object poses for registration, and visible object crops for grasp or placement reasoning. This is particularly useful in industrial settings where new objects may need to be introduced without rebuilding the entire perception pipeline from scratch. Instead of collecting a full real dataset for every new part, object-specific models and simulated scenes can be used to generate task-relevant training data.

However, sim-to-real transfer should be validated carefully. High performance on synthetic validation data does not guarantee performance on real scans. Industrial evaluation should therefore include tests on real point clouds, real backgrounds, realistic object arrangements and downstream perception outputs such as instance quality, pose accuracy or registration stability. This connects the dataset question back to the broader thesis argument: adaptability depends not only on generating data quickly, but on generating data that is physically plausible, sensor-aware and useful for the complete perception-to-action pipeline.

2.8 ROS-Based Modular Robotic Systems and Digital Twins

The previous sections discussed the perception, registration, grasping and dataset requirements needed for adaptable pick-and-place. These capabilities must eventually be integrated into a robotic system that can communicate with sensors, planners, robot controllers and user interfaces. This is where modular robotic software frameworks become important. In manufacturing, the challenge is not only to develop accurate perception or planning algorithms, but also to connect them in a way that remains reusable, testable and compatible with industrial execution [112].

Robot Operating System (ROS) has become one of the most widely used frameworks for robotic software development because it supports modular communication between independent software components [112], [113]. In a ROS-based architecture, perception, transformation management, planning, control and visualisation can be developed as separate modules while exchanging data through standardised messages and services.

In ROS, a robotic application is organised as a network of nodes. Each node is an executable process responsible for a specific function, such as reading sensor data, processing a point cloud, publishing robot states, controlling a device, planning motion or visualising the system. These nodes exchange information through standard ROS communication mechanisms, mainly topics, services and actions. Topics implement asynchronous publish–subscribe communication and are suitable for continuous data streams, such as sensor data, robot states, point clouds and visualisation markers.

Services implement synchronous request–response communication and are used when one node requires a specific operation to be performed before continuing. Actions are used for longer-running tasks by allowing a goal to be sent, feedback to be monitored and a result to be returned. Together, nodes and communication mechanisms allow robotic systems to be divided into separate software modules while maintaining coordinated behaviour across perception, planning, control and visualisation.

This modularity is important for adaptable pick-and-place because new perception models, object descriptions, planning modules or user interfaces can be integrated without redesigning the entire system [90]. Figure 2.10 illustrates this role of ROS as a central integration layer, connecting perception, computer vision, motion planning, robot control, hardware drivers, simulation, data logging and visualisation within a shared robotic software ecosystem.

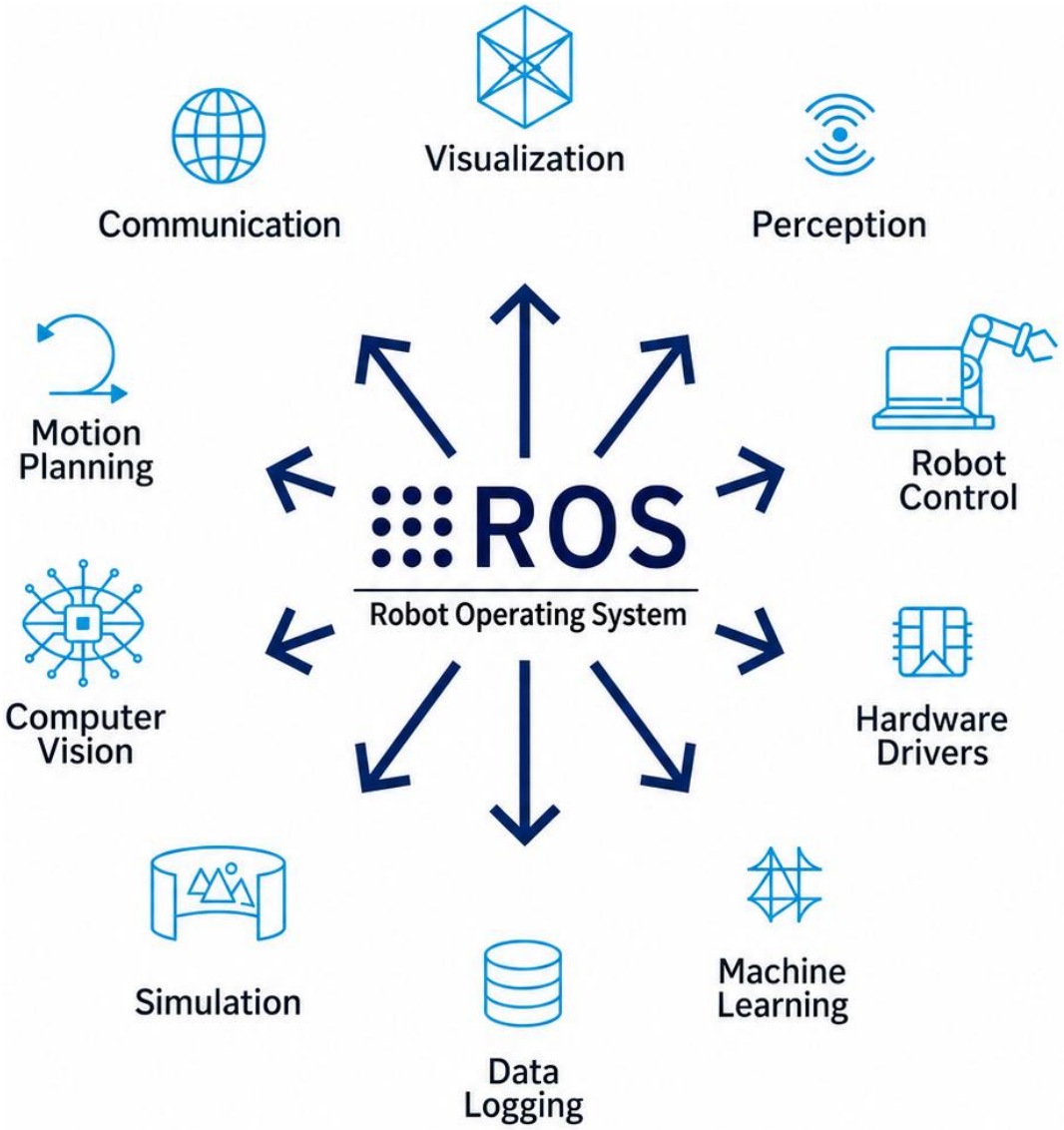


Figure 2.10: Overview of ROS as a modular robotics framework.

For industrial robotics, however, modularity alone is not sufficient. Manufacturing cells require repeatability, safety, controller reliability and compatibility with existing industrial hardware [31]. ROS-Industrial addresses this gap by extending ROS concepts toward industrial robot drivers, tooling, process control and manufacturing-oriented integration. In this sense, ROS-based systems provide a useful

middle ground between research flexibility and industrial reliability. They allow advanced perception and planning methods to be tested and integrated while preserving a structured connection to robot control and execution.

2.8.1 ROS and ROS-Industrial for manufacturing robotics

ROS is best understood as a middleware ecosystem rather than a single robot-control program. Its main value lies in supporting distributed modules that communicate through common interfaces. This structure makes it suitable for robotic cells where 3D sensing, object localisation, pose estimation, motion planning and execution must operate together but may be developed independently. For adaptable manufacturing cells, this is important because individual modules can be replaced or extended as the task changes.

On the other hand, ROS-Industrial builds on this idea by focusing on the needs of industrial automation. It provides tools, drivers and conventions for connecting ROS-based applications with industrial manipulators and manufacturing equipment. Literature on ROS-Industrial emphasises its role in bringing ROS capabilities into factory environments while recognising that low-level robot control and safety are often still handled by the industrial controller [113]. This supports a layered integration model, where ROS can manage perception, planning and task-level coordination, while deterministic execution remains tied to the robot's control system. Figure 2.11 illustrates this industrial extension of the ROS ecosystem, showing how ROS-Industrial connects robot hardware interfaces, industrial 3D sensors, calibration tools, bridges, developer tools, applications and human interfaces.

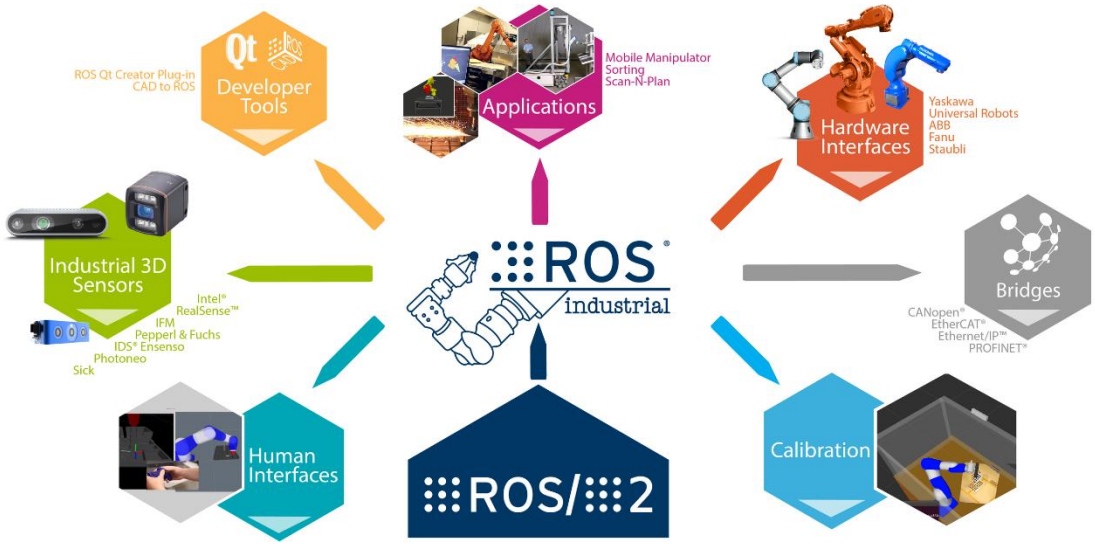


Figure 2.11: Overview of the ROS-Industrial ecosystem, showing its role in connecting ROS/ROS 2 with industrial robot hardware interfaces, 3D sensors, calibration tools, communication bridges, developer tools, applications and human interfaces. Adapted from [113].

This distinction is important for the type of system considered in this thesis. Adaptability depends on the ability to reconfigure object models, perception resources and task definitions. Reliability depends on maintaining controlled execution, collision checking and predictable robot behaviour. A modular ROS-based architecture can support both aims if high-level flexibility is separated from safety-critical low-level control.

2.8.2 Digital twins, motion planning, and virtual commissioning

Digital twins and digital counterparts are increasingly used in manufacturing to connect physical production systems with virtual representations. In this context, a digital twin can be understood as a virtual representation of a physical production system that is connected to the real system through data exchange, allowing the system to be monitored, simulated, analysed or optimised. This distinguishes a digital twin from a static digital model, since digital-twin definitions in manufacturing emphasise the integration between physical and virtual systems and, in stricter definitions, automatic data exchange between them [114], [115]. Reviews of digital twins describe their role in modelling, monitoring, simulation, control, validation, optimisation and lifecycle support for industrial systems [114], [116], [117], [118]. In robotic manufacturing, this idea becomes especially relevant because the virtual model can represent the robot, gripper, cell geometry, object models, sensors and process constraints. *Tipary et al.* apply this reasoning directly to flexible robotic pick-and-place cells, arguing that digital-twin-based development can reduce the work required in the physical cell and support faster commissioning or reconfiguration [16]. More recently, Singh et al. demonstrate the use of Unity and ROS as digital and communication layers for a robotic-arm digital twin in a smart manufacturing cell, with MoveIt used for motion planning and execution [119].

While Section 2.6.3 focused on motion feasibility as a robotics problem, this section examines the digital infrastructure that supports it. There, the focus was the robotics problem of whether a grasp can be reached and executed without collision across the full pick-to-place sequence. Here, the focus is the software and modelling infrastructure that allows those checks to be performed as part of an integrated robotic system. In ROS-based systems, MoveIt provides motion-planning pipelines, inverse kinematics, collision checking and trajectory generation, linking virtual robot models and environment representations to executable motions [90], [120]. More recent work also shows that MoveIt continues to be extended for advanced industrial manipulation needs, including more specialised process and execution requirements [31].

Virtual commissioning extends the value of digital models by allowing robot tasks, cell layouts and control logic to be tested before deployment on the physical system. The virtual-commissioning literature presents this to reduce commissioning time, detect design or control errors earlier and lower the risk of testing unsafe or infeasible behaviours directly on real equipment [121], [122]. In robotic cells, this is particularly useful because motion feasibility, collision constraints and task sequences can be assessed before the physical cell is fully operational. However, the literature also emphasises that the usefulness of virtual models depends on their consistency with the real system. Calibration errors, simplified collision geometry, inaccurate object models or outdated layouts can reduce the reliability of virtual testing. Digital twins should be understood as tools for reducing risk and improving integration, not as replacements for real validation. Thus, virtual plans and robot states must remain sufficiently aligned with the physical cell through calibration, state feedback and consistent environment modelling.

For adaptable industrial pick-and-place, this literature supports the need for a modular digital infrastructure. Perception outputs, object templates, grasp candidates and motion plans must be represented in a shared system where they can be checked, visualised and updated as tasks change. This connects ROS-based modularity with the broader digital-twin literature, although the present work is better understood as using

a ROS-based digital counterpart rather than a full digital twin. Through MoveIt and RViz, the physical and virtual robot states can remain linked for planning, visualisation and bidirectional control, while more advanced digital-twin functions such as lifecycle monitoring, predictive analytics or continuous process optimisation remain outside the scope of this work. Adaptability is therefore supported not by a single algorithm, but by the ability to maintain a consistent relationship between the physical cell, its virtual representation and the software modules that coordinate perception, planning and execution.

2.9 User-Guided Configuration and Human Involvement in Robotic Systems

The previous sections described the technical components needed for adaptable pick-and-place, including perception, registration, grasp planning, dataset generation and ROS-based integration. However, adaptability in industrial robotics also depends on how human knowledge is introduced into the system. In many manufacturing environments, the object set, production goals and placement requirements are known in advance, even when the position and orientation of each part during execution are uncertain. This means that full autonomy is not always necessary. A practical alternative is to use human input to define the task while allowing the robotic system to automate perception, planning and execution within defined limits.

As discussed in Section 2.3.3, HITL robot learning, adaptive automation and low-code or no-code programming represent different ways of involving humans in robotic systems. Here, they are considered from the perspective of system reconfiguration, where the aim is to increase autonomy while preserving human knowledge, supervision and responsibility [32], [33], [35], [36], [37].

2.9.1 Reducing expert dependence in robot programming

Conventional robot programming remains a barrier to flexible and multi-product manufacturing because the programming process often requires specialist knowledge, programming time and detailed process tuning. This limitation becomes more evident when new parts, layouts or task variants must be introduced frequently. Recent work addresses this problem through more intuitive programming and reconfiguration methods, including no-code and low-code interfaces, augmented-reality tools, programming by demonstration and multimodal interaction. These methods may be used individually or combined, but they share the same aim of allowing users to specify robotic tasks without writing low-level robot code [26], [27], [33], [37].

Such methods change how task knowledge is provided to the robotic system. Instead of manually writing robot programmes, the user may define objects, target poses, constraints or desired behaviours through a graphical interface, demonstration, spatial interaction, voice command or high-level instruction. This does not remove the need for human expertise. Rather, it shifts the human role from low-level robot programming toward higher-level task specification.

This shift toward higher-level task specification is particularly relevant because many tasks can be described using structured information. The user may know which object should be handled, where it should be placed and what process constraints are acceptable. The challenge is therefore not always to make the robot discover the task

independently, but to provide interfaces that convert user-defined task intent into perception, planning and execution resources.

2.9.2 User-guided configuration and bounded autonomy

User-guided configuration provides a practical form of bounded autonomy. The system does not attempt to solve every aspect of the task independently. Instead, it receives task-specific information from the user and then operates autonomously within that defined scope. This approach is appropriate for industrial pick-and-place because the environment is often partly structured, the objects are usually known, and the required placements are defined by the process.

This differs from continuous HITL control. In HITL robot learning, the human may remain involved during training or execution by providing demonstrations, corrections or feedback [32]. In user-guided configuration, the human role is concentrated mainly before execution. The system can then process repeated task instances without requiring the operator to approve each perception, grasping or motion decision. When uncertainty or failure occurs, industrial systems may reject, ignore or recirculate the object rather than requesting immediate human intervention. This still reflects bounded autonomy because the system acts within predefined limits, but it should not be described as continuous HITL control.

This distinction is central to the positioning of the thesis. The proposed approach does not aim for open-ended autonomy, nor does it require continuous operator supervision. It follows the literature trend toward reducing expert programming effort while preserving human-defined task intent. Industrial adaptability is therefore framed as the ability to combine user-defined configuration, modular software and autonomous execution within reliable boundaries.

2.10 Hybrid Robotic Systems: Combining AI with Industrial Reliability

The literature reviewed in this chapter suggests that industrial pick-and-place cannot be addressed effectively by relying only on conventional automation or only on end-to-end AI. Traditional robotic cells remain attractive because they provide repeatability, predictable cycle times and controlled execution. However, they become difficult to reconfigure when products, layouts or object presentation conditions change. AI-based methods, in contrast, can improve perception, segmentation, registration and grasp generation under uncertainty, but they introduce challenges related to data requirements, generalisation, explainability and validation [4], [13], [62].

Hybrid robotic systems respond to this trade-off by combining intelligent perception or decision-making with deterministic planning, structured task constraints and controlled execution. This view is consistent with the broader literature on adaptive automation, human-centred robot programming, ROS-based integration and reconfigurable manufacturing systems [29], [31], [32], [35], [37]. In such systems, AI is not expected to replace the full industrial control stack. Instead, it is used where it is most beneficial, such as object localisation, candidate generation or scene interpretation, while motion planning, collision checking, task constraints and human-defined goals provide boundaries for safe and repeatable operation.

2.10.1 Limits of purely traditional and purely AI-based systems

Purely traditional systems are reliable when the environment is structured and the task is stable. This is why fixtures, feeders, custom tooling and offline programming remain common in industrial automation. However, this reliability is achieved by reducing uncertainty before the robot acts. When object pose, part type or layout changes frequently, the engineering effort required for reprogramming, recalibration or feeder redesign can become a major limitation [15], [16], [29].

Purely AI-based systems address some of these limitations by learning from data and adapting to variation during perception or decision-making. In robotic pick-and-place, this includes learning-based segmentation, registration and grasp prediction. However, these systems can be difficult to justify in industrial settings when their decisions are hard to interpret, their training data are limited, or their performance changes under unseen conditions. Reviews of AI in manufacturing identify black-box behaviour, data quality, reliability, safety and regulatory uncertainty as major barriers to adoption [4], [13]. Therefore, the literature does not support a simple replacement of traditional automation by AI, but rather a careful integration of AI into systems that remain verifiable and controllable.

2.10.2 Hybrid architectures for bounded autonomy

Hybrid architectures combine deterministic control with intelligent adaptability. In robotics, this usually means that AI-based modules support perception, candidate generation or high-level reasoning, while more deterministic components handle motion planning, collision checking, safety constraints and execution. This layered structure is consistent with the literature on adaptive automation, where functions may be allocated between human and machine depending on task conditions, system state or operator needs [35], [36].

Recent reconfigurable manufacturing work follows a similar direction. *Asif et al.* propose the R3M framework, which uses modular robot skills, ROS 2 and Gazebo-based virtual trials to support plug-and-produce adaptation in robot manufacturing cells [29]. In that architecture, new robot skills are not simply executed after being generated. They are first checked through risk assessment and virtual testing, which reflects the principle of bounded autonomy. The system can adapt, but its actions remain constrained by validation steps before physical execution.

This type of design is also supported by the wider literature on trustworthy industrial AI. Reviews identify black-box behaviour, data quality, reliability and safety concerns as major barriers to industrial AI adoption [13]. Human-centred automation studies similarly show that preserving operator authority can improve perceived autonomy and satisfaction without reducing performance [36]. Therefore, hybrid architectures are increasingly important because they allow AI to improve flexibility and adaptability while maintaining the verification, transparency and control expected in industrial environments.

2.11 Research Gaps and Positioning of This Thesis

The literature reveals several gaps at the intersection of pick-and-place, flexibility, adaptability and reliability. First, a large body of work focuses on improving individual stages of the robotic pipeline, such as segmentation [64], [123], pose estimation and registration [80], [84], grasp detection [89], [96], [124] or motion

planning [120], [125], while fewer studies demonstrate how these stages can be integrated into a complete and reconfigurable robotic cell [16], [29], [126]. This is important because reliable industrial pick-and-place depends on the consistency of the full pipeline, not only on the performance of an individual module.

Second, systems that support new products often still require significant engineering effort. Advanced reconfigurable frameworks reduce adaptation effort through modular skills, digital models and structured process knowledge, but they still depend on formal task representations and engineering input when new parts or processes are introduced [29]. This means that adaptability is improved, but not fully automated.

Third, low-effort part addition remains a difficult problem because perception resources are often object-specific. Many pipelines require manually collected real scans, annotated datasets, hand-coded templates or carefully tuned parameters for each new object. Synthetic data can reduce this burden, but generic synthetic scenes may not match the real sensing conditions of the robotic cell. This creates a gap for dataset-generation methods that can use the target object model to automatically produce physically plausible, sensor-aware and task-relevant training data for perception and registration.

Fourth, the literature increasingly recognises the value of bounded autonomy, modular software and digital representations, but these ideas are often studied separately. Human input is discussed in low-code programming and human-centred automation, digital representations are discussed in digital-twin and virtual-commissioning literature, and AI methods are often evaluated as separate perception or grasping modules. Fewer works connect these elements into a single configurable system for adaptable industrial pick-and-place.

Overall, the gap addressed in this work is the need for a semi-autonomous robotic system that can combine user-defined task information, autonomous dataset generation from target object models, modular ROS-based integration, computer-vision-based localisation, object-specific geometric knowledge and reliable motion execution. The aim is not to replace industrial structure with unconstrained AI, but to reduce the effort required to introduce new objects and reconfigure a pick-and-place cell while preserving the reliability expected in manufacturing environments.

2.11.1 Positioning of the proposed system and mapping to SRQs

Building on the gaps identified above, this subsection positions the proposed system in relation to the sub-research questions defined in Chapter 1. The aim is not to describe the methodology in detail, but to show how the proposed system responds to the limitations identified in the literature.

Reflecting **SRQ1 on modular system architecture**, the proposed system builds on ROS-based integration to connect the physical cell, digital counterpart, user interface, perception, registration, motion planning and execution modules. As discussed in Section 2.8, ROS and ROS-Industrial concepts support modular communication between robotic software components, while MoveIt provides motion planning and collision-checking capabilities. In this thesis, this modular foundation is important because it allows the same robotic-cell architecture to support both the foundational pick-and-place experiment and the later extension to complex industrial parts.

Reflecting **SRQ2 on system adaptability through user-guided configuration**, the proposed system uses operator-defined object models and placement requirements as the starting point for reconfiguration. Instead of requiring direct robot reprogramming for each new part, the system uses the provided CAD models and slot definitions to trigger task-specific hybrid dataset generation and perception configuration. This connects to the literature on reconfigurable manufacturing, low-code robot programming and human-guided automation, where the objective is to reduce the effort required to adapt robotic systems to new products or tasks without removing human knowledge from the process.

Reflecting **SRQ3 on flexibility in random object flows**, the system addresses the need to detect, localise, classify and pick objects arriving in random positions and orientations. The literature reviewed in Sections 2.4 and 2.5 shows that 3D sensing, point-cloud segmentation and pose estimation are central to this problem because grasping and placement depend on reliable object-level scene understanding. In this thesis, this requirement is addressed across two experimental stages, beginning with classical geometric perception for simpler object geometries and extending to deep-learning-based instance segmentation and registration for complex industrial parts. This progression supports both object localisation and semantic class matching for assignment to available placement slots.

Reflecting **SRQ4 on reliability under changing object and scene conditions**, the proposed system does not rely on perception outputs alone. The literature reviewed in Sections 2.6 and 2.10 shows that industrial pick-and-place requires validation across the complete action sequence, including object pose estimation, grasp selection, reachability, collision checking and trajectory feasibility. In this thesis, data-driven perception is combined with object-specific geometric knowledge, template-based localisation and planning-based validation so that unsuitable detections, uncertain object poses or infeasible motions can be rejected before execution.

Reflecting **SRQ5 on the balance between autonomy and user input**, the proposed system follows a bounded-autonomy approach. As discussed in Sections 2.9 and 2.10, industrial robotic systems benefit from automation, but they must remain understandable, configurable and recoverable by human operators. In this thesis, the user provides high-level task information, such as target objects, CAD models, placement slots and initial parameters, while the system automates dataset generation, perception, pose estimation, object-to-slot assignment, motion validation and execution within predefined limits. This positions the system between manually programmed industrial automation and unconstrained end-to-end AI.

The contribution of this study is therefore to synthesise these ideas into a working system for adaptable industrial pick-and-place. The proposed approach combines ROS-based modular integration, user-guided configuration, automatic hybrid dataset generation from target object models, dense 3D perception, template-based localisation, object-to-slot assignment and reliable motion execution. In doing so, it aims to reduce the engineering effort required to introduce new objects while preserving the repeatability and safety expected in manufacturing environments.

Chapter 3: FOUNDATIONAL ROBOTIC CELL DEVELOPMENT AND VALIDATION

3.1 Introduction

This chapter presents the first experimental stage of the research, which focused on the development and validation of a foundational robotic cell for random-flow pick-and-place tasks. The purpose of this stage was to establish the physical, digital and software infrastructure needed to support later extensions of the system, particularly the introduction of complex industrial parts and learning-based perception methods.

Industrial pick-and-place systems are traditionally designed for structured environments, where object positions, robot trajectories and destination poses are predefined. Although this approach provides high reliability in repetitive manufacturing operations, it limits the ability of the system to respond to objects arriving in random positions and orientations. To address this limitation, this first experiment integrated 3D computer vision, ROS-based communication, a digital representation of the cell and MoveIt-based motion planning into a functional robotic platform.

The experimental system was built around a minimum robotic cell composed of an ABB IRB 1200-5/09 six-axis industrial robot with an IRC5 Compact controller, a Photoneo PhoXi 3D Scanner S structured-light scanner, a belt conveyor driven by an Interroll RollerDrive, a GIMATIC PB-0013 pneumatic gripper, a stationary placing table and a digital counterpart of the physical environment. At this stage, the perception problem was intentionally constrained to simple rectangular objects. Point-cloud processing, background removal, DBSCAN clustering and PCA-based pose estimation were used to localise objects on the conveyor and generate robot targets for top-down pick-and-place execution.

This chapter validates the foundational robotic-cell infrastructure developed for the first experimental stage. Section 3.2 defines the aim and objectives of the experiment. Section 3.3 then describes the robotic-cell development and serves as the materials and methods description for this chapter, specifying the equipment, model designations, manufacturers and software versions used throughout the experimental stage. Sections 3.4 and 3.5 describe the vision-based point-cloud processing method, and the ROS computer-vision pick-and-place pipeline. Sections 3.6 and 3.7 present the experimental protocol and evaluation metrics. Section 3.8 discusses the results, including orientation-estimation accuracy and pick-and-place performance. Finally, Section 3.9 concludes the chapter and identifies how this foundation supports the second experimental stage.

3.2 Aims and Objectives

The aim of the first experiment was to develop and validate a foundational ROS-based robotic cell capable of performing random-flow pick-and-place tasks using 3D vision and classical point-cloud processing. This experiment was designed to verify the physical, digital and software infrastructure required to localise objects on a conveyor belt and execute robotic displacement to a predefined placing area. The specific objectives of this experiment were to:

1. Develop a physical robotic cell suitable for random-flow pick-and-place experiments, including the integration of an industrial robotic arm, 3D vision sensor, conveyor belt, end-effector and placing area.

2. Create a digital counterpart of the robotic cell, representing the robot, surrounding structure and relevant workspace elements for visualisation, development and motion-planning support.
3. Integrate the physical and digital layers through a ROS-based software architecture, enabling communication between perception, robot control, visualisation and planning modules.
4. Implement a vision-based localisation method capable of processing point-cloud data from the 3D scanner, removing background information, clustering detected objects and estimating their position and orientation.
5. Develop a basic pick-and-place pipeline in which detected object poses are converted into robot targets and executed using MoveIt-based motion planning.
6. Validate the system experimentally using simple rectangular objects moving in random-flow conditions on a conveyor belt.
7. Evaluate the accuracy and reliability of the proposed system through object orientation estimation and pick-and-place success rate.

This experiment was not intended to address the full complexity of adaptive manipulation for industrial parts. Instead, it established the baseline robotic-cell framework required for the remaining research. By validating the complete workflow from 3D scan acquisition to robotic execution using simple geometric objects, this stage confirmed that the core platform was suitable for further development. The outputs of this experiment, including the physical cell, digital counterpart, ROS communication structure, MoveIt planning interface and basic pick-and-place logic, were later reused and extended in the second experimental stage, where more complex perception and localisation methods were introduced.

3.3 Robotic Cell Development

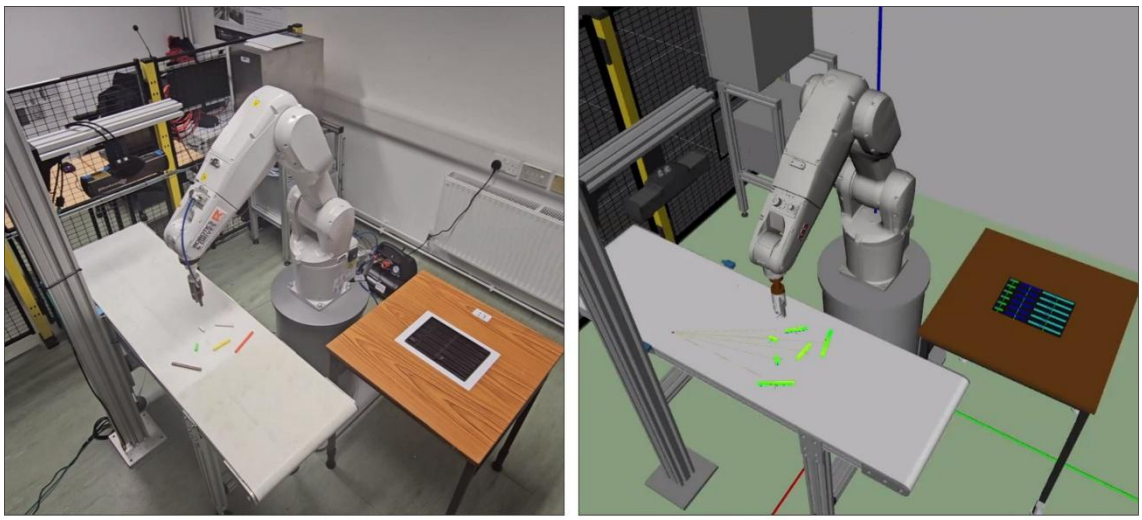
The robotic cell used in this experiment was composed of a physical layer, a digital layer and a ROS-based communication structure. The physical layer consisted of an ABB IRB 1200-5/09 industrial robotic arm with an ABB IRC5 Compact controller, a Photoneo PhoXi 3D Scanner S, a GIMATIC PB-0013 pneumatic gripper, a belt conveyor driven by an Interroll RollerDrive and DriveControl 20 module, and a stationary placing table.

Table 3.1 lists the model designations, manufacturers and principal specifications of these components, together with the computing hardware and software versions used throughout this experimental stage. The digital layer reproduced the main cell components in ROS for visualisation, planning and collision checking, while the communication structure connected the perception package, robot controller, peripheral devices and motion-planning tools. Section 3.3.1 describes each physical component and the reasons for its selection in more detail.

Table 3.1: Equipment, computing hardware and software used in the foundational robotic cell.

Component	Model / designation	Supplier	Principal specification
Industrial robot	IRB 1200-5/09	ABB	6 axes, ~900 mm reach, 5 kg payload
Robot controller	IRC5 Compact	ABB	Joint-level control, safety supervision, digital I/O
3D scanner	PhoXi 3D Scanner S	Photoneo	Structured light, up to 3.2 MP, 384-520 mm range, 442 mm optimum
End-effector	PB-0013 pneumatic gripper	GIMATIC	Single-acting, spring return, 3/2-way valve
Conveyor	Custom	Robotics & Drives Services Ltd (RDS)	Belt transport through scanning region
Conveyor drive	RollerDrive + DriveControl 20	Interroll	Speed, direction and start/stop via IRC5 digital outputs
Object detection sensor	WSE9 through-beam photoelectric sensor	SICK	Scan trigger on beam interruption
Placing table	Standard table	-	Fixed placement location within the robot working envelope
Placement tray	Custom, designed in Fusion 360	-	Object-specific slots, 1 mm clearance per side
Workstation	Intel Core i7-11800H, 32 GB RAM, NVIDIA RTX 3060 6 GB	-	Hosts the ROS workspace
Software	ROS Noetic, MoveIt, RViz, abb_driver, Python 3.8.10, C++ 9.4.0, Autodesk Fusion 360	-	Integration, planning, perception, CAD

Figure 3.1 shows the relationship between the physical robotic cell and its digital counterpart. The physical cell, shown in Figure 3.1(a), contains the ABB robotic arm, 3D scanner, conveyor belt, placing table and surrounding support structure listed in Table 3.1. The digital cell, shown in Figure 3.1(b), reproduces the same main components in the ROS visualisation environment, allowing the robot model, scanner position, conveyor, detected objects and placement area to be represented in a common virtual workspace. This figure provides an overview of the experimental platform before the physical and digital layers are described separately in the following sections.



(a) (b)

Figure 3.1: Physical and digital layers of the robotic cell used in this experiment. (a) Physical robotic cell including the ABB IRB 1200-5/09 robot, PhoXi 3D scanner, conveyor belt, gripper and placing table. (b) Digital counterpart of the cell in the ROS/RViz environment, showing the robot model, scanner structure, conveyor region, detected objects and predefined placement area.

3.3.1 Physical Robotic Cell

The ABB IRB 1200-5/09 was used as the manipulation unit. This six-axis industrial robot has a maximum reach of approximately 900 mm, a payload capacity of 5 kg and a compact design suitable for integration into confined robotic cells. As shown in Figure 3.2, the robot provides a relatively large operating envelope compared with its physical footprint, which was important for positioning the conveyor scanning region and the placing area within the same reachable workspace. The robot also provides high positioning repeatability and is designed for applications requiring accurate and reliable manipulation, such as material handling, machine tending and assembly operations. These characteristics made it suitable for the pick-and-place task investigated in this work, where objects had to be detected, grasped and transferred between locations with consistent accuracy.

The robot was mounted next to the conveyor belt so that both the object acquisition area and the placing area were located within its reachable workspace. The selected payload capacity was considerably higher than the mass of the test objects, ensuring stable operation while accommodating the weight of the pneumatic gripper and any additional tooling requirements. The six degrees of freedom also provided sufficient flexibility to approach objects from different orientations and to generate collision-free trajectories within the robotic cell.

The robot was connected to an ABB IRC5 Compact controller, which acted as the main interface between the physical robot cell and the external ROS-based control system. In addition to executing the robot motions, the controller handled the low-level robot control functions, including joint-level motion execution, safety supervision and communication with external devices. The IRC5 controller also provided digital input and output interfaces used to connect the peripheral components of the robotic cell. In this setup, digital inputs were used to receive signals from external sensors, such as the retro-reflective sensor used to detect when an object interrupted the light beam. Digital outputs were used to control the main actuated peripherals, including the conveyor

drive and the pneumatic valve responsible for opening and closing the gripper. This allowed the robot controller to coordinate not only the robot motion but also the object detection trigger, conveyor operation and gripper actuation required for the pick-and-place task.

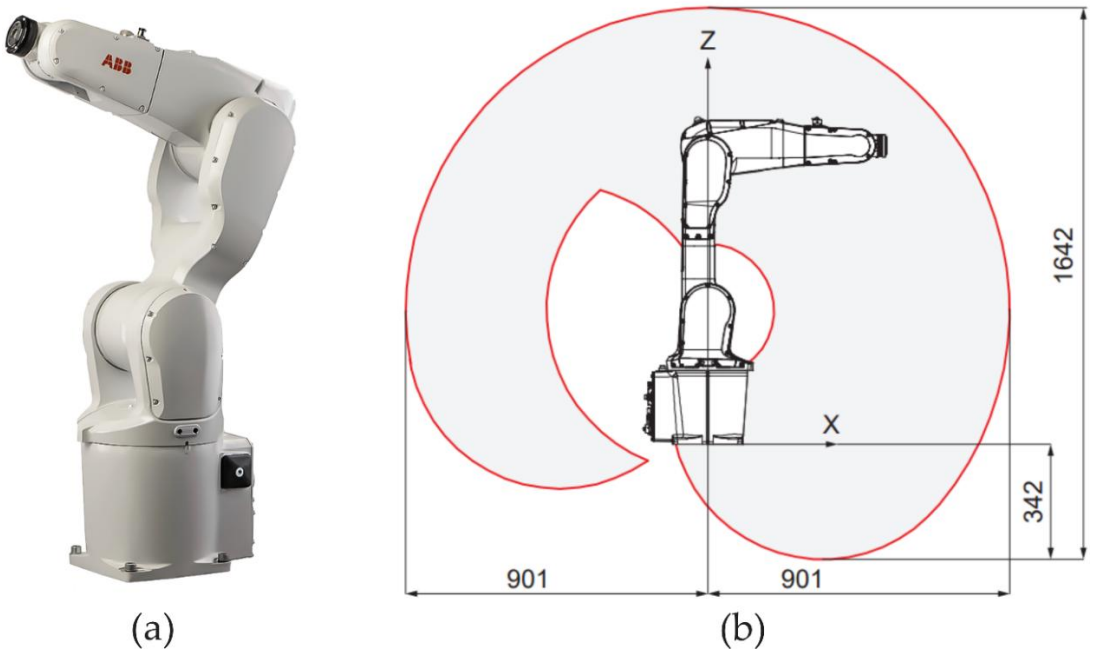


Figure 3.2: ABB IRB 1200-5/09 robotic arm used in the experimental cell. (a) Physical appearance of the robot. (b) Workspace envelope of the robot, showing its approximate reach and vertical operating range.

The vision system was based on a PhoXi 3D Scanner S from Photoneo. The scanner was used to acquire three-dimensional point clouds of the conveyor region where the objects were located. The PhoXi 3D Scanner S is an industrial structured-light scanner designed for high-accuracy object localisation and inspection tasks. Figure 3.3(a) shows the manufacturer-specified scanning volume within the operating range, while Figure 3.3(b) shows a sample pipe-fitting point cloud supplied by Photoneo with the PhoXi Control software to illustrate the type of dense 3D surface representation produced by the scanner.

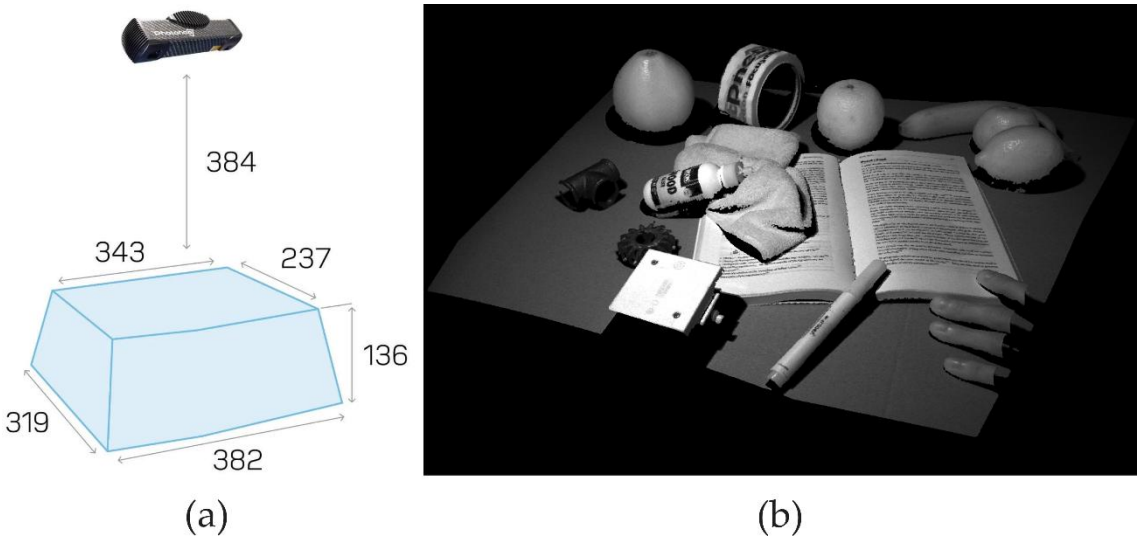


Figure 3.3: PhoXi 3D Scanner S operating characteristics. (a) Manufacturer-specified field of view and working-volume dimensions. (b) Sample pipe-fitting point cloud supplied by Photoneo with PhoXi Control [61], illustrating the dense 3D surface information produced by the scanner.

The scanner provides a resolution of up to 3.2 MP and operates within a scanning range of 384 mm to 520 mm, with an optimal scanning distance of 442 mm. It generates dense point-cloud data with high spatial accuracy, making it suitable for applications that require reliable object detection and pose estimation in robotic manipulation scenarios.

The scanner was selected because the developed computer-vision package relied on geometric information extracted from point clouds rather than conventional two-dimensional image features. Accurate depth measurements were therefore required to determine the position and orientation of objects on the conveyor belt and to provide target poses for the robotic arm. The use of a structured-light 3D scanner also reduced the influence of variations in object colour, texture and ambient lighting conditions, which can affect image-based detection methods.

To maximise the quality of the acquired point clouds, the scanner was installed according to the manufacturer's recommended operating distance. Following this working distance, the scanner was positioned 1370 mm above the floor and 430 mm above the conveyor belt surface. This configuration provided full coverage of the conveyor inspection area while maintaining sufficient point-cloud density and measurement accuracy for the subsequent object-recognition and pose-estimation processes.

The end-effector was a GIMATIC PB-0013 pneumatic gripper mounted on the sixth joint of the robotic arm. The gripper was selected due to its compact design and suitability for handling the test objects used in the experiment. It was actuated using compressed air through a 3/2-way valve connected to the robot controller I/O system. The gripper operated as a single-acting mechanism with spring return: the fingers closed when air pressure was applied and opened automatically when the pressure was released. This configuration provided a simple and reliable grasping solution while reducing the number of pneumatic control components required.

The conveyor belt was used to transport the test objects through the scanning and picking region of the robotic cell. It was built for this cell by Robotics & Drives Services Ltd (RDS), Mullingar, Ireland, rather than being supplied as a standard catalogue product, and therefore carries no manufacturer model designation. The belt had a width of 0.42 m and a length of 1.4 m. It was positioned within the robot workspace so that objects could be detected by the 3D scanner and subsequently reached by the robotic arm. The conveyor was driven using an Interroll RollerDrive controlled by an Interroll DriveControl 20 module. This drive unit provided the interface for setting the conveyor motion, including speed, direction and start/stop operation. In the implemented cell, these control signals were connected to the IRC5 digital outputs, allowing the conveyor to be coordinated with the robot task sequence. A stationary table was placed next to the conveyor and served as the destination area for the pick-and-place operations. The table was located within the robot working envelope and provided a fixed and repeatable placement location for all experimental trials.

3.3.2 Digital Counterpart

The digital counterpart of the robotic cell was developed in ROS and consisted of a virtual representation of the robot, end-effector, conveyor belt, placing table and additional cell structures required for visualisation, simulation and motion planning. The digital environment was designed to mirror the physical layout of the robotic cell and to

provide a common framework for visualisation, collision checking and trajectory generation. During operation, information was exchanged bidirectionally between the physical and digital environments. Robot state information, such as joint positions, was transmitted from the physical robot to the digital model, allowing the virtual representation to replicate the robot's movements in real time. Conversely, trajectories generated in the digital environment were transmitted to the physical robot for execution, enabling the physical system to follow motions planned in the virtual model.

Despite this bidirectional exchange of information, the term digital counterpart is used instead of digital twin. A digital twin generally requires a more comprehensive and continuously synchronised representation of the physical system, often including real-time modelling of system behaviour, process dynamics, environmental interactions and, in some cases, physics-based simulation and predictive capabilities. In the present work, the virtual model was primarily employed for visualisation, motion planning and collision detection. Although it maintained synchronisation of robot states and supported the execution of planned motions, it did not replicate all physical processes and behaviours of the robotic cell in real time and therefore does not fully satisfy the broader definition of a digital twin.

For each component, the Unified Robot Description Format (URDF) description included visual, collision and inertial properties. Visual meshes were used for graphical rendering in RViz, whereas simplified collision geometries were defined to support efficient collision detection during motion planning. Inertial parameters were included where required by the robot model to maintain consistency with ROS-based simulation and kinematic calculations.

The digital counterpart was integrated with the ROS transform (TF) framework, which maintained the spatial relationships between all coordinate frames in the robotic cell. Coordinate frames were defined for the robot base, robot joints, end-effector, conveyor belt, scanner and robotic cell structures. This enabled the perception system, robot controller and motion-planning framework to operate within a common reference structure and ensured that object poses estimated by the vision system could be transformed into robot-referenced coordinates.

RViz was used as the primary visualisation environment. Through RViz, the operator could observe the robot state, coordinate frames, planning scene, detected objects and generated trajectories. The software also provided visual feedback regarding the position of the robot relative to surrounding structures and allowed verification of planned motions before execution.

The digital counterpart was further integrated with the MoveIt planning framework. The robot description, kinematic configuration and collision models were imported into MoveIt through the MoveIt Setup Assistant, which generated the required configuration packages for motion planning. The resulting planning scene contained both the robot model and the relevant environmental obstacles, enabling collision-aware trajectory generation during the execution of the pick-and-place task.

3.3.3 ROS and ROS-Industrial Integration

The robotic cell software architecture was implemented in ROS Noetic and organised into functional packages for perception, robot communication, motion planning, visualisation and device control. ROS was used as the integration layer

between the digital cell representation, the motion-planning framework, the vision package and the ABB IRC5 controller. Communication between modules followed the standard ROS communication mechanisms introduced in Section 2.8. In this implementation, topics were used for continuous data exchange, such as robot state updates, point-cloud data and visualisation information. Services were used for request-based operations, such as triggering specific processing or control routines, while action-style communication was associated with motion-planning and execution tasks that required goal submission, feedback and completion status.

The ROS workspace was hosted on an external computer connected to the IRC5 controller, as shown in Figure 3.4. This computer was a standard laptop with an 11th Gen Intel Core i7-11800H processor operating at 2.30 GHz, 32 GB of RAM and an NVIDIA GeForce RTX 3060 GPU with 6 GB of memory. Although the system included a GPU, the computer-vision algorithm was executed on the CPU only. This choice was made to keep the implementation simple and compatible with the existing ROS-based processing pipeline. Since the point-cloud processing method used in this experiment was based on deterministic operations, such as filtering, DBSCAN clustering, PCA and feature comparison, GPU acceleration was not required for the validation presented in this chapter.

ROS-Industrial was used to establish communication between the ABB IRB 1200 robot and the ROS environment. The integration was implemented using the ABB ROS-Industrial driver (*abb_driver*), which provides the required software interface between ROS applications and ABB IRC5 controllers [127]. As illustrated in Figure 3.4, the connection between the external computer and the IRC5 controller was divided into two communication routes. The first route used a Transmission Control Protocol/Internet Protocol (TCP/IP) socket connection between a C++ socket client running in ROS and a socket server implemented in the Robotics Application Programming Interactive Dialogue (RAPID) language. This connection was used to exchange robot motion information. The IRC5 controller transmitted joint-position and robot-state data to the ROS environment, allowing the digital robot model to be updated in RViz. In the opposite direction, the ROS system transmitted the robot target sequence generated by the motion-planning pipeline. These targets were decoded by the RAPID program and executed by the physical robot.

Although Figure 3.4 includes the computer-vision package as part of the robotic system architecture, the internal processing performed by this package is described later in the thesis. In this section, the package is shown only as a modular ROS component that provides object-related information to the rest of the system. The physical and digital layers of the robotic cell, including the robot description, RViz visualisation, socket communication and controller input/output (I/O) communication, can operate independently of a specific vision algorithm. For autonomous pick-and-place execution, however, a perception module is required to process acquired point clouds and provide object information to the motion-planning.

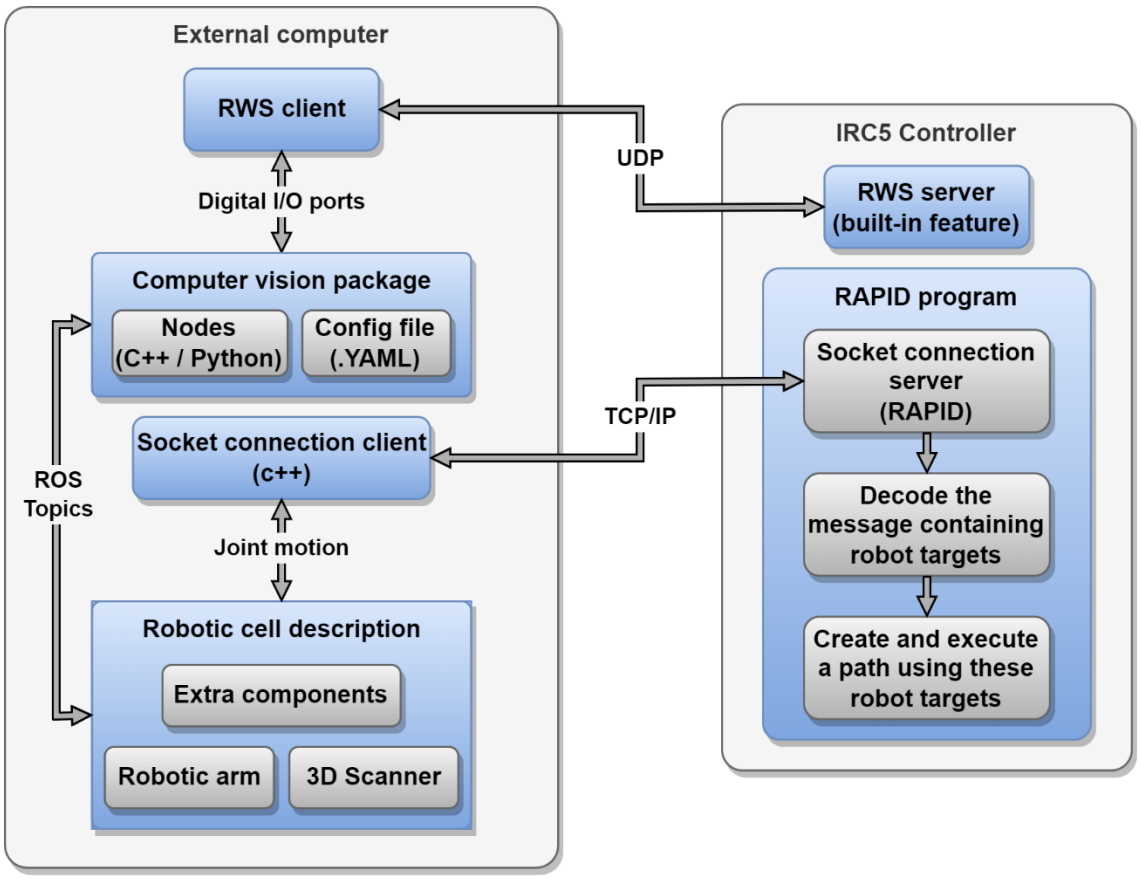


Figure 3.4: The ROS workspace structure used to connect the external computer with the ABB IRC5 controller. The external computer hosted the ROS workspace, including the computer-vision package, robotic-cell description, socket communication client, RViz and MoveIt, while the IRC5 controller handled low-level robot execution through a RAPID program.

The second communication route used ABB Robot Web Services (RWS), which is a built-in Representational State Transfer (REST)-based communication interface available on the IRC5 controller [128]. Access to RWS was implemented through the ROS package *abb_librws*, an open-source C++ library developed within ROS-Industrial that provides a convenient interface for interacting with ABB controllers via RWS [129]. In this implementation, *abb_librws* was used by the ROS-based system to access controller data over the network and to read or modify digital I/O signal states. This provided a separate communication layer for peripheral coordination, while robot motion data were handled through the TCP/IP socket connection with the RAPID program. By separating motion communication from I/O communication, the system could exchange robot targets through the socket interface while using RWS through *abb_librws* for controller-level signal operations and status monitoring.

Within the ROS workspace, the main modules exchanged information through ROS topics and network interfaces. The robotic-cell description provided the digital model used for visualisation and planning, the socket client handled robot-motion communication, and the RWS client handled controller I/O communication. This modular structure allowed the same robotic-cell architecture to be used with different perception implementations. Therefore, the computer-vision package developed for the first experiment could later be replaced by the new point-cloud processing package described in the following chapter, while preserving the overall ROS integration, robot communication and pick-and-place execution structure.

3.3.4 MoveIt-Based Motion Planning

MoveIt was used as the motion-planning framework for the robotic cell. Within ROS, MoveIt provides tools for inverse kinematics, motion planning, collision checking and trajectory generation. Its role in this experiment was to connect the digital robot model with executable robot motion, allowing target poses generated by the system to be converted into feasible joint-space trajectories for the ABB IRB 1200 robot.

The MoveIt configuration was generated from the robot description used in the digital counterpart of the cell. This configuration defined the robot kinematic chain, planning groups, joint limits, end-effector frame and collision geometry required for motion planning. The surrounding cell components, such as the conveyor, placing area and fixed structures, were included in the planning scene so that robot trajectories could be checked against the known physical constraints of the workspace.

MoveIt supports different motion-planning algorithms through planning plugins. Common planners include the Open Motion Planning Library (OMPL) [125], Stochastic Trajectory Optimization for Motion Planning (STOMP) [130], and Covariant Hamiltonian Optimization for Motion Planning (CHOMP) [100]. In this experiment, OMPL was selected as the motion planner. OMPL is a sampling-based motion-planning framework that generates collision-free paths by exploring the robot configuration space rather than explicitly searching all possible motions. This approach is particularly suitable for industrial manipulators operating in constrained environments, where feasible solutions must be found efficiently while respecting joint limits and workspace obstacles.

Compared with optimization-based planners such as STOMP and CHOMP, OMPL generally provides faster planning performance and robust integration with MoveIt for standard pick-and-place applications and its capabilities have been proven over the years in many related works [131], [132]. Since the task considered in this work involves repeated point-to-point motions between known pick and placement locations, the primary requirement is reliable generation of collision-free trajectories rather than trajectory optimization. For this reason, OMPL was considered the most appropriate planner for the experimental setup, providing a practical balance between computational efficiency, planning reliability, inverse kinematics integration, and collision checking capabilities.

In this section, MoveIt is described only as the motion-planning layer of the robotic cell. The generation of object poses from point-cloud data is presented in Section 3.4, while the conversion of these poses into pick-and-place targets and the complete execution sequence are described in Section 3.5.

3.4 Vision-Based Point Cloud Processing

The objective of the vision algorithm developed for this first experiment was to process raw 3D point-cloud data and extract the geometric information required to identify and localise objects on the picking area. The method focused exclusively on point-cloud processing, transforming an unstructured scene into a set of segmented object candidates with associated position, orientation and shape information. These outputs were later used by the robotic system for manipulation and sorting.

The algorithm was designed for simple geometric objects, particularly rectangular parts, whose shape could be reliably characterised using classical geometric analysis. Rather than relying on ML techniques, the approach used a sequence of

deterministic point-cloud processing operations including filtering, clustering, orientation estimation and shape matching. This approach was appropriate for the first experimental stage, where the objective was to validate the basic perception pipeline under controlled conditions.

The processing pipeline consisted of five main stages. First, a raw point cloud representing the scanned scene was provided as input to the algorithm. The method operates directly on point-cloud data represented as an $N\ x\ 3$ array of Cartesian coordinates (X, Y, Z), making it independent of any specific sensor and therefore applicable to point clouds acquired using different 3D scanning technologies. Second, the point cloud was filtered to remove irrelevant regions and isolate the workspace containing the objects of interest. Third, Density-Based Spatial Clustering of Applications with Noise (DBSCAN) was applied to separate individual object candidates [133], [134]. Fourth, Principal Component Analysis (PCA) was performed on each cluster to estimate its centroid and dominant orientation [135]. Finally, each segmented object was compared with predefined reference geometries using a shape-matching procedure to determine its identity and sorting category.

The following sections describe each stage of the point-cloud processing pipeline in detail, from the initial scene representation to the final object classification step. More advanced perception methods for complex industrial parts are introduced in the second experimental stage described in Chapter 4.

3.4.1 Point Cloud Acquisition

The input to the localisation algorithm was a raw point cloud generated by the PhoXi 3D Scanner S. For all experiments presented in this work, point-cloud data were acquired using the PhoXi Control application. However, the localisation method is not dependent on this specific sensor and can be applied to any 3D vision system capable of generating point-cloud representations of the scene.

The acquired scene contained not only the target objects but also the conveyor surface and any other visible structures within the scanning region. Consequently, the raw point cloud included a significant amount of information that was not relevant for object localisation. Directly applying object detection techniques to the complete scene would increase computational cost and reduce robustness due to the presence of background points and measurement noise.

For this reason, the raw point cloud served only as the starting point of the processing pipeline. A sequence of filtering and segmentation operations was subsequently applied to isolate the object regions and prepare the data for geometric analysis and classification.

3.4.2 Background Removal and Preprocessing

The raw point cloud acquired by the 3D scanner contained both useful object points and unwanted background points. Since the purpose of the localisation method was to estimate the position and orientation of the target objects, points corresponding to the conveyor surface and other non-object regions were removed before clustering.

The preprocessing stage reduced the scene point cloud to a set of candidate object points by removing the surface and keeping only the points belonging to potential

objects. Since the raw acquisition contained both the objects and the supporting surface, two filtering approaches were evaluated.

The first approach was a hard-coded height filter. In this method, the axis perpendicular to the picking surface was predefined according to the point-cloud reference frame, typically the Z-axis. A threshold value was then defined along this axis, and all points below that value were removed. The raw point cloud PC can be represented as a set of N points:

$$PC = \{p_i \in \mathbb{R}^3 \mid p_i(x_i, y_i, z_i), i = 1, 2, \cdots, N\} \quad 3.1$$

Using the predefined vertical axis, the filtered point cloud PC_f was obtained as:

$$PC_f = \{p_i \in PC \mid z_i > z_{th}\} \quad 3.2$$

where z_{th} is the height threshold used to remove points corresponding to the surface while retaining points above it. This approach is simple and computationally efficient, but it assumes that the selected axis correctly represents the surface normal and requires recalibration whenever the surface height changes.

The second approach was based on surface-normal estimation. If the supporting surface was sufficiently flat and visible in the point cloud, its normal vector could be estimated directly from the acquired data. To do this, a KD-tree was used to efficiently search for the local neighbouring points around each point in the cloud. For each point p_i, its local neighbourhood K_i was defined using the k-nearest neighbouring points:

$$K_i = \{p_j \in PC \mid p_j \; is \; one \; of \; the \; k \; nearest \; neighbours \; of \; p_i\} \quad 3.3$$

The local surface geometry around p_i was then estimated from the covariance matrix of its neighbouring points:

$$\Sigma_i = \frac{1}{k} \sum_{p_j \in K_i} (p_j - \bar{p_i})(p_j - \bar{p_i})^T \quad 3.4$$

where $\bar{p_i}$ is the mean position of the neighbouring points in K_i, and Σ_i is the local covariance matrix around point p_i. The eigenvector associated with the smallest eigenvalue of Σ_i was taken as the estimated local surface normal, since it represents the direction with the least variation among the neighbouring points.

Once the normal vector n of the supporting surface was estimated, the surface could be represented as a plane defined by n and a point p_0 lying on the plane. The signed distance of each point p_i from the plane is given by:

$$d_i = n \cdot (p_i - p_0) \quad 3.5$$

The filtered point cloud can then be defined as:

$$PC_f = \{p_i \in PC \mid d_i > d_{th}\} \quad 3.6$$

where d_{th} is the minimum distance from the surface required for a point to be retained as a candidate object point. The advantage of this method is that it adapts to changes in surface orientation and position. However, its performance depends on the visibility of

the surface and can be affected when large portions of the surface are occluded by objects.

For the experiments presented in this work, the hard-coded height filter was selected because the conveyor surface remained fixed throughout all tests and the simpler implementation provided reliable and repeatable results.

After background removal, the remaining points represented possible objects located on the conveyor. This filtered point cloud was used as the input for DBSCAN clustering. The quality of this preprocessing step directly influenced the stability of object detection because clustering can only correctly isolate objects when most irrelevant background points have been removed and the object geometry has been sufficiently preserved.

3.4.3 DBSCAN-Based Object Clustering

The filtered point cloud was segmented into candidate objects using Density-Based Spatial Clustering of Applications with Noise. DBSCAN was selected because it can group points according to spatial density without requiring the number of clusters to be known in advance. This is suitable for random-flow conveyor scenarios where the number of visible objects may change between scans.

DBSCAN uses two main parameters, epsilon (ε) and minimum number of points ($MinPts$). The ε parameter defines the maximum distance between two points for them to be considered neighbours. The $MinPts$ parameter defines the minimum number of neighbouring points required for a point to be treated as part of a dense region.

For a point $p = (x_p, y_p, z_p)$ belonging to a point cloud PC, the ε-neighbourhood of p is defined as:

$$N_\varepsilon(p) = \{q \in PC \mid d(p,q) \leq \varepsilon\} \quad 3.7$$

where $d(p,q)$ is the Euclidean distance between points p and q:

$$d(p,q) = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2 + (z_p - z_q)^2} \quad 3.8$$

Based on this neighbourhood, each point is classified as a core point, border point, or noise point. A point is a core point when it has at least $MinPts$ points inside its ε-neighbourhood. A border point does not satisfy the $MinPts$ requirement itself but lies within the ε-neighbourhood of a core point. A noise point satisfies neither condition and is not assigned to a cluster. The clustering process began by marking all points as unvisited. The algorithm then iterated through the point cloud. When an unvisited point was encountered, its ε-neighbourhood was calculated. If the point satisfied the core-point condition, a new cluster was created and expanded by checking neighbouring points. If a neighbouring point was also a core point, its neighbours were added to the same cluster. If the point did not satisfy the core-point condition and was not reachable from any core point, it was labelled as noise. At the end of the process, the filtered point cloud was divided into one or more clusters, where each cluster represented a potential object on the conveyor. Given a set of clusters:

$$C = \{C_1, C_2, \cdots, C_n\} \quad 3.9$$

each cluster C_i was non-empty, clusters were mutually disjoint,

$$C_i \cap C_j = \emptyset \quad for\ i \neq j \tag{3.10}$$

and their union formed a subset of the original point cloud:

$$\bigcup_{i=1}^{n} C_i \subseteq PC \tag{3.11}$$

This means that all clustered points belonged to the original point cloud, while some points could remain unclustered as noise. The resulting clusters were then analysed individually to estimate their centroid and orientation.

The use of DBSCAN provided flexibility because the clustering behaviour could be adjusted through ε and $MinPts$. Increasing or decreasing these values changed the sensitivity of the algorithm to point density and separation between objects. This made it possible to tune the method according to the scanner resolution, object size, and expected spacing between objects on the conveyor.

3.4.4 PCA-Based Centroid and Orientation Estimation

After clustering, Principal Component Analysis (PCA) was applied to each object cluster to extract its centroid and dominant orientation. PCA was used because the objects considered in this experiment had simple geometric shapes with clear principal axes. For rectangular objects, the longest dimension of the point cloud provides a useful indication of the object orientation on the conveyor plane.

For a cluster C containing n points, each point p_i can be represented as:

$$p_i = [x_i,\ y_i,\ z_i] \tag{3.12}$$

The centroid $\mathcal{G}$ of the cluster was calculated as the mean of all points in the three Cartesian dimensions:

$$\mathcal{G} = \frac{1}{n}\sum_{i=1}^{n} p_i = \left[\frac{1}{n}\sum_{i=1}^{n} x_i,\ \frac{1}{n}\sum_{i=1}^{n} y_i,\ \frac{1}{n}\sum_{i=1}^{n} z_i\right] \tag{3.13}$$

The centroid represented the estimated position of the detected object. This position was later used to define the robot target for picking.

To estimate the object orientation, the cluster was first centred by subtracting the centroid from each point:

$$p'_i = p_i - \mathcal{G} \tag{3.14}$$

The centred cluster was then represented as a matrix C_o:

$$C_o = \begin{bmatrix} p'_1 \\ p'_2 \\ \vdots \\ p'_n \end{bmatrix} \tag{3.15}$$

The covariance matrix was calculated as:

$$\Sigma = \frac{1}{n-1} {C_o}^T \times C_o = \frac{1}{n-1} \sum_{i=1}^{n} (p_i - \mathcal{G})(p_i - \mathcal{G})^T \quad 3.16$$

where C_o is the centred cluster matrix. The eigenvectors and eigenvalues of the covariance matrix were then calculated as:

$$\Sigma v = \lambda v \quad 3.17$$

The eigenvectors define the principal directions of variation in the cluster, while the eigenvalues indicate the amount of variation along each direction. The eigenvector associated with the largest eigenvalue corresponds to the longest direction of the object. For the rectangular objects used in this experiment, this direction was used to estimate the object orientation. The orthogonal principal direction represented the shorter dimension of the object.

The output of this stage was therefore a set of geometric descriptors for each detected object, including centroid, eigenvectors, and eigenvalues. The centroid provided the object position, while the principal eigenvector provided the orientation required for alignment during top-down picking. These values were also used in the subsequent object matching and sorting stage.

3.4.5 Object Shape Matching and Sorting

The final stage of the localisation method compared each detected object with a set of predefined target objects stored in the system. Each detected object cluster was treated as a source point cloud, while each predefined object model was treated as a target point cloud. The purpose of this stage was to determine which known object type best matched each detected cluster and to assign the corresponding placement location for sorting.

The target representations were generated from object mesh files stored in the system. For each configured object type, the corresponding STL mesh was loaded and converted into a point cloud by uniformly sampling points from the mesh surface. The generated target point cloud was then rotated and scaled according to the system configuration so that it matched the reference frame and scale used by the perception pipeline. Since the experiment focused on top-down picking, only the upper surface of the target object was retained for comparison. This was achieved by selecting points close to the maximum height of the sampled target point cloud:

$$PC_t^{TOP} = \{ p_i \in PC_t \mid z_i > z_{max} - \delta_z \} \quad 3.18$$

where PC t is the sampled target point cloud, PC t top is the retained upper-surface target point cloud, z max is the maximum height value in the target point cloud, and δ z is a small height tolerance.

A centroid-based alignment was then used to place the source and target point clouds in a common reference position. The centroid was calculated using Equation 3.13, and each point was translated relative to this centroid using the centring operation defined in Equation 3.14. This moved each point cloud to a local origin and allowed the source and target objects to be compared independently of their absolute position in the workspace.

In addition to centroid alignment, the point clouds were transformed using their principal axes. The PCA information extracted in the previous stage was used for the detected source objects, while PCA was computed directly for the predefined target point clouds. This transformation aligned each object with a local coordinate frame based on its dominant geometric directions. As a result, the comparison focused on the object shape rather than its position or orientation in the scene.

After alignment, a feature vector was extracted from each source and target point cloud. The implemented matching node used three geometric features: the object width, the object height, and the number of points in the point cloud. The width and height were calculated from the range of point coordinates along the local x- and y-axes:

$$w = x_{max} - x_{min} \quad and \quad h = y_{max} - y_{min} \tag{3.19}$$

The feature vector for each object was therefore defined as:

$$f = [w, \ h, \ N_p] \tag{3.20}$$

where w is the width, h is the height, and N_p is the number of points in the point cloud.

For each detected source object, the feature vector was compared with the feature vectors of all predefined target objects. The absolute error between a source feature vector f_s and a target feature vector f_t was calculated as:

$$e_{s,t} = |f_s - f_t| \tag{3.21}$$

Since the features had different numerical scales, each feature error was normalised using min-max normalisation:

$$\hat{e}_{s,t} = \frac{e_{s,t} - e_{min}}{e_{max} - e_{min}} \tag{3.22}$$

where $e_{s,t}$ is the error between the source object s and target object t before normalisation, e_{min} and e_{max} are the minimum and maximum error values for that feature, and $\hat{e}_{s,t}$ is the normalised source-target error. This normalisation was applied separately to each feature across the compared source-target pairs.

The normalised feature errors were then combined to produce a single matching score for each source-target pair. This score was calculated as the sum of the normalised errors:

$$D_t = \sum_{m=1}^{M} \hat{e}_{t,m} \tag{3.23}$$

where D_t is the total matching error for the target t, M is the number of extracted features, and $\hat{e}_{t,m}$ is the normalised error for feature m. A lower value of D_t indicates a better match between the detected object and the predefined target object.

The target object with the lowest total matching error was selected as the best match:

$$t_{best} = arg \min_{t} D_t \tag{3.24}$$

where t_{best} is the selected target object. In other words, the detected object was assigned to the predefined target whose feature vector produced the smallest normalised error.

The matching process was implemented as a ROS node. The node subscribed to the processed object point clouds and to the position-orientation data extracted from the previous stages. It then generated the geometric features for each detected object, compared them against the predefined target features, and identified the target with the lowest matching error. The index of this best-matching target was published for each detected source object and used by the sorting system to select the corresponding placement location.

The method linked perception with sorting. The detected object was not only localised in the scanning area but also matched to a predefined object type and assigned to the correct target placement position. The system supported multiple predefined object types, which were configured by loading the required mesh information and corresponding placement data into the system.

In this first experiment, the object matching and sorting strategy was suitable because the tested parts had simple rectangular geometries and clear differences in size. The approach allowed the system to detect objects in random positions and orientations, classify them according to their expected shape, and provide the necessary information for pick-and-place execution. However, because the method relied on geometric clustering, principal-axis alignment, and simple feature-based shape comparison, it was best suited to objects with simple and distinguishable geometries. This limitation motivated the later extension of the system to learning-based perception and template-based registration for complex industrial parts.

3.5 ROS Computer-Vision Package and Pick-and-Place Execution Pipeline

After the vision-based point-cloud processing method described in Section 3.4, the next stage is to integrate the perception outputs into the ROS-based robotic system. In the implemented workspace, this is mainly achieved through the *computer_vision_pkg* package. Although named as a computer-vision package, this package is not limited to point-cloud processing alone. It also contains the application-level nodes used for scan triggering, segmented-object data publication, detected-object transform (TF) generation, target-object matching, placement-frame generation, RViz marker publication, robot-control coordination and pick-and-place execution.

Figure 3.5 illustrates the operational flow of the implemented autonomous pick-and-place system. The process begins with system initialisation and conveyor operation, followed by object detection, point-cloud acquisition and processing, target generation, robot execution and cycle repetition. The point-cloud processing stage shown in the figure corresponds to the perception pipeline described in Section 3.4, while the remainder of this section focuses on how these perception outputs are integrated with the ROS workspace, robot communication framework and autonomous pick-and-place execution process.

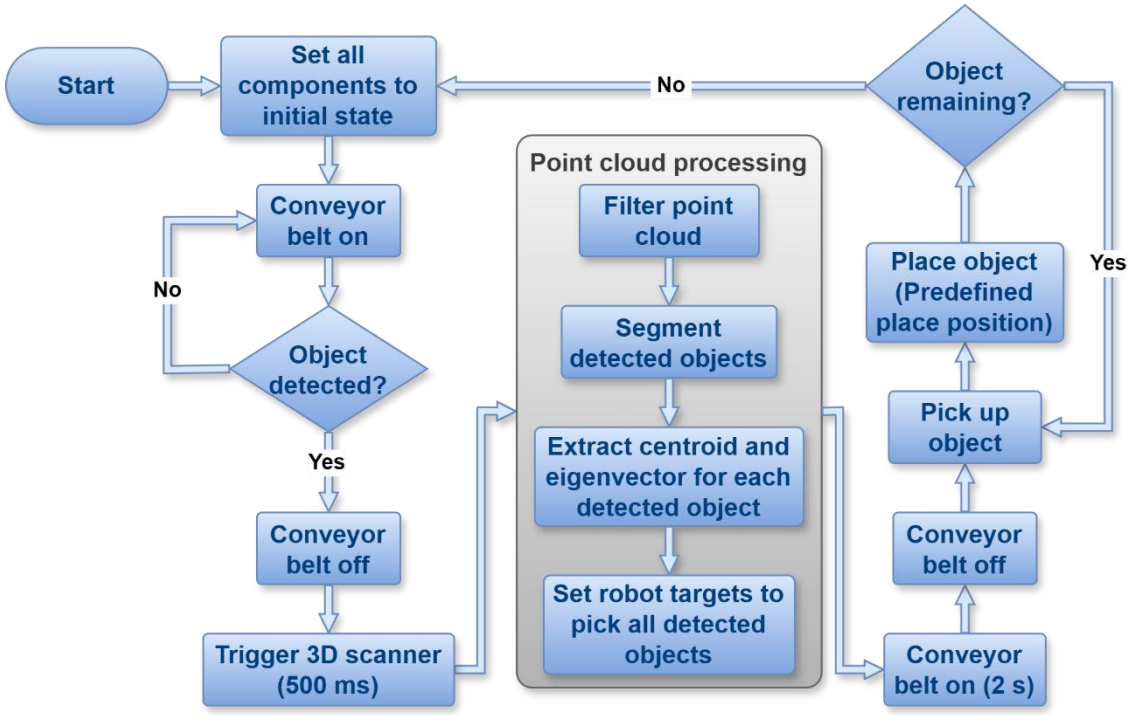


Figure 3.5: Autonomous pick-and-place process flowchart. The system initialises all components, activates the conveyor belt, detects objects using retroreflective sensors, stops the conveyor, triggers the 3D scanner, processes the point cloud, generates robot targets for the detected objects, executes pick-and-place actions and repeats the sequence until no objects remain in the scanning region.

3.5.1 ROS Workspace Organisation

The ROS workspace is organised into separate packages according to their function in the robotic cell. This separation allows the robot description, scanner interface, robotic cell environment, communication layer and application-level task logic to be developed and maintained as distinct modules. Figure 3.6 provides a simplified tree representation of the main workspace structure, highlighting the ABB robot packages and the internal organisation of the computer-vision package used in this system.

The *abb_robot_arm* directory contains the ABB robot-related packages used in the robotic cell. These packages were adapted from the ROS-Industrial ABB repositories and provide the basis of the robot description, motion-planning configuration, and controller-communication infrastructure used in this work. The main components were adapted from the abb repository, which provides the ABB IRB 1200 support and MoveIt configuration packages, and the *abb_driver* package, which provides the TCP/IP communication interface between ROS and the ABB IRC5C controller. The ABB support package provides the robot description, including the URDF model, visual meshes, and collision meshes of the ABB IRB 1200. The MoveIt configuration packages define the robot planning groups, joint limits, kinematic parameters, and planning pipelines used for trajectory generation. Together, these packages enable trajectories generated in the ROS environment to be communicated to the physical robot for execution.



Figure 3.6: Simplified ROS workspace structure used in this experiment, showing the main ABB robot packages and the configuration files and scripts within the computer-vision package.

The *phoxi_camera* package provides the ROS interface for the PhoXi 3D Scanner S. This package includes the scanner description, configuration files, launch files, messages, services and source files required to connect to the camera and acquire point-cloud frames. During operation, the scanner output was published as a point-cloud topic and used as the input to the object-localisation pipeline.

The *pick_and_place_env* package describes the fixed robotic-cell environment and peripheral-device interfaces. It contains URDF and Xacro (XML Macros) descriptions and meshes for the robotic cell components, including the conveyor, gripper-related components, calibration tool, support structures and safety elements. It also includes nodes used to interact with the conveyor, sensor and gripper signals through the ABB controller I/O interface.

The *computer_vision_pkg* package contains the application-level implementation of the random-flow pick-and-place task. As shown in Figure 3.6, this package is organised around two main elements. These are a set of YAML configuration files and a collection of Python scripts implementing the main processing and execution logic. The configuration files store the system, point-cloud, picking and placing parameters, while the scripts implement functions such as point-cloud acquisition and processing, object transform generation, pattern matching, target marker publication, target TF publication and robot control. In this system, *computer_vision_pkg* acts as the main coordination package between perception outputs and robotic execution.

This organisation separates the low-level robot interface from the task-level logic. The ABB robot driver, scanner interface and robotic cell description provide the infrastructure required by the cell, while *computer_vision_pkg* implements the specific pick-and-place behaviour used in this experiment.

3.5.2 Computer-Vision Package Architecture

The computer_vision_pkg package was implemented using Python and C++ scripts. Python scripts are mainly used for high-level task logic, point-cloud processing, TF generation, object matching and robot-control coordination. C++ is used where direct communication or efficient execution was required. The implementation reported in the paper used Python 3.8.10 and C++ 9.4.0.

The package contains four main types of resources. These are configuration files, target meshes, custom messages and execution scripts. The configuration files store the parameters required to adapt the package to the robotic cell. The mesh directory contains STL files for the target objects and tray. The custom messages define the data structures used to pass processed point-cloud and object information between ROS nodes. The scripts implement the perception, matching, TF publication and execution logic. The main configuration files used in this experiment are summarised in Table 3.2. These configuration files control the parameters required to adapt the package to the robotic cell and controlled the behaviour of the perception, picking and placing stages of the system:

Table 3.2: Summary of the main YAML configuration files used by the computer-vision package and their functions.

YAML configuration files	Purpose
general_sys_params	Stores system-level parameters such as conveyor speed, stopping delay, scanning time, reverse-conveyor distance and whether the conveyor should return to its initial position.
point_cloud_params	Stores point-cloud filtering, segmentation and downsampling parameters, including axis thresholds, DBSCAN parameters and voxel size.
picking_task_params	Stores detected-object frame settings, parent frame names and picking/pre-picking offsets.
placing_task_params	Stores tray frame information, target object names, object mesh references, placement-frame definitions, placement offsets, marker settings and target slot positions.

The use of YAML configuration files allows the package to be adapted without modifying the core source code. For example, target object names, mesh paths, placement locations and pose offsets can be changed directly in the configuration files. This was important because the system needs to support different target object types and predefined placement positions within the same robotic-cell structure.

The package also defines custom ROS messages to exchange information between processing stages. The *PointCloudArray* message is used to publish multiple segmented point clouds from objects. The *PosOriObjectsArray* message is used to publish the number of detected objects, their centroid data, eigenvectors, eigenvalues and PCA-related component points. The *BestMatchArray* message is used to publish the matching result between detected source objects and predefined target objects.

This modular implementation allows each stage of the application to be executed as a ROS node and to communicate with the others through topics, services and TF frames. The package provides the link between the point-cloud processing algorithm, the digital cell representation, the ABB controller communication layer and the physical pick-and-place task.

Table 3.3: Summary of the main scripts and nodes used by the package.

Python modules	Main role in the system
main_script	Supervises the high-level task state, including conveyor activation, sensor detection, scanner triggering and object-queue monitoring.
get_poitcloud_topic	Subscribes to the scanner point-cloud topic, filters the raw point cloud, segments objects, downsamples each object point cloud, extracts centroid/orientation data and publishes processed outputs.
generate_obj_tf	Converts detected object pose information into TF frames used as pick targets.
basic_pattern_matching	Loads predefined target meshes, extracts target features, compares detected objects with configured targets and publishes the best-match result.
object_target_tf_publisher	Publishes TF frames for the tray and predefined object placement positions.
object_target_marker_publisher	Publishes RViz markers for the target objects and tray visualisation.
robot_control	Uses the detected-object TFs, target TFs and best-match information to execute the pick-and-place sequence through MoveIt and the ABB communication interface.
utils	Provides helper functions used across the package, including configuration loading and data decoding.

3.5.3 System Initialisation and Object Detection

The system execution begins by setting the robotic cell components to their initial state. The main application node initialises the ROS node, loads the system parameters from the general configuration file, subscribes to the conveyor sensor and object-queue topics, and creates publishers for the conveyor state, conveyor direction and gripper state.

At initialisation, the conveyor and gripper states are set according to the task requirements. The conveyor is then activated to move objects into the scanner region. Object detection is performed using the retroreflective sensor connected to the controller I/O system. The sensor state is published to ROS and monitored by the main application node.

While no object is detected, the conveyor continues moving. When the sensor detects that an object had entered the scanning region, the system waits for a short stopping delay and then stops the conveyor. This delay allows the object to reach the desired scanning position before acquisition. The conveyor stop is necessary because the structured-light scanner requires the object to remain stationary during point-cloud capture.

Once the conveyor is stopped, the scanner is triggered through the PhoXi camera service interface. The system then waits for the configured scanning time to allow the point cloud to be acquired and published. In the implementation, the scan acquisition and conveyor timing are controlled through parameters such as conveyor speed, stopping delay and scanning time. After the scan is captured, the conveyor can be briefly reactivated to adjust object position before the robot execution sequence began.

This state-machine logic ensures that the object was detected, stopped, scanned and processed before the robot attempted to pick it. It also allows the system to restart the conveyor once the detected-object queue had been cleared.

3.5.4 Point-Cloud Processing Outputs and ROS Messages

The point-cloud processing node subscribes to the raw scanner point-cloud topic. When a point cloud is received, it is converted into a numerical point array and processed using the method described in Section 3.4. The processing sequence includes filtering, DBSCAN-based segmentation, voxel downsampling, centroid extraction and PCA-based orientation estimation.

After filtering, the node publishes a filtered point cloud for visualisation or debugging. After segmentation, each detected object cluster is converted into a separate point-cloud message and published as part of a *PointCloudArray* message. This allows downstream nodes to access the individual object point clouds rather than the full raw scene.

The node also publishes the geometric data extracted from the detected objects. For each object, the published data include the centroid, eigenvectors, eigenvalues and a PCA component point used to derive orientation. These values are packed into the *PosOriObjectsArray* message and published on the object-position/orientation topic.

This design separates point-cloud processing from later task stages. The processing node does not directly control the robot. Instead, it generates structured perception outputs that could be consumed by the TF generation node, object-matching node and robot-control node.

3.5.5 Detected-Object TF Generation

After the centroid and orientation data are published, the detected-object TF generation node converts this information into coordinate frames. For each detected object, the node decodes the object-position/orientation message, calculates the object yaw angle from the PCA direction and generates a quaternion representation of the object orientation.

A separate TF frame is then created for each detected object. The child frame name uses a common prefix defined in the picking configuration file, producing frames such as *object_to_pick_frame_1*, *object_to_pick_frame_2*, and so on. These frames are published relative to the scanner coordinate frame.

The use of TF frames allows detected objects to be represented in the same coordinate-management system used by the robot model and digital cell. Instead of passing only numerical coordinates between nodes, each detected object becomes a spatial frame that could be transformed, visualised and used as a target for robot motion.

The detected-object TF publisher continues broadcasting the frames while objects remained in the queue. When the object queue is cleared, the node stops publishing the previous object frames and waits for the next scan to provide new detections.

3.5.6 Target Object Matching and Slot Allocation

Object matching is performed by the pattern-matching node. This node subscribes to both the processed object point-cloud array and the object-position/orientation data. It also loads the predefined target object meshes from the configuration file.

For each configured target object, the corresponding STL mesh is loaded and sampled into a point cloud. The target point cloud is transformed according to the configuration settings, including rotation and scale. Since the experiment used top-down picking and the scanner observed the upper visible surface of each object, only the top surface of the target point cloud is retained for comparison.

The node extracts simple geometric features from both source and target point clouds. Each detected object is treated as a source point cloud, while each predefined object mesh is treated as a target. Before feature extraction, point clouds are aligned to a local coordinate frame based on the centroid and PCA directions. This reduces the influence of the absolute object position and orientation in the scanner frame.

The implemented matching procedure uses object width, object height and number of points as the feature vector. For each source object, the extracted feature vector is compared against all target feature vectors. The errors are normalised and combined into a matching score. The target object with the lowest score is selected as the best match, and the corresponding target index is published through the *BestMatchArray* message.

This matching result links each detected object to one of the predefined object categories configured in the system. Once the object category is known, the robot-control node can select the correct placement row or group of slots associated with that object type.

The placement targets are defined through the placing configuration file, which specifies the tray frame, object names, object mesh settings, placement offsets and marker colours. This configuration-based approach makes the system reconfigurable, allowing different trays, bases or object sets to be used without modifying the underlying software architecture.

The target TF publisher creates the tray frame and the individual placement frames according to the positions defined in the configuration. In parallel, the marker publisher displays the tray and target objects in RViz using the corresponding STL meshes. This allows the configured placement positions to be visualised in the digital environment and used as place targets during execution

3.5.7 Calibration and Coordinate Consistency

Coordinate consistency is required because the point cloud, detected-object frames, robot model and placement targets are defined in different coordinate references. The scanner produces object data in its own reference frame, while the robot executes motions relative to the robotic cell frame. The TF framework is used to maintain the relationship between these frames.

The scanner was calibrated according to the Photoneo calibration procedure, using the official Photoneo calibration sheet provided in the scanner documentation and included in Appendix A. The PhoXi scanner supports different coordinate-space

definitions. CameraSpace defines the point cloud with its origin located at the scanner. RobotSpace defines the point-cloud data relative to a robot coordinate system. CustomSpace allows the user to define an application-specific coordinate frame. MarkerSpace defines the point cloud relative to a physical marker pattern placed in the scene.

In this system, MarkerSpace is used because it provides a stable reference frame based on the marker plate rather than the scanner body. This means that if the scanner position is adjusted or recalibrated, the acquired point cloud can still be returned in the coordinate system defined by the marker pattern. This is useful for the robotic cell because the detected-object TF frames are published relative to a repeatable physical reference in the scanning area. This improves consistency between point-cloud acquisition, digital visualisation and robot target generation.

3.5.8 Robot Control and Pick-and-Place Execution

The robot-control node uses the outputs of the previous stages to execute the pick-and-place task. It subscribes to the matching result and uses TF lookups to obtain the detected-object pick frames and the predefined placement frames. The matching result determines which group of placement slots should be used for each detected object.

The system maintains the availability of target placement slots so that objects are placed into free positions. Once a detected object is assigned to a target type, the robot-control logic selects an available placement frame associated with that object type. This avoids assigning multiple detected objects to the same placement position. The pick-and-place motion is executed using a constrained top-down strategy. For each detected object, the robot first moves to a pre-pick pose above the object.

The pre-pick pose is generated using the detected-object frame and the configured offset. The robot then moves to the picking pose, aligned with the estimated object orientation. At the picking pose, the gripper is actuated through the controller I/O interface to grasp the object.

After the object is grasped, the robot moves away from the conveyor and towards the predefined placement position. A pre-place pose is used before the final placement pose to reduce the risk of collision or inaccurate release. The robot then moves to the place pose, opens the gripper and releases the object. After placement, the system updates the slot state and continues with the next detected object if any remained in the queue.

Motion planning and collision checking during this process are handled by MoveIt, as described in Section 3.3.4. The computer-vision package provides the object-related frames and matching decisions, while MoveIt calculates the robot motion required to reach the corresponding pick and place poses. The generated robot target sequence is transmitted to the ABB IRC5 controller through the ROS-Industrial communication interface, where the RAPID program executes the physical motion.

When all objects detected in the current scan have been processed, the robot-control node publishes the updated object-queue state. The main application node then resumes conveyor operation and returns the system to the object-detection stage. If configured, the conveyor can also be reversed by a predefined distance to return to the initial position before continuing the next cycle.

3.5.9 Execution Sequence Summary

The complete execution sequence can be summarised as follows. First, the system initialises the robot, conveyor, gripper, scanner interface and ROS nodes. Second, the conveyor moves objects into the scanning region. Third, the retroreflective sensor detects an object and triggers the stop-and-scan sequence. Fourth, the PhoXi scanner acquires a point cloud and publishes it to ROS. Fifth, the point-cloud processing node filters the scene, segments candidate objects and extracts centroid and orientation data. Sixth, detected-object TFs are generated, and object matching is performed against the configured target meshes. Seventh, the robot-control node selects pick frames and available placement frames. Finally, MoveIt generates the robot motion, and the ABB controller executes the pick-and-place task.

This pipeline closes the loop between sensing, perception, target generation and robot execution. The system does not rely on fixed object pick positions on the conveyor. Instead, each pick target is generated from the detected object pose, while each place target is selected from predefined configured positions. This provides the level of flexibility required for the first experimental stage while keeping the task constrained to simple rectangular objects and top-down grasping.

3.6 Experimental Protocol

The experimental protocol was designed to validate the robotic cell, computer-vision package and pick-and-place execution pipeline developed in the previous sections. The experiment evaluated whether the system could detect rectangular objects arriving on a conveyor belt, estimate their position and orientation from 3D point-cloud data, and execute pick-and-place operations into predefined tray positions.

The protocol was divided into two complementary evaluations. The first evaluation analysed the accuracy of the vision-based orientation estimation method by comparing the real object angle with the angle estimated by the computer-vision algorithm. The second evaluation analysed the reliability of the full pick-and-place task by recording whether each detected object was successfully picked from the conveyor and placed into the correct tray slot.

3.6.1 Experimental Setup

The experiment was conducted using the robotic cell described in Section 3.3. The cell consisted of the ABB IRB 1200 robot, PhoXi 3D Scanner S, conveyor belt, pneumatic gripper and placing table. The objects were transported by the conveyor through the scanner field of view. Once detected and scanned, their positions and orientations were processed by the computer-vision package and converted into robot targets. The robot then picked the objects from the conveyor and placed them into a tray located on the table near the robot.

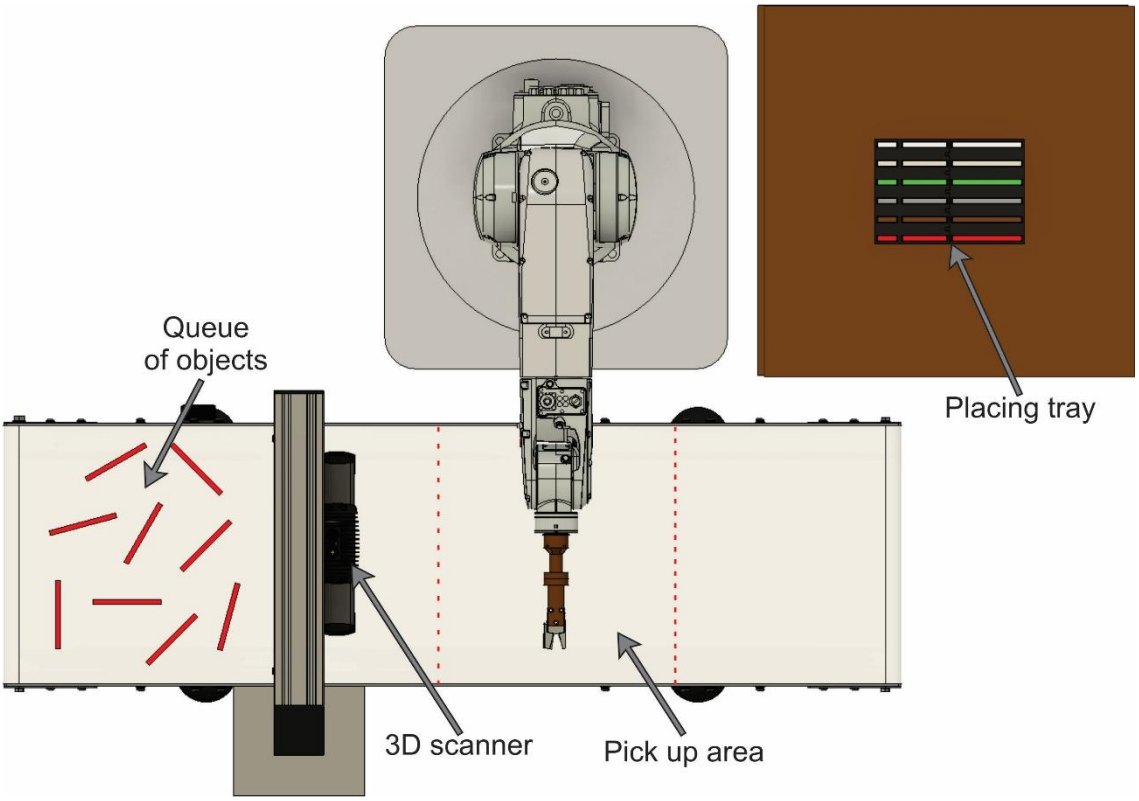


Figure 3.7: Pick-and-place experimental setup. Objects moved along the conveyor through the scanner region for 3D scanning, then were picked by the robot and placed into the corresponding tray slots.

The experiment followed a stop-and-scan strategy. The conveyor moved the objects until the retroreflective sensor detected their presence in the scanning region. The conveyor was then stopped, the 3D scanner was triggered, and the acquired point cloud was processed. Keeping the conveyor stationary during acquisition and picking reduced the risk of mismatch between the scanned object pose and the physical object position.

3.6.2 Test Objects and Placement Tray

Three object types were used in the experiment. All objects had rectangular-prism geometry but differed in length. This allowed the system to be tested with objects that had similar shape characteristics but different dimensions. The object types were defined as small, medium and large.

The small object measured 8 × 8 × 30 mm. The medium object measured 8 × 8 × 70 mm. The large object measured 8 × 8 × 110 mm. These objects were selected because their rectangular geometry was suitable for the PCA-based orientation estimation method described in Section 3.4, while their different lengths allowed the shape-matching and sorting procedure to distinguish between object categories.

A placement tray was designed with customised slots for each object type. Each slot provided 1 mm of clearance on all sides to allow the robot to place the objects without requiring unrealistically tight positioning. The resulting slot dimensions were 10 × 10 × 32 mm for the small objects, 10 × 10 × 72 mm for the medium objects, and 10 × 10 × 112 mm for the large objects.

Table 3.4: Summary the object and tray-slot dimensions used in the experiment.

Object type	Object dimensions	Tray-slot dimensions
Small rectangular object	8 × 8 × 30 mm	10 × 10 × 32 mm
Medium rectangular object	8 × 8 × 70 mm	10 × 10 × 72 mm
Large rectangular object	8 × 8 × 110 mm	10 × 10 × 112 mm

The tray was positioned on the table within the robot workspace. The placement locations were predefined in the system configuration files and represented digitally in RViz through target frames and markers. During execution, the matching result from the computer-vision package determined which group of tray slots was associated with each detected object type.

3.6.3 Orientation Estimation Test

The first part of the experimental protocol evaluated the accuracy of the orientation estimated by the computer-vision algorithm. During each trial, the system recorded the detected object position and orientation. The estimated orientation was then compared with the known real orientation of the object.

Each experimental batch consisted of 12 objects of the same type placed on the conveyor at predefined orientations. The orientations ranged from 0° to 165° in increments of 15°. Therefore, the tested angles were:

To maintain consistency between repetitions, a mould with 12 slots was used to position the objects at the required angles. This ensured that the real object orientation was known for each sample and that the angle distribution was consistent across repeated batches.

For each object type, the 12-angle batch was repeated 60 times. This resulted in 720 orientation measurements per object type. Since three object types were tested, the total number of pick-and-place executions across the experiment was 2160.

Table 3.5: Summary of the orientation-test protocol.

Parameter	Value
Object types	Small, medium and large rectangular objects
Number of orientations per batch	12
Angle range	0° to 165°
Angle increment	15°
Repetitions per object type	60
Executions per object type	720
Total executions	2160

The orientation-estimation test was important because the pick strategy depended on aligning the gripper with the detected object. Since the robot used a top-down grasping approach, errors in the estimated planar orientation could affect the quality of the grasp and, consequently, the success of the full pick-and-place task.

3.6.4 Pick-and-Place Success Test

The second part of the experimental protocol evaluated the success of the complete pick-and-place operation. This test used the same batches and repetitions described in Section 3.6.3, but the focus was on the physical outcome of each task rather than only the estimated orientation.

A pick-and-place execution was considered successful when the robot correctly picked the object from the conveyor and placed it into the corresponding tray slot. The object had to be grasped from the detected position, transported without being dropped, and released into the correct slot associated with its object type.

A failure was recorded when one of the following conditions occurred:

1. The robot failed to pick the object from the conveyor.
2. The object was grasped but fell during transfer.
3. The object was placed incorrectly in the tray.
4. The object was placed in a mismatched slot.

This success test evaluated the complete system rather than a single module. A successful execution required correct object detection, accurate pose estimation, correct object matching, correct target-slot assignment, feasible robot motion, stable grasping and accurate placement. Therefore, the pick-and-place success test was used as the main practical validation of the integrated robotic cell developed in this chapter.

3.7 Evaluation Metrics

The evaluation of this first experiment focused on two aspects of system performance. The first aspect was the accuracy of the computer-vision algorithm in estimating the orientation of objects on the conveyor. The second aspect was the reliability of the complete pick-and-place operation, including object detection, pose estimation, grasping, transfer and placement.

The metrics were selected to reflect the objective of this experimental stage. Since the objects had simple rectangular geometries, the main perception requirement was accurate planar orientation estimation. Moreover, since the robotic task required physical displacement from the conveyor to the tray, the main execution requirement was successful completion of the pick-and-place sequence.

3.7.1 Orientation Error

Orientation error was used to evaluate the accuracy of the PCA-based orientation estimation method described in Section 3.4. For each detected object, the real object angle was compared with the angle estimated by the computer-vision algorithm. The signed orientation error was calculated as:

$$e_{\theta,i} = \hat{\theta}_i - \theta_i \tag{3.25}$$

where $e_{\theta,i}$ is the orientation error for object i, $\hat{\theta}_i$ is the estimated orientation angle and θ_i is the real object angle.

The signed error indicates whether the algorithm overestimated or underestimated the real angle. A value close to zero indicates that the estimated orientation is close to the real orientation. Positive values indicate that the estimated angle is larger than the real angle, while negative values indicate that the estimated

angle is smaller than the real angle. For each object type, the mean signed error was calculated as:

$$\bar{e}_\theta = \frac{1}{n}\sum_{i=1}^{n} e_{\theta,i} \tag{3.26}$$

where n is the number of evaluated samples for that object type. The mean signed error was used to identify systematic bias in the orientation estimation. A mean value close to zero indicates that the algorithm does not consistently overestimate or underestimate the object angle. The standard deviation of the orientation error was calculated as:

$$\sigma_\theta = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left(e_{\theta,i} - \bar{e}_\theta\right)^2} \tag{3.27}$$

The standard deviation was used to evaluate the consistency of the orientation estimation. A lower standard deviation indicates that the error values are more tightly distributed around the mean. In addition to signed error, absolute error was used for angle-specific visual analysis. The absolute orientation error was calculated as:

$$\left|e_{\theta,i}\right| = \left|\hat{\theta}_i - \theta_i\right| \tag{3.28}$$

This metric removes the sign of the error and shows the magnitude of the deviation between the estimated and real angles. It was used to compare the error associated with each predefined object angle.

3.7.2 Pick-and-Place Success and Failure Rate

Pick-and-place success rate was used to evaluate the reliability of the complete robotic system. This metric considered the full task sequence rather than only the output of the vision algorithm. A successful trial required the robot to pick the object from the conveyor and place it into the correct tray slot associated with that object type. A trial was classified as successful when all the following conditions were satisfied:

1. The object was detected and assigned to the correct object type.
2. The robot reached the detected pick position.
3. The gripper successfully grasped the object.
4. The object remained stable during transfer.
5. The object was placed in the correct predefined tray slot.

A trial was classified as a failure when one or more of these conditions were not satisfied. Failures included missed or unstable picking, object dropping during transfer, incorrect placement inside the tray and placement into a mismatched slot. For each object type, the success rate was calculated as:

$$S = \frac{N_s}{N_t} \times 100 \tag{3.29}$$

where S is the success rate, N_s is the number of successful trials and N_t is the total number of trials. The failure rate was calculated as:

$$F = \frac{N_f}{N_t} \times 100 \tag{3.30}$$

where F is the failure rate and N_f is the number of failed trials. Since each trial was classified as either successful or failed, the relationship between success rate and failure rate was:

$$S + F = 100 \tag{3.31}$$

The success and failure rates were calculated separately for the small, medium and large object types. They were also used to evaluate the overall reliability of the integrated system across all tested objects.

3.7.3 Statistical Analysis

Statistical analysis was used to compare the orientation-error distributions obtained for the different object types. The purpose of this analysis was to determine whether the observed differences in error distribution were likely to be associated with object size.

The *Mann–Whitney U* test was selected to compare the angle-error distributions between object types. This test is suitable when comparing two independent samples without assuming that the data follow a normal distribution. In this experiment, it was used to compare the orientation-error distributions obtained for the small, medium and large objects. The pairwise comparisons were defined as follows:

1. Small object errors compared with medium object errors.
2. Small object errors compared with large object errors.
3. Medium object errors compared with large object errors.

The null hypothesis stated that the two compared error distributions came from the same underlying distribution. The alternative hypothesis stated that the two compared error distributions were different.

The *Mann–Whitney U* statistic was calculated using the ranked values of the two samples. For two object groups with sample sizes n_1 and n_2, the U statistic for the first group is given by:

$$U_1 = n_1 n_2 + \frac{n_1(n_1 + 1)}{2} - R_1 \tag{3.32}$$

where R_1 is the sum of ranks for the first group. The corresponding statistic for the second group is given by:

$$U_2 = n_1 n_2 - U_1 \tag{3.33}$$

The smaller of U_1 and U_2 was used as the test statistic. The statistical analysis provided support for interpreting whether differences in orientation-estimation behaviour were associated with object size. This was relevant because PCA-based orientation estimation depends on the distribution of points in each object cluster. Longer rectangular objects provide a clearer dominant axis, while shorter objects can produce more ambiguous principal directions.

3.8 Results and Discussion

This section presents and discusses the results obtained from this first experiment. The analysis is organised according to the two evaluation dimensions defined in Section 3.7. The first part examines the accuracy of the vision-based orientation estimation method. The second part evaluates the success and failure rate of the complete pick-and-place operation. The results are discussed immediately after each set of findings to connect the measured performance with the behaviour of the implemented robotic system.

3.8.1 Orientation Estimation Accuracy

The first analysis evaluated the accuracy of the computer-vision algorithm in estimating object orientation. The estimated angle generated by the PCA-based method was compared with the real object angle defined during the experimental protocol. Figure 3.8 presents the distribution of the signed angle error for the three object types.

Across all object types, the obtained orientation errors ranged from −2.558° to 2.025°. The mean error was close to zero for all objects, indicating that the algorithm did not produce a strong systematic bias towards overestimation or underestimation. The small object produced a mean error of −0.240° with a standard deviation of 0.536°. The medium object produced a mean error of −0.131° with a standard deviation of 0.282°. The large object produced a mean error of −0.138° with a standard deviation of 0.211°.

Table 3.6: Mean orientation error and error dispersion obtained for the small, medium and large objects.

Type	Mean orientation error (°)	Standard deviation (°)
Small	−0.240	0.536
Medium	−0.131	0.282
Large	−0.138	0.211

These results show that the PCA-based orientation estimation method achieved high accuracy for the three rectangular object types. The mean error remained below 0.25° in magnitude for all object categories. This indicates that the vision algorithm was able to provide orientation estimates with sufficient accuracy for the top-down pick-and-place strategy used in this experiment.

The standard deviation values also indicate consistent estimation behaviour. The small object showed the highest variability, while the medium and large objects produced narrower error distributions. This suggests that object size influenced the stability of the PCA result. Since PCA identifies the principal direction of variation in a point cloud, longer rectangular objects provide a clearer dominant axis. As object length increases, the difference between the long and short dimensions becomes more pronounced, making it easier for PCA to identify the main orientation of the object.

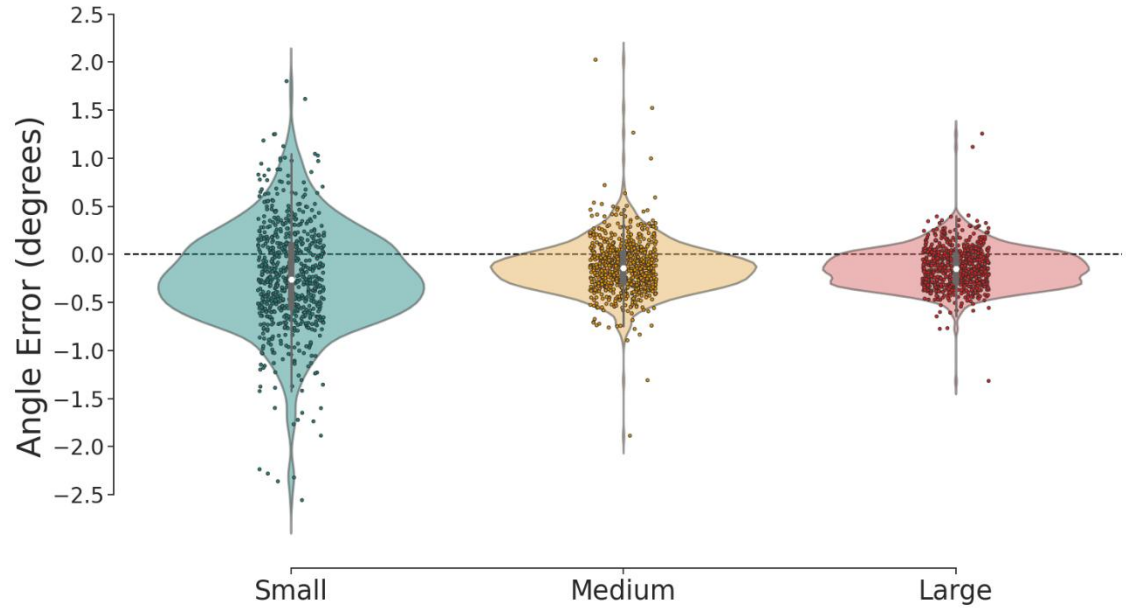


Figure 3.8: Violin plot of the orientation error for the small, medium and large rectangular objects.

The small object produced the largest standard deviation because its geometry provided less separation between dominant and secondary directions. This effect becomes even more pronounced for objects with nearly equal dimensions. For example, in a square object where both sides have the same length, there is no clearly dominant axis in the point distribution. In such cases, the principal components can become ambiguous, and small variations in the measured points may cause the estimated orientation to shift significantly. The diagonal directions represent the largest spread of points and may be selected as dominant directions by the PCA. Relative to the object edges, this corresponds to an orientation offset of up to 45 degrees, illustrating the maximum ambiguity that can occur when the geometry does not strongly favour a single principal direction. Therefore, the higher variability observed for the small object is consistent with the way PCA extracts orientation from geometric point distributions.

3.8.2 Error Distribution Across Object Orientations

A second analysis examined the orientation error for each predefined object angle. Figure 3.9 presents the absolute angle error for the 12 tested orientations, ranging from 0 to 165 degrees in increments of 15 degrees. In this representation, negative signed errors were converted into positive values to compare the magnitude of the deviation at each tested angle.

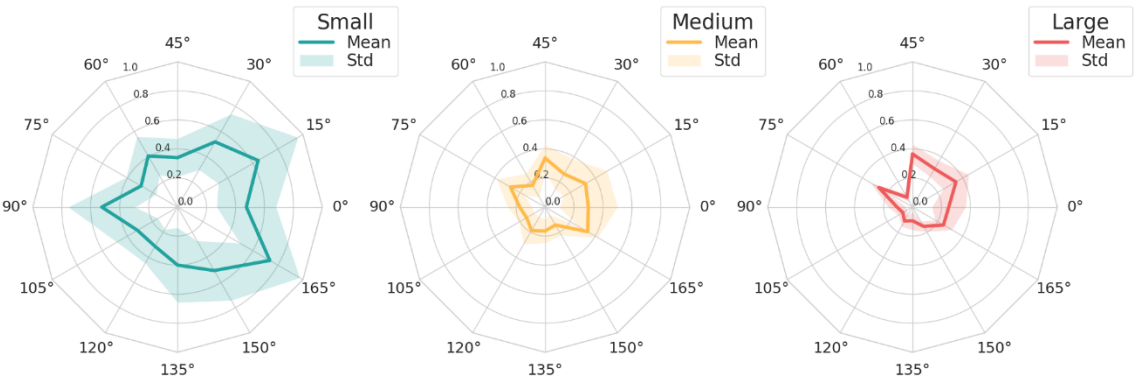


Figure 3.9: Absolute orientation error for each predefined object angle from 0 to 165 degrees in increments of 15 degrees.

The error distribution across tested angles confirms the trend observed in Figure 3.8. The small object displayed more visible variation across the tested angles, while the medium and large objects showed more similar and more compact error behaviour. This reinforces the interpretation that the PCA-based method is more stable when the object has a clear dominant geometric axis.

The Mann-Whitney U test was used to compare the orientation-error distributions between object types. The comparison supported the visual interpretation that the small-object error distribution differed more clearly from the medium and large object distributions. In contrast, the distinction between the medium and large object distributions was less evident. This result is consistent with the dimensional characteristics of the tested objects. The medium and large objects both had a stronger length-to-width difference than the small object, which made their principal axis easier to identify.

These findings show that the classical point-cloud method was effective for the simple rectangular objects used in this first experimental stage. However, they also indicate a limitation of relying only on PCA for orientation estimation. PCA performs well when the object shape has a clear dominant direction, but its reliability can decrease when the object geometry is short, nearly symmetric, noisy or partially segmented. This limitation is important for the thesis because the second experimental stage deals with more complex industrial parts, where the dominant axis may not be sufficient to describe the object pose.

3.8.3 Pick-and-Place Success and Failure Analysis

The second evaluation analysed the reliability of the complete pick-and-place task. This analysis considered the full execution pipeline, including object detection, point-cloud processing, object matching, robot target generation, motion planning, grasping, transfer and placement.

Each object type was tested over 720 pick-and-place executions. The small object produced 27 failures, the medium object produced 42 failures, and the large object produced 56 failures. This corresponded to failure rates of 3.75%, 5.83% and 7.78% respectively. Across all 2160 executions, the total number of failures was 125, producing an overall failure rate of 5.79% and an overall success rate of 94.21%. Figure 3.10 presents the success and failure behaviour observed during the pick-and-place trials.

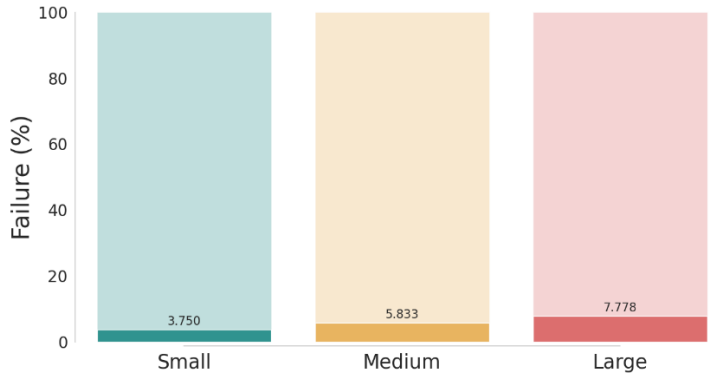


Figure 3.10: Pick-and-place success and failure rates for the small, medium and large rectangular objects. Each object type was evaluated over 720 executions. Failure rates increased from 3.75% for the small object to 5.83% for the medium object and 7.78% for the large object, corresponding to success rates of 96.25%, 94.17% and 92.22%, respectively. Across all 2160 executions, the overall pick-and-place success rate was 94.21%.

The results show that the robotic cell achieved reliable execution for the simple rectangular objects tested in this experiment. The overall success rate of 94.21% indicates that the integrated system was able to detect objects in random orientations, estimate their pose, generate robot targets and execute physical displacement from the conveyor to the tray.

Table 3.7: Number of executions, failures and success rates for each object type.

Type	Total executions	Failures	Failure rate (%)	Success rate (%)
Small	720	27	3.75	96.25
Medium	720	42	5.83	94.17
Large	720	56	7.78	92.22
Overall	2160	125	5.79	94.21

The failure rate increased with object size. The large object produced the highest failure rate, followed by the medium object and then the small object. This trend differs from the orientation-estimation results, where larger objects produced more stable angle estimates. Therefore, the main source of failure was not the vision algorithm alone. Instead, the failures were mainly associated with physical placement into the tray.

Most failures occurred during the placing stage, where objects contacted the edges of the tray slots. This was caused by the strict placement tolerance defined by the tray design. Each slot provided only 1 mm of clearance on each side. This small clearance increased the difficulty of placement because even minor deviations in object position, gripper alignment, robot execution, or release behaviour could cause contact with the tray edge.

The larger object was more affected by this condition because its longer length increased the consequence of small angular or positional deviations during insertion. Even when the estimated orientation was accurate, a slight misalignment during the approach or release could cause the object to strike the slot boundary. This explains why the large object had the most stable orientation-estimation distribution but the highest pick-and-place failure rate.

The small object had the highest orientation variability but the lowest physical failure rate. This suggests that the placement task was less sensitive to its angular deviation because the shorter object had less length over which orientation error could accumulate during insertion. Therefore, the relationship between perception accuracy and task success was not linear. The physical geometry of the object and the clearance of the placement slot strongly influenced execution reliability.

3.8.4 Overall Discussion and Transition to the Next Study

The results presented in this first experiment should be interpreted as the initial validation of the robotic-cell foundation developed in this thesis. The experiment addresses the first step towards the **research question** by testing whether a ROS-based pick-and-place system can respond to random object poses while maintaining reliable execution in a controlled manufacturing cell. This links directly to **SRQ1**, since the experiment required the physical cell, digital counterpart, perception module, motion-planning framework and robot-execution layer to operate as one integrated architecture. It also provides the first experimental response to **SRQ3** by evaluating whether objects arriving in random positions and orientations could be detected, localised, classified and

picked using 3D point-cloud data. Conventional industrial pick-and-place systems achieve reliability by reducing uncertainty through fixed object presentation, predefined trajectories, structured layouts and extensive programming. This approach remains effective in stable production environments, but it becomes less suitable when objects arrive in random positions and orientations or when the target parts change frequently [14], [15], [16], [17]. In this experiment, the robotic cell did not rely on fixed pick positions on the conveyor. Instead, the system detected each object, estimated its pose from the acquired point cloud and generated the corresponding pick target during operation.

This is important because random-flow environments introduce a different type of challenge from conventional structured automation. In a structured cell, object positions and robot motions are defined before execution. In a random-flow cell, the system must perceive the current scene and adapt the pick operation to the detected object pose. This agrees with the literature reviewed in Section 2.2.3, where random object flows are described as whole-system problems that require coordination between localisation, pose estimation, grasp planning, motion execution and industrial communication [3], [17], [18], [25]. The results of this experiment validate not only an orientation-estimation method, but also the integration chain required to convert perception into physical robot action.

The orientation-estimation results showed that the classical point-cloud pipeline was suitable for the simple rectangular objects used in this first stage. The combination of background removal, DBSCAN clustering, PCA-based orientation estimation and feature-based matching produced small orientation errors for all three object types. This confirms that deterministic geometric methods can still be effective when the workspace is controlled and the object geometry provides clear features for analysis. In this experiment, the rectangular objects had well-defined dimensions and a visible dominant axis. This made PCA an appropriate method for estimating planar orientation.

However, the same results also reveal the limits of this approach. PCA estimates orientation from the distribution of points in each cluster, which means that its reliability depends strongly on object geometry and point-cloud quality. The large and medium objects produced more stable estimates because their longer shape made the dominant axis clearer. The small object produced greater variability because its shorter geometry reduced the distinction between the main and secondary directions. This finding is consistent with the interpretation that classical geometric methods work well when the object has simple and distinguishable shape characteristics, but become less reliable when the object is short, symmetric, partially occluded or geometrically complex.

The pick-and-place results further show that successful robotic manipulation cannot be evaluated only through perception accuracy. The system achieved a high overall success rate, but the failure distribution showed that most failures were associated with the physical placing stage rather than with object localisation alone. The large object had the most stable orientation-estimation behaviour, but it also produced the highest failure rate. This indicates that task success was influenced by the interaction between object length, gripper alignment, robot motion, release behaviour and tray clearance. The 1 mm clearance in the placement slots made the task sensitive to small pose deviations and mechanical contact with the slot edges. Therefore, the experiment

confirms that perception accuracy is necessary but not sufficient for reliable industrial pick-and-place.

This distinction is important for the thesis because it supports the argument that industrial robotic systems must be evaluated as complete perception-to-action pipelines. Many pick-and-place studies focus mainly on the grasp-detection or pose-estimation problem. These contributions are important, but an industrial system also requires communication with the robot controller, coordination with peripheral devices, target-frame generation, motion planning, collision checking and physical execution. In this chapter, the computer-vision package acted as the bridge between the perception algorithm and the physical robotic task. This supports the broader position of the thesis that adaptable pick-and-place requires modular system integration rather than isolated algorithmic improvement.

The modularity of the implemented system is one of the main outcomes of this experiment and provides direct evidence for **SRQ1**. The ROS workspace separated robot description, scanner interface, cell representation, communication and task-level logic into distinct packages. This reflects the role of ROS discussed in Section 2.8, where modular communication allows perception, transformation management, planning, control and visualisation to be developed independently while still operating as one coordinated system [112], [113]. In the developed cell, ROS and ROS-Industrial allowed high-level perception and planning to be separated from low-level robot execution. This layered structure is important for industrial reliability because the robot controller remains responsible for physical execution, while ROS manages perception, target generation, planning and coordination. The use of configuration files also provided an initial link to **SRQ5**, since task information such as object data, placement positions, scanner parameters and execution settings could be defined before operation while the system carried out the repeated perception-to-action sequence within predefined limits.

Another design decision in this experiment was the use of ROS rather than ROS 2. At the time of development, ROS provided a mature and widely adopted ecosystem for industrial robotics, particularly through ROS-Industrial, MoveIt, robot drivers and supporting packages that had already been validated in manufacturing applications [112], [113]. The implemented cell relied heavily on this established software stack, and selecting ROS reduced integration risk during the initial validation of the perception-to-action pipeline. Since the primary objective of the experiment was to evaluate object localisation, pose estimation and robotic execution in a random-flow environment, software stability and compatibility with existing industrial packages were prioritised over adopting newer middleware features.

ROS 2 was developed to address several limitations of ROS, particularly in relation to real-time systems, communication over non-ideal networks, security and deployment in production environments [136], [137]. Unlike ROS, which uses a centralised ROS Master for node discovery and communication management, ROS 2 is built on the Data Distribution Service (DDS) middleware and supports decentralised communication [138]. This architecture removes the dependency on a central ROS Master and provides configurable Quality of Service (QoS) policies that allow communication behaviour to be tailored to application requirements [138], [139]. ROS 2 also offers improved support for multi-machine systems, deterministic communication

and real-time execution, which are increasingly important for advanced industrial automation.

If ROS 2 had been used in this experiment, the overall system architecture would have remained largely unchanged because the perception, transformation, planning and execution functions are conceptually independent of the underlying middleware. However, several practical differences could have been expected. Communication between the scanner interface, perception nodes and robot-control components could potentially benefit from lower latency and more predictable message delivery through DDS QoS settings. The decentralised architecture would also improve fault tolerance in larger deployments where multiple computers, robots or sensors operate simultaneously. In addition, ROS 2 provides stronger long-term support for industrial applications through enhanced security mechanisms and improved integration with modern real-time operating systems.

Despite these advantages, the use of ROS 2 would not necessarily have changed the experimental outcomes reported in this chapter. The dominant factors affecting performance were the perception algorithms, object geometry, mechanical placement tolerances and execution sequence rather than middleware limitations. The stop-and-scan conveyor strategy also reduced the importance of strict real-time communication because object detection and motion planning occurred while the conveyor was stationary. Consequently, ROS provided sufficient performance for validating the proposed workflow, while ROS 2 represents a logical direction for future development as the system evolves towards higher throughput, greater scalability and more complex industrial deployments.

The use of configuration files also supported reconfigurability. Object information, placement positions, scanner parameters and task settings could be modified without changing the core source code. This agrees with the user-guided configuration concept discussed in Section 2.9. In this type of workflow, the user provides task-specific information before execution, while the system performs repeated perception, target generation and execution steps autonomously within defined limits. The first experiment therefore showed an early form of bounded autonomy. The system did not attempt open-ended decision-making, but it could operate within a controlled industrial cell using user-defined object and placement information.

At the same time, the experiment showed that the system was flexible but not fully adaptive. It was flexible because it handled variation in object position and orientation on the conveyor. Each pick target was generated from the detected object pose rather than from a preprogrammed location, which provides an initial response to **SRQ3** for simple geometric objects. However, the system was not yet fully adaptive in the sense required by **SRQ2**, because the perception method still depended on simple predefined geometries, hand-crafted features and manual configuration. If the rectangular objects were replaced by complex injection-moulded assemblies, the same DBSCAN, PCA and shape-matching pipeline would not be sufficient to segment, recognise and estimate the pose of the parts reliably. The results also provide only a partial response to **SRQ4**. The experiment showed reliable execution under controlled conditions, but it also demonstrated that reliability depends on more than perception accuracy alone. It depends on the interaction between object geometry, pose estimation, grasping, motion planning, placement tolerance and physical execution.

The limitations observed in this experiment define the need for the second experiment. The first experiment validated the stable part of the robotic cell, including the robot, scanner, conveyor, gripper, digital counterpart, ROS workspace, controller communication, TF-based target representation, MoveIt planning and physical execution sequence. These components form the structured industrial foundation of the research. However, the perception method remained dependent on simple object geometries, hand-crafted features and predefined shape-matching rules. This made the approach effective for rectangular objects with clear dominant axes, but less suitable for complex injection-moulded assemblies, where segmentation, recognition and pose estimation are more difficult.

This transition reflects the wider research gap identified in Chapter 2. Traditional systems remain reliable when the environment is structured, but they are difficult to reconfigure when object types, layouts or presentation conditions change [14], [15], [16]. Learning-based perception can increase flexibility and support more complex object understanding, but it also introduces concerns related to data requirements, explainability, validation and predictable behaviour in industrial environments [4], [13], [62]. The thesis therefore follows a hybrid approach. Learning-based perception is introduced where adaptability is most needed, while deterministic planning, collision checking, object-specific templates, user-defined goals and controlled execution provide boundaries for reliable operation.

The next experiment described in Chapter 4 addresses these limitations by extending the validated robotic infrastructure with a more adaptable perception framework for complex industrial components. The focus shifts from geometric object localisation towards learning-based instance segmentation and template-based registration. This provides a natural progression from the proof-of-concept demonstrated in this experiment towards a more flexible and reconfigurable robotic pick-and-place solution.

Overall, this study provides the experimental baseline required for the rest of the thesis. It demonstrates that the robotic cell can close the loop between sensing, perception, target generation, motion planning and physical pick-and-place execution. In relation to the research question, this experiment validates the architectural and execution foundation needed to combine random-flow flexibility with reliable operation in a controlled manufacturing cell. However, it also shows that flexibility in object pose is only the first step. A more complete adaptable system must also support new object geometries, new placement configurations and reduced expert intervention. The next experiment addresses this remaining challenge by extending the validated robotic infrastructure with hybrid dataset generation, learning-based instance segmentation and template-based registration for complex industrial parts.

3.9 Conclusion

This chapter presented the development and validation of the first experimental stage of the research. The objective was to establish a foundational robotic cell capable of performing random-flow pick-and-place tasks using 3D vision, ROS-based communication, a digital cell representation and MoveIt-based motion planning.

The experiment demonstrated that the system could detect rectangular objects on a conveyor, estimate their position and orientation from point-cloud data, generate robot targets and execute pick-and-place operations into predefined tray locations. The

classical point-cloud processing method, based on filtering, DBSCAN clustering, PCA-based orientation estimation and simple geometric matching, was suitable for the simple rectangular objects used in this stage.

The results confirmed the reliability of the integrated system. The PCA-based orientation method produced small errors, ranging from −2.558° to 2.025°, and mean errors remained close to zero. Across 2160 pick-and-place executions, the system achieved a success rate of 94.21%, with an overall failure rate of 5.79%. Most failures occurred during placement, showing that physical tolerance in the tray slots was a major constraint.

The experiment also identified limitations that motivate the second experimental stage. The perception method depended on clear object dimensions and a dominant geometric axis, while the top-down picking strategy was mainly suitable for simple rectangular objects. More complex industrial components require a more adaptable perception and configuration workflow.

Overall, this chapter validated the physical robotic cell, digital counterpart, ROS communication structure, computer-vision package, TF-based target generation and MoveIt execution pipeline. These outputs form the baseline for the next chapter, where the system is extended towards complex industrial parts using hybrid dataset generation, configurable perception models and template-based localisation.

Chapter 4: EXTENSION TO COMPLEX INDUSTRIAL PARTS

4.1 Introduction

Chapter 3 presented the first experimental stage of this research, in which the robotic cell was developed and validated using simple rectangular objects arriving in random positions and orientations. Building on that implementation, this chapter extends the robotic system through a more configurable perception and task-preparation workflow. In particular, the updated approach combines user-defined object models, hybrid point-cloud dataset generation, learning-based instance segmentation, and template-to-scan registration. As a result, these methods support the introduction and localisation of complex target parts while still retaining the structured planning and execution mechanisms of the robotic cell described in Chapter 3.

The overall structure of the second experimental stage is summarised in Figure 4.1. The system is organised around three interconnected elements. The first element is the revised robotic cell, which retains the principal equipment used in Chapter 3 while incorporating a relocated physical arrangement and a custom two-finger electronic gripper. The second element is the hybrid dataset generation methodology, which combines object models, physics-based object arrangement, and information acquired from the physical workspace. This process produces object-level single-instance samples as well as scene-level multi-instance samples. The object-level samples can support tasks such as classification, point-cloud completion, and registration, however the experimental work presented in this chapter focuses primarily on template-to-scan registration. In parallel, the scene-level samples are used to support semantic and instance segmentation.

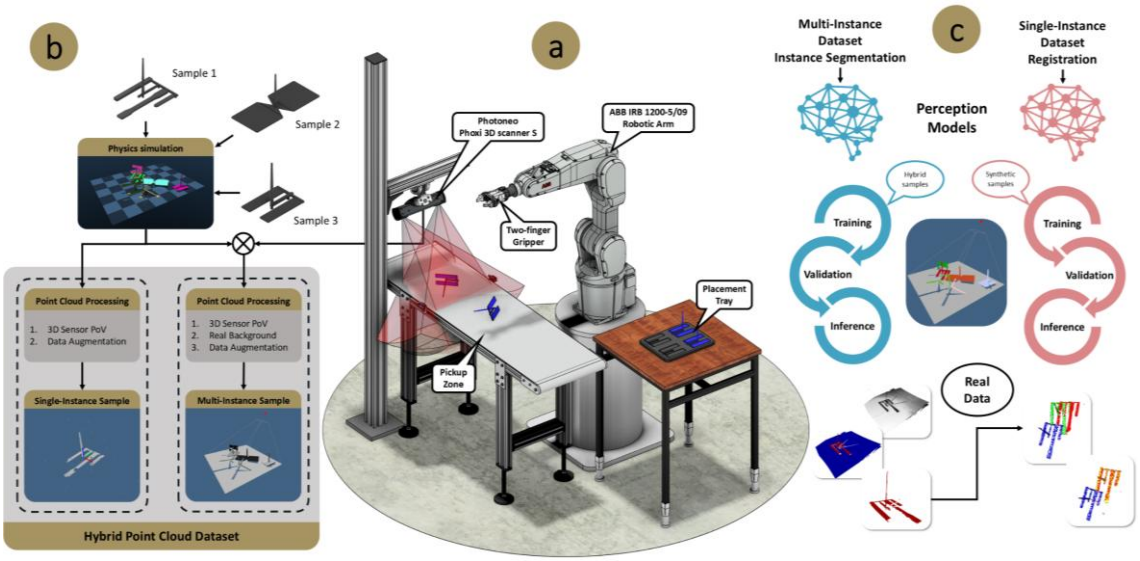


Figure 4.1: Overview of the second experimental stage. (a) Revised robotic cell and physical data sources, (b) hybrid point-cloud dataset generation, and (c) perception-method development, evaluation and inference for semantic and instance segmentation, and template-to-scan registration.

The third element is the perception-method workflow. In this context, the generated datasets are first prepared for model training, validation, and testing, while, in parallel, real scanner observations are used to evaluate the resulting segmentation and registration behaviour. Subsequently, during online operation, the segmentation method separates and identifies the visible object instances, after which the registration method estimates the pose of each selected object relative to its geometric template. Finally, the resulting information is transferred to the robotic planning and execution

pipeline. Each of these three elements is further expanded in the subsequent sections of this chapter, where their implementation, configuration, and experimental role are described in detail.

These elements operate within a bounded and user-guided workflow rather than a fully unstructured autonomous process. In particular, the robot, scanner, conveyor, and working area remain predefined, while the user provides the target object models, placement information, and selected perception settings. Based on these inputs, the system generates the required perception resources and subsequently uses them within a controlled sequence of scanning, object identification, pose estimation, trajectory validation, and physical execution.

The results of the first experiment showed that flexibility in object position and orientation represented only one part of the wider research problem. The system used a top-down picking strategy in which the robot approached each object from above. This was appropriate for the rectangular objects because their regular geometry provided consistent grasping surfaces and allowed their planar orientation to be estimated from dominant geometric axes. More complex industrial components may not provide the same grasping conditions. Their geometry and orientation can require different approach directions, grasp orientations and placement constraints.

Section 4.2 defines the aim and objectives of the second experiment. Section 4.3 then explains its relationship with the first experimental stage and describes the main system adaptations. After that, Sections 4.4 to 4.7 present the hybrid dataset generation, dataset preparation, instance segmentation, and registration methods. In addition, Sections 4.8 and 4.9 describe the user-guided configuration workflow and the target industrial use case. Moreover, Sections 4.10 and 4.11 define the offline and online validation protocols, respectively. Finally, Section 4.12 presents the results and discussion, and Section 4.13 concludes the chapter.

4.2 Aims and Objectives

The aim of the second experiment was to extend the foundational robotic cell developed in Chapter 3 to support more complex industrial parts and a more adaptable perception and configuration workflow. This experiment addressed the limitations of the first experimental stage by reducing dependence on simple geometric features and by introducing a system in which new target objects, placement locations, dataset generation parameters and perception models could be configured through a user-guided workflow.

The second experiment did not aim to develop a new segmentation or registration model architecture. Instead, it focused on developing the surrounding robotic and software framework required to generate task-specific point-cloud datasets, configure model training, load selected perception models, execute inference and connect the resulting object information to the robotic pick-and-place pipeline. In this way, the experiment extended the system from a fixed classical point-cloud pipeline towards a more reconfigurable perception framework capable of supporting interchangeable models within different processing tasks, including instance segmentation and template-to-scan registration. The specific objectives of this experiment were as follows:

1. Adapt the robotic system for the second experimental stage, including the end-effector, perception workflow, task configuration process and online pick-and-place execution pipeline.
2. Develop a ROS-based hybrid dataset generation package capable of using object mesh models, physics-based object arrangement and real background scans to create task-specific point-cloud datasets for complex industrial parts.
3. Define the dataset preparation workflow required to support both scene-level instance segmentation and object-level template-to-scan registration.
4. Develop a configurable point-cloud processing framework in Python and PyTorch that can load selected perception models through a common pipeline structure rather than being limited to a single fixed model implementation.
5. Implement the model training workflow for selected segmentation and registration models, including dataset loading, model configuration, training parameter definition, checkpoint handling and training feedback.
6. Implement the inference workflow used during system execution, including point-cloud input handling, instance segmentation, object template registration, registered pose generation and publication of perception outputs through ROS messages and transforms.
7. Develop a user-guided configuration interface that allows the user to load target object CAD models, define placement poses, select perception models, set basic model and training parameters, generate configuration files and initialise the system workflow.
8. Integrate the generated datasets, trained perception models, user-defined task configuration and ROS-based execution modules within a common workspace so that the system can progress from configuration to perception and pick-and-place execution.
9. Define and carry out offline validation using hybrid and real point-cloud data to evaluate the behaviour of the perception and registration pipeline before online robotic execution.
10. Define and carry out online validation to assess how the updated perception workflow integrates with the robotic cell during pick-and-place execution with complex industrial parts.

The second experiment was implemented as a controlled extension of the system developed in Chapter 3. The following section identifies the elements retained from the first experimental stage and the adaptations required before the updated perception and manipulation workflow could be evaluated.

4.3 Relationship with Chapter 3 Experiment

The two experimental stages were designed as connected parts of the same system-development process. The first experiment established the operational baseline by validating communication between the scanner, perception package, transformation framework, motion-planning interface and physical robot. On the other hand, the second experiment retained this validated execution structure and changed the elements required to introduce a new object set and a more configurable perception workflow.

This relationship allowed the second experiment to focus on system extension rather than repeating the integration work performed in Chapter 3. The robot communication interface, scanner acquisition process, ROS transformation structure,

MoveIt planning environment and low-level ABB execution framework continued to provide the connection between perception and physical manipulation. The main changes occurred in the object-specific layers responsible for system configuration, dataset generation, object identification, pose estimation and grasp execution.

The continuity between the experiments concerned the system architecture and principal equipment rather than an unchanged physical installation. Before the second experiment, the robotic cell was moved from its original location and reassembled in a second area. The ABB robot, PhoXi scanner, conveyor, placing table, safety fence and other principal components were retained, but the arrangement of the cell was modified. The revised assembly therefore required the digital representation and the spatial relationships between the cell components to be updated before online operation.

The experiment described in this chapter also introduced a more explicit separation between perception validation and robotic execution. The segmentation and registration methods were first evaluated offline using generated point clouds and real scanner samples. This allowed the behaviour of the perception pipeline to be examined independently from grasping, trajectory planning and physical placement. The selected configuration was then connected to the robotic cell during the online validation stage.

This experimental relationship provided a controlled basis for comparison. The first stage demonstrated that the robotic infrastructure could execute a complete pick-and-place sequence under constrained conditions. The second stage examined whether the same infrastructure could support changes in object geometry, perception method, task configuration and end-effector while preserving the established planning and execution structure.

4.3.1 System Adaptations

The first adaptation concerned the physical location and arrangement of the robotic cell. The equipment used in Experiment 1 was transferred to a second location and assembled in a revised configuration. Although the same robot, scanner, conveyor, placing table and safety structures were used, their positions and orientations within the workspace were not assumed to remain the same as those used in Chapter 3.

Consequently, the digital counterpart was revised to represent the new physical assembly. The positions of the robot, scanner, conveyor, placing area, safety fence and other fixed structures were updated within the digital cell. The scanner working region and the relationships between the relevant coordinate frames were also checked against the physical installation. These changes ensured that the visualisation, transformation and motion-planning layers corresponded to the cell used during the second experiment. Figure 4.2 compares the digital robotic-cell configurations used in the two experimental stages. The first model represents the assembly used for the first experiment, while the second model represents the relocated cell and the changes introduced for the second experiment.

A second hardware adaptation involved replacing the pneumatic gripper used in the first experiment with a custom two-finger electronic gripper. This custom gripper was designed to provide software-controlled opening and closing actions and to support the manipulation sequence required for the injection-moulded assemblies.

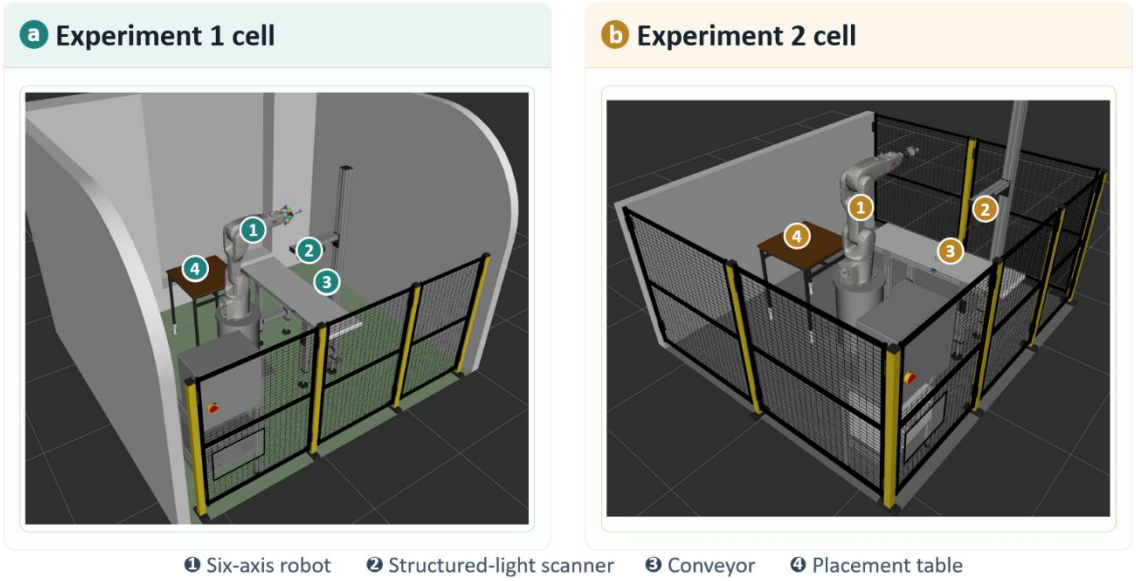


Figure 4.2: Digital robotic-cell configurations used in the (a) first and (b) second experiments.

To achieve this, the gripper was developed as a parallel two-finger mechanism with its own actuation, embedded control and ROS communication interface. It was then mounted on the ABB robot and represented in the updated digital robot model, as shown in Figure 4.2(b). In this way, its geometry and connection to the robot flange could be considered during visualisation and motion planning. Furthermore, gripper commands were incorporated into the pick-and-place sequence so that opening, closing and release operations could be coordinated with robot motion.

The complete mechanical design, electronic system, firmware, communication interface and ROS integration of the gripper are presented in Appendix B. Within this chapter, however, the gripper is discussed only where its operation affects grasp-pose generation, trajectory validation and online pick-and-place execution.

The main software adaptation involved the reorganisation of the perception and task-configuration layers. While the robot communication, scanner acquisition, coordinate-transformation and MoveIt interfaces were retained due to their prior validation, the single computer-vision package used in the first experimental stage was replaced by a set of specialised and interconnected modules.

As a result, the revised architecture separated hybrid dataset generation, point-cloud processing, user-guided configuration, system initialisation and robotic control into distinct components. This modular structure allowed the data-generation, segmentation and registration processes to be configured independently, while still maintaining the same interfaces for scanner input, coordinate transformations, motion planning and robot execution.

More specifically, the hybrid dataset generation module created labelled point-cloud data from object meshes and real background scans. In parallel, the point-cloud processing module supported both model training and inference. Meanwhile, the configuration and initialisation modules prepared the selected objects, placement information and perception resources required for execution. Finally, the robotic-control module established the connection between the resulting object poses and the manipulation pipeline. The detailed operation of these modules and their interactions is presented in the following sections.

For completeness and reproducibility, the software developed for both experimental stages is available at the following repositories: https://gitlab.com/tus4/robotenv_ws and https://gitlab.com/tus4/robotenv_v2_ws.

4.4 Hybrid Dataset Generation Methodology

The updated perception workflow required labelled point-cloud data for two related tasks. The first involved separating and semantically identifying individual objects within a complete scanner scene. The second involved aligning each separated object with its corresponding geometric template. Although these datasets could have been obtained from real scanner observations, their collection would have required repeated physical object arrangement, scanning and point-level annotation. A hybrid dataset generation methodology was developed to produce task-specific point-cloud data automatically from target object meshes and real background scans acquired from the physical robotic cell.

The methodology combines physics-based object arrangement, mesh-derived point clouds and real scans of the empty working area. In general, object instances are first arranged within a simulated environment to obtain realistic spatial configurations. These configurations are then used to generate synthetic point-cloud representations of the objects, which are subsequently combined with real background scans captured from the physical workspace. Additional variations are introduced to approximate the characteristics of scanner observations.

The method is described as hybrid because the objects and their annotations are generated computationally, while the background geometry is acquired using the physical scanner. The synthetic component provides controlled variation, known object identities and exact transformation information. The real component introduces the conveyor surface, fixed cell structures and background characteristics present during physical operation. This combination reduces the dependence on manually labelled real data while producing scenes that contain information from the actual robotic environment.

Each generated arrangement produces two related forms of data. The scene-level representation contains all visible objects and the surrounding background together with semantic and instance labels and is used for instance segmentation. The object-level representation contains an individual visible object observation, its complete geometric template and its known transformation, and is used for template-to-scan registration.

To integrate the methodology with the wider robotic system, it was implemented as a ROS package. The package coordinates the generation stages and allows the process to be configured, initiated and monitored through the same software environment used by the remaining perception and control modules. The implementation separates the generation parameters from the processing code, allowing datasets for different object sets to be created without modifying the underlying generation functions.

The generation process is controlled through a YAML configuration file. This file defines the object meshes, semantic classes, mesh scale, maximum number of object copies, simulation limits, scanner resolution, sensor position and background scans. It

also contains parameters controlling visibility, point density, intensity variation and measurement noise.

Once the configuration has been defined, generation is initiated through a ROS action. The action receives the path of the selected configuration file and starts the processing sequence. During execution, intermediate feedback reports the current stage and the percentage completed. When the process is complete, the action returns the directory containing the generated dataset, allowing dataset generation to operate as part of the user-guided system workflow.

The following subsections describe the individual stages of the methodology in more detail.

4.4.1 Template Object Preparation

The template object preparation stage defines how each target object is represented within the dataset generation process. Its purpose is to convert the original CAD mesh into a consistent and computationally efficient point-cloud representation that can be used both for scene generation and for template-to-scan registration. This ensures that all subsequent stages operate on a unified geometric representation of each object.

The template-preparation stage begins with the object mesh models selected for the task. Each mesh is associated with a semantic class in the configuration file so that its identity is preserved throughout simulation, annotation and dataset preparation. In the experimental implementation, the objects were represented using STL files.

Before point-cloud generation, the mesh coordinates are converted to the measurement units used by the robotic system. This conversion is required because CAD models may be exported in units that differ from those used by MuJoCo, the scanner point clouds and the robot coordinate system. Mesh scaling is controlled through the configuration file to support models created in different measurement units. For the object models used in this experiment, the scale factor was set to 10^{-3} to express the coordinates in metres. Storing this value in the configuration file also allows models created in other units to be introduced without modifying the generation code.

After scaling, points are sampled uniformly across the triangular surfaces of each mesh to produce a dense point-cloud representation. The initial point set is then reduced through voxel downsampling, which limits the computational requirements of the subsequent stages while retaining the relevant object geometry. A foreground voxel resolution of 1 mm was used in the generation configuration.

The resulting point cloud is centred by subtracting the mean coordinates of its sampled points. The translation applied during this operation is retained as a transformation matrix because the mesh body and the centred point-cloud template are represented in different reference positions. MuJoCo determines the final pose of the mesh body, whereas the synthetic scene is created by applying that pose to the centred template. The centring transformation is combined with the simulated object transformation to maintain consistency between the generated points and the recorded ground-truth pose.

The centred point cloud is retained as the complete geometric template of the object. During registration, this representation forms the source point cloud, while the

visible and partially occluded observation generated within the scene forms the target point cloud. Consequently, the same user-provided mesh supports point-cloud generation, semantic labelling and template-to-scan registration.

4.4.2 Physics-Based Object Dropping in MuJoCo

The physics-based object dropping stage determines the spatial arrangement of the objects used in each generated scene. MuJoCo is used to produce physically plausible configurations by modelling gravity, object motion and contact interactions [104]. For each scene, the generator creates a temporary simulation environment containing a support surface, bounded spatial limits and the required object bodies. The number of instances assigned to each object type can be varied within the maximum values defined in the configuration file.

Before describing the object-dropping procedure, it is useful to clarify the selection of MuJoCo as the physics engine. Other platforms, such as Gazebo, PyBullet and NVIDIA Isaac Sim, also support rigid-body and contact simulation. However, this work required only object placement, simulation of gravity and contact interactions, and access to final poses.

MuJoCo was selected because it provides rigid-body dynamics and contact simulation through native Python bindings with direct access to the simulation state. It also offered a relatively lightweight solution for the required object-settling task because a complete rendering, sensor-simulation or graphical robotic environment was not needed. These characteristics allowed the object environments to be created, simulated and analysed within the same Python-based dataset generation workflow, with the final object poses extracted as ground-truth transformations.

The broader rendering, sensing and complete robotic-cell simulation capabilities available in other platforms were not required. The simulation was used only to generate physically plausible object arrangements, while sensor effects and real background integration were handled during later stages. A comparative evaluation of different physics engines was therefore outside the scope of this research.

At the beginning of the simulation, the objects are positioned above the support surface. Their initial locations are selected randomly within a bounded three-dimensional region centred over the working area. Each object is also assigned a random normalised quaternion, allowing its initial orientation to vary around all three rotational axes rather than being restricted to planar rotation.

Once the objects have been initialised, the simulation advances for a fixed number of steps. During this period, gravity and contact interactions cause the objects to fall, collide and settle on the supporting surface. Consequently, the final object poses are influenced by the object geometry, contact with the support surface and interaction with neighbouring parts. This produces more physically plausible spatial configurations than assigning independent random transformations directly to the object meshes.

After the settling stage, the final position and orientation of each object are extracted from the MuJoCo state. These values are combined into a homogeneous transformation matrix containing the three-dimensional rotation and translation. The corresponding inverse transformation is also calculated and stored. Together, these matrices preserve the exact relationship between the centred object template and its

position within the generated scene, providing the ground-truth pose information required for dataset extraction and registration evaluation.

The MuJoCo environment is used specifically to determine the arrangement of the objects. It is not intended to reproduce the complete scanner or robotic cell. Viewpoint visibility, point density, appearance variation and scanner-related measurement effects are therefore introduced during the subsequent point-cloud generation stages.

4.4.3 Synthetic Foreground Point-Cloud Generation

In the context of this methodology, the foreground refers to the point-cloud data belonging to the target objects. In contrast, the background represents the surrounding working area, including the support surface and any fixed structures visible within the scanner region. These definitions are used throughout the remaining dataset-generation stages.

After the objects have settled in MuJoCo, their final poses are used to construct the synthetic foreground. For each object instance, the rotation and translation obtained from the simulation are applied to its centred point-cloud template. This operation places the sampled object points in the same spatial arrangement produced during the physics simulation.

Each transformed object instance is initially assigned a uniform greyscale value selected randomly during scene generation. This introduces basic appearance variation between different objects and generated arrangements. The greyscale values are modified further during the lighting and measurement-variation stage described in Section 4.4.5. Meanwhile, the semantic class and instance identity remain associated with the transformed points so that the required ground-truth annotations can be generated automatically.

Once all object templates have been transformed, their points are combined to form a single synthetic foreground. At this stage, each object is still represented by its complete sampled surface. As a result, points that would be hidden from the scanner by the object itself, neighbouring objects or the support surface remain present. The complete geometry is retained temporarily so that visibility filtering can later be applied to the complete arrangement from the configured scanner viewpoint.

The resulting foreground represents the geometry and spatial arrangement of the target objects without including the surrounding workspace. It is then passed to the background-integration stage, where it is combined with either a simulated background or a real background point cloud acquired from the physical robotic cell.

4.4.4 Background Integration

After the synthetic foreground has been constructed, the surrounding working area is added to produce the complete scene. The methodology supports two background sources. A real scan of the empty working area can be used to generate a hybrid scene, while a computationally generated background can be used to produce a fully synthetic scene.

For hybrid scene generation, the background is acquired by scanning the empty pick-up area with the PhoXi scanner before the target objects are introduced. These scans contain the visible conveyor surface and any fixed structures observed from the

scanner position. Since no target objects are present during acquisition, the complete scan can be treated as background data.

The real background scans are stored as Polygon File Format (PLY) point-cloud files and loaded from the directory defined in the generation configuration. Before integration, each scan is voxel downsampled to reduce unnecessary point density and to maintain a consistent spatial resolution. The point coordinates are then converted to metres and centred in the horizontal plane so that the measured workspace corresponds to the simulated object-arrangement region.

When multiple real background scans are available, one is randomly selected for each generated scene. This introduces natural variation because the visible portion of the conveyor may differ slightly between acquisitions. Sensor noise and changes in acquisition conditions may also affect the distribution and intensity of the captured points, while the main geometry of the physical workspace remains unchanged. The selected scan is then combined with the synthetic foreground to form a hybrid point-cloud scene.

Alternatively, a simulated background can be used when a real scan is not required or available. The background may consist of a simple rectangular support surface generated directly as a point cloud. It may also be created from a geometric surface model, such as an STL representation of the conveyor, which is sampled to produce the required point cloud. In either case, the resulting scene is fully synthetic because both the target-object foreground and the surrounding background are generated computationally.

After either background has been selected, its points are merged with the synthetic foreground. The combined point cloud is subsequently shuffled so that foreground and background points are not retained as separate ordered groups. It is then passed to the visibility and measurement-variation stages described in the following subsection.

4.4.5 Lighting, Noise, and Self-Occlusion Simulation

Although the combined point cloud contains the required object and background geometry, it remains cleaner and more complete than a point cloud acquired directly from the scanner. Additional processing is therefore applied to introduce appearance variation, viewpoint visibility, missing points and measurement disturbances.

First, a simplified lighting model modifies the greyscale values associated with the points. Surface normals are estimated from the local point neighbourhood and oriented towards the configured camera position. Ambient, diffuse and specular components are then calculated using the point normals, viewing direction and configured light position. In this way, surfaces facing the light and scanner can receive different values from surfaces viewed at less favourable angles. This stage is not intended to reproduce the complete optical behaviour of the PhoXi scanner. Instead, it introduces controlled appearance variation related to the object geometry and viewpoint.

After the appearance values have been calculated, visibility is determined from the scanner position. The Hidden Point Removal method is applied to the complete point cloud to retain the points that are visible from the configured viewpoint [109]. This operation removes points hidden behind the outer visible surface of each object and

points blocked by neighbouring objects. Consequently, complete mesh-derived templates are converted into partial observations that more closely resemble the data available during physical scanning.

Further point removal is then introduced to avoid producing samples with uniform and complete point coverage. A random proportion of the points can be retained across the complete scene, while distance-based thinning reduces the probability of retaining points farther from the scanner. Localised regions can also be removed to create small holes in the observed surfaces. The relevant parameters are sampled from ranges defined in the configuration file so that the amount and location of missing data vary between samples.

Finally, geometric noise and outliers are added to the remaining points. Radial displacement moves points along the viewing direction using a magnitude that varies with their distance from the scanner. A smaller tangential displacement is applied across the viewing direction to avoid limiting the disturbance to depth alone. In addition, isolated points and small clusters can be introduced within the observed region. Together, these operations approximate common point-cloud imperfections without claiming to reproduce the complete measurement process of the physical sensor.

4.4.6 Semantic and Instance Ground Truth Generation

Because the generated point clouds are intended for supervised model training and quantitative evaluation, the semantic and instance annotations must remain aligned with the point data throughout the generation process. These annotations are created automatically from the object information provided in the configuration and from the individual object copies created for each simulated arrangement.

Each object type is associated with a semantic class that identifies its category. In addition, every physical copy within a scene receives a separate instance identifier. All points sampled from the same object copy therefore share both its semantic class and its instance identifier. Objects of the same type have the same semantic label but retain different instance labels, allowing neighbouring copies of the same part to be distinguished.

Background points are assigned a reserved non-object label when either background source is combined with the synthetic foreground. The same treatment is applied to artificial outliers introduced during the noise stage. After the foreground and background points have been combined and shuffled, the indices belonging to each object instance are recalculated and stored with the scene information.

The labels are then processed together with the point coordinates. When Hidden Point Removal or point dropout removes a point, the corresponding semantic and instance entries are removed using the same index or mask. When noise changes the position of an existing object point, its original labels are retained. Newly generated outliers remain marked as non-object points. This preserves point-to-label correspondence after all visibility and measurement variations have been applied.

For each scene-level sample, the generator stores the final point coordinates, greyscale values, semantic labels and instance labels. The forward and inverse transformations associated with each object are also retained. These records allow individual visible observations to be extracted later and paired with their complete templates for registration. Consequently, the same generated arrangement provides

point-level annotations for segmentation and exact transformation information for object-pose evaluation without requiring manual labelling.

4.5 Dataset Preparation for Perception Models

The generation methodology described in Section 4.4 produces two complementary forms of point-cloud data. The first preserves the complete observed scene and its point-level annotations. The second isolates individual object observations and pairs them with their corresponding geometric templates and known poses. Since segmentation and registration operate on different input structures, these outputs are prepared and stored separately.

Scene-level samples are organised for semantic and instance segmentation, while object-level samples are organised for template-to-scan registration. Although their structures differ, both originate from the same simulated object arrangements. Consequently, the object identities, visible geometry and pose information remain consistent between the two dataset forms.

PyTorch was used as the common framework for storing and loading the prepared data. During saving, the point-cloud arrays and their associated annotations are converted into tensors and grouped within structured sample records. Each record is then stored as an individual PyTorch file. This approach preserves the relationship between the different data fields while allowing samples to be loaded directly during model training, validation and testing.

The prepared samples are organised into training, validation and test subsets. The proportion assigned to each subset is defined through the dataset configuration, allowing flexible control over the dataset split. The samples are shuffled before the subsets are created to reduce ordering effects. Scene-level and object-level data are divided separately because a single generated scene can contain several visible object instances.

The stored records provide a common dataset representation rather than the final input format of a particular model. For trainable methods, each model implementation contains a PyTorch Dataset class that reads the relevant sample type and converts it into the structure required by that model. This class defines how individual samples are accessed and can also apply model-dependent operations such as sampling, augmentation, scaling, normalisation or feature preparation.

The Dataset objects are subsequently used with PyTorch data loaders, which retrieve samples and assemble them into batches for training or evaluation. Where point clouds contain different numbers of points or require specialised structures, the selected implementation can also define its own batch assembly procedure. In this way, dataset storage remains consistent, while input preparation can be adapted to different perception methods.

The neural-network or processing model remains separate from the Dataset class. The Dataset is responsible for reading and preparing the data, while the model receives the resulting tensors and produces the task-specific predictions. This separation allows the same underlying datasets to support alternative model implementations without requiring the generation methodology or stored sample structure to be

redesigned. The selected models and their particular preparation requirements are described in Sections 4.6 and 4.7.

4.5.1 Scene-Level Dataset for Instance Segmentation

Each scene-level sample represents the complete visible working area after background integration, visibility filtering and measurement variation. The stored data include the final point coordinates, three-channel greyscale appearance values, semantic labels and instance labels. Together, these elements provide the scene context and point-level annotations required for semantic and instance segmentation.

The semantic label associated with each point identifies its object category. In contrast, the instance label identifies the individual object copy to which the point belongs. Separate objects of the same type therefore share a semantic class while retaining different instance identities. Background points and artificial outliers remain associated with the reserved non-object label established during dataset generation.

Before storage, the coordinates and appearance values are converted into floating-point tensors. The semantic and instance annotations are converted into integer tensors because they represent discrete labels. These tensors are grouped within the same scene record so that a given point index refers to the corresponding coordinate, appearance value and annotations.

A semantic-class mapping is also stored with the scene dataset. This mapping relates the numerical semantic labels to the background and target object categories. As a result, the object classes do not need to be fixed directly within an individual segmentation model, and the same dataset structure can be used for different target-object sets.

The scene records are saved individually within their assigned training, validation and test subsets. A segmentation Dataset class then loads the appropriate records according to the selected operating mode. During training and validation, it returns the scene geometry, appearance information and point-level labels. During inference, it receives an unlabelled scene containing only the point coordinates and appearance information.

Any conversion beyond this common representation is defined by the selected segmentation implementation. Depending on its input requirements, the Dataset class may organise points spatially, reduce their density, construct additional features or apply geometric variation during training. It may also define a specialised batching operation when scenes contain different numbers of points or cannot be combined using the default PyTorch procedure.

The prepared batches are then passed to the selected segmentation model. The model returns point-level semantic predictions and information used to separate individual object instances. Keeping scene storage, data preparation and prediction as separate stages allows alternative segmentation models to use the same generated scene dataset.

4.5.2 Object-Level Dataset for Registration

The object-level dataset is derived from the same generated arrangements as the scene-level data. However, each visible object instance is extracted separately, and

the surrounding background is excluded. This produces an individual observation that can be paired directly with the geometric template of the corresponding object.

Each registration pair contains a complete centred template and a partial scene observation. The complete template represents the object in its local coordinate frame and forms the source point cloud. The partial observation contains the points that remain visible after viewpoint filtering, point removal and measurement variation. It remains positioned within the generated scene and forms the target point cloud.

In addition to this pair, the stored sample retains the object category and instance identifiers. It also contains the complete object representation in the generated scene before visibility filtering and the appearance information associated with the object points. These records preserve the relationship between the original template, its simulated placement and the final partial observation.

The known simulated pose is stored together with each object sample as rotation and translation information. This transformation defines the exact relationship between the centred template and the object position within the scene. It therefore provides the reference pose required during registration training and quantitative evaluation.

As with the scene-level data, the numerical point-cloud and transformation fields are converted into PyTorch tensors before each object record is saved. Identification fields are retained alongside these tensors so that the object category, individual instance and corresponding transformation remain available when the sample is loaded.

For trainable registration methods, a registration Dataset class reads the object-level records and returns a source and target pair together with the associated reference transformation. The selected implementation determines whether the pair requires point sampling, point-order randomisation, scaling, normalisation or feature calculation. These operations are performed when the sample is loaded rather than being permanently applied to the stored data.

The resulting samples are provided to a PyTorch data loader for batching and delivery to the selected registration model. The model then estimates the transformation between the complete template and the partial observation. Since the stored object records remain independent of a specific model input format, the same dataset can also be accessed by alternative learning-based, classical or combined registration methods.

4.6 Instance Segmentation Methodology

The first perception task involved separating the target objects from the surrounding scene and distinguishing individual objects from one another. This required both semantic segmentation and instance segmentation. Semantic segmentation identifies the category associated with each point, while instance segmentation separates different physical objects even when they belong to the same category.

This distinction is important when several objects of the same semantic class are present within one scene. A semantic prediction identifies the category associated with each point, but it does not separate the points belonging to different physical

objects. The instance segmentation stage therefore groups the predicted object points into individual instances and retains the semantic identity associated with each group.

The segmented instances form the connection between scene-level perception and template-to-scan registration. The segmentation method operates on the complete scanner scene, while the registration methods described in Section 4.7 operate on one object instance at a time. As a result, errors such as merged objects, incomplete instances or retained background points can directly affect the subsequent pose estimate.

The methodology was implemented around a common segmentation task in which a complete scene is supplied as input and a separated set of object instances is returned. Within this task, model-specific input conversion, training and proposal generation are handled by the selected segmentation implementation. This preserves a consistent relationship between dataset preparation, model execution and the registration stage without requiring the wider perception methodology to depend on one input structure.

4.6.1 Segmentation Model Selection

The segmentation model was selected according to the requirements of the robotic task. It needed to process three-dimensional point clouds directly, identify the semantic class of each target object and separate neighbouring instances of the same class. It also needed to preserve the points assigned to each detected instance because those points would later be used for registration.

PointGroup was selected as the implemented segmentation model [63]. It is a bottom-up instance segmentation method that combines point-level semantic prediction with geometric grouping. This approach was suitable for the generated scene-level dataset because the data already contained point coordinates, appearance values, semantic labels and instance labels.

Rather than predicting complete object masks directly, PointGroup first extracts features from the scene and predicts a semantic class for each point. In parallel, it estimates an offset that shifts each object point towards the centre of its corresponding instance. The original point positions and the shifted positions are then used as two complementary representations for grouping. Points that may be difficult to separate in the original scene can become easier to distinguish after they have been shifted towards their predicted instance centres.

The resulting groups are treated as candidate object instances. A separate scoring stage estimates the quality of each candidate before overlapping proposals are compared and filtered. This sequence allows the method to produce both point-level semantic predictions and object-level instance proposals from the same scene.

PointGroup was adopted as an existing segmentation architecture rather than developed as a new model contribution. The contribution of this research lies in the surrounding methodology that generates task-specific data, prepares the model inputs and connects the resulting object instances to registration and robotic execution. The wider processing structure remains open to alternative segmentation models that can accept the common scene data and return equivalent instance and semantic information.

4.6.2 Model Input Preparation

The input preparation begins with the scene-level records described in Section 4.5.1. Each record contains the point coordinates, three-channel greyscale appearance values and the point-level annotations required during training. However, PointGroup processes the scene through sparse three-dimensional convolutions and therefore requires the irregular point cloud to be converted into a sparse spatial representation.

The scene coordinates are first scaled into the processing grid and translated so that the occupied region begins at a valid grid location. They are then converted into discrete voxel coordinates. When multiple points occupy the same voxel, their input features are combined to produce a single sparse voxel feature. At the same time, mappings between the original points and the sparse voxels are retained. These mappings allow features calculated within the sparse network to be returned to the individual scene points.

The greyscale appearance values form the primary point features. Depending on the selected model configuration, the spatial coordinates can also be included as additional features. This combination allows the network to use both measured appearance and three-dimensional position while learning the semantic and geometric relationships within the scene.

For training and validation, the semantic and instance annotations are converted into the indexing convention expected by the model. The instance labels are also used to calculate the centre, spatial bounds and number of points associated with every visible object. From this information, the required point-to-centre offsets can be derived and used as supervision during training.

Geometric augmentation is applied only when training data are loaded. The available operations include small coordinate perturbations, reflection along a horizontal direction, rotation around the vertical axis and elastic spatial distortion. These transformations increase the geometric variation presented to the model while preserving the semantic class and instance identity of each point. Validation and inference data are processed without these random changes so that the model output corresponds directly to the supplied scene.

During inference, the input contains only point coordinates and appearance values because ground-truth labels are not available. The scanner scene is converted into the same sparse representation used during training. The retained point-to-voxel mappings then allow the resulting semantic scores, offsets and instance assignments to be associated with the original scene points.

4.6.3 Training Procedure

The training procedure uses the labelled scene records and the model-specific input preparation described in the previous subsection. Before training begins, the model architecture, dataset location, optimisation method, runtime device and checkpoint behaviour are defined through configuration files. This allows the training procedure to be adapted without changing the model or dataset code.

For each training batch, the sparse scene representation is supplied to the segmentation model. The model produces semantic scores for the individual points and an offset vector that indicates the predicted direction from each point towards its

instance centre. These predictions are compared with the semantic labels and instance information stored in the dataset.

The training objective combines several loss components. The semantic classification loss measures whether each point has been assigned to the correct object category or background class. The offset magnitude loss measures the difference between the predicted and reference point-to-centre vectors. In addition, the offset direction loss encourages each predicted vector to point towards the correct instance centre.

Training proceeds through two connected stages. During the initial stage, the network learns the semantic and offset predictions needed to form meaningful point groups. Once these predictions have developed, candidate instances are generated from the original and shifted point coordinates. A proposal score loss is then added so that the model can learn to distinguish stronger object proposals from incomplete or incorrect groups.

For each batch, the combined loss is propagated through the network, and the trainable weights are updated. The same model is evaluated on the validation subset without applying weight updates. Training and validation losses are recorded throughout the process, while model checkpoints are stored according to the configured saving strategy. The exact training settings used for the experimental evaluation are reported in the offline validation protocol.

4.6.4 Instance Segments and Semantic Class Predictions

During inference, PointGroup first produces a semantic score for each input point. The class with the highest score is retained as the point-level semantic prediction. Background predictions are separated from target-object predictions before the instance grouping stage begins.

For points predicted as target objects, the model also produces an offset towards the estimated instance centre. Candidate groups are then formed using both the original point coordinates and the offset-shifted coordinates. The two sets are combined because each provides different information about the possible object boundaries. The original coordinates preserve the measured scene geometry, while the shifted coordinates bring points from the same predicted instance closer together.

Each candidate instance receives a proposal score. Candidates that do not satisfy the configured confidence or point-count requirements are removed. The remaining proposals are compared according to their point overlap, after which non-maximum suppression removes duplicate groups that represent the same object.

Every accepted instance retains the original point coordinates and appearance values assigned to it. It also contains its predicted semantic identity and proposal score. Points that are not assigned to an accepted object instance remain within a separate background representation.

The final segmentation output therefore consists of a set of object-level point clouds and the remaining scene background. Each object instance can then be matched with the geometric template associated with its predicted semantic class. This output provides the target point clouds required by the registration methodology described in the following section.

4.7 Registration Methodology

Instance segmentation identifies the visible objects and separates their points, but it does not provide the complete pose required for manipulation. Registration addresses this requirement by aligning the geometric template of each detected object with its partial point-cloud observation.

The registration methodology was designed to support classical, learning-based and combined approaches. Each approach receives the same general source and target representation and returns a rigid transformation. This common formulation allows different registration strategies to be examined without changing the preceding dataset generation or segmentation stages.

Several registration configurations were considered because no single method provides the same behaviour under all conditions. Local geometric optimisation can provide accurate refinement when the initial alignment is suitable, while global or learning-based methods can provide a stronger initial estimate when the template and observation begin far apart. The methodology therefore examines both individual methods and coarse-to-fine combinations.

4.7.1 Template-to-Scan Registration Problem

For each detected object, the source point cloud is the complete centred template prepared from the corresponding mesh. The target point cloud is the partial observation produced by instance segmentation. Since the semantic class is already predicted during segmentation, the registration stage can select the appropriate template before estimating the object pose.

The registration problem seeks a three-dimensional rotation and translation that map a source point into the coordinate frame of the target observation:

$$\mathbf{p}t \approx \mathbf{R}\mathbf{p}_s + \mathbf{t} \tag{4.1}$$

where $\mathbf{p}_s$ and $\mathbf{p}_t$ represent points in the source and target point clouds, respectively, $\mathbf{R}$ represents the three-dimensional rotation matrix, and $\mathbf{t}$ represents the translation vector.

The rotation and translation together describe the six-degree-of-freedom pose of the object. They are combined into a homogeneous transformation matrix so that the complete template can be transformed into the scanner scene.

The alignment is challenging because the two point clouds do not contain identical information. The source represents the complete object surface, whereas the target contains only the portion visible from the scanner viewpoint. The target may also contain missing regions, uneven point density, measurement noise and segmentation errors. In addition, symmetric or repeated geometric features can support more than one visually similar alignment.

The registration method must therefore identify a transformation that is consistent with the visible geometry rather than attempting to create a point-to-point match across the complete template. This requirement motivated the use of global alignment methods for initialisation and local optimisation for final refinement.

4.7.2 Registration Method Selection

The selected methods represent different approaches to rigid point-cloud alignment. ICP was included as a classical geometric baseline [66]. ICP repeatedly establishes local point correspondences and updates the transformation to reduce the alignment error. It can provide accurate results when the two point clouds are already sufficiently close, although its local optimisation makes it sensitive to the initial pose.

DCP was selected as an end-to-end learning-based alternative [80]. DCP uses learned point-cloud embeddings and attention-based correspondence estimation before recovering the rigid transformation through singular value decomposition. It provides a direct estimate of the rotation and translation without requiring the source and target to begin with a close initial alignment. The implementation used in this work retained this general DCP architecture while adapting the feature normalisation and correspondence-training procedure, as described in Section 4.7.4.

RANSAC was included as a robust global estimation method [77]. In this approach, geometric or learned descriptors are calculated around selected points and used to propose correspondences between the source and target. RANSAC repeatedly tests subsets of those correspondences and rejects combinations that are inconsistent with a common rigid transformation. This produces a coarse pose that can be refined through local optimisation.

General and Distinctive 3D local descriptors (GeDi) were selected as the learned local descriptor method [140]. It represents local surface regions using descriptors designed to remain distinctive under changes in rotation, scale, occlusion and acquisition domain. These descriptors can be matched between the complete template and the partial observation before RANSAC estimates the initial transformation.

Together, these methods provide a comparison between local geometric optimisation, direct learned registration and descriptor-based global alignment. They also support combined configurations in which a broad initial estimate is followed by a more precise geometric refinement.

4.7.3 Coarse-to-Fine Registration Configurations

The registration study was organised around several configurations so that the contribution of initial alignment and local refinement could be examined separately. The simplest configuration used ICP alone. This provided a classical baseline but also represented the condition in which local refinement was applied without a dedicated global initialisation stage.

A second configuration used DCP to estimate the transformation directly. This allowed the learned rotation and translation to be evaluated before geometric refinement. A related configuration used the DCP estimate as the initial transformation for ICP. In this arrangement, DCP provides the coarse alignment and ICP reduces the remaining local geometric error.

A further configuration used descriptor correspondences with RANSAC to obtain the initial transformation. ICP was then applied to refine the coarse result. This configuration represented a classical coarse-to-fine approach in which correspondence rejection provides the broader pose estimate and local optimisation improves the final alignment.

The final configuration replaced the geometric descriptor stage with GeDi descriptors. Local descriptors were calculated for sampled regions of the template and observed instance before their correspondences were supplied to RANSAC. The resulting transformation was then refined with ICP. This configuration combined learned local geometry, robust correspondence estimation and classical local optimisation.

Across the combined configurations, the coarse stage addresses the limited capture range of ICP by bringing the template closer to the observation. The fine stage then uses the measured point geometry to improve the alignment. This separation also allows the effect of each component to be examined during offline validation.

4.7.4 Training and Inference Procedure

The learning-based registration procedure uses the object-level source and target pairs described in Section 4.5.2. Before a pair is supplied to a trainable model, it is converted into the input structure required by that implementation. This includes point sampling, point-order randomisation and normalisation using a common object scale. The source represents the complete object template, while the target represents the corresponding partial observation.

DCP retains the learned feature extraction, attention-based correspondence estimation and rigid-transformation recovery of the original architecture [80]. However, the training procedure is adapted to the registration conditions considered in this work. The generated object-level dataset contains wide rotational variation and pairs complete source templates with partial target observations. *Dang et al.* identified large rotation ranges and differences between source and target feature distributions as important limitations for SVD-supervised learning-based registration and proposed Match Normalization together with direct supervision of point correspondences [81]. Following this approach, Match Normalization is incorporated into the paired feature extraction, while the model is trained using an NLL correspondence objective rather than a pose loss propagated through the SVD stage.

Reference correspondences are generated from the known transformation stored with each training sample. The source points are first transformed using this reference pose, after which mutually consistent nearest-neighbour pairs within a configured geometric threshold are treated as valid correspondences. Because the complete source and partial target do not contain identical visible geometry, points without a valid correspondence are assigned to explicit outlier states. The model converts the learned source-target similarities into a normalised correspondence assignment that includes these unmatched conditions.

For the set of reference assignments $\mathcal{G}$, the correspondence objective is expressed as:

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{|\mathcal{G}|} \sum_{(i,j)\in\mathcal{G}} \log P_{ij}, \tag{4.2}$$

where P_{ij} is the predicted probability assigned to the reference correspondence between source point i and target point j. The set $\mathcal{G}$ also includes assignments to the outlier state when no valid correspondence exists. The loss therefore penalises the model when insufficient probability is assigned either to the correct point correspondence or to the appropriate unmatched condition.

This adaptation changes the training supervision rather than the required registration output. Rotation and translation estimates are still calculated during training for diagnostic purposes, but their pose error is not back-propagated through the SVD operation. During inference, the strongest mutually consistent correspondences are retained and used in a confidence-weighted SVD solution to recover the rotation and translation. SVD therefore remains part of pose estimation, while the NLL correspondence objective provides the optimisation signal used to train the model.

Validation follows the same correspondence objective without updating the model weights. Training and validation NLL are recorded throughout optimisation, and the checkpoint retained for subsequent evaluation is selected using validation loss. The exact training conditions and repeated-seed procedure used in the final experiment are presented in Section 4.11.1.

GeDi is used through pretrained descriptor weights rather than retrained using the generated object-level dataset. During inference, descriptors are calculated for sampled points from both the template and the observed instance. These descriptors provide the candidate correspondences used by RANSAC to estimate the coarse transformation. ICP and RANSAC do not require a learning stage because their behaviour is controlled through geometric and optimisation parameters.

During online inference, the semantic identity of each segmented instance determines which canonical template is loaded. The source template and target observation are then prepared according to the selected registration configuration. For coarse-to-fine configurations, the transformation returned by the initial method is supplied to ICP, which refines the template against the measured target geometry. The final transformation is returned in the common form described in Section 4.7.5.

4.7.5 Estimated Transformation Output

Each successful registration produces a homogeneous transformation matrix containing the estimated rotation and translation from the local template frame to the observed scene frame.

$$\boldsymbol{T} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0^T} & 1 \end{bmatrix} \tag{4.3}$$

where $\mathbf{T}$ represents the transformation from the local template frame to the observed scene frame, while $\mathbf{R}$ and $\mathbf{t}$ represent the estimated rotation and translation, respectively.

The semantic identity of the object is retained with the transformation so that the pose remains associated with the correct template and downstream task information. Where the registration input was normalised, the translation component is restored to the physical units used by the scanner and robotic system before the result is returned.

The transformed template is also compared geometrically with the observed instance. Occupied spatial regions are examined at multiple resolutions to determine how much of the target is covered by the aligned template and how much of their occupied volume overlaps. These measurements provide a consistency check that is independent of the internal confidence produced by a learning-based model.

A registration can be rejected when the transformed template does not provide sufficient agreement with the observed geometry. The numerical acceptance criteria are defined as part of the validation protocol because they depend on the object set, point-cloud resolution and required manipulation tolerance.

Object symmetry must also be considered when interpreting the result. Some objects may support several rotations that produce similar visible geometry. In these cases, a transformation can be geometrically valid even when its rotation differs from one recorded reference pose. The evaluation procedure must distinguish a genuinely incorrect alignment from an equivalent pose caused by object symmetry.

The accepted transformation represents the estimated pose of the object within the observed scene. It becomes the input to grasp-pose generation and subsequent trajectory validation, which are described as part of the online system pipeline.

4.8 ROS-Based System Implementation

The dataset generation, perception and robotic-control methods described in the preceding sections were developed as separate functional components. However, the second experimental stage also required these components to exchange information and operate within one coordinated workflow. The integration challenge was to connect task configuration, dataset generation, model training, scanner acquisition, point-cloud processing, coordinate transformations, motion planning and physical execution without concentrating the complete system within a single program.

ROS was used as the integration layer for this purpose. The workspace separates the main functions into specialised packages and connects them through configuration files, topics, services, actions, custom messages and coordinate transformations. As a result, individual components can be started, replaced or reconfigured independently while maintaining defined interfaces with the rest of the system.

This structure supports the wider research requirements in two ways. First, it preserves flexibility during operation because scanner data, object instances, registered poses and placement targets are exchanged through common representations. Second, it supports adaptability and reconfiguration because object sets, perception methods, placement definitions and control parameters can be changed through configuration rather than being embedded within the source code.

The following subsections describe the organisation of the workspace, the responsibilities assigned to its main packages and the movement of information through the integrated system.

4.8.1 Workspace and Package Organisation

The second experimental stage was implemented within the *robotenv_v2_ws* workspace. Its organisation retained the hardware integration and motion-planning functions established during the first experiment while introducing a new set of packages for dataset generation, perception, task configuration and robotic control. The workspace represents an extension of the original software structure rather than a separate implementation.

The *robotenv_ws* workspace used during the first experimental stage contained four principal software groups. The *abb_robot_arm* collection provided the robot

description, ABB controller interface and MoveIt configuration. The *phoxi_camera* package provided communication with the three-dimensional scanner. The *pick_and_place_env* package represented the robotic cell and connected the robot, conveyor, scanner and planning environment. Finally, the *computer_vision_pkg* contained the object-processing and manipulation functions developed for the rectangular test objects.

The second workspace retained the *abb_robot_arm*, *phoxi_camera* and *pick_and_place_env* groups because their principal functions had already been validated during the first experiment. However, these groups were adapted where required by the relocated cell, revised digital model and new electronic gripper. In contrast, the original *computer_vision_pkg* was not retained as the main perception layer. Its fixed object-processing structure was replaced by the specialised packages contained within the new perception directory.

The resulting workspace can be considered as two connected software layers. The first layer contains the retained and adapted robotic infrastructure. The second contains the new perception, configuration and task-execution components developed for the second experimental stage.

The retained robotic infrastructure is organised mainly through the *abb_robot_arm*, *phoxi_camera* and *pick_and_place_env* package groups. These packages continue to provide the robot description and communication interfaces, scanner acquisition, coordinate transformations, digital-cell representation, conveyor communication and MoveIt-based planning functions established during the first experimental stage.

Although their general responsibilities remained unchanged, their configurations were adapted to the cell used during the second experiment. The digital environment and spatial relationships were updated to represent the relocated installation, while the robot description and planning configuration were revised to include a custom two-finger electronic gripper developed for the second experimental stage, referred to in this thesis as the WiLi-Gripper. These changes allowed the new perception and task-configuration components to use the previously validated hardware and motion-planning interfaces without reproducing their underlying implementation.

The software supporting the WiLi-Gripper was added to provide the geometric representation and control interfaces required to include the new end-effector within the robot model and pick-and-place sequence. Further details of its mechanical, electronic and communication design are provided in Appendix B.

The retained packages continued to manage the physical robotic infrastructure, while the main software changes were concentrated in the perception and task-specific layers. The original *computer_vision_pkg* used during the first experiment was replaced by the specialised packages contained within the new perception directory.

The main research extensions are organised within the perception directory. This directory contains the *hybrid_dataset_generation*, *point_cloud_processing*, *smart_pick_and_place_interface*, *smart_pick_and_place_control* and *smart_pick_and_place_bringup* packages. Together, these packages replace the centralised processing structure used during the first experiment with a set of components that can be configured and operated independently.

The hybrid_dataset_generation package implements the methodology described in Section 4.4. Its internal structure separates dataset configurations, generation modules, launch files and the ROS action interface. The Python modules perform template preparation, physics-based arrangement, point-cloud construction, background integration, visibility processing and ground-truth generation. The action server receives the selected dataset configuration, coordinates these stages and returns the generated output location.

The package stores the generation parameters separately from the processing functions. Object sets, simulation limits, background sources and measurement variations can be changed without modifying the implementation. The action-based structure also allows generation to be started from the graphical interface while progress information is returned during execution.

The *point_cloud_processing* package contains the common framework used for perception-model training and inference. It provides separate executable nodes for these two operating modes, together with the pipeline configurations, task implementations, messages, services and trained model resources required by the selected methods.

Within the package, segmentation and registration are organised as separate processing tasks. Each task contains common interfaces and registered model implementations so that the training and inference nodes do not need to be rewritten when a different compatible model is selected. The package also contains the functions used to publish processed point clouds, communicate registered instances and broadcast the resulting object transformations.

This organisation separates the wider perception sequence from the internal requirements of an individual model. The pipeline configuration determines which tasks are enabled and which implementation is assigned to each task. The selected wrappers then load the required datasets, model parameters and processing functions. As a result, model changes remain contained within the point-cloud processing package.

The *smart_pick_and_place_interface* package contains the graphical user interface and the supporting task-configuration functions. In addition to the main interface executable, it contains configuration resources, custom messages, services and helper modules for placement definition and target-frame generation.

The package receives the object models, placement poses, dataset settings and model selections provided by the user. It stores this information in structured configuration files and converts the active placement task into coordinate frames, visual markers and labelled placement-slot information. The four interface tabs and their user-facing functions are described in Section 4.9.

The *smart_pick_and_place_control* package contains the manipulation functions introduced for the second experimental stage. Its main control node receives the registered object poses produced by the perception pipeline and the placement slots generated from the user-defined task. It then connects this information to the robot model, end-effector and MoveIt planning environment.

The package contains the control configuration, launch files and supporting functions required for object selection, grasp-pose generation, placement assignment,

pose transformation and trajectory evaluation. It also includes a symmetry-checking function that supports the interpretation of object orientations where more than one equivalent pose may exist. The detailed online sequence implemented by this package is presented in Section 4.12.

The *smart_pick_and_place_bringup* package provides the main integration point for the complete system. Its launch structure can start the dataset generator, graphical interface, point-cloud processing nodes and robotic-control component according to the required operating mode. These components can be enabled independently so that dataset generation and model development do not require the complete physical cell to be active.

For online operation, the bring-up package also starts the conveyor input and output nodes retained within *pick_and_place_env*. A coordination node links the conveyor sensor, conveyor movement and scanner acquisition sequence. It controls when the conveyor is stopped, when the scanner measurement is requested and when the resulting point cloud becomes available to the perception pipeline.

This workspace organisation preserves a clear boundary between the validated infrastructure and the functions introduced during the second experimental stage. Robot communication, scanner access, conveyor control, cell representation and MoveIt integration remain within the retained package groups. Dataset generation, configurable perception, task definition and object-level manipulation are contained within the new packages.

The structure also clarifies the transition from the first experiment. Rather than modifying the original *computer_vision_pkg* to accommodate every new function, its responsibilities were redistributed across packages with defined roles. The interactions between these retained and newly introduced components are described in the following subsection.

4.8.2 Package Responsibilities and Common Interfaces

The packages communicate through interfaces selected according to the type and duration of the exchanged information. Configuration files preserve task and model settings between sessions. ROS topics carry continuous sensor data, perception results and system states. Services perform discrete operations such as loading a processing pipeline or applying a placement task. Actions are used for longer operations that require progress feedback and cancellation. The ROS transformation system maintains the spatial relationships between the scanner, detected objects, placement targets and robot.

Dataset generation is initiated through an action interface because the process can contain many simulated arrangements and may require a substantial execution time. The action goal contains the selected configuration path. During execution, the generator reports the current stage, completed sample count, total sample count and percentage completed. When generation finishes, the result identifies whether the process succeeded and provides the location of the generated data. This interface allows the graphical application or another ROS component to initiate generation without becoming responsible for the underlying processing sequence.

The point-cloud processing package uses a pipeline configuration to define its operating mode, active tasks, dataset association and selected model implementations.

A registry maps each configured model name to a wrapper that follows the common task interface. The wrapper is responsible for loading the model-specific configuration, preparing the implementation and exposing the required training or inference operation. This arrangement separates model selection from the main processing nodes.

Training and inference are handled by separate ROS nodes because they have different input and execution requirements. The training node loads the selected pipeline, creates the enabled model wrappers and invokes their training procedures. Training progress is returned through a feedback topic containing the active task, selected model, training stage and recorded performance values. The inference node loads the same type of pipeline configuration but operates on scanner point clouds rather than labelled datasets.

During inference, the incoming ROS point cloud is converted into the common scene representation. The enabled segmentation method separates the scene into object instances and background. Optional classification or completion stages can then process the separated instances when required by the selected pipeline. Registration subsequently estimates a transformation for each accepted instance. The node publishes the segmented point clouds for visualisation, a processing summary for monitoring and a structured message containing the registered instances.

Each registered-instance message retains the semantic identity, instance identifier, parent reference frame, child frame name and estimated transformation. A separate broadcaster converts these results into ROS coordinate frames so that the registered object poses can be inspected and accessed by downstream components. This avoids requiring the robotic-control package to reproduce the internal calculations of the perception methods.

Placement information follows a corresponding structure. When a task configuration is applied, the interface package creates a parent frame for the placement support and a child frame for each placement slot. It also publishes mesh markers so that the configured arrangement can be inspected in RViz. A placement-slot message links each child frame with its semantic label and slot index.

The robotic-control package subscribes to the placement-slot and registered-instance messages. The semantic label provides the connection between a detected object and the placement locations defined for its class. The control node maintains the availability of these slots and publishes occupancy updates after a placement has been completed or when the task is reset. TF is used to resolve the detected object and target-slot poses relative to the robot base before motion planning begins.

The package then uses the established MoveIt interfaces for forward and inverse kinematics, robot-state validation, collision-scene management and trajectory planning. In this way, the perception packages provide object-level geometric information while the control package remains responsible for determining whether a manipulation sequence is feasible and safe to execute.

4.8.3 Data Flow and System Initialisation

System initialisation begins through the bring-up launch structure. The launch configuration determines whether the dataset generator, graphical interface and point-cloud processing pipeline are required. It also selects the configuration set associated with the current object or task definition. When the physical cell is used, the conveyor

communication nodes and scan coordinator are started together with the perception and robotic-control components.

Each component then loads the configuration associated with its responsibility. The interface loads its mesh paths, scaling rules and display settings. The perception pipeline loads the enabled tasks, selected model names and corresponding model configurations. The robotic-control node loads its planning groups, reference frames, gripper settings, motion limits and grasp parameters. Since these files are resolved independently, one part of the system can be reconfigured without requiring all other components to use the same internal parameter structure.

Before robotic execution is enabled, a placement task must be applied through the user interface. This operation generates the parent and slot coordinate frames, publishes their visual markers and sends the list of available slots to the robotic-control node. Receiving a valid slot list enables the placement workflow. Removing the task clears the slot list, resets occupancy information and prevents new perception results from being processed for execution.

The perception pipeline is initialised by loading the selected pipeline configuration. The enabled model wrappers are then configured, and their trained parameters or descriptor resources are loaded. The same loading mechanism can be used by the training node or inference node depending on the operating mode. This enables a revised model combination to be introduced without changing the scanner subscription or result-publication interfaces.

During physical operation, the conveyor sensor identifies when an object has reached the acquisition region. The scan coordinator then stops the conveyor, waits for the configured settling condition and requests a frame from the 3D scanner. The resulting point cloud is published through the scanner interface and received by the point-cloud processing node.

The processing node applies the selected segmentation and registration stages and publishes the resulting object instances and estimated transformations. These transformations are also broadcast as object coordinate frames. The robotic-control node receives the registered instances together with the available placement slots and associates each object with a free slot belonging to the same semantic class.

The object and slot poses are transformed into the robot reference frame before grasp and placement candidates are created. MoveIt is then used to examine inverse kinematics, robot-state validity, collision conditions and trajectory feasibility. Only candidates that satisfy these checks are passed to the physical execution sequence.

System feedback is distributed throughout this process. Dataset generation reports its current stage and completion percentage. Model training reports the active task and training progress. The inference node publishes the detected instances, selected models and stage execution status. The control node publishes placement-slot occupancy and execution state. These outputs provide the information required by the graphical interface and support later evaluation of the complete workflow.

This section has described the technical integration of the system components. The following section presents the user-facing workflow through which task-specific information is introduced and the integrated functions are prepared for operation.

4.9 User-Guided Configuration Workflow

The ROS architecture described in Section 4.8 provides the technical connections between dataset generation, perception, task definition and robotic execution. To support task reconfiguration without requiring changes to the underlying software, the configuration workflow was implemented through a graphical user interface. The interface allows the user to define the object set, prepare the dataset and model resources, and initialise the task-execution settings required by the proposed system.

The interface is organised into four tabs named Placement Config, Dataset Setup, Model Tuning and System Control. Each tab corresponds to a different stage of system preparation, although they share the same object identities and task configuration. The user therefore defines the high-level task through a common interface, while the corresponding configuration files and ROS requests are generated for the individual system components.

Figure 4.3 shows the Placement Config tab, which provides the spatial task-definition interface used to configure the working environment, placement-support geometry and object placement poses. The left-hand side contains a three-dimensional mesh viewer for visual inspection of the loaded geometry, while the right-hand side contains a slot-configuration table in which the position, orientation and associated object identity of each placement slot are recorded. Additional controls allow configurations to be loaded, saved and reset, and the corresponding TF frames and visual markers to be generated for use by the robotic pipeline.

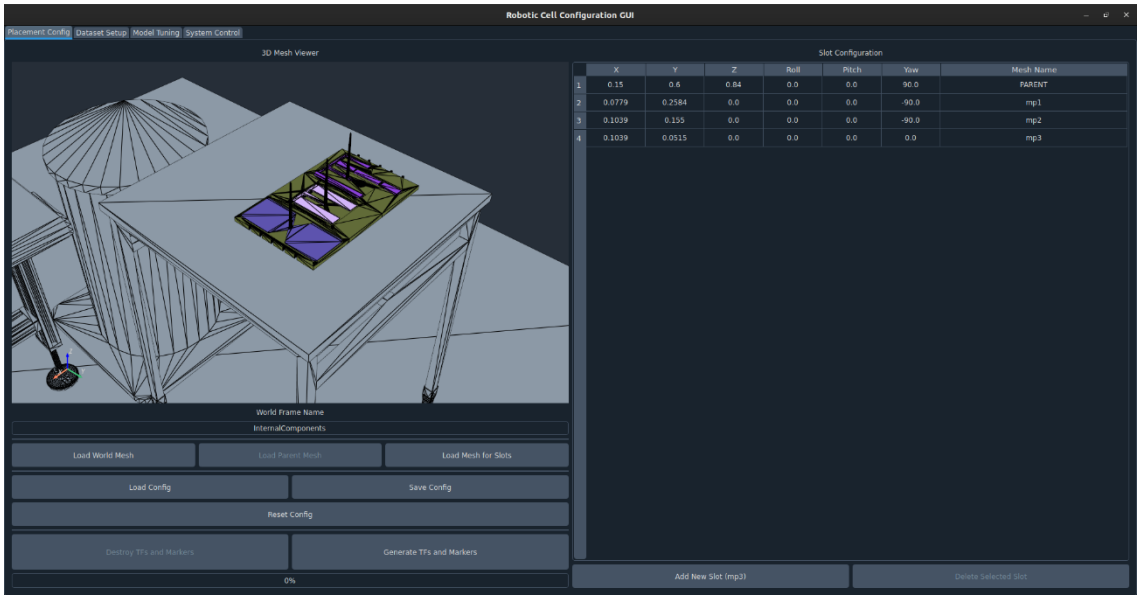


Figure 4.3: Placement Config tab of the user-guided configuration interface, showing the 3D task viewer, slot-definition table and controls used to define and apply object placement poses.

The Dataset Setup tab prepares the data-generation process described in Section 4.4. Separate interface areas are provided for object-level and scene-level datasets because the two data forms have different generation requirements. The user selects the object definitions, background source, spatial limits, sampling settings, simulation settings and output location. Existing configurations can be loaded and modified, while new configurations can be stored for later reuse. Dataset generation is initiated through the ROS action interface, and progress is displayed as the individual

stages are completed. Generated samples can also be loaded by index or selected randomly for inspection within the point-cloud viewer.

The Model Tuning tab connects the generated data with the trainable perception methods. The user selects the perception task, available model implementation, dataset location and model configuration. Training settings and checkpoint behaviour can then be reviewed or modified before the training node is started. During execution, the tab displays the current task, model state, training progress and validation feedback. Once training is complete, the selected checkpoint and model configuration are associated with the pipeline definition used for later inference.

The System Control tab applies the prepared resources to the integrated robotic workflow. It loads the selected placement, perception and robot-control configurations and verifies that the required ROS components are available. The tab provides controls for enabling or stopping acquisition, perception and robotic execution while displaying the status of each processing stage. Detected instances, registered poses, available placement slots and slot occupancy are also presented so that the user can follow the progression of the task. Reset and shutdown functions return the system to a controlled state when the task is changed or execution is interrupted.

Together, the four tabs provide a sequence from object and task definition to dataset generation, model preparation and physical operation. The interface does not replace the automated functions of the individual packages. Instead, it provides a consistent point from which those functions can be configured and initiated without requiring direct modification of their source code.

4.9.1 Loading Target Object CAD Models

The placement workflow begins using the Placement Config interface shown in Figure 4.3 by defining the geometric reference used for the task. The user first enters the name of the main reference frame and loads a mesh representing the wider working environment. This model provides visual context within the interface but does not itself define the placement poses.

A second mesh is then loaded to represent the placement support, fixture or tray. This geometry is treated as the parent of the placement arrangement. Its six-degree-of-freedom pose defines the relationship between the placement region and the wider reference frame. Establishing this parent relationship prevents every slot from being defined independently in the global cell frame.

Once the parent geometry has been loaded, the user selects the CAD models associated with the target objects. Each file is read, converted according to the configured mesh scale and displayed in the three-dimensional viewer. The name of the model and file path are retained as part of the task definition. They provide the link between the geometric representation, semantic object identity and placement slots created for that object.

Several target models can be loaded within the same task. After a model has been selected, the interface retains its geometry so that additional placement copies can be created without reloading the source file. A consistent display colour is assigned to the copies of each model, which makes different object classes easier to distinguish within the completed arrangement.

The interface creates a three-level geometric structure. The wider environment is referenced to the selected world frame. The placement support is positioned within that environment. Each target-object pose is then defined relative to the placement support. This hierarchy preserves the local arrangement when the complete support is moved to another position within the cell.

A previously saved task can also be loaded at this stage. The interface reconstructs the environment mesh, parent geometry, target-object models and previously defined poses from the stored configuration. The user can then review or modify the arrangement without repeating the complete setup procedure.

4.9.2 Defining Placement Slots

After a target-object model has been loaded, one or more placement slots can be created for that object. Each slot defines the intended final pose of an object rather than a robot joint configuration or predetermined trajectory. The planning system remains responsible for finding a feasible motion between the detected object and this target pose.

The slot-configuration table shown in Figure 4.3 records the translation and orientation of each slot together with the associated mesh identity. Translation is represented along the three spatial axes, while orientation is defined through roll, pitch and yaw. The first table entry represents the parent geometry, and the following entries represent the local poses of the individual placement slots.

When a pose value is changed, the corresponding mesh is updated in the three-dimensional viewer. The local slot transformation is combined with the current parent transformation before the geometry is displayed. This provides immediate visual feedback while preserving the distinction between the global placement-support pose and the local arrangement of its slots.

Additional slots can be created for the currently selected model, while existing slots can be deleted or modified. The user can also load another object model and add slots belonging to that class within the same placement arrangement. Copies of the same object retain a common semantic identity but receive separate slot entries and visual representations.

The hierarchical representation simplifies later task changes. Moving or rotating the parent geometry updates the complete group of placement targets, whereas modifying one slot affects only the associated object copy. This reduces the need to recalculate every target when the placement support is repositioned within the cell.

The completed arrangement can be inspected from different viewpoints before it is applied. This review helps identify incorrect model selection, unsuitable orientation, unintended overlap or inconsistent spacing. However, the visual inspection does not establish robot reachability or collision-free motion. These conditions are evaluated later through the planning and trajectory-validation stages.

Each slot is assigned an ordered frame identifier when the task is applied. The semantic identity links the slot to its target-object class, while the index distinguishes separate locations belonging to that class. The robotic-control component uses this information to identify an available target and maintain its occupancy state during execution.

4.9.3 Generating and Applying Task Configurations

Once the object models and placement slots have been defined, the interface converts the task into structured YAML configurations. Separating this information from the processing code allows a task to be saved, reviewed and reused without changing the dataset generation, perception or robotic-control implementations.

The editable placement configuration stores the main reference frame, environment model, parent geometry, parent pose and individual slot entries. Each slot retains its local pose, mesh identity and mesh path. This form is used when the task must be reconstructed within the graphical interface because every object copy remains available as a separate editable entry.

When the task is applied, the interface creates a second operational representation. Slot entries associated with the same object model are grouped under one semantic identity, while their individual poses are retained as separate copies. This structure provides the relationship required by the downstream ROS nodes without removing the individual placement locations.

The operational configuration is sent to the target-frame service. The parent pose is first converted into a transformation relative to the selected world frame. A child frame is then created for each placement slot relative to the parent. Corresponding mesh markers are published so that the active task can be inspected in RViz or another ROS visualisation tool. The complete placement-configuration sequence, from loading the task geometry to visualising the applied configuration in RViz, is summarised in Figure 4.4.

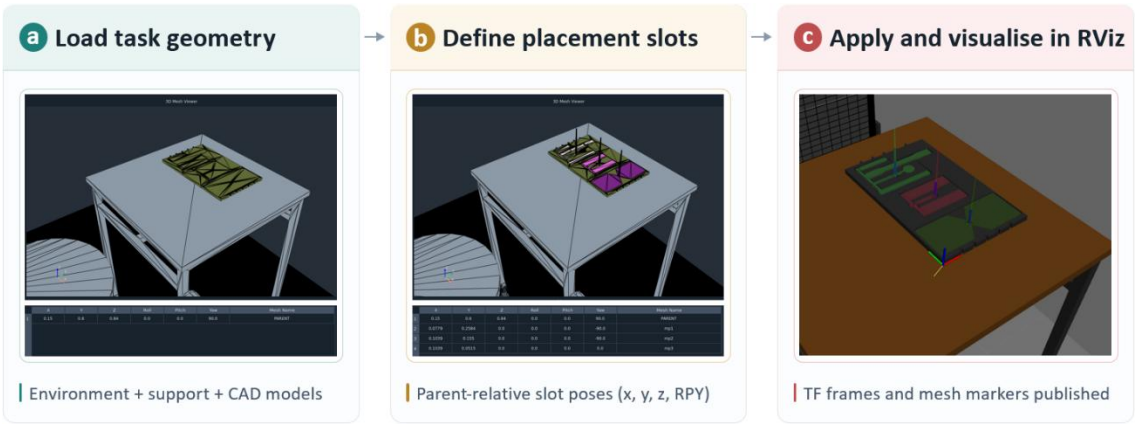


Figure 4.4: User-guided placement-configuration workflow. (a) Loading the task geometry, (b) defining parent-relative placement slots, and (c) visualising the applied configuration through ROS TF frames and mesh markers in RViz.

At the same time, the interface publishes the placement-slot list. Each entry contains the object label, target frame and slot index. Since this message is retained by the ROS communication layer, components that start later can still obtain the current placement task. The robotic-control node uses the list to initialise its slot manager and becomes active only when at least one valid slot is available.

The Dataset Setup tab uses the same object definitions to create the configurations required for object-level and scene-level dataset generation. Once the generation parameters have been reviewed, the configuration path is sent to the dataset action server. Progress feedback is returned to the interface until generation finishes or is cancelled, after which the output directory is recorded for model preparation.

The Model Tuning tab associates the generated dataset with the selected task and model implementation. It records the required model configuration, training resources and checkpoint selection before loading the pipeline through the training interface. When the resulting model is approved for inference, the corresponding task and model entry is stored in the operational pipeline configuration.

The System Control tab then loads the active placement configuration, perception pipeline and robotic-control settings. This step establishes a consistent relationship between the semantic classes used by the dataset, the model outputs, the geometric templates and the available placement slots. The user can verify the component states before enabling scanner acquisition and physical execution.

A task can be saved for later reuse or loaded and modified when the object set or placement arrangement changes. Applying a revised configuration replaces the active frames, markers and slot list. Removing the task destroys the generated placement representations, publishes an empty slot list and returns the robotic-control component to its disabled state.

Through this workflow, the user provides the information that must remain task-specific, while the system performs the conversions needed by the individual ROS components. The object identities, dataset settings, model selections and placement definitions are therefore introduced through one controlled process rather than being repeated within separate source files.

4.10 Target Industrial Use Case

The robotic system developed in this research was intended to support configurable pick-and-place tasks involving known but changeable object sets. For each task, the user provides the relevant object models, perception resources and placement definitions. During operation, the system determines the identity and pose of the configured objects from the acquired point cloud before planning their transfer.

A suitable use case was required to evaluate these functions with objects that were more geometrically complex than those used in Chapter 3. The selected objects needed to present irregular shapes, different stable orientations, partial scanner observations and repeated or symmetric geometric features. At the same time, they needed to remain part of a bounded task in which the supported object classes and destination locations were defined before execution.

Runner-connected injection-moulded assemblies were selected because they combined industrial relevance with the geometric and operational conditions required for the second experimental stage. The use case emerged from wider polymer smart-manufacturing activities conducted within the Materials Research Institute at the Athlone Campus of the Technological University of the Shannon. These activities considered injection moulding, conveyor transport, three-dimensional imaging, robotic handling and subsequent processing as connected manufacturing stages. Within this context, the moulded assemblies provided a suitable application through which the developed dataset generation, perception, task-configuration and robotic-control methods could be evaluated together.

After moulding, the polymer feed system remained attached to the moulded products, forming a complete structure comprising the sprue, runners, gates and

connected products. Throughout this chapter, the term injection-moulded assembly refers to this complete runner-connected structure before the individual moulded products are separated. The subsequent separation of the moulded products from the gates and runner system is referred to as degating or gate removal [141], [142], [143].

The selected use case was examined through complementary offline and online validation stages. The offline stage evaluated segmentation and registration using generated datasets and real scanner observations of the selected assemblies. The online stage introduced the assemblies onto the conveyor and connected the perception output to grasp generation, motion planning and placement. The corresponding procedures are described in Sections 4.11 and 4.12. The injection-moulding machine and the subsequent inspection or degating processes were not included within the experimental cell.

4.10.1 Selected Injection-Moulded Assemblies

Three geometrically distinct injection-moulded assemblies were selected for the experiment. Each assembly consisted of several connected features forming one rigid structure. For dataset generation, perception and robotic manipulation, the complete assembly was treated as a single object rather than as a collection of separate moulded products.

The study did not require the intended function of the individual features to be identified. Instead, the assemblies were selected according to the geometric conditions they introduced into the validation. These included irregular branching structures, variations in width and thickness, repeated features, approximate symmetry and different possible resting orientations. These characteristics provided more demanding conditions for instance segmentation and six-degree-of-freedom registration than the regular rectangular objects examined in Chapter 3.

For clarity and consistency throughout the remainder of this chapter, the three assemblies are designated as Assembly Type 1 (AT1), Assembly Type 2 (AT2) and Assembly Type 3 (AT3). These designations refer to their experimental geometry and do not imply a particular mould classification, material test or subsequent manufacturing operation. Each assembly type was assigned a separate semantic class, while different physical copies of the same type retained separate instance identities.

Figure 4.5 presents the CAD representations of the three assemblies. The models show the complete connected structures processed by the perception and robotic-control components. The assembly types are displayed separately so that their overall geometry, repeated features and symmetry relationships can be compared.

AT1, shown in Figure 4.5(a), contained four dissimilar branches connected to a common central structure. These comprised a rectangular product measuring approximately 127x13x6 mm, a circular product measuring approximately 25 mm in diameter and 2 mm in thickness, an elongated dumbbell-shaped product measuring approximately 165 mm in length and 4 mm in thickness, and a rounded rectangular product measuring approximately 70x12x2 mm. Their different dimensions and positions produced an irregular object with narrow connections, curved boundaries and a non-uniform distribution of geometry.

AT2, shown in Figure 4.5(b), contained two identical rectangular plate-like products measuring approximately 103x26x2 mm. They were positioned on opposite

sides of the central connection, producing approximate reflection symmetry about one principal axis. Their thin geometry also produced different visible profiles depending on the resting orientation of the complete assembly.

AT3, shown in Figure 4.5(c), contained two identical square plate-like products measuring approximately 80x80x2 mm. The arrangement produced approximate symmetry about two principal axes when considered through its main planar geometry. The central and connecting features provided additional local three-dimensional detail within the complete assembly.

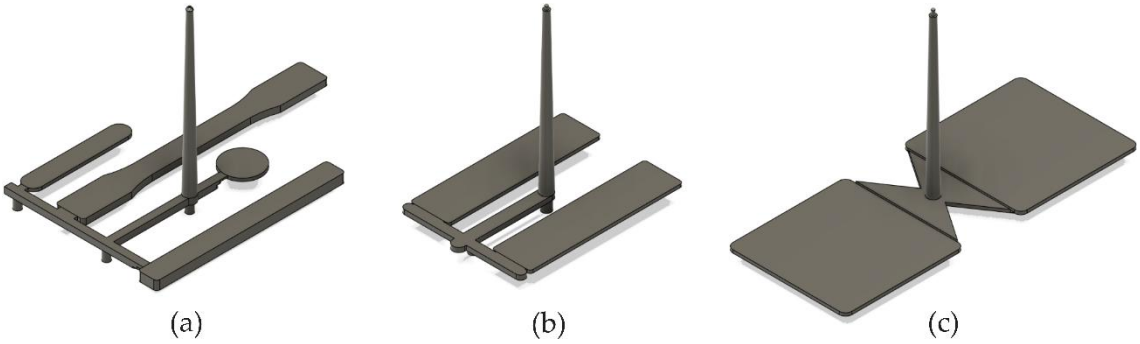


Figure 4.5: CAD representations of the three injection-moulded assemblies used during the second experimental stage. (a) AT1 with four dissimilar connected products. (b) AT2 with two identical rectangular plate-like products. (c) AT3 with two identical square plate-like products.

These geometric differences influenced the visible point clouds produced from different viewpoints and resting orientations. However, the material properties and intended mechanical tests of the individual products were not evaluated. Within this research, the complete assemblies were selected according to their relevance to object segmentation, pose registration and robotic manipulation.

4.11 Offline Validation Protocol

The offline validation was designed to evaluate the perception methodology independently of robot motion planning and physical manipulation. While the online validation presented in Section 4.12 examines the ability of the complete robotic system to execute the target pick-and-place task, the offline validation isolates the principal perception stages and compares their outputs against known reference information. This separation enables errors associated with segmentation and point-cloud registration to be examined independently before their combined effect on the complete perception pipeline is considered.

The validation was organised into the three experimental branches summarised in Figure 4.6. The first evaluated instance segmentation and domain transfer, with particular attention to whether the integration of measured background information within the hybrid dataset improved the real-world generalisation of PointGroup compared with training using fully synthetic scenes. The second evaluated registration independently from segmentation by comparing five registration configurations using canonical object templates and reference object instances. These comprised ICP, DCP, DCP followed by ICP, RANSAC followed by ICP, and GeDi followed by RANSAC and ICP. The third evaluated the complete perception sequence using the selected Hybrid PointGroup

followed by DCP and ICP configuration, where object instances predicted directly from the real-scanner scenes were passed to the registration stage.

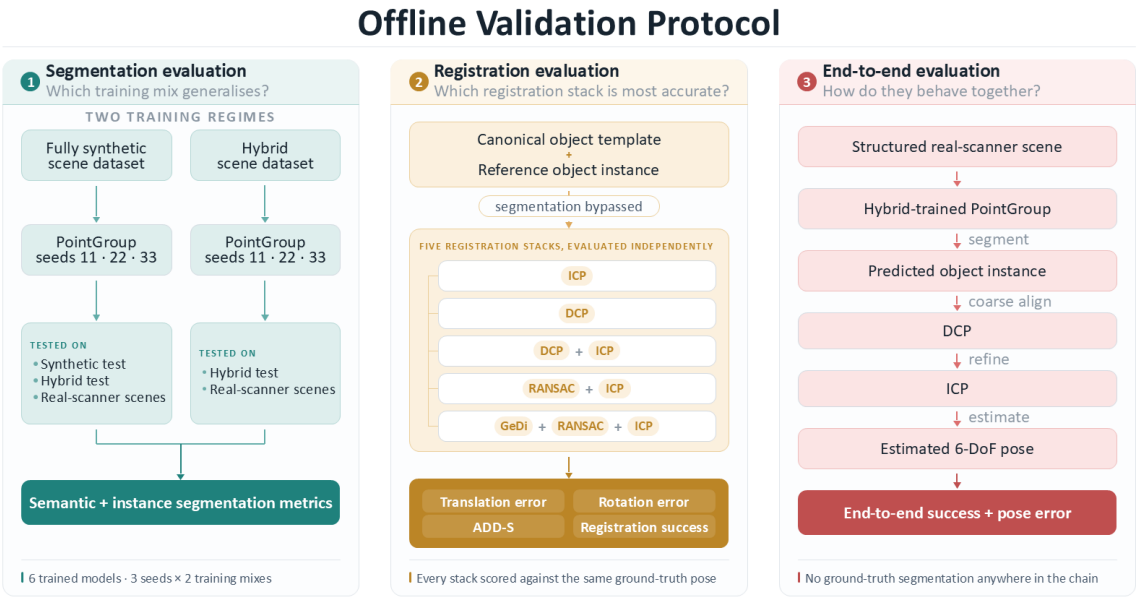


Figure 4.6: Overview of the offline validation protocol. PointGroup models trained using fully synthetic and hybrid datasets are evaluated across generated and real-scanner scenes. Registration is assessed independently using five configurations, followed by end-to-end evaluation of the selected Hybrid PointGroup followed by DCP and ICP pipeline.

PointGroup was evaluated at both semantic and instance levels because these outputs represent distinct requirements of the perception pipeline. The architecture predicts point-level semantic information and groups points into object-instance proposals, with candidate instances subsequently assigned confidence scores through ScoreNet [64]. The original PointGroup evaluation also uses overlap-based instance measures including average precision at IoU thresholds of 0.25 and 0.50, providing an established basis for the instance-level metrics adopted in this work.

Registration was first evaluated independently from segmentation using reference object point clouds extracted from the structured real-scan dataset. This experiment determines the registration capability of each method when the correct target points are available and therefore prevents segmentation errors from influencing the comparison.

DCP was based on the learned embedding and attention-based correspondence architecture introduced by *Wang et al.* [80]. The final implementation incorporated the adapted training procedure described in Section 4.7.4, including Match Normalization and direct NLL supervision of the correspondence assignment rather than back-propagating a pose-level objective through the differentiable SVD stage. This adaptation was motivated by the findings of *Dang et al.* [81], who identified sensitivity to wide rotational variation and differences between source and target feature distributions as important limitations for learning-based point-cloud registration. GeDi was evaluated as a pretrained local geometric descriptor and was used to establish feature correspondences before RANSAC-based global alignment and ICP refinement. GeDi was designed to provide scale- and rotation-invariant local descriptors with transfer across sensing domains [140].

The final offline experiment connected the selected segmentation and registration components. Predicted instances produced by the hybrid-trained

PointGroup model were supplied to DCP and subsequently refined using ICP. Unlike the independent registration experiment, this configuration preserves errors introduced by segmentation and therefore evaluates the combined behaviour of the perception sequence before its integration with physical robot execution.

4.11.1 Experimental Datasets and Training Conditions

The offline experiments used separate scene-level and object-level datasets according to the requirements of the segmentation and registration stages introduced in Sections 4.6 and 4.7. Scene-level datasets were used for PointGroup, whereas isolated object observations were used for DCP. The structured real-scan dataset described in Section 4.11.2 was retained exclusively for evaluation and was not used for network training, validation-based checkpoint selection or model optimisation.

Two scene-generation conditions were considered for PointGroup. The fully synthetic condition combined generated object observations with a computationally generated representation of the supporting environment. The hybrid condition used the same object-generation methodology but integrated the generated object observations with point-cloud measurements acquired from the physical conveyor environment. The principal experimental difference between the two conditions was therefore the representation of the scene background, while the object-generation procedure, target classes and automatic annotation process remained consistent.

The comparison was designed to isolate the contribution of measured background information rather than differences in network architecture or training procedure. PointGroup was therefore trained independently under the fully synthetic and hybrid conditions using the same architecture, preprocessing procedure, optimisation strategy and training hyperparameters. Each condition contained 1000 generated scenes, divided into 900 training scenes, 50 validation scenes and 50 test scenes. This corresponded to a 90%, 5% and 5% partition while maintaining identical dataset sizes between the two experimental conditions.

Neither PointGroup condition was initialised using pretrained weights associated with the target objects. The two conditions used the same model initialisation procedure and differed only in the scene dataset supplied during training. Three independent training runs were performed for each condition using random seeds 11, 22 and 33. Within each run, the model checkpoint used for subsequent evaluation was selected according to validation loss. The real-scan dataset was not used during this selection process.

Repeating the training procedure across different random seeds enabled the variability associated with stochastic neural-network optimisation to be represented in the final results rather than relying on a single trained instance. Previous studies have shown that model initialisation and optimisation can introduce meaningful run-to-run variation even when the architecture and principal hyperparameters remain unchanged [144], [145].

The registration data were treated separately because registration operates on isolated object observations rather than complete scenes. Each object-level sample contained a complete centred source template, a partial target observation and the known rigid transformation relating the two point clouds. Background points used during scene composition were not retained as part of the object-level registration

representation. Consequently, the distinction between fully synthetic and hybrid background generation was not considered an experimental factor for DCP.

The object-level observations generated under the two scene-generation conditions were combined while preserving their original data partitions. Training observations from the two collections were merged only with one another, with the same procedure independently applied to the validation and test subsets. The combined dataset contained 5625 training observations, 312 validation observations and 313 test observations. No observations were transferred between the original data partitions during this process.

DCP operates on point clouds sampled to 1,024 points. Samples that did not contain sufficient points for this sampling procedure were excluded by the dataset loader. This removed five observations from the training subset and one observation from the validation subset, resulting in 5620 training observations and 311 validation observations used during optimisation and checkpoint selection. The same point-count eligibility criterion was retained for subsequent evaluation. The composition of the scene-level and object-level datasets used during offline validation is summarised in Table 4.1.

Table 4.1: Composition of the generated datasets used during offline validation.

Dataset	Representation	Training	Validation	Test
Fully synthetic scene dataset	Generated objects and generated environment	900 scenes	50 scenes	50 scenes
Hybrid scene dataset	Generated objects and measured real background	900 scenes	50 scenes	50 scenes
Combined registration dataset	Complete template and partial object observation	5625 objects	312 objects	313 objects

Pooling the object-level observations increased the range of available object poses and partial observations without introducing background type as an artificial registration condition. This distinction was appropriate because the hybrid-versus-synthetic comparison was designed to investigate scene-level segmentation transfer, whereas registration was concerned with alignment between a complete object template and a partial target observation.

The adapted DCP model described in Section 4.7 was trained from scratch using the combined object-level dataset. Three independent runs were performed using seeds 11, 22 and 33. The best checkpoint within each run was selected using the corresponding validation subset. No published DCP weights were used to initialise the models included in the final evaluation.

GeDi was treated differently because the descriptor was evaluated using its published pretrained model rather than being retrained using the generated object-level observations. The GeDi and DCP conditions therefore do not represent equivalent training strategies. DCP represents a task-specific learned registration model trained using the generated dataset, while GeDi represents a pretrained descriptor designed to generalise across point-cloud acquisition domains. *Poiesi et al.* describe GeDi as a local descriptor constructed to remain robust to geometric transformations, occlusion and

changes in sensing domain [140]. The training and evaluation conditions associated with each perception model are summarised in Table 4.2.

Table 4.2: Model training and evaluation conditions used during offline validation.

Model condition	Training data	Training runs	Evaluation data	Purpose
PointGroup synthetic	Fully synthetic scenes	3	Synthetic, hybrid and real scenes	Evaluate segmentation and domain transfer
PointGroup hybrid	Hybrid scenes	3	Hybrid and real scenes	Evaluate effect of real-background integration
DCP	Combined object-level dataset	3	Generated objects and real reference instances	Evaluate task-specific registration
GeDi	Pretrained model	–	Real reference instances	Evaluate pretrained geometric registration

4.11.2 Structured Real-Scan Validation Dataset

A structured real-scan dataset was created to provide a controlled evaluation of transfer from generated training data to point-cloud measurements obtained from the physical robotic cell. In contrast to the randomly arranged scenes produced during dataset generation, the real validation scenes were acquired using predefined resting configurations and regular angular increments. This acquisition procedure introduced controlled variation in object orientation and visibility while retaining a consistent experimental structure across the three target assembly types.

The dataset contained AT1, AT2 and AT3, as introduced in Section 4.10.1. Each scanner scene contained one instance of each assembly type, resulting in three reference objects per scene. The objects were evaluated using three predefined resting configurations selected to represent distinct stable arrangements of the assemblies on the supporting surface. For clarity, the three predefined resting configurations are referred to as RC1, RC2 and RC3 throughout the subsequent evaluation. Within each resting configuration, measurements were obtained at successive angular increments of 15° over a complete in-plane rotation. The set of evaluated orientations was defined as

$$\Theta = \{15k^\circ \mid k = 0,1,\dots,23\} \qquad 4.4$$

which corresponds to 24 angular conditions ranging from 0 to 345 degrees for each resting configuration.

Figure 4.7 illustrates this structured acquisition procedure. A representative scene is shown for each of the three resting configurations, while the circular sampling diagrams indicate the 24 angular conditions acquired for each configuration. Each resting configuration therefore contributed 24 scanner scenes. Across RC1, RC2 and RC3, this produced 72 scenes, with every scene containing one instance of AT1, AT2 and AT3.

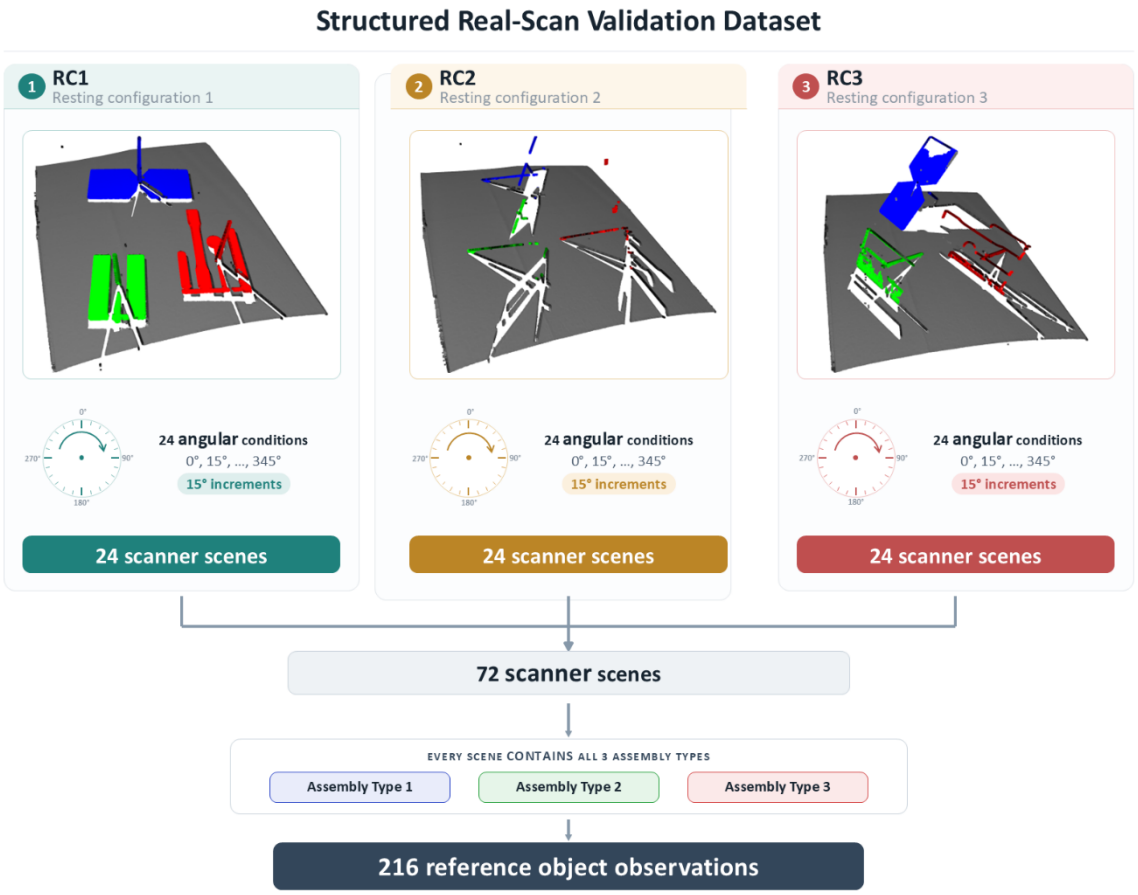


Figure 4.7: Structure of the real-scanner validation dataset. The three resting configurations are each sampled at 24 angular conditions between 0 ◦ and 345 ◦ using 15 ◦ increments. Every scene contains one instance of each of the three target assembly types, resulting in 72 scenes and 216 reference object observations.

The resulting dataset therefore contained 72 scanner scenes and 216 individual object observations. Its complete composition, including the available point-level annotations and reference transformations, is summarised in Table 4.3.

Table 4.3: Composition of the structured real-scanner validation dataset.

Property	Value
Target assembly classes	3
Resting configurations	3
Angular increment	15°
Angular conditions per configuration	24
Total scanner scenes	72
Objects per scene	3
Total reference object observations	216
Physical colour variants	3
Point-level semantic labels	Available
Point-level instance labels	Available
Reference 6-DoF transformations	Available

Three physical colour variants were represented during the structured acquisition, corresponding to white, black and silver specimens. Colour followed the

predefined acquisition sequence and was retained as scene metadata rather than being treated as an independently randomised experimental factor. Since colour was partially coupled to the angular acquisition sequence, no independent statistical comparison between physical colour conditions was performed.

Each scene was stored together with point-level semantic and instance reference labels. Semantic labels identified background points and points belonging to each of the three target assembly classes. Instance labels retained the identity of the individual physical objects. Although each structured scene contained one object from each semantic class, semantic and instance information were stored separately to preserve compatibility with the general instance-segmentation representation used by PointGroup.

The reference labels were derived geometrically using the known CAD models and their positioned transformations relative to the measured point clouds. Measured points were assigned semantic and instance labels according to their geometric association with the positioned object models. The resulting annotations for all 72 scenes were visually inspected before inclusion in the evaluation dataset.

The original scanner appearance information was retained as model input. Colours used solely to visualise the semantic and instance reference annotations were not supplied to PointGroup during inference. Instead, these annotation colours were converted to numerical reference labels and used only during evaluation. This separation prevented ground-truth annotation information from being introduced as an input feature.

A reference six-degree-of-freedom transformation was retained for each of the 216 object observations. The transformation from the local object frame O to the common validation reference frame V was represented as

$$T_{V \leftarrow O} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \tag{4.5}$$

where $R \in \mathbb{R}^{3\times3}$ represents the object rotation and $t \in \mathbb{R}^3$ represents its translation.

The individual transformations were obtained from the CAD assembly representation and converted to the common reference frame used by the validation point clouds. Each positioned object model was visually compared with the corresponding measured scene before the transformation was accepted as reference data. These transformations subsequently provided the ground-truth poses used during registration evaluation.

Figure 4.8 illustrates the different representations retained for a representative validation scene. Figure 4.8(a) shows the original measured point cloud, while Figure 4.8(b) shows the corresponding semantic and instance reference annotation. Figure 4.8(c) uses the same measured background but removes the points assigned to the three object instances and replaces them with the canonical CAD meshes transformed using their reference 6-DoF poses. This final representation is included only to visualise the correspondence between the stored reference poses and the physical object locations and was not used as an input to the perception models.

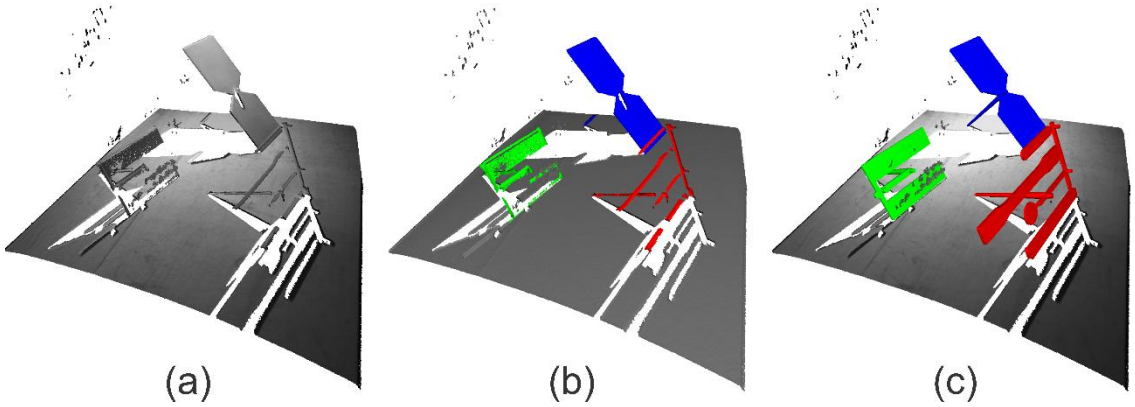


Figure 4.8: Example of the structured real-scan reference data. (a) Original measured point cloud. (b) Semantic and instance reference annotation. (c) Canonical CAD meshes transformed using the stored reference 6-DoF poses and shown within the measured scene.

4.11.3 Segmentation Evaluation

The segmentation evaluation was designed to assess both the performance of PointGroup within the generated data domain and its ability to generalise to point clouds acquired from the physical robotic cell. Particular attention was given to the effect of incorporating the real environment into the hybrid training dataset. The fully synthetic and hybrid PointGroup models were evaluated under a set of controlled within-domain and cross-domain conditions before their performance was compared using the structured real-scan dataset.

PointGroup produces point-level semantic predictions together with grouped object-instance proposals. The evaluation consequently considered both semantic classification and instance segmentation. At the semantic level, predicted point labels were compared directly with the corresponding reference semantic labels. At the instance level, the predicted object proposals were compared with the reference object instances according to their spatial overlap and predicted class. This distinction is important because correct semantic classification does not necessarily imply correct object separation. Points may be assigned to the correct assembly class while individual physical objects remain incomplete, fragmented or incorrectly grouped. The separate evaluation of semantic and instance performance therefore provides a more complete assessment of the segmentation stage. This follows the general evaluation approach used for PointGroup, which reports instance-level performance based on overlap between predicted and reference object instances [64].

Five evaluation conditions were considered. The PointGroup models trained using fully synthetic scenes were first evaluated using the fully synthetic test subset. This provides an in-domain reference indicating how well the model performs when the evaluation scenes follow the same generation condition as the training data. The same models were then evaluated using the hybrid test subset. This cross-domain comparison introduces the real background while retaining generated object observations and provides an intermediate assessment of sensitivity to the change in background representation.

The fully synthetic PointGroup models were subsequently evaluated using the 72 structured real scanner scenes described in Section 4.11.2. This represents the principal domain-transfer condition because neither the object observations nor the

complete measured scenes used during this evaluation were included during training or checkpoint selection.

The PointGroup models trained using hybrid scenes were evaluated first using the hybrid test subset and subsequently using the same 72 structured real scanner scenes. Evaluation on the hybrid test subset provides the corresponding in-domain result for the hybrid training condition. Evaluation on the real scanner dataset measures whether the integration of measured environmental information during dataset generation improved transfer to physical scanner measurements. The complete set of segmentation evaluations is summarised in Table 4.4.

Table 4.4: PointGroup segmentation evaluation conditions.

Training condition	Evaluation dataset	Evaluation type	Purpose
Fully synthetic	Fully synthetic test scenes	In-domain	Establish performance within the fully generated domain
Fully synthetic	Hybrid test scenes	Cross-domain generated data	Examine sensitivity to measured background information
Fully synthetic	Real scanner scenes	Synthetic-to-real transfer	Measure generalisation from fully generated scenes to physical measurements
Hybrid	Hybrid test scenes	In-domain	Establish performance within the hybrid generation domain
Hybrid	Real scanner scenes	Hybrid-to-real transfer	Measure generalisation from hybrid scenes to physical measurements

The hybrid test scenes used to evaluate the fully synthetic models were identical to those used to evaluate the hybrid-trained models. Similarly, both training conditions were evaluated using the same structured real-scanner scenes and the same reference annotations. Keeping the evaluation data fixed provides a paired comparison between the two training strategies and prevents differences in scene composition from influencing the comparison.

The primary comparison for assessing the contribution of the hybrid dataset was performed on the real-scanner dataset. For each random seed, the fully synthetic and hybrid PointGroup models were evaluated using the same 72 scenes. This comparison directly measures whether replacing the generated representation of the supporting environment with real background information during training resulted in improved segmentation performance on physical scanner data.

Each of the three independently trained models associated with seeds 11, 22 and 33 was evaluated separately. Metrics were first calculated for each trained model and subsequently aggregated across the three runs. This preserves the variation associated with independent model optimisation rather than reporting the performance of a single selected training run. The statistical aggregation procedure is defined in Section 4.11.6.

For the real-scanner evaluation, the original measured scene was supplied to PointGroup without using the reference semantic or instance labels as model inputs. Predicted semantic labels and object proposals were retained in the coordinate system of the corresponding scene and compared with the reference annotations described in

Section 4.11.2. No correction of predicted labels or object instances was performed before metric calculation.

Semantic performance was quantified using class-level Intersection over Union, precision, recall and F1 score, together with mean Intersection over Union across the target classes. Instance segmentation was assessed using average precision at overlap thresholds of 0.25 and 0.50. These thresholds are consistent with those commonly reported for PointGroup and related 3D instance-segmentation evaluation, where predicted instances are associated with reference instances according to their Intersection over Union [64]. The mathematical definitions of these metrics and the aggregation procedure used across classes, scenes and training seeds are presented together in Section 4.11.6.

4.11.4 Registration Evaluation

The registration evaluation was designed to assess pose-estimation performance independently from the segmentation stage. For this experiment, the reference instance point clouds from the structured real-scan dataset were used directly as registration targets. Therefore, predicted PointGroup instances were excluded from this stage. This separation ensures that differences between registration methods are caused by their ability to align the object geometry rather than by incomplete, fragmented or contaminated segmentation results.

For each of the 216 reference object observations, the canonical CAD-derived object representation was used as the source point cloud and the corresponding reference instance extracted from the real scanner scene was used as the target point cloud. The known six-degree-of-freedom transformation described in Section 4.11.2 provided the reference pose against which the estimated transformation was evaluated. The same source and target data were supplied to all registration configurations wherever applicable, providing a common basis for comparison.

Five registration configurations were evaluated. These consisted of ICP, DCP, DCP followed by ICP, RANSAC followed by ICP, and GeDi followed by RANSAC and ICP. The configurations represent different approaches to obtaining the initial rigid alignment and to refining the final pose estimate.

ICP was included as the conventional local-registration baseline. The method iteratively establishes closest-point associations between two point sets and updates the rigid transformation to reduce their alignment error. ICP can provide accurate local refinement when a suitable initial alignment is available, although its iterative optimisation can converge to an incorrect local solution when the initial pose is poor [67].

DCP was evaluated as the task-specific learned registration method described in Section 4.7. The model directly predicts the rigid alignment between the complete source template and the partial target observation using learned geometric representations and correspondence estimation. The DCP architecture was originally proposed as a learned alternative to traditional closest-point registration and combines point-cloud embeddings with attention-based correspondence reasoning [80]. In this work, DCP refers to the adapted implementation and training procedure described in Section 4.7.4.

The DCP followed by ICP configuration was included to determine whether the learned DCP transformation provides a useful coarse alignment for subsequent local refinement. DCP first estimates the object pose from the complete template and partial observation. The resulting transformation is then supplied as the initialisation for ICP, which performs the final local alignment. This configuration separates the role of DCP as a learned global pose estimator from the role of ICP as a geometric refinement stage.

RANSAC followed by ICP provides a non-DCP global-registration alternative. RANSAC estimates an initial rigid transformation from candidate geometric correspondences while rejecting correspondences that are inconsistent with the proposed transformation. The method is designed to remain robust when the correspondence set contains a substantial proportion of outliers [78]. The resulting coarse transformation is subsequently refined using ICP.

The final configuration combined GeDi, RANSAC and ICP. GeDi was used to compute local geometric descriptors for the source and target point clouds, from which candidate correspondences were established. The descriptors were then supplied to RANSAC to estimate the coarse rigid transformation, followed by ICP refinement. GeDi was designed to produce local descriptors that are robust to rotation, occlusion and changes between sensing domains, making it suitable for correspondence estimation between CAD-derived geometry and partial scanner observations [140].

Table 4.5: Registration configurations used during offline validation.

Configuration	Initial alignment	Purpose
ICP	ICP registration initialisation	Local registration baseline
DCP	Learned DCP transformation	Standalone learned registration
DCP + ICP	Learned DCP transformation	DCP initialisation with refinement
RANSAC + ICP	RANSAC global alignment	Global registration with refinement
GeDi + RANSAC + ICP	GeDi correspondences and RANSAC global alignment	Pretrained learned descriptors within a correspondence-based registration pipeline

The comparison between DCP and DCP followed by ICP determines whether geometric refinement improves the transformation predicted by the learned model. The comparison between RANSAC followed by ICP and GeDi followed by RANSAC and ICP examines the contribution of the pretrained GeDi descriptors to correspondence-based global alignment. ICP provides the corresponding local-registration reference.

The three independently trained DCP models associated with seeds 11, 22 and 33 were evaluated separately using the same 216 reference object observations. The same DCP checkpoints were also used to initialise the corresponding DCP followed by ICP evaluations. Results from these runs were subsequently aggregated across seeds as described in Section 4.11.6. GeDi was evaluated using its published pretrained model and was therefore not associated with multiple independently trained checkpoints.

Registration performance was evaluated from the difference between the estimated transformation and the known reference transformation for each object observation. Translation error, geodesic rotation error, symmetry-aware surface error and registration success rate were retained as the principal registration measures. Their

mathematical definitions and statistical aggregation are provided in Section 4.11.6 so that the metric definitions remain common to all registration configurations.

4.11.5 End-to-End Perception Evaluation

The end-to-end offline evaluation was designed to assess the combined behaviour of segmentation and registration when reference instance point clouds were no longer provided to the registration stage. In contrast to the independent registration evaluation described in Section 4.11.4, the target object point clouds were obtained directly from PointGroup predictions. This allows segmentation errors, including incomplete, fragmented or contaminated object instances, to propagate naturally into the subsequent pose-estimation stage.

The end-to-end experiment was intentionally restricted to the perception configuration adopted for the final system rather than repeating every possible combination of segmentation and registration methods. The hybrid-trained PointGroup model was used to generate semantic and instance predictions from the structured real-scanner scenes. The predicted object instances were then supplied to DCP together with the corresponding canonical object templates. The transformation estimated by DCP was subsequently used to initialise ICP, which produced the final refined pose estimate. This configuration was selected based on the preceding component-level evaluations and corresponds to the perception pipeline subsequently used during the online experiments presented in Section 4.12. The evaluated sequence was therefore

$$\text{Hybrid PointGroup} \rightarrow \text{DCP} \rightarrow \text{ICP} \tag{4.6}$$

Restricting the end-to-end experiment to this configuration avoided duplicating the component-level comparisons already defined in Sections 4.11.3 and 4.11.4. The segmentation evaluation separately examines the effect of the fully synthetic and hybrid PointGroup training conditions, while the registration evaluation separately compares the five registration configurations using reference object instances. The purpose of the end-to-end experiment was therefore not to repeat these comparisons, but to determine how the selected perception sequence performs when its components operate consecutively.

The complete set of 72 structured real-scanner scenes was used for this evaluation, corresponding to 216 reference object observations. PointGroup predictions were passed to the registration pipeline without manual correction. Reference semantic labels, instance labels and object transformations were not used to construct the predicted registration inputs. Instead, they were used only after inference to associate the resulting predictions with the corresponding reference objects and to calculate the evaluation metrics.

This distinction ensures that the end-to-end evaluation reflects the actual behaviour of the perception pipeline. A missing object prediction, an incorrectly identified instance or an instance that could not produce a valid pose estimate was retained as an end-to-end failure rather than being removed from the evaluation. Similarly, errors caused by incomplete or incorrectly grouped predicted point clouds were allowed to influence the DCP and ICP stages.

Three end-to-end model combinations were evaluated using the independently trained networks associated with seeds 11, 22 and 33. For consistency, models obtained using the same seed were paired, resulting in the following combinations

$$\begin{aligned} &\mathrm{PointGroup}_{11} \rightarrow \mathrm{DCP}_{11} \rightarrow \mathrm{ICP} \\ &\mathrm{PointGroup}_{22} \rightarrow \mathrm{DCP}_{22} \rightarrow \mathrm{ICP} \\ &\mathrm{PointGroup}_{33} \rightarrow \mathrm{DCP}_{33} \rightarrow \mathrm{ICP} \end{aligned} \tag{4.7}$$

This produced three complete realisations of the selected perception pipeline without introducing an unnecessary evaluation of all nine possible PointGroup and DCP seed combinations. Performance was subsequently aggregated across the three runs according to the statistical procedure described in Section 4.11.6.

The principal measure of the end-to-end experiment was object-level perception success. An object was considered successfully processed when the segmentation stage produced a valid prediction corresponding to the reference object and the final DCP followed by ICP transformation satisfied the registration success criterion defined in Section 4.11.6. Translation, rotation and symmetry-aware surface errors were additionally retained for successfully associated pose estimates. Failed or missing instances remained part of the denominator when calculating the overall end-to-end success rate.

4.11.6 Evaluation Metrics

The offline evaluation used a common set of metrics to quantify semantic segmentation, instance segmentation, point-cloud registration and the complete perception pipeline. Metrics were selected to describe the principal requirements of each stage while avoiding unnecessary duplication between measures that represent equivalent information. Segmentation was evaluated using point-level classification and instance-overlap measures, while registration was evaluated from the difference between the estimated and reference object poses. The same metric definitions were applied consistently to all methods within each comparison.

4.11.6.1 Semantic segmentation metrics

For each target class c, semantic predictions were compared with the corresponding reference point labels. The number of correctly predicted points belonging to class c was denoted by TP_c (true positives). Points incorrectly assigned to the class were denoted by FP_c (false positives), while reference points belonging to the class but assigned another label were denoted by FN_c (false negatives).

Precision (P) was defined as:

$$P_c = \frac{TP_c}{TP_c + FP_c} \tag{4.8}$$

And represents the proportion of points predicted as class c that were correctly classified.

Recall (R) was defined as:

$$R_c = \frac{TP_c}{TP_c + FN_c} \tag{4.9}$$

and represents the proportion of reference points belonging to class c that were correctly recovered.

The $F1$ score combines precision and recall through their harmonic mean:

$$F1_c = 2\frac{P_c R_c}{P_c + R_c} \tag{4.10}$$

IoU measures the overlap between the predicted and reference point sets for each semantic class and was calculated as:

$$IoU_c = \frac{TP_c}{TP_c + FP_c + FN_c} \tag{4.11}$$

The overall semantic segmentation performance was summarised using mean IoU:

$$mIoU = \frac{1}{C}\sum_{c=1}^{C} IoU_c \tag{4.12}$$

where $C = 3$ corresponds to the three target assembly classes. The background was not included when calculating the headline $mIoU$ because the objective of the segmentation stage is the identification of the target assemblies. Where required, background performance can be reported separately.

These metrics provide complementary information. Precision describes contamination of predicted object classes, recall describes the proportion of reference points belonging to each class that were recovered, and IoU accounts for both missing and incorrectly included points within a single overlap measure.

4.11.6.2 Instance segmentation metrics

Instance segmentation was evaluated by comparing each predicted object proposal with the corresponding reference instances. The overlap between a predicted instance P and reference instance G was calculated as:

$$IoU(P, G) = \frac{|P \cap G|}{|P \cup G|} \tag{4.13}$$

A predicted instance was considered a valid match when it had the correct semantic class and its IoU with the corresponding reference instance exceeded an overlap threshold τ, evaluated at $\tau = 0.25$ and $\tau = 0.50$, producing AP25 and AP50 (average precision) respectively. These thresholds follow the evaluation protocol reported for PointGroup, where AP25 and AP50 correspond to average precision calculated using IoU thresholds of 25% and 50%.

For a given threshold, average precision (AP) was interpreted as the area under the precision-recall curve:

$$AP_\tau = \int_0^1 P_\tau(R)dR \tag{4.14}$$

where $P_\tau(R)$ represents precision as a function of recall when instances are matched using threshold τ. AP was calculated for each target class and averaged across the three assembly classes.

4.11.6.3 Registration metrics

Registration performance was evaluated by comparing the estimated object transformation with the corresponding reference transformation. Using the homogeneous transformation representation introduced in Equation 4.3, the estimated and reference transformations are denoted by $\hat{\mathbf{T}}$ and $\mathbf{T}$, respectively. Their rotation and translation components are denoted by $\hat{\mathbf{R}}$, $\hat{\mathbf{t}}$ and $\mathbf{R}$, $\mathbf{t}$.

Translation error was calculated as the Euclidean distance between the estimated and reference translation vectors:

$$e_t = \|\hat{\mathbf{t}} - \mathbf{t}\|_2 \tag{4.15}$$

where e_t represents the translation error and was reported in millimetres. This metric provides a direct measure of the positional difference between the estimated and reference object poses.

Rotation error was calculated using the geodesic angular distance between the estimated and reference rotation matrices:

$$e_R = \cos^{-1}\left(\frac{\mathrm{tr}\left(\hat{\mathbf{R}}\mathbf{R}^{\mathrm{T}}\right) - 1}{2}\right) \tag{4.16}$$

where e_R represents the angular difference between the estimated and reference orientations and was reported in degrees. This formulation measures the magnitude of the three-dimensional rotation required to align the estimated orientation with the reference orientation without relying on separate Euler-angle differences.

Translation and rotation errors provide an intuitive representation of pose accuracy, but they can be misleading for objects containing rotational or approximate geometric symmetries. Two estimated poses may contain substantially different rotation matrices while producing almost identical physical alignments of the object surface. A symmetry-aware surface measure was therefore included in the registration evaluation to complement the direct translation and rotation errors.

Pose accuracy was evaluated using the Average Closest Point Distance for symmetric objects, commonly referred to as $ADD-S$. $ADD-S$ is a symmetry-aware extension of the Average Distance of Model Points metric used in established six-degree-of-freedom object-pose evaluation. While the standard ADD formulation compares corresponding model points transformed by the estimated and reference poses, $ADD-S$ replaces this fixed correspondence with a nearest-neighbour comparison. This formulation was explicitly adopted for symmetric objects by *Xiang et al.* in the PoseCNN evaluation protocol [146].

For a set of points M sampled from the canonical object mesh, the $ADD-S$ error was defined as:

$$e_{ADD-S} = \frac{1}{|M|}\sum_{\mathbf{x}\in M}\min_{\mathbf{y}\in M}\left\|\hat{\mathbf{R}}\mathbf{x} + \hat{\mathbf{t}} - (\mathbf{R}\mathbf{y} + \mathbf{t})\right\|_2 \tag{4.17}$$

where M represents the set of points sampled from the canonical object mesh and $|M|$ is the number of points in this set. The vectors x and y represent points from the same canonical object model. The matrices R and $\hat{\mathrm{R}}$ represent the reference and estimated rotations, respectively, while t and $\hat{t}$ represent the corresponding reference and estimated translation vectors. The Euclidean norm $\|\cdot\|_2$ measures the distance between the transformed model points. For each point x transformed according to the reference pose, the operation $min_{\mathbf{y}\in M}$ identifies the closest point from the same object model after transformation by the estimated pose.

Therefore, $ADD-S$ measures the average distance between the object surface under the reference pose and the closest surface points under the estimated pose. Because it uses nearest-point matching, it can assign a low error to poses that are geometrically equivalent even when their rotations differ. This is useful for the moulded elements, where symmetry can produce different orientation estimates that still represent a valid physical alignment.

To express the surface error relative to the physical dimensions of each object, the object diameter d was calculated from the canonical mesh as:

$$d = \max_{\mathbf{x},\mathbf{y}\in M} \|\mathbf{x}-\mathbf{y}\|_2 \tag{4.18}$$

where d represents the maximum Euclidean distance between any two sampled points on the canonical object model. A registration was considered successful when the $ADD-S$ error was below 10% of the corresponding object diameter:

$$e_{ADD-S} < 0.1d \tag{4.19}$$

The use of a threshold relative to the object diameter follows established six-degree-of-freedom pose-estimation evaluation practice. PoseCNN, for example, evaluates pose correctness using an average model-point distance threshold corresponding to 10% of the three-dimensional model diameter and applies the closest-point ADD-S formulation when object symmetry makes direct point correspondence ambiguous [146].

The registration success rate was then calculated as:

$$SR_{\mathrm{reg}} = \frac{N_{\mathrm{successful}}}{N_{\mathrm{evaluated}}} \times 100 \tag{4.20}$$

where $N_{\mathrm{evaluated}}$ represents the total number of reference object observations included in the registration experiment.

Together, the three registration metrics provide complementary information about the estimated pose. Translation error quantifies positional accuracy, rotation error quantifies orientation accuracy and $ADD-S$ evaluates the resulting geometric alignment while accounting for object symmetry. The $ADD-S$ threshold additionally provides a common criterion from which the registration success rate was calculated. This combination allows numerical pose differences to be distinguished from geometrically valid registrations caused by symmetric or near-symmetric object geometry.

4.11.6.4 Treatment of target coverage

During development, an asymmetric overlap measure referred to as Intersection over Coverage (IoC) was considered as an alternative to IoU for complete-to-partial registration. The motivation was that the target scanner observation contains only the visible portion of the object, whereas the source representation contains the complete object geometry. Thus, penalising source regions that cannot be observed in the target can make conventional IoU difficult to interpret. The proposed coverage measure was:

$$IoC = \frac{|V_T \cap V_S|}{|V_T|} \tag{4.21}$$

where V_T represents the occupied target region and V_S represents the occupied region of the transformed source. When the target occupancy is treated as the reference positive set, however:

$$TP = |V_T \cap V_S| \tag{4.22}$$

and:

$$TP + FN = |V_T| \tag{4.23}$$

which gives:

$$IoC = \frac{TP}{TP + FN} = R \tag{4.24}$$

The proposed measure is mathematically equivalent to recall, as defined in Equation 4.9, under this occupancy representation. Therefore, it was not retained as an additional evaluation metric, avoiding the reporting of the same information under two different names. The asymmetric interpretation remains useful, as it expresses the proportion of the partial target observation explained by the transformed source; however, recall provides the established terminology for this quantity.

4.11.6.5 End-to-end success rate

The end-to-end experiment considered both segmentation and registration. A reference object was considered successfully processed when a correctly classified valid instance corresponding to the object was produced by PointGroup and the final pose obtained from DCP followed by ICP satisfied the registration success criterion defined above. The overall end-to-end success rate was calculated as:

$$SR = \frac{N_{\text{successfully segmented and registered}}}{N_{\text{reference objects}}} \times 100 \tag{4.25}$$

For the structured real-scanner dataset $N_{\text{reference objects}}$ was equal to 216 for each complete evaluation run. Objects that were not detected, were incorrectly classified, did not produce a valid instance, or failed to satisfy the final registration criterion remained in the denominator. This prevents registration accuracy from being artificially increased by excluding difficult objects that failed earlier in the perception pipeline.

4.11.6.6 Statistical reporting

For PointGroup and DCP-based experiments, results were first calculated independently for models trained using seeds 11, 22 and 33. For a metric x, the mean across the three independently trained models was calculated as:

$$\bar{x} = \frac{1}{S}\sum_{s=1}^{S} x_s \tag{4.26}$$

where S is equal to 3.

Variation between independently trained models was reported using the sample standard deviation:

$$\sigma_x = \sqrt{\frac{\sum_{s=1}^{S}(x_s - \bar{x})^2}{S-1}} \tag{4.27}$$

and repeated-model results were therefore reported in the form:

$$\bar{x} \pm \sigma_x \tag{4.28}$$

The mean and standard deviation describe variation between independently trained model instances rather than variation between individual point-cloud observations.

For registration, object-level translation and rotation errors can contain a small number of substantially larger failures. The mean alone may provide an incomplete representation of registration behaviour. Translation and rotation errors were consequently summarised using the mean, median and 90th percentile for each evaluation condition. The median represents typical performance, while the 90th percentile provides a compact indication of the upper error distribution without requiring a large number of additional statistics.

For DCP and DCP followed by ICP, these statistics were calculated independently for each trained seed before being aggregated across the three runs. Methods that did not involve independently trained models, including ICP, RANSAC followed by ICP and the pretrained GeDi pipeline, were reported using their fixed evaluation configuration and were not assigned artificial between-training-run standard deviations.

4.12 Online Validation Protocol

The online validation was conducted to assess the behaviour of the proposed perception methodology when integrated with the complete physical robotic system. While the offline experiments in Section 4.11 evaluate segmentation and registration under controlled conditions, the online experiment examines whether the selected perception pipeline can provide sufficiently reliable information for autonomous pick-and-place execution. Thus, the purpose of this stage was not to repeat the component-level benchmarking, but to demonstrate the operation of the complete sequence from physical sensing to robot manipulation.

The perception configuration selected from the offline evaluation was retained for the online experiment. PointGroup trained using the hybrid scene dataset was used for semantic and instance segmentation, followed by DCP and ICP for object-pose

estimation. The resulting object identity and validated six-degree-of-freedom pose were then passed to the task-execution layer, which generated the required grasp and placement targets. Robot motion was planned and collision-checked through MoveIt before being transmitted to the ABB controller through the ROS-Industrial interface. This connected the learning-based perception methodology with the physical robot, gripper, conveyor and scanner within a single operational pipeline.

The online experiment represents a system-level physical validation rather than an independent evaluation of each perception component. The system incorporated validation checks before allowing a detected object to proceed to physical manipulation. Perception outputs that did not satisfy the configured validity checks were rejected before a picking motion was attempted. In particular, registration quality was evaluated from the geometric agreement between the transformed object template and the observed instance. This was important because the estimated transformation directly determined the subsequent grasp target, and an unreliable pose could therefore result in an incorrect or unsafe picking motion. Objects passing the perception checks continued to grasp generation and motion planning, where failures could still occur because of trajectory feasibility or physical manipulation. All rejected and failed cases were retained as experimental outcomes rather than manually corrected.

The manipulation strategy was intentionally constrained to the requirements of the target application. The object geometry and identity were known in advance, while the position and orientation of each object within the conveyor workspace were determined online from the scanner measurements. The estimated object pose was used together with the configured grasp constraints and offsets to generate the required robot grasp target, after which the robot followed a staged pick-and-place sequence containing approach, grasp, retreat, transfer and placement motions. This model-based and pose-conditioned strategy is consistent with the use of prior object knowledge for industrial manipulation, where restricting the grasp family can provide a practical alternative to unconstrained grasp search when repeatable handling of known objects is required. The implementation of the robot-control layer and the corresponding staged execution sequence are described in Section 4.12.2.

A total of 120 physical pick-and-place trials were performed using different object configurations and arrangements within the conveyor workspace. Each trial exercised the complete online pipeline beginning with object detection and scan acquisition and ending with the physical placement attempt. Performance was recorded at successive stages of the process so that failures associated with perception, pose estimation, motion execution and final manipulation could be distinguished. As explained in Section 4.12.3, severely under-observed single-view object poses were intentionally avoided during the online trials because their effect had already been examined in the offline end-to-end evaluation. The experimental procedure and evaluation measures are described in Sections 4.12.3 and 4.12.4.

Figure 4.9 provides an overview of the integrated experimental environment used during the online validation. The physical robotic cell is shown from the front, the conveyor workspace is shown from above, and the corresponding RViz scene presents the digital counterpart used during planning and execution. Together, these views identify the principal components involved in the experiment, including the PhoXi scanner, ABB robot with the custom gripper, conveyor, placing table and placement tray.



Figure 4.9: Overview of the integrated robotic cell used during the online validation. The physical cell is shown from the front, the conveyor workspace is shown from above, and the corresponding RViz scene presents the digital representation used during planning and execution. The views highlight the main experimental components, including the PhoXi scanner, ABB robot with the custom gripper, conveyor, placing table and placement tray.

4.12.1 Integrated Online Pick-and-Place Pipeline

The online pick-and-place pipeline extended the execution structure established in Chapter 3 by replacing the original geometry-based perception stages with the segmentation and registration methodology developed in this chapter. The previously validated interfaces for conveyor control, scan acquisition, coordinate transformation, motion planning and ABB robot communication were retained. The principal change therefore occurred between point-cloud acquisition and robot-target generation, where PointGroup and the selected DCP with ICP registration configuration provided the object information required for manipulation. This allowed the perception methodology evaluated offline in Section 4.11 to be incorporated into the complete physical system while retaining the established planning and execution infrastructure.

Figure 4.10 summarises the resulting online sequence. The structure was adapted from the pick-and-place flowchart presented for the first experimental stage in Chapter 3. At system initialisation, the robotic cell components are returned to their initial state and the conveyor is activated. When an object is detected, the conveyor is stopped and the PhoXi scanner is triggered to acquire the point cloud. The acquired data then enter the point-cloud processing stage, where PointGroup performs instance segmentation and the corresponding object template is registered to each detected instance. The resulting registration is validated before manipulation is allowed to proceed. Invalid registrations are rejected, while valid registrations are used to generate the corresponding grasp and placement targets. MoveIt then performs motion planning before the robot picks the object and places it in its predefined slot. After each processed object, the system checks whether another object remains in the current scene. If so,

the sequence continues with the next object. Otherwise, the robotic cell returns to its initial state and the conveyor cycle restarts.

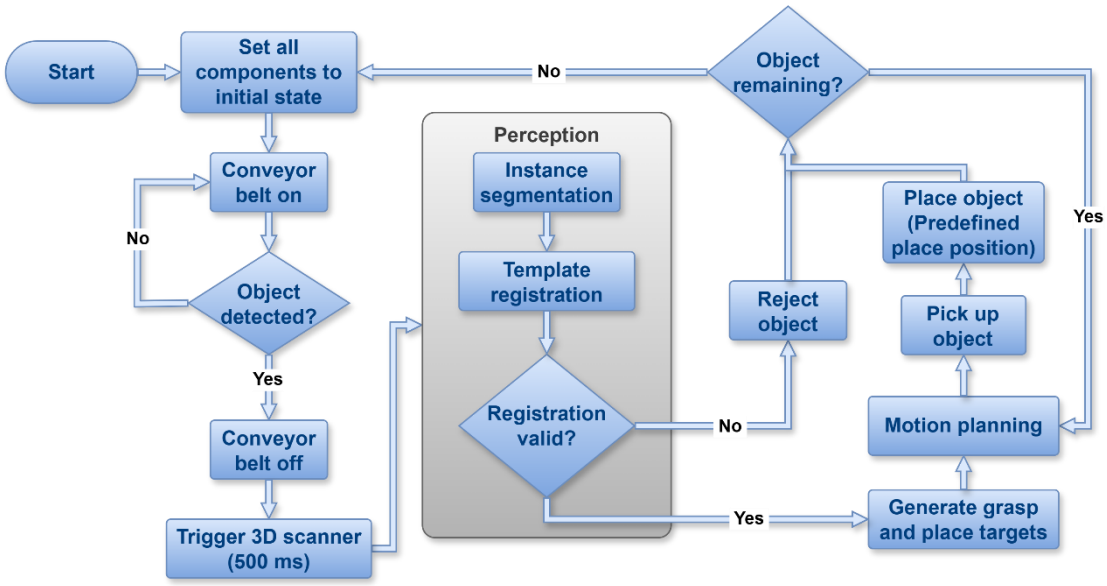


Figure 4.10: Flowchart of the integrated online pick-and-place sequence. After sensor-triggered point-cloud acquisition, the perception stage performs instance segmentation, template registration and registration validation. Valid objects proceed to grasp and placement target generation, motion planning and physical pick-and-place execution, while rejected objects are excluded from manipulation.

The acquired point cloud was first converted into the representation required by the perception pipeline. PointGroup then generated semantic predictions and separated the detected target points into object instances. Each accepted instance therefore contained both an individual observed point cloud and a predicted semantic class. The predicted class determined which of the three configured object templates was supplied to the subsequent registration stage. This differs from the first experimental stage, where object position and orientation were derived from centroid and principal-axis information and object identity was determined using manually defined geometric features. For each predicted instance, the corresponding complete object template was registered against the partial point-cloud observation. The DCP implementation produced the initial rigid transformation, which was subsequently refined using ICP. The quality of the resulting alignment was then assessed using the IoC measure. Because a reliable object pose was essential for the subsequent manipulation stage, the registration result was accepted for execution only when the IoC exceeded the configured threshold. Configuring this threshold within the user interface allowed the rejection criterion to be adjusted between tasks. The accepted transformation represented the estimated pose of the object within the common robotic-cell reference structure and provided the spatial information required by the subsequent manipulation stage.

The estimated object pose was subsequently converted into the grasp targets required by the robot-control procedure. Placement targets were obtained from the predefined positions associated with the corresponding object class on the placement tray. The manipulation layer therefore combined an online perception-derived pick pose with a predefined task-specific placement pose. MoveIt was used to determine feasible robot trajectories between the required task poses while considering the kinematic and collision constraints represented by the digital robotic cell. The resulting trajectories

were transmitted through the established ROS-Industrial and ABB communication interfaces for execution by the physical robot. The previous robotic architecture similarly separated application-level target generation from MoveIt motion planning and ABB execution.

This sequence is illustrated further in Figure 4.11, which shows representative stages of one online trial. Figure 4.11(a) presents the scene immediately after scan acquisition, with the physical cell shown above and the corresponding raw point cloud shown below in RViz. Figure 4.11(b) shows the subsequent perception output, where the detected object instances are separated and visualised in different colours. Figure 4.11(c) shows the robot after placing object AT3 in its assigned slot, while the corresponding RViz view presents the digital scene used by the execution layer. The remaining objects were then processed by repeating the same sequence until the conveyor workspace was cleared.

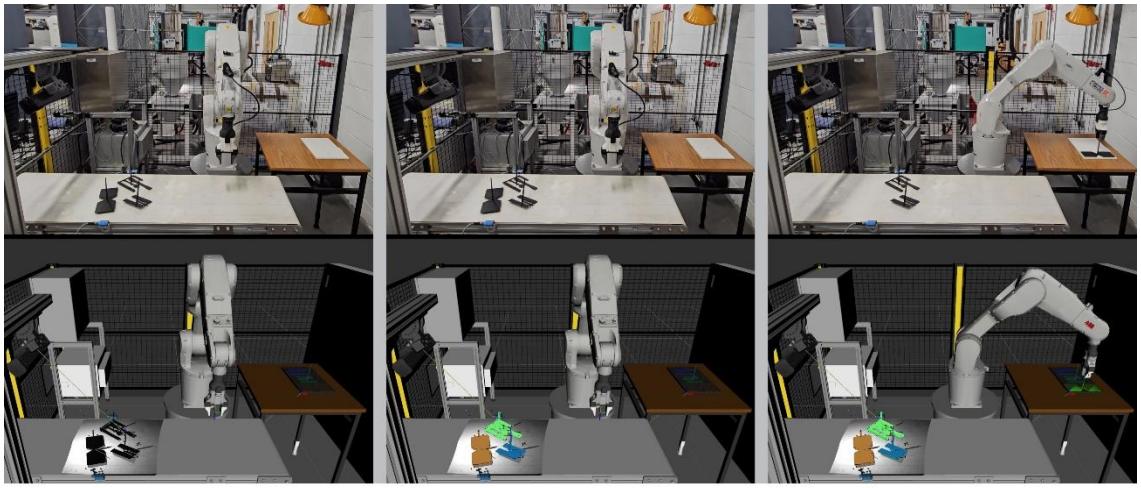


Figure 4.11: Representative stages of one online pick-and-place trial. (a) shows the scene after scan acquisition, including the physical view and the corresponding raw point cloud in RViz. (b) shows the perception result, where the detected object instances are separated and displayed in different colours. (c) shows the robot after placing object AT3 in its assigned slot, together with the corresponding digital scene in RViz.

The complete operation therefore formed a closed perception-to-action loop in which scan acquisition produced the input to segmentation, segmentation determined the object instance and semantic identity, registration determined its pose, and the resulting pose was converted into a physical manipulation target. After the selected object had been processed, the system continued with any remaining object instances before returning to the conveyor and acquisition stage. Importantly, perception and manipulation errors were not manually corrected during this process. Consequently, inaccuracies produced during segmentation or registration could propagate into grasp generation, trajectory planning and physical placement, allowing the online experiment to evaluate the behaviour of the complete integrated system rather than the perception algorithms in isolation.

4.12.2 Pose-Conditioned Grasp and Motion Execution

Once an object had passed the perception and registration validation stages, its estimated pose was used to generate the corresponding manipulation targets. The grasping method exploited the fact that the geometry of each target object was known in advance. Rather than searching for an unrestricted grasp directly from the measured

point cloud, a grasp was defined relative to the registered object frame and subsequently transformed into the robot planning frame. This provided a model-based and pose-conditioned grasping strategy in which the grasp followed changes in the detected position and orientation of the object.

The relationship between the registered object pose and the required grasp pose can be represented as:

$$T_{\text{grasp}} = T_{\text{object}} T_{\text{offset}} \tag{4.29}$$

where T_{object} represents the registered pose of the object after conversion to the robot planning frame, T_{offset} represents the configured translation and rotation between the object reference frame and the desired gripper tool centre point, and T_{grasp} represents the resulting robot grasp target. In this way, the object pose was determined online while the grasp relationship relative to the known object geometry remained task-specific and configurable.

Multiple candidate gripper orientations were generated around the registered object frame to increase the probability of finding a feasible manipulation pose. For the injection-moulded assemblies used in the online experiment, yaw offsets were evaluated at 30° intervals between −60° and 90°, including 0°. The gripper approach direction was defined relative to the registered object frame rather than to a fixed picking orientation in the conveyor workspace. Each candidate therefore preserved the estimated object pose while varying the gripper orientation available to the motion planner.

The picking motion was divided into separate approach and grasp stages. For the experimental configuration, a pre-grasp pose was generated 0.10 m from the final grasp position along the configured approach direction. The robot first planned a motion to this intermediate pose before approaching the object. The WiLi-Gripper was opened before the final approach and commanded to close once the grasp pose was reached. Contact feedback from the force-sensitive resistor was used during closing, as described in Appendix B. After grasping, the object was lifted by 0.12 m before the transfer motion began. The use of an intermediate approach and subsequent lift reduced the need to plan direct motions between the surrounding workspace and the object contact position.

The predicted semantic class determined the corresponding placement target configured for that object on the placement tray. Before reaching the final placement pose, the robot moved to a pre-place position located 0.10 m from the target along the configured placement approach direction. It then approached the final pose, opened the gripper to release the object and withdrew from the placement region. The resulting manipulation sequence therefore consisted of approach, grasp, lift, transfer, placement approach, release and retreat stages. This staged organisation follows the sequence-level manipulation principles discussed in Section 2.6.3, where feasibility must be considered across the complete pick-to-place operation rather than only at the grasp pose.

MoveIt provided the kinematic planning and collision-checking layer for the task poses generated by the robot-control procedure. For the experimental configuration, each planning request was allowed up to 5 s and five planning attempts, while velocity and acceleration scaling factors were both set to 0.3. Candidate grasp orientations could

therefore be evaluated against the constraints of the physical and digital cell before execution. If a feasible motion could not be obtained for the required manipulation sequence, the object was not picked, and the attempt was retained as a trajectory-related failure in the online evaluation.

The numerical values used for grasp orientation, approach distance, lifting distance, placement approach and MoveIt planning were not fixed properties of the proposed manipulation method. They were stored as task configuration parameters and could be modified through the user-guided configuration interface described earlier in this chapter. The values reported above correspond to the configuration selected for the injection-moulded assemblies and robotic cell used in the online experiment. Different object geometries, grippers or workspace arrangements may therefore require different values.

4.12.3 Experimental Procedure

The online experiment consisted of 120 physical pick-and-place trials using AT1, AT2 and AT3, as defined in Section 4.10.1. The trials were distributed equally across the three assembly types, resulting in 40 evaluated trials for each assembly type. The purpose was to evaluate whether the selected perception and manipulation pipeline could operate repeatedly when the target object and its pose varied between executions, rather than testing the system at a small number of predefined picking positions.

To introduce variation in the observed geometry, the parts were evaluated under two principal resting conditions. These were denoted as sprue top and sprue sideways, corresponding to different orientations of the injection-moulded part relative to the conveyor surface. The two resting orientations produced different visible geometries from the scanner viewpoint and introduced additional variation for both instance segmentation and template registration. Across the complete experiment, 60 trials were performed in each resting condition.

Within these conditions, the parts were positioned at varying locations and orientations within the valid conveyor scanning and picking region. Their Cartesian position and planar orientation were not prescribed for individual trials. This randomised presentation was used to avoid repeatedly evaluating the system at known robot coordinates and to examine whether the perception-derived object pose could support manipulation across changes in object presentation. The randomisation was nevertheless bounded by the physical operating region of the robotic cell, since the objects had to remain visible to the scanner and within the intended robotic workspace.

Each trial followed the complete online sequence described in Section 4.12.1, beginning with object presentation and scan acquisition and continuing through perception, registration validation and, where the configured validity conditions were satisfied, physical manipulation. Trials rejected by the perception pipeline were recorded as experimental outcomes and were not manually corrected or repeated.

For objects proceeding to manipulation, the validated pose was used with the grasp configuration described in Section 4.12.2 to generate the picking target. MoveIt then attempted to plan the required approach, grasp, transfer and placement motions. The physical outcome was observed throughout the execution so that failures occurring

after perception could be distinguished from failures in registration, trajectory generation, grasping or placement.

Placement accuracy was evaluated using a task-specific tray containing a geometrically matched slot for each of the three injection-moulded assemblies. The tray followed the same constrained placement principle used in the first experimental stage, but its slot geometry was redesigned for AT1, AT2 and AT3. As shown in Figure 4.12(a), each object was assigned a dedicated shaped slot, while Figure 4.12(b) shows the three parts positioned within their corresponding destinations. Each slot provided approximately 2 mm of clearance around the corresponding part. A placement was therefore considered successful only when the object was transferred to its correct destination and entered the corresponding shaped slot. An object resting on the tray or positioned close to the slot without fitting within this tolerance was not considered a successful placement. This provided a stricter physical criterion than simply verifying that the object had reached the general placement area.

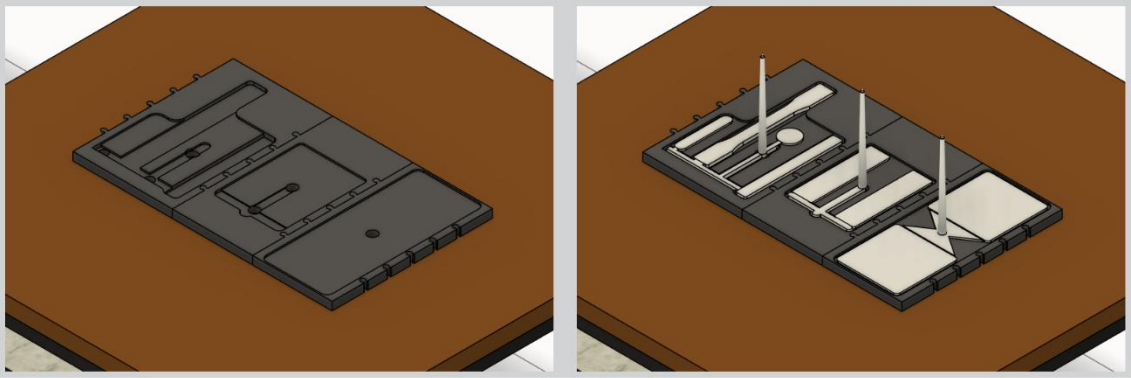


Figure 4.12: Placement tray used during the online experiment. (a) shows the empty tray containing the object-specific slots for AT1, AT2 and AT3, while (b) shows the three assemblies positioned within their corresponding destinations. Each slot provided approximately 2 mm of clearance around the corresponding object.

For every trial, the outcome was recorded progressively across the perception and physical execution stages. The recorded information distinguished whether the target object had been detected, whether its semantic class was correct, whether registration satisfied the required geometric agreement, and how far the subsequent physical pick-and-place operation progressed. This stage-wise recording allowed the final analysis to distinguish failures arising before robot execution from those caused by motion feasibility, grasp stability or placement accuracy. The definitions and quantitative measures used to summarise these outcomes are presented in Section 4.12.4.

4.12.4 Evaluation Metrics

The online evaluation was designed to measure both the performance of the perception stages and the outcome of the complete physical task. Unlike the offline validation in Section 4.11, which provides detailed segmentation and registration metrics, the online experiment focused on whether each trial progressed successfully through the successive stages required for physical manipulation. Therefore, the measures were organised according to target-object identification, semantic classification, registration validation and pick-and-place execution.

Target-object identification recorded whether the segmentation output contained a valid object instance corresponding to one of the three predefined target

parts. This measure was separated from semantic classification because detecting an object instance did not necessarily mean that it had been assigned to the correct object class. Correct classification was recorded when the predicted semantic class matched the physical object presented in the trial, corresponding to AT1, AT2 or AT3.

Registration performance was evaluated using the same IoC criterion employed by the online execution gate. A registration was classified as successful when:

$$\mathrm{IoC} > \tau_{\mathrm{IoC}} \tag{4.30}$$

where τ_{IoC} was set to 0.7 for the injection-moulded assemblies used in this experiment. The threshold was configurable through the user interface rather than fixed within the processing pipeline. Registrations below this value were rejected and did not proceed to physical manipulation. IoC therefore served both as an online acceptance criterion and as the registration success measure reported in the online evaluation.

For each of these successive stages, the pass rate was calculated as:

$$SR_k = \frac{N_{\mathrm{pass},k}}{N_{\mathrm{trials}}} \times 100 \tag{4.31}$$

where $N_{\mathrm{pass},k}$ is the number of trials satisfying the criterion for stage k, and N_{trials} is the total number of physical trials. Using the complete trial set as the denominator preserves the end-to-end character of the evaluation by showing how failures at earlier stages affect the number of trials available for subsequent manipulation.

For trials that progressed to robotic execution, the physical outcome was assigned to one of four categories:

1. **No feasible trajectory found** indicates that the perception stages produced an accepted object pose, but MoveIt could not obtain a suitable motion for the required picking sequence.
2. **Picked but incomplete** indicates that the object was successfully grasped but the manipulation sequence was not completed, for example because the object slipped during transfer.
3. **Picked but misplaced** indicates that the object was successfully grasped and transported to the placement region but did not enter the required shaped slot.
4. **Successfully placed** indicates full task completion, where the object was picked, transported and inserted into the correct tray slot within the physical clearance defined in Section 4.12.3.

The principal system-level measure was the overall pick-and-place success rate:

$$SR_{\mathrm{PnP}} = \frac{N_{\mathrm{placed}}}{N_{\mathrm{trials}}} \times 100 \tag{4.32}$$

where N_{placed} is the number of trials resulting in successful placement. This measure represents the strictest evaluation of the integrated system because successful completion depends on the preceding perception, registration, trajectory-planning, grasping, transfer and placement stages.

In addition to the individual stage outcomes, a weighted task-progress score was recorded as a secondary descriptive measure. The score was introduced to

represent how far each trial progressed through the complete pipeline rather than treating all unsuccessful trials as equivalent. It was calculated as:

$$S = 0.2I + 0.2C + 0.2A + 0.4(0.2T + 0.4U + 0.6M + P) \quad 4.33$$

where I, C and A are binary indicators for successful target-object identification, correct semantic classification and acceptable registration, respectively. A value of 1 indicates that the corresponding perception stage was successfully completed, while 0 indicates failure. The variables T, U, M and P represent the terminal physical-execution outcomes of no feasible trajectory, incomplete execution after picking, misplaced object and successful placement, respectively.

The score assigns 60% of its total value to the three perception stages and 40% to physical pick-and-place execution. This weighting reflects the emphasis of the experiment, where the main methodological contribution concerns the perception pipeline used to identify, classify and estimate the pose of the target parts. The physical manipulation stage instead provides system-level validation of whether these outputs can support successful robotic execution. Equal weights of 0.2 were therefore assigned to identification, classification and registration.

Within the physical-execution component, T, U, M and P are binary and mutually exclusive outcome indicators. At most one can take the value 1 in a trial, while trials rejected before physical execution have all four set to 0. Their weights increase according to the amount of the manipulation sequence successfully completed. A trajectory failure receives a relative weight of 0.2, an incomplete execution after picking receives 0.4, a misplaced object receives 0.6, and successful placement receives 1.0. These stages are sequential, since incomplete execution can only occur after the system has progressed beyond trajectory generation, while misplacement requires successful picking and transfer to the placement region. After application of the 0.4 execution weighting, the four outcomes contribute 0.08, 0.16, 0.24 and 0.40 to the final score, respectively.

The maximum score is 1.0, obtained when all three perception stages are successful and the object is correctly placed. Since the weighting scheme was defined specifically for the objectives of this experiment rather than derived from a standard robotic performance metric, the score was treated as a supplementary task-progress measure. The primary evaluation remained the individual perception-stage pass rates, physical failure categories and overall pick-and-place success rate.

4.13 Results and Discussion

This section presents and discusses the results obtained during the second experimental stage. The analysis begins with the training behaviour of the learning-based segmentation and registration models. It then evaluates the perception methodology offline, first considering instance segmentation and domain transfer, followed by the comparison of the registration configurations and the selected end-to-end perception pipeline. Finally, the online results examine the behaviour of the selected perception and manipulation configuration when integrated with the physical robotic cell.

The results are presented progressively so that the behaviour of the individual perception components can be distinguished from that of the integrated system. This

distinction is important because a failure observed during end-to-end operation may originate from scene acquisition, instance segmentation, pose registration, trajectory generation or physical manipulation. The component-level experiments therefore provide the basis for interpreting the subsequent end-to-end and online results.

4.13.1 Model Training

The learning-based components evaluated in this chapter were trained independently using the three random seeds defined in Section 4.11.1. PointGroup was trained separately using the fully synthetic and hybrid scene datasets, while the adapted DCP model was trained using the combined object-level registration dataset. The training curves were examined to assess the consistency of the optimisation behaviour across the independent runs and to identify any substantial divergence between the training and validation data before evaluating the resulting checkpoints on the offline datasets.

Figure 4.13 presents the training and validation behaviour of PointGroup using the fully synthetic scene dataset. The three runs followed similar optimisation trends. Semantic loss decreased rapidly during the initial training period and continued to reduce throughout the remaining epochs, with the corresponding validation curves following a similar pattern. By the later stages of training, the semantic loss had reached a low and relatively stable level for all three seeds.

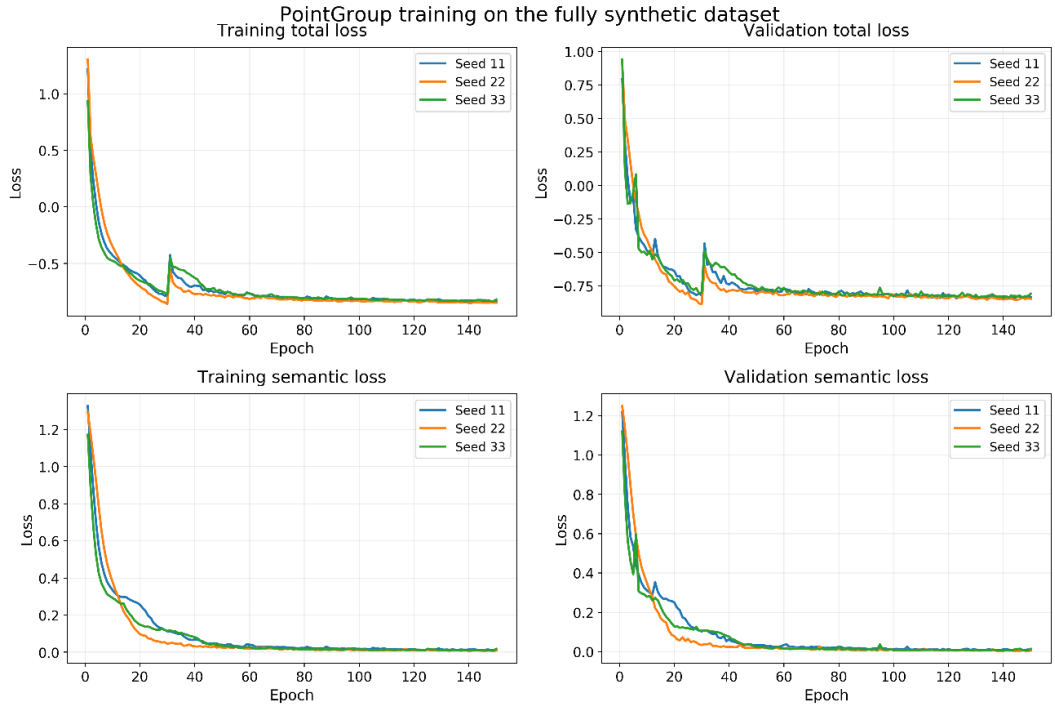


Figure 4.13: PointGroup training and validation behaviour using the fully synthetic scene dataset for random seeds 11, 22 and 33. The figure shows the total optimisation objective and semantic-loss component over 150 epochs. The proposal-scoring component is introduced after the initial 30-epoch preparation stage. Negative values of the total objective result from the directional point-offset component of the PointGroup loss formulation.

The total PointGroup objective becomes negative during the later stages of training. This behaviour results from the formulation of the point-offset direction component and does not indicate an error in optimisation. The directional term is based on the negative similarity between the normalised predicted and reference offset vectors. As the predicted direction becomes increasingly aligned with the reference direction, this component approaches −1. Once the positive semantic and offset-

magnitude losses have decreased sufficiently, the negative directional contribution can therefore cause the combined objective to take negative values. The semantic-loss curves remain positive and continue to decrease towards zero, providing a more direct indication of the point-level classification behaviour.

A further change in the total-loss curves is visible after approximately 30 epochs. This corresponds to the PointGroup training procedure rather than instability in optimisation. During the initial preparation period, optimisation is performed without the proposal-scoring component. After this stage, the proposal score loss is activated and contributes to the total objective. The semantic-loss curves remain continuous through this transition and continue to decrease, indicating that semantic optimisation remained stable while the additional instance-level objective was introduced.

The fully synthetic runs show no sustained divergence between training and validation semantic loss. The similarity between the three independent runs also indicates that the generated scene representation could be learned consistently under different model initialisations. However, the training curves alone do not indicate whether this representation transfers successfully to measurements obtained from the physical scanner. This question is examined directly in Section 4.13.2.1 using the independent real-scanner validation dataset.

Figure 4.14 presents the corresponding PointGroup behaviour when trained using the hybrid scene dataset. A similar overall optimisation pattern was observed. Training and validation semantic loss decreased substantially during the first part of training and subsequently approached a low and stable level. The three seeds remained close throughout most of the optimisation process, with only small variations during the earlier epochs.

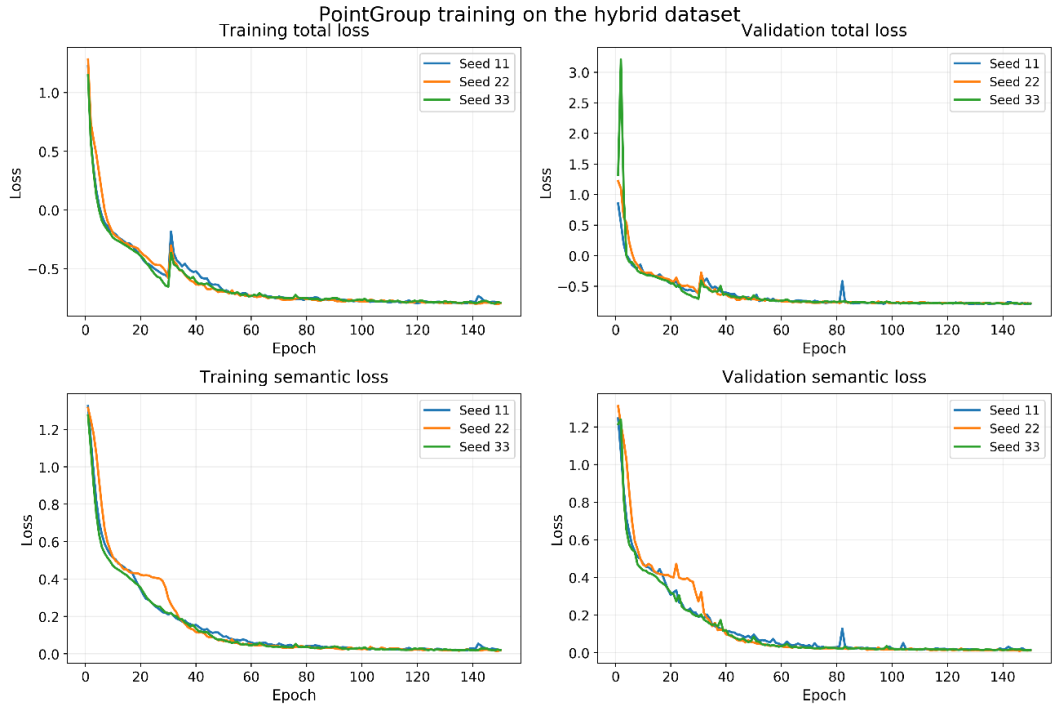


Figure 4.14: PointGroup training and validation behaviour using the hybrid scene dataset for random seeds 11, 22 and 33. The figure shows the total optimisation objective and semantic-loss component over 150 epochs. The proposal-scoring component is introduced after the initial 30-epoch preparation stage. Negative values of the total objective result from the directional point-offset component of the PointGroup loss formulation.

As with the fully synthetic condition, the negative total loss observed during later hybrid training results from the directional point-offset component of the optimisation objective. The corresponding semantic losses remain positive and converge towards low values. The similarity between the training and validation trends provides no indication of substantial divergence between the two data partitions during optimisation.

The repeatability of the hybrid training behaviour across the three seeds is particularly relevant to the subsequent domain-transfer evaluation. Differences between the fully synthetic and hybrid conditions on the real-scanner dataset can therefore be examined across independently trained models rather than being based on a single favourable training run. The real-scanner data were not used to select any of these models. Each checkpoint used in the subsequent evaluation was selected according to the corresponding generated validation data, as defined in Section 4.11.1.

The optimisation behaviour of the adapted DCP model is presented in Figure 4.15. The training NLL decreased substantially during the initial epochs for all three runs before gradually approaching stable values. A similar trend was observed for the validation data, with the validation curves remaining close to their corresponding training curves throughout optimisation.

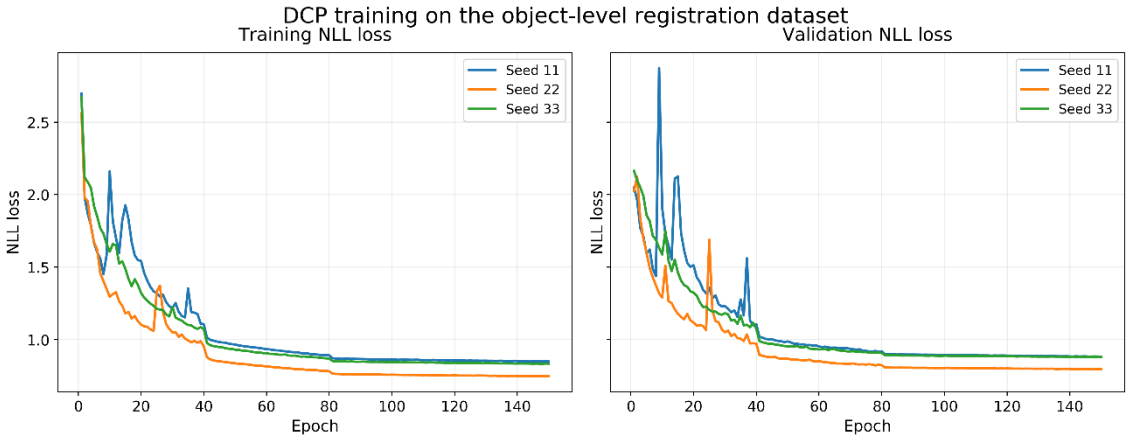


Figure 4.15: Training and validation behaviour of the adapted DCP model using the combined object-level registration dataset for random seeds 11, 22 and 33. The curves show the NLL correspondence loss over 150 epochs.

Differences were observed between the independently trained DCP models. Seed 22 converged to a lower NLL value than seeds 11 and 33, while the latter two reached similar final values. Nevertheless, all three runs exhibited the same overall convergence pattern and no sustained increase in validation loss was observed during the later stages of optimisation. This indicates that the training procedure remained stable across the three initialisations without clear evidence of increasing divergence between the training and validation subsets.

The difference in final NLL between the runs was retained in the subsequent evaluation rather than selecting a model according to its performance on the real-scanner dataset. The checkpoint for each seed was selected using its corresponding generated validation subset, and no published DCP weights were used to initialise the models included in the final evaluation. The registration experiments presented in

Section 4.13.2.2 therefore determine whether the differences observed during optimisation translated into differences in final pose-estimation performance.

Overall, the PointGroup and DCP training curves show stable optimisation across the independently trained models. The training analysis is not used as evidence of real-world perception performance, since successful optimisation on generated data does not necessarily imply successful transfer to physical measurements. Instead, it establishes that the models reached suitable states for subsequent evaluation while preserving the separation between model development and the independent real-scanner experiments. The following sections therefore focus on whether the trained models generalised beyond their generated training and validation data.

4.13.2 Offline

The offline results were analysed according to the three evaluation stages defined in Section 4.11. The first analysis examined the transfer of PointGroup from generated training data to real scanner measurements. The second isolated the registration stage using reference object instances. Then, the final analysis combined the selected segmentation and registration methods to evaluate the complete perception sequence.

4.13.2.1 Instance Segmentation and Domain Transfer

The first offline experiment examined how the PointGroup models performed within the generated-data domain and how this performance transferred to measurements acquired from the physical robotic cell. Particular attention was given to the comparison between fully synthetic and hybrid training because the two conditions used the same target classes and object-generation procedure, while differing principally in their representation of the surrounding environment. The results aggregated across the three independently trained models are presented in Table 4.6.

Table 4.6: PointGroup segmentation performance across the generated and real evaluation datasets. Results are reported as mean ± standard deviation across seeds 11, 22 and 33.

Training condition	Evaluation dataset	mIoU (%)	Macro F1 (%)	mAP25 (%)	mAP50 (%)
Fully synthetic	Synthetic test	98.05 ± 0.33	99.01 ± 0.17	93.66 ± 1.14	93.66 ± 1.14
Fully synthetic	Hybrid test	63.85 ± 3.57	75.19 ± 3.64	73.60 ± 4.11	68.35 ± 7.24
Fully synthetic	Real scanner	39.95 ± 4.00	53.71 ± 3.44	54.27 ± 3.50	49.28 ± 3.74
Hybrid	Hybrid test	97.09 ± 0.32	98.52 ± 0.16	91.30 ± 1.70	90.56 ± 1.62
Hybrid	Real scanner	81.14 ± 1.92	89.53 ± 1.16	74.08 ± 0.26	73.92 ± 0.25

Both training conditions achieved high performance when evaluated within their corresponding generated domain. The fully synthetic models achieved an mIoU of 98.05% on the synthetic test data, while the hybrid models achieved 97.09% on the hybrid test data. The corresponding macro F1 scores were above 98% in both cases. These results show that PointGroup could learn the three target assembly classes effectively from either generated dataset. The main difference between the training strategies only became apparent when the models were evaluated outside their original training domain.

The fully synthetic models showed a substantial reduction when evaluated on hybrid scenes. Their mIoU decreased from 98.05% on the synthetic test set to 63.85% on the hybrid test set. This comparison is particularly informative because the hybrid scenes continued to use generated target objects but introduced the measured physical background. Thus, the result indicates that changing the representation of the surrounding environment alone introduced a considerable domain difference for the synthetic-trained model.

The effect became stronger when the same models were evaluated on the independent real-scanner dataset. The fully synthetic models obtained an mIoU of 39.95%, a macro F1 score of 53.71% and an mAP50 of 49.28%. This contrasts with their strong performance on generated synthetic scenes and demonstrates that high performance within the simulated domain did not directly translate to physical scanner measurements. This behaviour is consistent with the sim-to-real problem discussed in Section 2.7, where differences in background geometry, point density, missing surfaces, visibility and sensor characteristics can cause models trained on generated data to perform considerably worse after deployment.

The hybrid-trained models transferred substantially better to the same 72 real-scanner scenes. Their real-data mIoU reached 81.14%, compared with 39.95% for the fully synthetic models. This represents an absolute improvement of 41.19 percentage points. The improvement was also present at the instance level, where mAP50 increased from 49.28% to 73.92%. Importantly, both training conditions were evaluated using exactly the same real scenes and reference annotations, so these differences cannot be attributed to changes in the evaluation data.

The result supports the motivation for incorporating measured environmental information into the generated dataset. Previous work has shown that synthetic point clouds can support sim-to-real transfer in industrial perception, while also emphasising the need to process generated observations so that they better represent the conditions encountered in the physical environment [101]. More general sim-to-real approaches such as domain randomisation similarly seek to reduce dependence on idealised simulation by exposing learning models to greater variation during training [102], [103]. The approach evaluated here follows a complementary direction by incorporating information measured directly from the target robotic cell into the generated scenes. A direct numerical comparison with these studies would not be meaningful because the applications, datasets and model architectures differ, but the present results support the same general conclusion that the characteristics of the generated training domain strongly influence subsequent real-world performance.

The effect of the measured background can be examined more directly through the segmentation of the background class. Although background was not included in the headline object mIoU, it provides useful diagnostic information about the transfer behaviour. On the real-scanner data, the synthetic-trained models achieved a background precision of 98.71%, but background recall was only 51.90%. Therefore, points classified as background were generally correct, but a large proportion of the actual physical background was not recognised as such and was instead absorbed into the predicted target classes. The hybrid-trained models increased background recall to 97.83% while maintaining a precision of 99.34%.

Figure 4.16 illustrates this effect using one structured real-scanner scene captured in resting configuration 1 at a rotation of 210°. Figure 4.16(a) shows the original scanner observation, while Figure 4.16(b) shows the corresponding reference annotation. Figure 4.16(c) presents the prediction produced by the PointGroup model trained on the fully synthetic scene dataset, and Figure 4.16(d) presents the prediction produced by the PointGroup model trained on the hybrid scene dataset. The comparison shows that both models identify the principal object regions, but the hybrid-trained model preserves a cleaner separation between object and background points. In contrast, the synthetic-trained model absorbs larger regions of the supporting surface into the predicted object classes. This visual result is consistent with the quantitative results presented in Table 4.6 and with the background-class analysis discussed above, where the hybrid-trained model achieved substantially stronger performance on the real-scanner data.

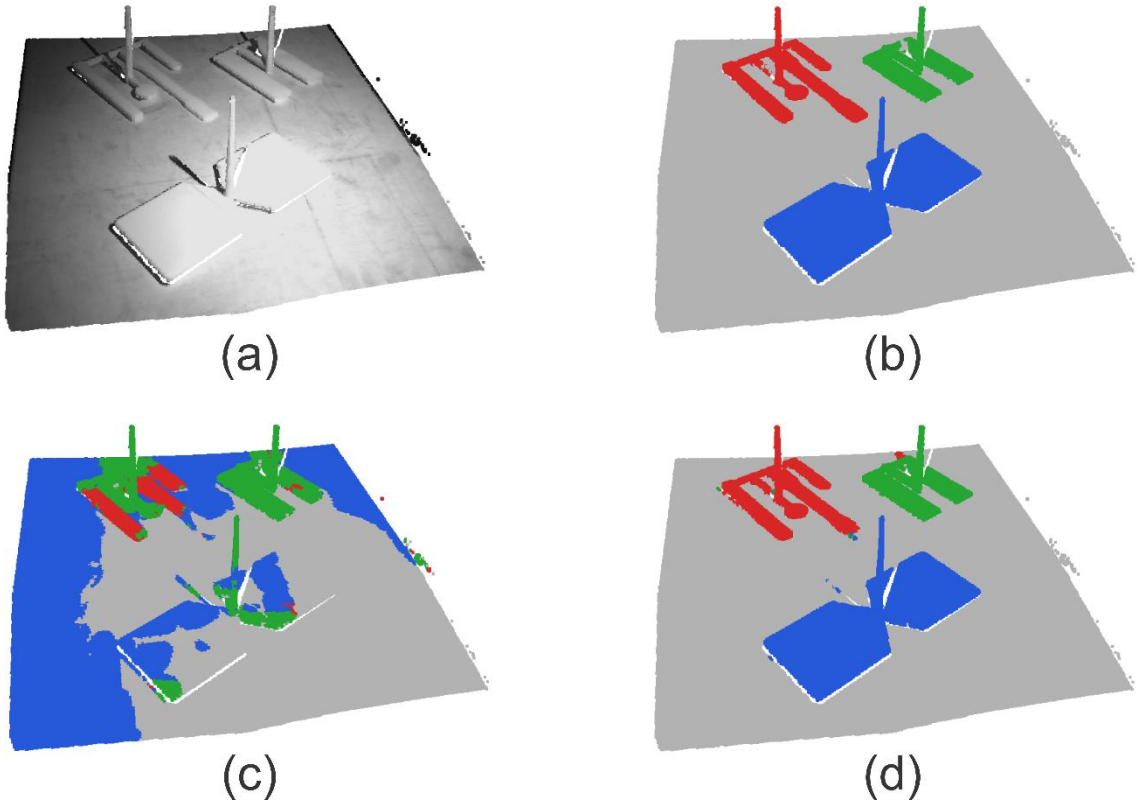


Figure 4.16: Example real-scanner scene used to compare synthetic-trained and hybrid-trained PointGroup predictions. (a) shows the original scanner observation from resting configuration 1 at a rotation of 210°. (b) shows the corresponding reference semantic and instance annotation. (c) shows the prediction produced by the PointGroup model trained on the fully synthetic scene dataset. (d) shows the prediction produced by the PointGroup model trained on the hybrid scene dataset.

The improvement was also observed across all three assembly classes rather than being dominated by a single object type. With hybrid training, the real-scanner IoU values for AT1, AT2 and AT3 were 81.74%, 77.39% and 84.29%, respectively. Their corresponding AP50 values were 75.72%, 71.06% and 74.99%. AT2 produced slightly lower results, but the three classes remained within a relatively narrow range. In contrast, the fully synthetic models produced considerably lower and more variable performance across the three assembly types. The hybrid approach improved both the overall level of real-world segmentation and the consistency of the results across the different target geometries.

The remaining difference between semantic and instance performance is also important. The hybrid-trained models achieved 81.14% semantic mIoU on the real scenes, while mAP50 was 73.92%. At an instance IoU threshold of 0.5, precision remained high for all three assembly types, ranging from approximately 93.5% to 99.4%. Instance recall was lower, ranging from approximately 72.2% to 75.9%. This indicates that the main remaining instance-level limitation was not the generation of a large number of incorrect object proposals. Instead, some physical objects did not produce a sufficiently complete or coherent instance prediction.

The structured acquisition procedure helps explain where these missing instances occurred. The three resting configurations exposed different amounts of object geometry to the single scanner viewpoint. For the hybrid-trained models, RC1 produced the strongest real-scanner performance, with an mIoU of 87.74% and an instance recall of 95.37%. RC3 produced an mIoU of 79.72% and an instance recall of 75.93%. RC2 was the most difficult condition. Its semantic mIoU remained 73.32%, but instance recall decreased to 51.85%.

Figure 4.17 provides representative examples of the three resting configurations used during structured real-scanner validation. The examples correspond to RC1 at a rotation of 165°, RC2 at a rotation of 240°, and RC3 at a rotation of 120°. Although all three scenes contain the same total number of acquired scene points, the number of valid object points differs substantially between them. RC1 contains about 1.38 million valid object points in RC1, 28.3 thousand in RC2 and 153.4 thousand in RC3. This comparison helps explain the stronger segmentation performance observed for RC1, and the weaker performance observed for RC2 and RC3. In these more challenging poses, the single scanner view captures a much smaller proportion of the object surfaces, which reduces the geometric evidence available to the segmentation model.

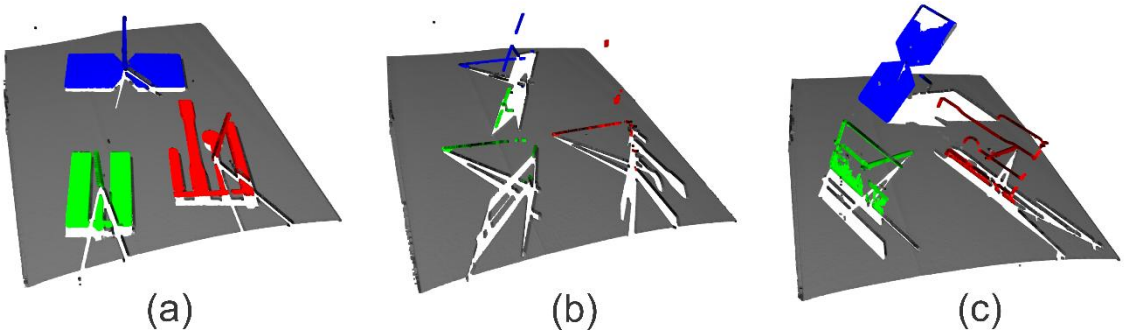


Figure 4.17: Effect of resting configuration on the geometry available to the segmentation model. Representative real-scanner observations are shown for RC1, RC2 and RC3, with the number of visible target points indicated for each example. Changes in resting configuration and orientation alter the amount of object surface visible from the fixed scanner viewpoint and can produce substantially more partial observations.

The difference between semantic and instance behaviour under RC2 is informative. Even when the visible points retain enough information for many of them to receive the correct semantic class, the observed geometry may be too incomplete to form a sufficiently complete object instance. The effect is caused by the combination of object shape, resting configuration and the fixed scanner viewpoint. At certain orientations, substantial portions of an assembly are self-occluded, leaving only a limited part of its geometry in the acquired point cloud. The segmentation model cannot recover geometric information that is absent from its input.

This limitation should be interpreted as a constraint of the complete single-view perception arrangement rather than solely as a limitation of the PointGroup architecture. It also suggests a practical direction for extending the system. Multiple scanner viewpoints could provide complementary observations of surfaces that are hidden from a single acquisition position. The resulting point clouds could be transformed into a common reference frame and fused before being supplied to the existing segmentation stage. The modular perception architecture developed in this work is compatible with such an extension because the downstream processing operates on the resulting scene point cloud rather than requiring a particular acquisition arrangement. Multi-view acquisition and point-cloud fusion were not evaluated in the present experiment and remain a direction for future work.

Overall, the segmentation results demonstrate two separate effects. First, the large difference between fully synthetic and hybrid training on the same real-scanner dataset confirms that incorporating measured environmental information substantially reduces the domain mismatch encountered by PointGroup. Second, the remaining failures are concentrated around difficult single-view observations where limited visible geometry reduces instance recovery. Therefore, the hybrid dataset addresses an important component of the sim-to-real problem, but it does not remove the physical visibility constraints imposed by the sensing configuration. The consequences of these missing or incomplete instances for the complete perception pipeline are examined later in Section 4.13.2.3.

4.13.2.2 Registration Performance

The second offline experiment evaluated registration independently from segmentation using the reference object instances extracted from the 72 real-scanner scenes. This allowed the five registration configurations to be compared using the same 216 partial object observations without errors from PointGroup influencing the registration input. The principal results are presented in Table 4.7 using the ADD-S measure and the corresponding registration success rate.

Table 4.7: Registration success and ADD-S performance on the 216 reference object observations. DCP and DCP + ICP results are reported as mean ± standard deviation across seeds 11, 22 and 33.

Registration method	Success rate (%)	ADD-S median (mm)	ADD-S P90 (mm)
ICP	38.89	21.63	433.22
DCP	87.19 ± 3.71	2.51 ± 0.33	19.78 ± 3.46
DCP + ICP	97.84 ± 0.53	1.41 ± 0.02	7.09 ± 2.67
RANSAC + ICP	29.17	42.11	378.38
GeDi + RANSAC + ICP	94.44	1.42	12.86

The strongest overall performance was obtained using DCP followed by ICP. This configuration achieved a registration success rate of 97.84 ± 0.53%, compared with 87.19 ± 3.71% for DCP alone. The median ADD-S error was reduced from 2.51 mm to 1.41 mm, while the 90th percentile decreased from 19.78 mm to 7.09 mm. The improvement therefore occurred both in the proportion of successful registrations and in the geometric accuracy of the resulting alignments.

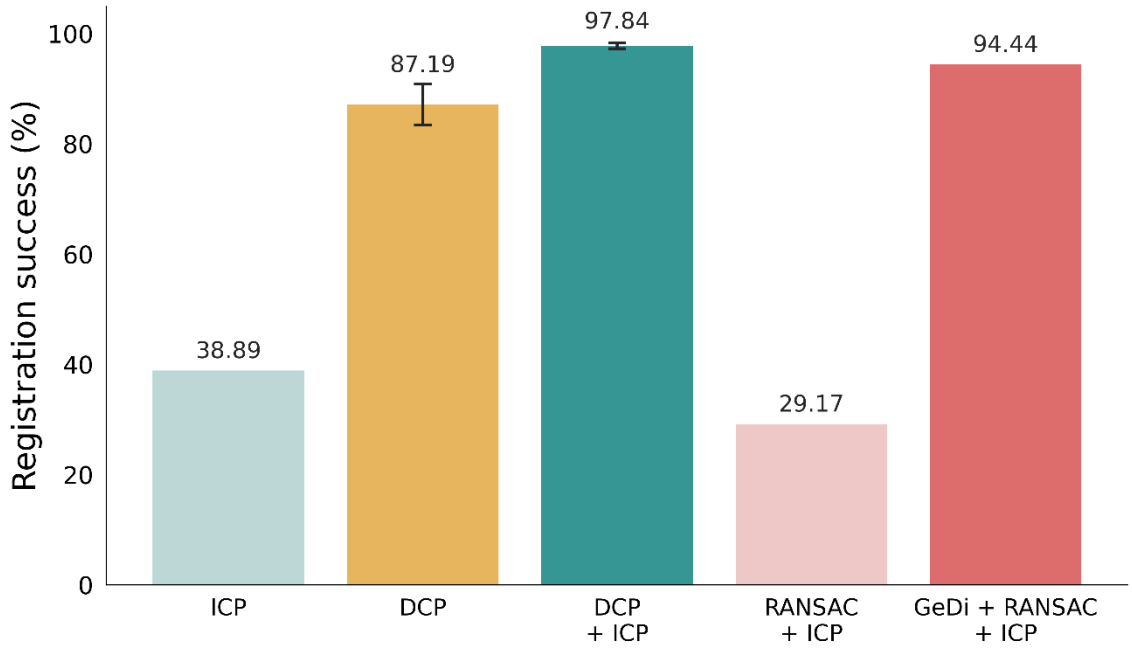


Figure 4.18: Registration success rate for the five configurations evaluated using the 216 reference object observations. DCP and DCP + ICP show the mean across models trained using seeds 11, 22 and 33, with error bars representing the standard deviation between the three runs. The remaining configurations were evaluated using a single fixed implementation.

The contribution of ICP can be examined directly because the DCP and DCP followed by ICP configurations used the same initial transformation before refinement. Across the three independently trained DCP models, the same 216 reference objects produced 648 checkpoint-object evaluations. ICP reduced ADD-S error in 582 of these cases and converted 69 registrations that failed the DCP-only success criterion into successful registrations. None of the registrations classified as successful using DCP became unsuccessful after ICP refinement.

This provides direct evidence for the coarse-to-fine registration strategy adopted in the final pipeline. DCP provides the broader learned transformation required to approach the correct object pose, while ICP subsequently performs local geometric refinement. This behaviour is consistent with the role of ICP described by *Besl and McKay*, where accurate local convergence depends on a sufficiently suitable initial alignment [67]. It also agrees with the original DCP formulation, where learned correspondence estimation can be followed by ICP to improve the final alignment rather than replacing classical geometric optimisation entirely.

The direct translation and rotation errors are presented separately in Table 4.8. Median and 90th percentile values are used in the table because several configurations produced strongly skewed error distributions. The complete analysis also considered the mean, as defined in Section 4.11.6, but the median provides a clearer indication of typical behaviour while the 90th percentile represents the upper part of the error distribution without being dominated by a small number of extreme failures.

The translation and rotation errors further demonstrate the effect of ICP refinement on the DCP output. Median translation error decreased from 7.32 mm with DCP to 2.91 mm after ICP refinement. Median rotation error similarly decreased from 4.92° to 1.84°. The 90th percentile translation error was also reduced from 35.51 mm to 19.85 mm. These improvements show that ICP not only recovered additional failed

registrations but also increased the accuracy of transformations that were already close to the correct alignment.

Table 4.8: Translation and rotation errors for the evaluated registration configurations. DCP and DCP + ICP results are reported as mean ± standard deviation across seeds 11, 22 and 33.

Registration method	Translation median (mm)	Translation P90 (mm)	Rotation median (°)	Rotation P90 (°)
ICP	39.52	480.49	140.28	178.96
DCP	7.32 ± 1.46	35.51 ± 3.79	4.92 ± 1.00	175.74 ± 1.56
DCP + ICP	2.91 ± 0.04	19.85 ± 1.67	1.84 ± 0.18	179.66 ± 0.07
RANSAC + ICP	97.15	456.90	133.20	179.18
GeDi + RANSAC + ICP	2.77	19.97	1.86	179.79

Standalone DCP nevertheless produced a strong result. Its registration success rate of 87.19% was considerably higher than those obtained using ICP alone or the evaluated RANSAC followed by ICP configuration. This indicates that the learned model was able to recover an appropriate global transformation for the majority of complete-to-partial registration cases before geometric refinement. This is particularly relevant to the present task because the source template represents the complete object while the scanner contains only the surfaces visible from a single viewpoint. The adapted correspondence-training procedure used in this work was intended to improve DCP under this type of partial observation and wide rotational variation, which are recognised challenges for learning-based registration.

GeDi followed by RANSAC and ICP also produced strong registration performance. It achieved a success rate of 94.44%, with a median translation error of 2.77 mm, a median rotation error of 1.86° and a median ADD-S error of 1.42 mm. These typical errors are very close to those obtained using DCP followed by ICP. However, the GeDi-based configuration produced a lower overall success rate and a higher 90th percentile ADD-S error of 12.86 mm compared with 7.09 mm for DCP followed by ICP.

The GeDi result is particularly relevant because the model was evaluated using its published pretrained descriptor rather than being retrained using the generated object-level dataset. GeDi was designed to provide local geometric descriptors that remain robust under changes in rotation, occlusion and acquisition domain, making it suitable for correspondence estimation between CAD-derived templates and partial scanner observations. The result therefore demonstrates that a strong registration solution can also be obtained without task-specific learning.

DCP and GeDi should not, however, be interpreted as equivalent training strategies. DCP was trained specifically using the object-level dataset generated for the target assemblies, while GeDi was used as part of a pretrained descriptor, RANSAC and ICP pipeline. The comparison therefore evaluates the complete registration configurations rather than isolating the contribution of the GeDi descriptor itself. Within this comparison, the task-specific DCP followed by ICP pipeline provided the strongest overall combination of registration success and upper-tail geometric accuracy, while the GeDi-based configuration provided a competitive alternative without additional training.

In contrast, ICP and RANSAC followed by ICP produced substantially lower registration success rates of 38.89% and 29.17%, respectively. Both configurations also exhibited strongly skewed error distributions. ICP produced a median translation error of 39.52 mm, while RANSAC followed by ICP produced 97.15 mm. A smaller number of severely incorrect transformations increased the corresponding mean translation errors to more than 2 m for both methods. These cases demonstrate why the mean alone provides an incomplete description of registration behaviour and support the use of the median and 90th percentile alongside it.

The poor performance of ICP alone is consistent with its dependence on a suitable initial alignment. Without a dedicated global registration stage, the optimisation may converge towards an incorrect local solution when the initial relative pose is too far from the target alignment. The RANSAC followed by ICP configuration attempted to address this through correspondence-based global alignment, but the evaluated geometric correspondences were not sufficiently reliable for many of the partial scanner observations. In comparison, both learned configurations provided substantially stronger initial correspondence information before geometric refinement.

The direct rotation errors require additional consideration because the target assemblies contain approximate geometric symmetries. This was particularly apparent for AT3. DCP followed by ICP frequently produced transformations whose rotation differed from the recorded reference by approximately 180°, even though the transformed object surface remained closely aligned with the scanner observation. Consequently, the 90th percentile rotation errors in Table 4.8 are high even for the strongest registration configurations.

This behaviour does not necessarily represent incorrect physical registration. For geometrically symmetric or approximately symmetric objects, multiple rotations can produce nearly equivalent surface alignments. The low ADD-S errors associated with these cases demonstrate why the symmetry-aware geometric measure was retained alongside direct rotation error. Rotation error describes the difference from one recorded reference orientation, while ADD-S better reflects whether the transformed object geometry occupies the correct physical region.

The object-level success rates also show that DCP followed by ICP remained consistent across the three assembly geometries. Registration success reached 99.07 ± 1.60% for AT1, 97.22 ± 1.39% for AT2 and 97.22 ± 2.41% for AT3. In comparison, standalone DCP decreased from 93.06% for AT1 to 80.56% for AT3. ICP refinement therefore reduced the differences between the object geometries in addition to improving the overall registration result.

A similar result was observed across the three resting configurations. DCP followed by ICP achieved 100% registration success for RC1, 94.44% for RC2 and 99.07% for RC3. This is particularly relevant when compared with the segmentation results in Section 4.13.2.1, where RC2 produced substantially lower instance recall. When the correct reference instance was supplied directly to registration, the selected DCP followed by ICP configuration remained successful for the large majority of RC2 observations. The difficulty associated with this resting configuration therefore appears to originate mainly before registration, where the fixed scanner viewpoint provides insufficient geometry for some object instances.

Overall, the registration experiment supports the use of a learned global estimate followed by local geometric refinement. DCP provided a substantially stronger initial transformation than the evaluated classical alternatives, while ICP further improved its accuracy and reliability across seeds, object types and resting configurations. GeDi followed by RANSAC and ICP demonstrated that a pretrained descriptor-based pipeline can also provide strong registration without task-specific training. However, DCP followed by ICP achieved the highest overall registration success and the lowest 90th percentile ADD-S error. This configuration was therefore retained for the end-to-end offline experiment and the subsequent physical robotic validation.

4.13.2.3 End-to-End Perception Performance

The final offline experiment evaluated the selected perception sequence using the hybrid-trained PointGroup model followed by DCP and ICP. In contrast to the independent registration experiment, the reference object instances were not supplied to the registration stage. Instead, DCP received the object instances predicted directly by PointGroup, allowing segmentation errors to propagate naturally through the complete perception pipeline. The three PointGroup and DCP models were paired according to their corresponding random seeds, giving three independent end-to-end evaluations of the same 216 reference objects.

Table 4.9: End-to-end perception performance for the selected Hybrid PointGroup followed by DCP and ICP configuration. Each run contains the same 216 reference objects from the 72 real-scanner scenes.

Seed	Usable predicted instances	Successful end-to-end registrations	End-to-end success rate (%)	Registration success after instance recovery (%)
11	161 / 216	160 / 216	74.07%	99.38%
22	160 / 216	160 / 216	74.07%	100.00%
33	162 / 216	162 / 216	75.00%	100.00%

The complete pipeline achieved an end-to-end success rate of 74.38 ± 0.53%. The result was highly consistent across the three independently trained model pairs, with less than one percentage point separating the individual runs. This stability is consistent with the relatively small seed variation observed for the hybrid-trained PointGroup models and the DCP followed by ICP registration configuration in the preceding component-level experiments.

The reduction from the 97.84% registration success obtained using reference instances to the 74.38% end-to-end result initially appears substantial. However, examining where the failures occurred shows that the reduction was almost entirely associated with the segmentation stage rather than with registration itself.

Across the three runs, the same 216 physical object observations were evaluated once using each seed pair, giving 648 checkpoint-object evaluations. PointGroup produced a usable predicted instance in 483 of these cases, corresponding to 74.54% of the complete evaluation set. DCP followed by ICP successfully registered 482 of those 483 instances. Registration therefore succeeded for:

$$\frac{482}{483} \times 100 = 99.79\% \tag{4.34}$$

of the objects that reached the registration stage.

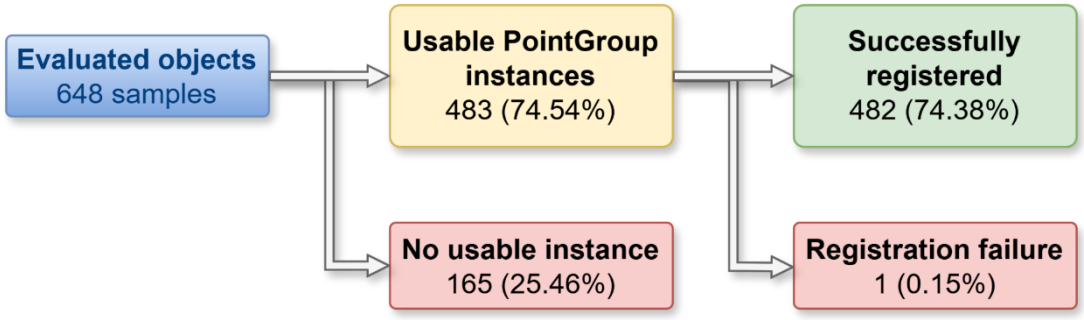


Figure 4.19: Progression through the selected end-to-end offline perception pipeline across the three seed-paired runs. Of the 648 checkpoint-object evaluations, PointGroup produced 483 usable instances, of which 482 were successfully registered using DCP followed by ICP. The remaining 165 evaluations did not produce a usable instance, while only one registration failed after a usable instance had been recovered.

This result is important for interpreting the complete perception pipeline. The lower end-to-end success rate does not indicate that DCP followed by ICP became unreliable when connected to PointGroup. Once a sufficiently complete object instance was available, the selected registration method remained successful in almost every case. The dominant limitation was therefore the recovery of the object instance from difficult real-scanner observations.

The successful end-to-end pose estimates also remained geometrically accurate. Across the three runs, the median translation error was 2.84 ± 0.15 mm, and the median rotation error was 1.62 ± 0.05°. The median ADD-S error was 1.36 ± 0.05 mm, while its 90th percentile remained at 3.50 ± 0.06 mm. These values are comparable to those obtained when DCP followed by ICP was evaluated using the reference object instances, indicating that the predicted PointGroup instances reaching the registration stage were generally of sufficient quality for accurate pose estimation.

As observed in Section 4.13.2.2, the 90th percentile rotation error remained close to 180°. This behaviour is again associated with approximately symmetric object geometry rather than an equivalent increase in surface-alignment error. The corresponding ADD-S values remained low, supporting the use of the symmetry-aware measure when assessing the final geometric alignment.

The end-to-end results were also relatively consistent across the three assembly types. Success rates of 75.93 ± 2.12%, 72.22 ± 1.39% and 75 ± 1.39% were obtained for AT1, AT2 and AT3, respectively. The relatively small difference between the classes suggests that no single target geometry dominated the overall reduction in performance.

A considerably larger difference was observed between the resting configurations. RC1 achieved an end-to-end success rate of 95.37 ± 2.12%, compared with 75.93 ± 0.8% for RC3 and 51.85 ± 0.8% for RC2. This follows the same pattern observed during the instance-segmentation analysis in Section 4.13.2.1. RC1 provided the most favourable single-view observations, while RC2 contained several orientations where only a limited portion of the object geometry was visible to the scanner.

The connection between the segmentation and end-to-end experiments is particularly clear for RC2. When reference object instances were supplied directly to DCP followed by ICP, registration success for RC2 remained 94.44%. When the same observations first had to pass through PointGroup, end-to-end success decreased to

51.85%. The registration experiment therefore demonstrates that the geometry present in the reference crop was generally sufficient for pose estimation. The difficulty occurred primarily when the partial scanner observation did not provide enough information for PointGroup to recover a usable instance.

This behaviour reflects a general challenge in point-cloud perception. A real scanner observes only surfaces visible from its acquisition viewpoint, and missing depth, occlusion and viewpoint-dependent visibility can therefore affect downstream learning-based processing. These limitations are recognised in point-cloud sim-to-real and registration research, where partial overlap and incomplete observations remain important difficulties [82], [83], [101]. The present results show how this problem propagates through an integrated perception sequence. An instance that cannot be recovered from the available measurement cannot subsequently be registered, regardless of the accuracy of the registration method itself.

The result also demonstrates the value of evaluating perception components both independently and as a connected pipeline. If only the end-to-end success rate had been considered, the 74.38% result could have been interpreted as a general limitation of the combined segmentation and registration methodology. The component-level experiments instead show that the two stages behave differently. Hybrid PointGroup substantially reduces the sim-to-real gap but remains sensitive to severely incomplete single-view observations, while DCP followed by ICP provides highly reliable pose estimation once a suitable instance is available. This distinction is relevant to industrial robotic perception because the quality of the final output depends not only on the accuracy of individual algorithms but also on how errors propagate between successive stages. The wider literature similarly identifies integration across perception, pose estimation and robotic execution as an important requirement beyond optimising individual modules in isolation.

The remaining limitation therefore points mainly towards improving scene acquisition rather than replacing the registration stage. As discussed in Section 4.13.2.1, additional viewpoints could provide complementary object surfaces when a single scanner position produces severe self-occlusion. The existing modular perception structure could accept a fused scene point cloud without changing the downstream segmentation and registration sequence, although multi-view acquisition was not evaluated in the present work.

Overall, the end-to-end offline experiment confirms that the selected Hybrid PointGroup followed by DCP and ICP pipeline can transfer from generated training data to real-scanner measurements and recover accurate object poses for the majority of the evaluated observations. Its main limitation is concentrated in object presentations where the fixed single-view scanner provides insufficient geometry for reliable instance recovery. Once a usable instance is produced, the registration stage remains highly reliable. These findings provide the basis for the online experiment presented in Section 4.13.3, where the selected perception pipeline is connected to grasp generation, motion planning and physical pick-and-place execution under observable operating conditions.

4.13.3 Online

The online experiment evaluated whether the perception configuration selected through the offline analysis could support repeated physical pick-and-place execution when integrated with the robotic cell. A total of 120 trials were performed,

with 40 trials for each of AT1, AT2 and AT3. The trials were also divided equally between the sprue-top and sprue-sideways resting conditions. Unlike the structured offline experiment, which deliberately included the complete range of single-view orientations, the online trials were conducted within the observable operating region of the scanner and robot. Object presentations known to produce severely incomplete single-view measurements were therefore avoided, as defined in Section 4.12.3. Figure 4.20 summarises the performance of the principal perception and task stages.

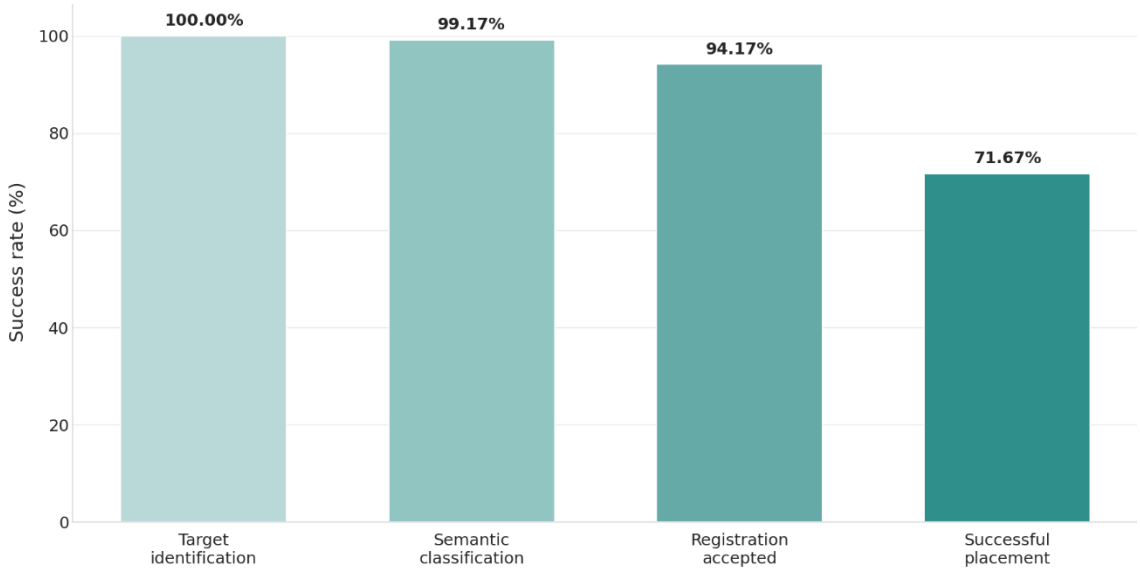


Figure 4.20: Performance of the principal stages during the 120 online pick-and-place trials. The figure shows target identification, semantic classification, registration acceptance and successful physical placement.

Target identification was successful in all 120 trials. Correct semantic classification was obtained in 119 trials, corresponding to 99.17%, while 113 registrations satisfied the configured online acceptance condition, giving a registration acceptance rate of 94.17%. The final pick-and-place success rate was 71.67%, with 86 of the 120 objects successfully transferred into their corresponding placement slots.

The higher perception performance observed during the online experiment compared with the 74.38% end-to-end offline result should not be interpreted as an improvement in the underlying models. The two experiments were intentionally designed for different purposes. The structured offline dataset included all 24 angular conditions for each resting configuration, including orientations where the fixed scanner viewpoint observed only a small portion of the object. These difficult observations were necessary to characterise the limitations of the perception pipeline. The online experiment instead evaluated integrated robotic execution under valid observable conditions. It therefore retained variation in position and orientation while excluding presentations known to provide severely insufficient single-view geometry.

This distinction reflects the bounded operating conditions commonly used in industrial robotic handling. Industrial bin-picking systems generally operate with known parts and controlled workspaces rather than attempting to handle every theoretically possible object presentation [25]. Practical feeding systems may also redistribute or constrain randomly presented parts so that sufficient geometry remains available for vision and picking [17]. The online experiment follows the same principle. The perception system remains responsible for identifying and localising randomly presented objects, but the physical cell retains an operational region within which adequate sensor

information is expected. The terminal outcomes of the 120 trials are summarised in Table 4.10.

Table 4.10: Terminal outcomes obtained during the 120 online pick-and-place trials.

Terminal outcome	Trials	Rate
No executable trajectory generated	8	6.67%
Picked but execution incomplete	3	2.50%
Picked but misplaced	23	19.17%
Successfully placed	86	71.67%

The eight trials in which no executable trajectory was generated include two different situations within the sequential pipeline. Seven trials were stopped because the registration result did not satisfy the configured acceptance condition and manipulation was therefore not allowed to continue. In one additional trial, registration was accepted but the motion-planning stage did not provide an executable trajectory. This separation is important because most trials within this category were intentionally rejected by the perception validation stage rather than representing failures of the robot planner.

The rejection of these seven registrations demonstrates the role of the online geometric validation stage. Rather than passing every perception output directly to the robot, the estimated transformation was checked before manipulation was permitted. This follows the bounded-autonomy approach adopted throughout the system, where data-driven perception provides flexibility while deterministic validation and motion planning constrain subsequent execution. Similar principles are used in reconfigurable manufacturing architectures where automatically generated robot behaviour is validated before physical execution [29]. The purpose of the acceptance stage is therefore not to maximise the number of robot motions, but to prevent uncertain perception results from automatically propagating into physical manipulation.

Of the 113 trials with an accepted registration, one did not obtain an executable motion plan. The remaining 112 trials progressed to physical manipulation. Among these, 86 were successfully placed, 23 reached the placement region but were misplaced, and three were interrupted after the object had been picked. Successful placement therefore occurred in 76.79% of the trials that progressed to physical manipulation.

The distinction between perception and manipulation outcomes is important. The online perception stages achieved high success rates, while most remaining failures occurred after a valid object pose had already been obtained. Misplacement was the dominant physical failure category, accounting for 23 trials. This is consistent with the strict placement criterion used in the experiment. Successful completion required the part to enter its corresponding shape-specific tray slot, which provided approximately 2 mm clearance. Moving the object to the general target region was therefore insufficient to satisfy the success condition.

Figure 4.21 illustrates a representative online perception result. The physical arrangement of the moulded assemblies is shown alongside the corresponding raw scanner observation and the output produced by PointGroup. The segmented instances

are separated from the surrounding scene and assigned their predicted semantic classes before being passed to the subsequent registration stage. This provides the link between the physical scanner measurement and the object-level information used by the remainder of the robotic pipeline.

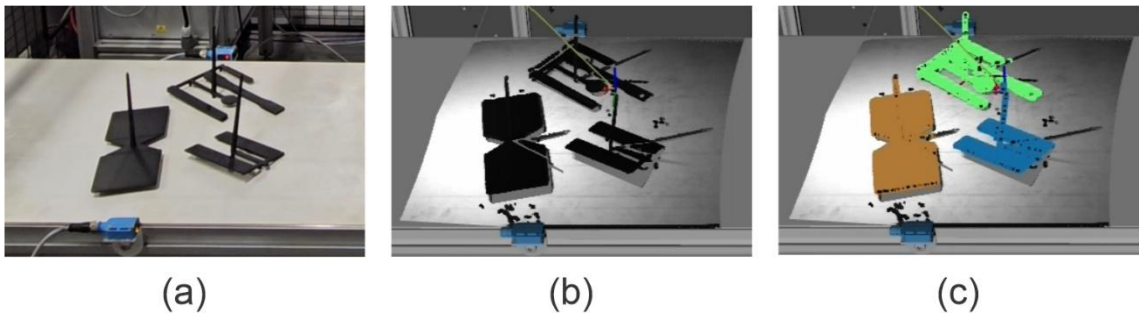


Figure 4.21: Representative example of the online perception process. (a) Physical arrangement of the moulded assemblies on the conveyor, (b) corresponding raw point-cloud observation displayed in RViz, and (c) PointGroup perception output showing the detected object instances coloured according to their predicted classes, with AT1 shown in green, AT2 in blue and AT3 in orange.

This sequence-level interpretation agrees with industrial pick-and-place literature, where localisation quality, grasp generation and collision-free motion are treated as connected requirements rather than independent objectives. *Ojer et al.* argue that industrial random-flow handling is best considered as a whole-system problem involving localisation, grasp calculation and motion simultaneously [25]. The present results support this view. High identification and registration performance did not automatically produce an equivalent final placement rate because additional uncertainties were introduced during grasping, object transfer and final insertion. Differences were also observed between the three assembly types. Figure 4.22 presents the successful placement rate for each object type and resting condition.

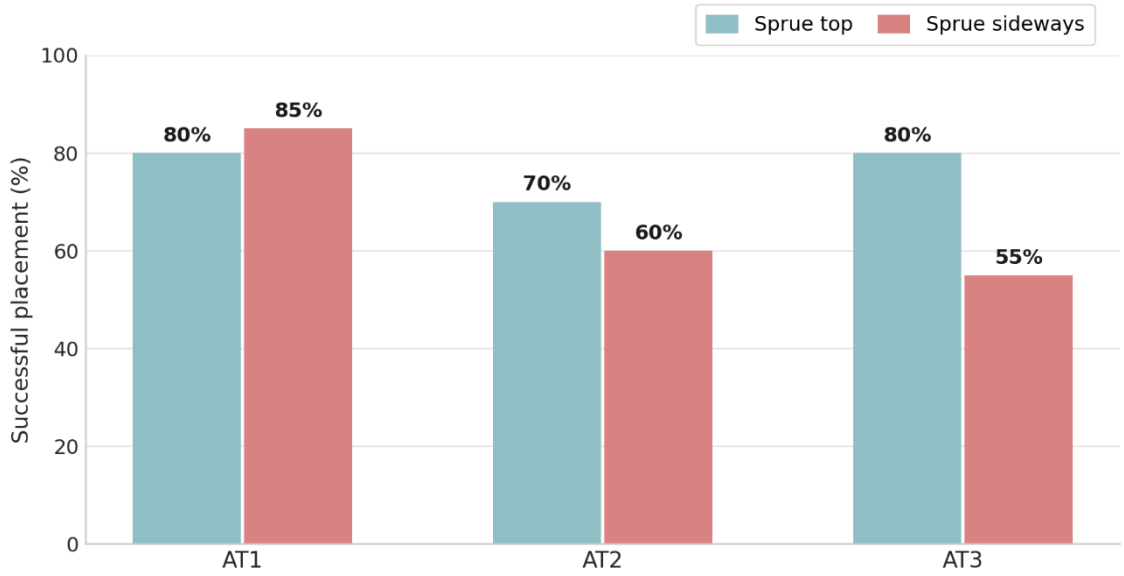


Figure 4.22: Successful physical placement for AT1, AT2 and AT3 under the sprue-top and sprue-sideways resting conditions. Each object and resting-condition combination contains 20 trials.

Across all 40 trials for each object, AT1 produced the strongest final placement performance with 33 successful trials, corresponding to 82.5%. AT2 was successfully placed in 26 trials, giving 65%, while AT3 was successfully placed in 27 trials, giving 67.5%. Registration acceptance remained comparatively high for all three objects at 95%

for AT1, 95% for AT2 and 92.5% for AT3. The larger difference in final placement performance therefore developed mainly after pose estimation.

The two resting conditions produced a similar pattern. Registration was accepted in 95% of sprue-top trials and 93.33% of sprue-sideways trials. However, successful placement decreased from 76.67% for sprue top to 66.67% for sprue sideways. The difference in final task success is therefore larger than the difference in registration acceptance.

The effect becomes clearer when object type and resting condition are considered together. AT1 achieved placement rates of 80% for sprue top and 85% for sprue sideways. AT2 decreased from 70% to 60%, while AT3 decreased from 80% to 55%. The weakest physical condition was therefore AT3 with the sprue sideways.

These results suggest that the resting orientation influenced more than the scanner observation alone. Once a pose had been accepted, the orientation of the part also changed the physical grasp configuration, the surfaces available to the gripper and the subsequent behaviour of the object during transfer and insertion. The lower placement rate for the sideways condition despite a relatively small change in registration acceptance supports this interpretation. The online limitation was therefore not concentrated in one perception stage but increasingly shifted towards physical interaction as the trial progressed through the pipeline.

The three incomplete executions also occurred in the sprue-sideways condition. These cases represented trials where the object was successfully picked but the complete manipulation sequence was not achieved. Although the number of cases is small, their concentration within the sideways configuration is consistent with the greater physical challenge associated with this presentation. A larger manipulation-focused experiment would be required before attributing this difference to a specific grasping mechanism or object feature.

The weighted task-progress score introduced in Section 4.12.4 produced an average value of 0.929 across the 120 trials. This value is higher than the strict placement success rate because unsuccessful trials retain credit for perception and intermediate task stages completed before the terminal outcome. The score is therefore useful for describing how far the system progressed through the overall sequence, but it should not replace the final placement rate as the principal measure of complete task success. A value of 71.67% remains the stricter representation of trials in which perception, registration, planning, grasping, transfer and final placement were all completed successfully.

Taken together with the offline experiments, the online results show a progressive change in the principal source of failure. The fully synthetic segmentation experiment first demonstrated the sim-to-real difficulty introduced by the physical environment. Hybrid training substantially reduced this gap. The end-to-end offline experiment then showed that difficult single-view observations could prevent PointGroup from producing usable instances, while registration remained highly reliable once an instance was recovered. During the online experiment, severely under-observed poses were excluded from the operational range and the perception stages consequently achieved high success. The dominant remaining failures then occurred during physical manipulation and accurate final placement.

This progression demonstrates why the system was evaluated at both component and integrated levels. A strong segmentation or registration metric alone cannot establish whether the complete robotic task is reliable. Conversely, a physical pick-and-place failure does not necessarily indicate that the perception method was incorrect. The staged evaluation allows these behaviours to be separated and shows how limitations move through the perception-to-action sequence.

Overall, the online experiment demonstrates that the selected hybrid perception pipeline could be integrated successfully with the ROS-based robotic architecture, pose-conditioned grasp generation, MoveIt motion planning and physical execution. The system identified all presented target objects, correctly classified 99.17%, accepted 94.17% of the estimated registrations and successfully completed 71.67% of the full pick-and-place trials under the defined operating conditions. The results therefore support the central objective of the developed framework, while also identifying precise physical placement and manipulation robustness as important areas for further development.

4.13.4 Overall Discussion

The experiments presented in this chapter were designed to evaluate more than the performance of individual segmentation or registration algorithms. The broader objective was to determine whether the robotic-cell foundation established in Chapter 3 could be extended into a more adaptable system in which new industrial parts could be introduced through object-specific information rather than extensive modification of the complete perception and robot-control pipeline.

The results of Chapter 4 indicate that adaptability, flexibility and controlled execution do not need to be addressed through a single fully autonomous model. Instead, they can be distributed across a modular pipeline in which different components address different forms of uncertainty. The generated data support adaptation to the target objects, learning-based perception handles variation in the observed scene, CAD-derived geometry constrains object localisation, geometric refinement improves pose accuracy, and explicit validation and motion planning determine whether the resulting action can proceed. This reflects the original position of the thesis that industrial adaptability can be increased without replacing the structured and verifiable elements that contribute to reliable robotic execution.

A central outcome of this chapter concerns the role of dataset generation in enabling this adaptability. The method developed in Section 4.4 does not generate idealised object point clouds directly from CAD surfaces. Object arrangements are first produced under physical constraints so that gravity, contact, collision and settling contribute to the resulting object poses. Visibility and acquisition conditions are then considered before the generated object observations are incorporated into the scene. The hybrid condition adds measured information from the physical environment, particularly the real conveyor background, while preserving automatically generated semantic, instance and pose information. These stages address different components of the sim-to-real problem. Physics-based arrangement reduces the occurrence of physically implausible training configurations, while incorporation of the measured environment reduces the difference between the generated scene context and that observed by the physical scanner.

This direction is consistent with a growing body of industrial robotic-perception research. *Kleeberger et al.* developed a large-scale industrial bin-picking dataset containing both synthetic and real scenes with corresponding instance segmentation and 6D pose information, illustrating the importance of providing learning-based methods with comparable information across simulated and physical domains [147]. *Li et al.* subsequently proposed S2R-Pick, in which an automated synthetic-data pipeline generates photorealistic labelled scenes used to train cascaded instance-segmentation and 6D localisation networks before deployment on real industrial inputs [126]. Their results demonstrate that task-specific synthetic generation can support complete recognition and localisation pipelines without requiring the equivalent volume of manually annotated real data.

The work of *Zhuang et al.* is particularly relevant to the present research because it applies the same general principle directly to point-cloud-based industrial bin picking [62]. Their method uses a physically simulated engine to generate synthetic point-cloud data and combines instance segmentation with subsequent 6D pose estimation. *Cordeiro et al.* similarly developed an automated CAD-driven dataset-generation framework that incorporates physics simulation and photorealistic scene generation and evaluated the resulting data across multiple instance-segmentation networks and a physical robotic use case [148]. These studies reinforce the importance of physically plausible automated dataset generation when manually acquiring and annotating object-specific industrial datasets would otherwise create substantial reconfiguration effort.

The contribution of the present work should therefore not be interpreted as the use of simulation alone. Rather, the proposed approach extends this direction by integrating physics-based object generation with measured environmental information within a task-specific 3D point-cloud workflow and connecting the resulting data directly to the configurable robotic system. The controlled comparison between the fully synthetic and hybrid PointGroup conditions provides particularly strong evidence for the measured-background component. Both models performed strongly within their corresponding generated domains, but real-scanner mIoU increased from 39.95% using fully synthetic training to 81.14% using hybrid training. Real-scanner mAP50 similarly increased from 49.28% to 73.92%. The 41.19 percentage-point improvement in mIoU demonstrates that the physical background was not simply a visual addition to the generated dataset. It materially changed how well the learned representation transferred to measurements from the robotic cell.

The experiment does not isolate the contribution of physics simulation in the same way because physics-based object generation was part of the wider generation methodology rather than the controlled variable between the two PointGroup conditions. Its role should therefore be interpreted through the design of the generation process and the supporting sim-to-real literature rather than through a separate numerical improvement. This distinction is important. The experimental evidence directly demonstrates the benefit of introducing measured scene information, while the use of physics ensures that the generated object arrangements remain consistent with the type of physically plausible training data advocated in related simulation-based robotic-perception research.

The findings are also consistent with *Wu et al.*, who demonstrated that synthetic point clouds can support sim-to-real transfer in an industrial disassembly application but emphasised the importance of generating and processing the synthetic data so that the learned representation remains useful on real measurements [101]. The results of the present work reinforce this conclusion. The important factor is not only the quantity of automatically generated data, but whether the generated observations reproduce the geometric, environmental and sensing conditions relevant to the final task.

Another important outcome is that the proposed pipeline is not defined by PointGroup and DCP. These methods were selected as the experimental implementations used to evaluate instance segmentation and learned registration, respectively. The objective stated in Section 4.2 was explicitly to develop a configurable perception framework rather than a new segmentation or registration architecture. Segmentation and registration are represented as separate processing tasks within the *point_cloud_processing* package, with model-specific requirements contained within their implementations. The training and inference nodes therefore operate through common task interfaces rather than being written specifically around one network.

This distinction is significant for system adaptability. The scene-level hybrid dataset contains the point and object information required by instance-segmentation methods, while the object-level dataset provides partial observations, templates and known transformations for registration training. PointGroup and DCP demonstrated that these generated resources can be used successfully within the implemented tasks, but compatible alternative methods can be introduced through new model implementations without changing the wider scanner, dataset-generation, ROS communication or robot-control interfaces. A new model architecture would still require an appropriate wrapper and compatibility with the task input and output definitions, but it would not require redesigning the complete robotic application. The fact that the model-loading mechanism is separated from scanner acquisition and result publication is an important aspect of this modularity.

The work of *Cordeiro et al.* provides useful support for this model-independent view of dataset generation [148]. Their automated generator was evaluated using three different instance-segmentation networks rather than being presented as data for only one specific architecture. The principle is similar in the present work. The generated data constitute a reusable perception resource, while the model is one configurable element of the processing pipeline. This is important for longer-term adaptability because improvements in segmentation or registration models should not require the surrounding robotic system to be redeveloped.

The user-guided configuration interface provides the corresponding mechanism at the system level. Reconfiguration is not performed by manually changing object identities, file paths, placement coordinates and model selections throughout separate source files. The interface connects placement definition, dataset generation, model preparation and system control through a common task description. Object CAD models and placement poses can be loaded and saved, dataset parameters can be configured and generation initiated, compatible perception models can be selected and trained, and the resulting configuration can then be applied to physical operation.

This means that the three injection-moulded assemblies used in the experiment should be understood as the validation objects rather than fixed objects embedded in the architecture. Within the supported workflow, another set of known target parts can be introduced by supplying their geometric models, defining their required placement locations, generating the corresponding data and preparing compatible perception models. The semantic relationships between generated annotations, trained models, object templates and placement slots are subsequently maintained through the configuration rather than repeated in separate implementations. Existing tasks can also be saved, restored and modified.

The experiments did not quantify the time required to reconfigure the complete system for a new product or compare this process directly with conventional robot programming. It would therefore be inappropriate to assign a numerical reduction in engineering effort. What has been demonstrated is a change in the form of the required intervention. Object-specific information is introduced through a controlled configuration workflow, while the reusable software performs the subsequent conversion into dataset, perception and robotic resources. This provides the basis for reduced direct reprogramming and is consistent with the bounded-autonomy objective established in Chapter 1.

4.14 Chapter Summary

This chapter extended the robotic-cell foundation developed in Chapter 3 towards the handling of more geometrically complex industrial parts. The extended system combined user-defined object and placement information, physics-based and hybrid point-cloud dataset generation, learning-based instance segmentation, template-based registration and pose-conditioned robotic execution within the existing ROS-based robotic-cell architecture.

The offline experiments demonstrated that incorporating measured environmental information into the generated training scenes substantially improved the transfer of PointGroup to physical scanner data. The registration experiments further showed that DCP provided a strong initial object alignment and that subsequent ICP refinement produced the most reliable overall registration configuration. When segmentation and registration were connected in the complete offline perception pipeline, the main remaining failures were associated with object observations containing insufficient visible geometry rather than with registration after a usable instance had been recovered.

The online experiment then demonstrated the selected perception pipeline as part of the physical robotic system. Under the defined observable operating conditions, the perception stages achieved high identification, classification and registration-acceptance performance, while the dominant remaining failures occurred during physical manipulation and precise placement. Therefore, Chapter 4 demonstrates the progression from CAD-defined target objects and automatically generated training data to real-scanner perception and physical robotic execution. Chapter 5 draws together these findings with the foundational experiment presented in Chapter 3 and considers the overall research questions, contributions, limitations and future development of the system.

Chapter 5: Conclusions

5.1 Introduction

This chapter brings together the findings from the two experimental stages and considers them in relation to the research aim, central research question and supporting research questions defined in Chapter 1. It then revisits the principal contributions of the research, discusses their significance for adaptable industrial automation, identifies the main limitations of the developed system and outlines directions for future work.

5.2 Research Question Synthesis

The findings from the two experimental stages can now be considered directly in relation to the central research question and the five supporting research questions. The central research question was:

RQ. How can a semi-autonomous robotic pick-and-place system be designed to combine flexibility in random object flows, adaptability to new object types and reliable execution in an industrial manufacturing cell?

- **SRQ1. Modular system architecture**:

How can a ROS-based robotic-cell architecture integrate the physical cell, digital counterpart, perception, planning and execution modules for adaptable pick-and-place?

The first experimental stage provided the initial response to **SRQ1** by establishing communication between the scanner, physical and digital cell, transformation system, MoveIt and ABB robot execution. Chapter 4 provides stronger evidence of modularity because this infrastructure was reused while the object-specific perception and manipulation layers were substantially extended. Dataset generation, configurable learning-based perception, the user interface and updated manipulation control were introduced without rebuilding the lower-level scanner, robot communication, transformation and planning foundations.

The resulting ROS workspace separates scanner and robot infrastructure from dataset generation, perception, task configuration and manipulation. This separation allowed components to be developed and validated independently while still communicating through defined ROS interfaces. It also allowed dataset generation and model training to operate without the complete physical cell being active. The Chapter 4 experiments therefore support **SRQ1** by demonstrating modularity through reuse under substantial system change, rather than only through the existence of separate software packages.

ROS was retained because, at the time the system was developed, it provided a mature ecosystem around ROS-Industrial, MoveIt and the available ABB and sensing interfaces. This reduced integration risk while the primary research effort remained focused on adaptable perception and physical execution. The successful extension from Chapter 3 to Chapter 4 indicates that this decision was sufficient for the experimental objectives.

- **SRQ2. System adaptability through user-guided configuration**:

How can a robotic pick-and-place system be reconfigured for new object types without requiring extensive manual reprogramming or expert intervention?

Chapter 4 provides the principal experimental response to **SRQ2**. The proposed workflow moves object-specific information away from the internal implementation of the robotic pipeline and into task definitions managed through the graphical interface. The user provides the information that cannot be inferred automatically, principally the target CAD models, placement arrangement and configuration parameters. These definitions are then used by the wider system to generate training data, configure perception, associate object classes with templates and placement slots, and prepare the task for execution.

The core adaptability mechanism is therefore the connection between configuration and data generation. Instead of assuming that a trained perception system already exists for every possible industrial part, the object model itself becomes an input to a procedure capable of creating the corresponding task-specific perception resources. The strong real-scanner performance obtained after hybrid training provides evidence that this generated data can produce useful perception models for physical operation, rather than serving only as simulated training material. This directly supports the objective of reducing dependence on manually acquired and annotated datasets when the target object set changes.

This approach is conceptually close to recent work on automated synthetic data generation for customised industrial scenarios. Li et al. demonstrate a synthetic-data-to-recognition-and-localisation pipeline for industrial bin picking, while *Cordeiro et al.* explicitly target the labour associated with constructing customised labelled datasets from new CAD-defined objects. The present work extends this idea into a user-guided ROS workflow in which dataset preparation is connected to model configuration, target definition and subsequent robot execution.

SRQ3. Flexibility in random object flows:

How can a robotic system detect, localise, classify and pick objects arriving in random positions and orientations, from simple geometric parts to complex industrial shapes?

The two experimental stages provide a progressive response to **SRQ3**. Chapter 3 demonstrated perception-guided picking of simple rectangular objects without relying on fixed conveyor pick coordinates. Chapter 4 replaced the geometry-dependent perception approach with instance segmentation and template-based 6-DoF registration suitable for more complex industrial parts. The structured offline evaluation then tested these parts across a substantially wider set of orientations than those required for online robotic operation.

The results also define an important boundary to this flexibility. Hybrid training substantially improved transfer to physical scans, and DCP followed by ICP achieved 97.84% registration success when reference object instances were available. In the complete offline pipeline, registration succeeded for 99.79% of the usable PointGroup instances. The main remaining perception failures therefore occurred when the fixed single-view scanner did not observe sufficient geometry for a valid instance to be recovered. Flexibility in object pose is consequently constrained by physical observability, not only by model capability.

The online experiment demonstrates the corresponding operational condition. Within the observable scanning and picking region, target identification reached 100%, semantic classification reached 99.17%, and 94.17% of registrations satisfied the online

geometric acceptance condition. These results show that the system can operate with variation in object position and orientation without returning to predefined pick coordinates, while retaining a bounded operating envelope appropriate to the sensing arrangement.

- **SRQ4. Reliability under changing object and scene conditions**:

How can the system maintain reliable performance across different object types and scene conditions while limiting dependence on unconstrained AI models that are difficult to validate in industrial settings?

The Chapter 4 results support a layered response to **SRQ4**. Reliability is not assigned to a single neural network. The hybrid dataset first attempts to make the learned perception problem representative of the physical cell. PointGroup identifies object instances and semantic classes. DCP estimates the broader template alignment. ICP provides explicit geometric refinement. The resulting transformation is checked using the configured geometric acceptance condition before manipulation is permitted. MoveIt then evaluates inverse kinematics, robot-state validity, collision conditions and trajectory feasibility before physical execution.

This architecture therefore uses AI where flexibility is required while retaining deterministic geometric and planning layers where explicit validation is possible. The approach is consistent with the bounded-autonomy position established in the literature review and with reconfigurable manufacturing systems such as the R3M framework, where adaptable robot behaviour remains subject to validation before physical execution [29].

The experimental results also demonstrate why this separation is necessary. The online perception stages performed strongly, but only 71.67% of the complete trials ended in successful placement. Most of the remaining reduction occurred after perception, particularly during precise placement. This should not be interpreted as evidence of production-level reliability. Instead, it demonstrates that a reliability-oriented architecture can identify where uncertainty enters the task. Improvements to perception alone would not eliminate failures caused by grasp stability, physical tolerance or final insertion accuracy.

The movement of the dominant failure source through the experiments is itself an important finding. Fully synthetic training initially exposed a large domain-transfer problem. Hybrid training substantially reduced it. Once a usable instance was recovered, registration became highly reliable. Under the more observable online conditions, the remaining limitation moved further downstream towards physical manipulation. The modular evaluation therefore makes it possible to determine which part of the complete system requires improvement instead of treating every failed pick-and-place trial as an undifferentiated system failure.

- **SRQ5. Balance between autonomy and user input**:

What level of user input is needed to support adaptable pick-and-place while keeping perception, planning and execution sufficiently automated?

The implemented workflow provides a practical response to **SRQ5**. The system is not autonomous in the sense of deciding which products should be handled or inferring the intended manufacturing task without prior information. The user remains

responsible for defining the high-level task and providing the object-specific knowledge required by the application. This includes the CAD representations, placement arrangement, selected perception methods and configurable operating parameters.

Once these inputs are defined, the software performs much of the repetitive technical preparation. It constructs the task configurations, initiates dataset generation, connects generated datasets to the selected model implementation, supports model training and checkpoint preparation, loads the inference pipeline, associates detected classes with object templates and placement slots, and coordinates perception and robotic execution. The graphical user interface (GUI) was therefore designed to minimise direct interaction with the underlying source code rather than remove the operator from the process completely.

This represents the bounded-autonomy position proposed in Chapter 1. Human knowledge is retained where it is useful and where the desired industrial task must be specified, while automation is used to reduce the repeated implementation work that otherwise appears when the object set changes. The balance is especially relevant to manufacturing, where completely unconstrained autonomous behaviour may offer greater theoretical flexibility but also makes validation, repeatability and recovery more difficult.

Taken together, the five supporting research questions provide a direct response to the central research question. The findings indicate that flexibility, adaptability and reliable execution do not need to be addressed through a single fully autonomous model. Instead, they can be distributed across a modular pipeline in which generated task-specific data support adaptation to new target objects, learning-based perception handles variation in the observed scene, CAD-derived geometry constrains object localisation, geometric refinement improves pose accuracy, and explicit validation and motion planning determine whether the resulting action can proceed. Thus, the research supports a semi-autonomous approach in which adaptability is increased without replacing the structured and verifiable elements that contribute to reliable industrial robotic execution.

5.3 Research Contributions and Industrial Significance

The primary contributions defined in Section 1.5 can now be considered in relation to the experimental evidence obtained across the thesis.

- **C1. Modular architecture for adaptable industrial pick-and-place:**

A modular semi-autonomous robotic-cell architecture was developed to integrate user configuration, dataset generation, perception, registration, planning and execution for industrial pick-and-place. The first experiment established the physical and digital robotic-cell foundation, while the second experiment demonstrated that the same lower-level scanner, transformation, planning and robot-control infrastructure could be retained while the perception and task-preparation layers were substantially extended. The contribution therefore lies not only in separating the software into modules, but in demonstrating that the architecture could be reused under a substantial change in object complexity and perception methodology.

- **C2. Hybrid point-cloud dataset generation for target industrial parts:**

Conclusions

A hybrid dataset generation method was developed to combine physics-based object arrangement, CAD-derived target geometry and measured background scans. The resulting workflow generated scene-level data for instance segmentation and object-level data for registration while preserving automatically generated annotations and known transformations. The controlled PointGroup experiment provided direct evidence for the measured-background component: real-scanner mIoU increased from 39.95% with fully synthetic training to 81.14% with hybrid training, while mAP50 increased from 49.28% to 73.92%. This demonstrates that incorporating information from the physical environment substantially improved transfer from generated training data to the real scanner.

- **C3. Perception and registration validation using hybrid and real data:**

Instance segmentation and template-based registration were evaluated using generated datasets together with a structured real-scanner dataset containing 72 scenes and 216 object observations. This allowed semantic and instance segmentation, registration and the complete perception sequence to be evaluated separately. DCP followed by ICP achieved the strongest overall registration performance, reaching 97.84% success when reference instances were supplied, while registration remained successful for 99.79% of the usable PointGroup predictions during end-to-end evaluation. The experiments therefore identified both the capability of the selected methods and the conditions under which perception performance remained limited by the available scanner observation.

- **C4. ROS-based physical and digital robotic-cell implementation:**

A physical robotic cell and corresponding digital representation were implemented using ROS, ROS-Industrial, MoveIt and RViz. The architecture connected point-cloud acquisition, transformation management, perception, target generation, collision-aware motion planning and ABB robot execution. The foundational experiment demonstrated reliable operation using simple rectangular objects, while the second online experiment showed that the same robotic infrastructure could support a substantially different perception and manipulation workflow involving complex injection-moulded assemblies and six-degree-of-freedom pose estimation.

- **C5. Experimental validation across two connected stages:**

The research was validated through two connected experimental stages rather than through isolated algorithmic tests. Chapter 3 established the foundational robotic-cell architecture and demonstrated random-pose pick-and-place using simple geometric objects. Chapter 4 then introduced more complex industrial parts, task-specific dataset generation, learning-based perception, template registration and updated manipulation. This progression allowed the research to identify how the dominant limitations changed as the task became more demanding, from geometry-dependent perception in the first experiment to sim-to-real transfer, single-view observability and physical placement in the second.

Taken together, these contributions are relevant to manufacturing environments where the robotic cell remains known, but the target product or task may change. The work does not demonstrate that complete industrial reconfiguration can occur without engineering input, nor was reconfiguration time compared quantitatively with conventional programming. Its industrial significance lies instead in changing how

object-specific information is introduced. CAD models, placement requirements and perception settings are provided through a controlled configuration workflow, while reusable software performs much of the subsequent dataset preparation, perception configuration and robotic coordination.

The resulting approach occupies a position between rigid task-specific automation and unconstrained autonomous robotics. It retains known cell geometry, explicit target definitions, deterministic registration refinement, geometric validation and conventional motion planning, while using generated data and learning-based perception where additional flexibility is required. This bounded structure provides a practical route towards increasing adaptability without removing the validation and predictability expected in industrial robotic applications.

5.4 Limitations and Future Work

The experiments also identified several limitations that define the next stages of development. The first concerns single-view observability. The structured real-scanner experiment demonstrated that certain resting configurations and orientations exposed insufficient object geometry for reliable instance recovery. Once a usable instance was available, DCP followed by ICP remained highly reliable, indicating that the principal limitation in these cases occurred during acquisition and segmentation rather than registration. Additional scanner viewpoints could provide complementary surfaces that are hidden from a single acquisition position. The resulting point clouds could be transformed into a common reference frame and fused before being supplied to the existing segmentation stage. The modular perception architecture is compatible with such an extension because the downstream processing operates on the resulting scene point cloud rather than requiring a particular acquisition arrangement.

A second limitation concerns physical manipulation and precise placement. Although the online perception stages performed strongly, 71.67% of the complete trials ended in successful placement, and the dominant remaining failures occurred after perception. Future work should therefore examine grasp stability, calibration accuracy, gripper-object interaction and the final placement strategy in greater detail. Alternative grasp candidates, improved object-specific approach selection, force or tactile feedback and active correction during placement could be investigated, particularly for tasks involving small slot clearances or mechanically difficult object orientations.

The current workflow also retains user-configured parameters for dataset generation, perception and manipulation. This is intentional because the system follows a bounded-autonomy approach and different object geometries can require different grasp offsets, approach directions and operating parameters. However, future versions could provide greater assistance in selecting these values. Optimisation methods or agent-assisted parameter recommendation could be used to propose suitable configurations from the target object geometry and previous task results while retaining operator verification before the parameters are applied. This would extend the user-guided approach rather than replacing it with unconstrained automatic decision making.

Execution time represents another limitation of the present evaluation. Industrial robotic handling is ultimately constrained not only by accuracy but also by cycle time and picking rate. The experiments intentionally prioritised accurate acquisition, perception transfer and system integration rather than minimum cycle time. The structured-light scanner was therefore used in a stop-and-scan sequence that

allowed the object to remain stationary during acquisition. A future industrial evaluation should measure the complete cycle and separately quantify acquisition, preprocessing, neural inference, registration, motion planning and physical manipulation time. This would allow the computational and mechanical bottlenecks to be identified in the same manner as the accuracy-related bottlenecks were identified in the present experiments. Faster sensing, parallel execution of independent stages, GPU optimisation and continuous conveyor operation could then be evaluated according to a defined throughput requirement.

ROS 2 also represents a logical future development as the architecture moves towards larger distributed cells and higher production throughput. ROS was retained during this research because it provided a mature ecosystem around ROS-Industrial, MoveIt and the available ABB and sensing interfaces. DDS-based communication, configurable Quality of Service, improved security, decentralised discovery and stronger support for modern real-time systems could become increasingly valuable in a future industrial implementation. The current stop-and-scan workflow reduced the need for strict communication timing, and the observed experimental limitations were dominated by perception, physical observability and manipulation rather than the ROS middleware. A migration to ROS 2 would therefore mainly address future scalability and deployment requirements rather than invalidate the architecture demonstrated in this thesis.

Finally, the experiments were conducted using a limited set of known target objects within one robotic cell. The three injection-moulded assemblies provided a useful validation case because they introduced irregular geometry, partial observations, repeated features and different resting configurations, but they do not establish generalisation to every industrial object family. Future validation should therefore include additional part geometries, longer-duration operation and different production environments. It would also be valuable to quantify the complete reconfiguration process for a new object set and compare the required engineering time directly with conventional robot-programming practice. This would provide quantitative evidence for the reduction in development effort that the current architecture is designed to support.

5.5 Final Conclusions

Taken together, the findings of this thesis support the central research argument that flexibility, adaptability and reliability do not need to be treated as mutually exclusive requirements. The proposed system retains the structured elements of an industrial robotic cell while moving object-specific perception and task information into configurable resources. Physics-based and hybrid dataset generation reduce dependence on manually collected training data, while the measured real background substantially improves transfer to the physical scanner. The perception architecture allows compatible models to be exchanged without redesigning the complete system, and the user-guided interface provides a controlled mechanism for introducing new object and placement information. Geometric validation and conventional robot planning then constrain the actions generated from the learned perception output.

The two experimental stages demonstrate this approach progressively. The first established that the physical and digital robotic-cell architecture could detect and manipulate objects arriving at random positions and orientations using a relatively simple geometric perception method. The second retained this foundation while

introducing complex industrial parts, automatically generated task-specific data, learning-based instance segmentation, six-degree-of-freedom registration and a more configurable manipulation workflow. The experiments therefore demonstrate not only individual perception or robot-control components, but their integration within a complete perception-to-action system.

The three moulded assemblies used in the second experiment represent a validation case for the developed workflow rather than the fixed scope of the system. The architecture is designed so that another known target-object set can be introduced through the same configuration, dataset-generation, model-preparation and execution sequence. The present work demonstrates that this workflow can progress from CAD-defined objects to generated training data, real-scanner perception and physical manipulation. It does not demonstrate arbitrary unknown-object manipulation, nor does it remove the need for task-specific geometric information and configuration. These boundaries are consistent with the intended scope of the thesis, which focuses on adaptable automation within a known manufacturing cell rather than unconstrained general-purpose robotics.

The main outcome is therefore not a specific PointGroup and DCP combination. It is the modular framework that allows generated task-specific data, selected perception models, object-specific geometry, user-defined manufacturing requirements and validated robotic execution to operate as parts of the same workflow. PointGroup and DCP provided the experimental means to validate this concept, while the interfaces surrounding them allow the perception methods to evolve independently from the wider robotic-cell architecture. In this way, the research addresses the gap identified in Chapter 2 by connecting dataset generation, intelligent perception, modular robot integration and human-guided reconfiguration within one experimentally validated approach to adaptable industrial pick-and-place.

References

[1] A. Gasparetto and L. Scalera, 'From the Unimate to the Delta Robot: The Early Decades of Industrial Robotics', 2019, pp. 284–295. doi: 10.1007/978-3-030-03538-9_23.

[2] C. O. Klingenberg, M. A. V. Borges, and J. A. do V. Antunes, 'Industry 4.0: What makes it a revolution? A historical framework to understand the phenomenon', *Technol. Soc.*, vol. 70, p. 102009, Aug. 2022, doi: 10.1016/j.techsoc.2022.102009.

[3] A. Cordeiro, L. F. Rocha, J. Boaventura-Cunha, D. Figueiredo, and J. P. Souza, 'A review of visual perception for robotic bin-picking', *Rob. Auton. Syst.*, vol. 196, p. 105236, Feb. 2026, doi: 10.1016/j.robot.2025.105236.

[4] R. X. Gao, J. Krüger, M. Merklein, H.-C. Möhring, and J. Váncza, 'Artificial Intelligence in manufacturing: State of the art, perspectives, and future directions', *CIRP Annals*, vol. 73, no. 2, pp. 723–749, 2024, doi: 10.1016/j.cirp.2024.04.101.

[5] Z. Xie, X. Liang, and C. Roberto, 'Learning-based robotic grasping: A review', *Front. Robot. AI*, vol. 10, Apr. 2023, doi: 10.3389/frobt.2023.1038658.

[6] A. Sethi and S. Sethi, 'Flexibility in manufacturing: A survey', *International Journal of Flexible Manufacturing Systems*, vol. 2, no. 4, Jul. 1990, doi: 10.1007/BF00186471.

[7] J. P. Shewchuk and C. L. Moodie, 'Definition and Classification of Manufacturing Flexibility Types and Measures', *International Journal of Flexible Manufacturing Systems*, vol. 10, no. 4, pp. 325–349, Oct. 1998, doi: 10.1023/A:1008062220281.

[8] R. Beach, A. P. Muhlemann, D. H. R. Price, A. Paterson, and J. A. Sharp, 'A review of manufacturing flexibility', *Eur. J. Oper. Res.*, vol. 122, no. 1, pp. 41–57, Apr. 2000, doi: 10.1016/S0377-2217(99)00062-4.

[9] Y. Koren and M. Shpitalni, 'Design of reconfigurable manufacturing systems', *J. Manuf. Syst.*, vol. 29, no. 4, pp. 130–141, Oct. 2010, doi: 10.1016/j.jmsy.2011.01.001.

[10] K. Khanna and R. Kumar, 'Reconfigurable manufacturing system: a state-of-the-art review', *Benchmarking: An International Journal*, vol. 26, no. 8, pp. 2608–2635, Sep. 2019, doi: 10.1108/BIJ-05-2018-0140.

[11] A. Gasparetto and L. Scalera, 'A Brief History of Industrial Robotics in the 20th Century', *Advances in Historical Studies*, vol. 08, no. 01, pp. 24–35, 2019, doi: 10.4236/ahs.2019.81002.

[12] T. Olsson *et al.*, 'Cost-efficient drilling using industrial robots with high-bandwidth force feedback', *Robot. Comput. Integr. Manuf.*, vol. 26, no. 1, pp. 24–38, Feb. 2010, doi: 10.1016/j.rcim.2009.01.002.

References

[13] M. N. Ahangar, Z. A. Farhat, and A. Sivanathan, 'AI Trustworthiness in Manufacturing: Challenges, Toolkits, and the Path to Industry 5.0', *Sensors*, vol. 25, no. 14, p. 4357, Jul. 2025, doi: 10.3390/s25144357.

[14] O. Heimann and J. Guhl, 'Industrial Robot Programming Methods: A Scoping Review', in *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, IEEE, Sep. 2020, pp. 696–703. doi: 10.1109/ETFA46521.2020.9211997.

[15] Z. Pan, J. Polden, N. Larkin, S. Van Duin, and J. Norrish, 'Recent progress on programming methods for industrial robots', *Robot. Comput. Integr. Manuf.*, vol. 28, no. 2, pp. 87–94, Apr. 2012, doi: 10.1016/j.rcim.2011.08.004.

[16] B. Tipary and G. Erdős, 'Generic development methodology for flexible robotic pick-and-place workcells based on Digital Twin', *Robot. Comput. Integr. Manuf.*, vol. 71, p. 102140, Oct. 2021, doi: 10.1016/j.rcim.2021.102140.

[17] N. I. Giannoccaro, G. Rausa, R. Rizzi, P. Visconti, and R. De Fazio, 'An Innovative Vision-Guided Feeding System for Robotic Picking of Different-Shaped Industrial Components Randomly Arranged', *Technologies (Basel).*, vol. 12, no. 9, p. 153, Sep. 2024, doi: 10.3390/technologies12090153.

[18] B. Tipary, A. Kovács, and F. G. Erdős, 'Planning and optimization of robotic pick-and-place operations in highly constrained industrial environments', *Assembly Automation*, vol. 41, no. 5, pp. 626–639, Sep. 2021, doi: 10.1108/AA-07-2020-0099.

[19] P. Tsarouchi, G. Michalos, S. Makris, and G. Chryssolouris, 'Vision System for Robotic Handling of Randomly Placed Objects', *Procedia CIRP*, vol. 9, pp. 61–66, 2013, doi: 10.1016/j.procir.2013.06.169.

[20] A. S. Ozer and I. Cinar, 'Real-Time and Fully Automated Robotic Stacking System with Deep Learning-Based Visual Perception', *Sensors*, vol. 25, no. 22, p. 6960, Nov. 2025, doi: 10.3390/s25226960.

[21] G. Du, K. Wang, S. Lian, and K. Zhao, 'Vision-based robotic grasping from object localization, object pose estimation to grasp estimation for parallel grippers: a review', *Artif. Intell. Rev.*, vol. 54, no. 3, pp. 1677–1734, Mar. 2021, doi: 10.1007/s10462-020-09888-5.

[22] International Federation of Robotics, 'Record of 4 Million Robots in Factories Worldwide'. Accessed: May 10, 2026. [Online]. Available: https://ifr.org/ifr-press-releases/news/record-of-4-million-robots-working-in-factories-worldwide

[23] A. Downs *et al.*, 'Assessing Industrial Robot agility through international competitions', *Robot. Comput. Integr. Manuf.*, vol. 70, p. 102113, Aug. 2021, doi: 10.1016/j.rcim.2020.102113.

[24] Z. L. Gan, S. N. Musa, and H. J. Yap, 'A Review of the High-Mix, Low-Volume Manufacturing Industry', *Applied Sciences*, vol. 13, no. 3, p. 1687, Jan. 2023, doi: 10.3390/app13031687.

[25] M. Ojer, H. Alvarez, X. Lin, and E. Lazkano, 'Advances in industrial bin picking through sim-to-real transfer and cycle optimization', *Int. J. Adv. Robot. Syst.*, vol. 22, no. 5, Sep. 2025, doi: 10.1177/17298806251374367.

[26] Y. Yin, P. Zheng, C. Li, and K. Wan, 'Enhancing human-guided robotic assembly: AR-assisted DT for skill-based and low-code programming', *J. Manuf. Syst.*, vol. 74, pp. 676–689, Jun. 2024, doi: 10.1016/j.jmsy.2024.04.016.

[27] F. Müller, M. Koch, and A. Hasse, 'User Study to Validate the Performance of an Offline Robot Programming Method That Enables Robot-Independent Kinesthetic Instruction through the Use of Augmented Reality and Motion Capturing', *Robotics*, vol. 13, no. 3, p. 35, Feb. 2024, doi: 10.3390/robotics13030035.

[28] D. Dimosthenopoulos, F. P. Basamakis, C. Glykos, A. C. Bavelos, G. Mountzouridis, and S. Makris, 'A Digital Twin-based paradigm for programming and control of cooperating robots in reconfigurable production systems', *Int. J. Comput. Integr. Manuf.*, vol. 38, no. 10, pp. 1443–1463, Oct. 2025, doi: 10.1080/0951192X.2024.2428683.

[29] S. Asif *et al.*, 'Rapid and automated configuration of robot manufacturing cells', *Robot. Comput. Integr. Manuf.*, vol. 92, p. 102862, Apr. 2025, doi: 10.1016/j.rcim.2024.102862.

[30] A. Napoleone, A.-L. Andersen, T. D. Brunoe, and K. Nielsen, 'Towards human-centric reconfigurable manufacturing systems: Literature review of reconfigurability enablers for reduced reconfiguration effort and classification frameworks', *J. Manuf. Syst.*, vol. 67, pp. 23–34, Apr. 2023, doi: 10.1016/j.jmsy.2022.12.014.

[31] P. Malvido Fresnillo, S. Vasudevan, W. M. Mohammed, J. L. Martinez Lastra, and J. A. Perez Garcia, 'Extending the motion planning framework—MoveIt with advanced manipulation functions for industrial applications', *Robot. Comput. Integr. Manuf.*, vol. 83, p. 102559, Oct. 2023, doi: 10.1016/j.rcim.2023.102559.

[32] H. Chen *et al.*, 'Human-in-the-Loop Robot Learning for Smart Manufacturing: A Human-Centric Perspective', *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 11062–11086, 2025, doi: 10.1109/TASE.2025.3528051.

[33] P. Wang, Z. Wang, J. Duan, Y. Miao, and W. Tian, 'Review of the development status of low-code programming software for robots', *Digital Engineering*, vol. 4, p. 100039, Mar. 2025, doi: 10.1016/j.dte.2025.100039.

[34] S. Liu and P. Liu, 'Benchmarking and optimization of robot motion planning with motion planning pipeline', *The International Journal of Advanced Manufacturing Technology*, vol. 118, no. 3, pp. 949–961, 2022.

[35] M. Bernabei and F. Costantino, 'Adaptive automation: Status of research and future challenges', *Robot. Comput. Integr. Manuf.*, vol. 88, p. 102724, Aug. 2024, doi: 10.1016/j.rcim.2024.102724.

[36] M. Rieth, L. Onnasch, and V. Hagemann, 'Adaptable automation for a more human-centered work design? Effects on human perception and behavior', *Int. J. Hum. Comput. Stud.*, vol. 186, p. 103246, Jun. 2024, doi: 10.1016/j.ijhcs.2024.103246.

References

[37] J. Halim, P. Eichler, S. Krusche, M. Bdiwi, and S. Ihlenfeldt, 'No-code robotic programming for agile production: A new markerless-approach for multimodal natural interaction in a human-robot collaboration context', *Front. Robot. AI*, vol. 9, Oct. 2022, doi: 10.3389/frobt.2022.1001955.

[38] W. Dong *et al.*, 'Towards a next-generation LLM empowered low-code programming industrial robotic system for human-centric smart manufacturing', *J. Manuf. Syst.*, vol. 83, pp. 675–686, Dec. 2025, doi: 10.1016/j.jmsy.2025.10.012.

[39] Sebastian. Thrun, Wolfram. Burgard, and Dieter. Fox, *Probabilistic robotics*. The MIT Press, 2006.

[40] Oussama. Khatib and Bruno. Siciliano, *Springer Handbook of Robotics*. Springer International Publishing : Imprint : Springer, 2016.

[41] C. Premebida, R. Ambrus, and Z.-C. Marton, 'Intelligent Robotic Perception Systems', in *Applications of Mobile Robots*, IntechOpen, 2019. doi: 10.5772/intechopen.79742.

[42] R. Chen, J. Xu, and S. Zhang, 'Comparative study on 3D optical sensors for short range applications', *Opt. Lasers Eng.*, vol. 149, p. 106763, Feb. 2022, doi: 10.1016/j.optlaseng.2021.106763.

[43] R. Horaud, M. Hansard, G. Evangelidis, and C. Ménier, 'An overview of depth cameras and range scanners based on time-of-flight technologies', *Mach. Vis. Appl.*, vol. 27, no. 7, pp. 1005–1020, Oct. 2016, doi: 10.1007/s00138-016-0784-4.

[44] T. Jia, X. Li, X. Yang, S. Lin, Y. Liu, and D. Chen, 'AdaptiveStereo: Depth estimation from adaptive structured light', *Opt. Laser Technol.*, vol. 169, p. 110076, Feb. 2024, doi: 10.1016/j.optlastec.2023.110076.

[45] M. Franaszek, P. Rachakonda, and K. S. Saidi, 'Filtering Organized 3D Point Clouds for Bin Picking Applications', *Applied Sciences*, vol. 14, no. 3, p. 961, Jan. 2024, doi: 10.3390/app14030961.

[46] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, 'Pointnet: Deep learning on point sets for 3d classification and segmentation', *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 652–660, 2017.

[47] C. R. Qi, L. Yi, H. Su, and L. Guibas, 'PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space', in *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/d8bf84be3800d12f74d8b05e9b89836f-Paper.pdf

[48] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, 'Dynamic Graph CNN for Learning on Point Clouds', *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, Oct. 2019, doi: 10.1145/3326362.

[49] B. Graham, M. Engelcke, and L. van der Maaten, '3D Semantic Segmentation with Submanifold Sparse Convolutional Networks', in *2018 IEEE/CVF Conference on*

Computer Vision and Pattern Recognition, IEEE, Jun. 2018, pp. 9224–9232. doi: 10.1109/CVPR.2018.00961.

[50] C. Choy, J. Gwak, and S. Savarese, '4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks', in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2019, pp. 3070–3079. doi: 10.1109/CVPR.2019.00319.

[51] B. Wu, A. Wan, X. Yue, and K. Keutzer, 'SqueezeSeg: Convolutional Neural Nets with Recurrent CRF for Real-Time Road-Object Segmentation from 3D LiDAR Point Cloud', in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2018, pp. 1887–1893. doi: 10.1109/ICRA.2018.8462926.

[52] T. Cortinhal, G. Tzelepis, and E. Erdal Aksoy, 'SalsaNext: Fast, Uncertainty-Aware Semantic Segmentation of LiDAR Point Clouds', 2020, pp. 207–222. doi: 10.1007/978-3-030-64559-5_16.

[53] H. Vassilev, M. Laska, and J. Blankenbach, 'Uncertainty-aware point cloud segmentation for infrastructure projects using Bayesian deep learning', *Autom. Constr.*, vol. 164, p. 105419, Aug. 2024, doi: 10.1016/j.autcon.2024.105419.

[54] Y. Sun, X. Zhang, and Y. Miao, 'A review of point cloud segmentation for understanding 3D indoor scenes', *Visual Intelligence*, vol. 2, no. 1, p. 14, Jun. 2024, doi: 10.1007/s44267-024-00046-x.

[55] S. Giancola, M. Valenti, and R. Sala, *A Survey on 3D Cameras: Metrological Comparison of Time-of-Flight, Structured-Light and Active Stereoscopy Technologies*. Cham: Springer International Publishing, 2018. doi: 10.1007/978-3-319-91761-0.

[56] S. Manzoor *et al.*, '3D Recognition Based on Sensor Modalities for Robotic Systems: A Survey', *Sensors*, vol. 21, no. 21, p. 7120, Oct. 2021, doi: 10.3390/s21217120.

[57] X. Huang *et al.*, 'Polarization structured light 3D depth image sensor for scenes with reflective surfaces', *Nat. Commun.*, vol. 14, no. 1, p. 6855, Oct. 2023, doi: 10.1038/s41467-023-42678-5.

[58] Photoneo, 'Scanning best practices', 3D Scanning Knowledge Base - Photoneo Wiki.

[59] Photoneo, 'The Revolution in Machine Vision: A Novel Approach Finally Enables 3D Scanning in Motion', https://www.photoneo.com/the-revolution-in-machine-vision.

[60] T. Kováčovský and J. Zizka, 'PhoXi 3D Camera', in *VISION Award 2018*, Photoneo, 2018.

[61] Photoneo, 'PhoXi 3D Scanner User Manual and Installation Instructions', Aug. 2020.

[62] C. Zhuang, S. Li, and H. Ding, 'Instance segmentation based 6D pose estimation of industrial objects using point clouds for robotic bin-picking', *Robot. Comput. Integr. Manuf.*, vol. 82, p. 102541, Aug. 2023, doi: 10.1016/j.rcim.2023.102541.

[63] Z. Liu and X. Yue, 'Enhancing Point Cloud Registration for Pipe Fittings: A Coarse-to-Fine Approach with DANIP Keypoint Detection and ICP Optimization', *Sensors*, vol. 25, no. 22, p. 7012, Nov. 2025, doi: 10.3390/s25227012.

[64] L. Jiang, H. Zhao, S. Shi, S. Liu, C.-W. Fu, and J. Jia, 'PointGroup: Dual-Set Point Grouping for 3D Instance Segmentation', in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2020, pp. 4866–4875. doi: 10.1109/CVPR42600.2020.00492.

[65] S. Chen, J. Fang, Q. Zhang, W. Liu, and X. Wang, 'Hierarchical Aggregation for 3D Instance Segmentation', in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, IEEE, Oct. 2021, pp. 15447–15456. doi: 10.1109/ICCV48922.2021.01518.

[66] A. Vaswani *et al.*, 'Attention is All you Need', in *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[67] P. J. Besl and N. D. McKay, 'Method for Registration of 3-D Shapes', P. S. Schenker, Ed., Apr. 1992, pp. 586–606. doi: 10.1117/12.57955.

[68] T. Hodaň *et al.*, 'BOP: Benchmark for 6D Object Pose Estimation', 2018, pp. 19–35. doi: 10.1007/978-3-030-01249-6_2.

[69] F. Gorschlüter, P. Rojtberg, and T. Pöllabauer, 'A Survey of 6D Object Detection Based on 3D Models for Industrial Applications', *J. Imaging*, vol. 8, no. 3, p. 53, Feb. 2022, doi: 10.3390/jimaging8030053.

[70] B. Drost, M. Ulrich, N. Navab, and S. Ilic, 'Model globally, match locally: Efficient and robust 3D object recognition', in *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, IEEE, Jun. 2010, pp. 998–1005. doi: 10.1109/CVPR.2010.5540108.

[71] Y.-X. Zhang *et al.*, 'Deep Learning-Based Point Cloud Registration: A Comprehensive Survey and Taxonomy', *Int. J. Comput. Vis.*, vol. 134, no. 4, p. 168, Apr. 2026, doi: 10.1007/s11263-025-02723-w.

[72] M. Lyu, J. Yang, Z. Qi, R. Xu, and J. Liu, 'Rigid pairwise 3D point cloud registration: A survey', *Pattern Recognit.*, vol. 151, p. 110408, Jul. 2024, doi: 10.1016/j.patcog.2024.110408.

[73] Y. Chen and G. Medioni, 'Object modelling by registration of multiple range images', *Image Vis. Comput.*, vol. 10, no. 3, pp. 145–155, Apr. 1992, doi: 10.1016/0262-8856(92)90066-C.

[74] A. Segal, D. Haehnel, and S. Thrun, 'Generalized-ICP', in *Robotics: Science and Systems V*, Robotics: Science and Systems Foundation, Jun. 2009. doi: 10.15607/RSS.2009.V.021.

[75] Q.-Y. Zhou, J. Park, and V. Koltun, 'Fast Global Registration', 2016, pp. 766–782. doi: 10.1007/978-3-319-46475-6_47.

[76] J. Li, Q. Hu, Y. Zhang, and M. Ai, 'Robust symmetric iterative closest point', *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 185, pp. 219–231, Mar. 2022, doi: 10.1016/j.isprsjprs.2022.01.019.

[77] R. B. Rusu, N. Blodow, and M. Beetz, 'Fast Point Feature Histograms (FPFH) for 3D registration', in *2009 IEEE International Conference on Robotics and Automation*, IEEE, May 2009, pp. 3212–3217. doi: 10.1109/ROBOT.2009.5152473.

[78] M. A. Fischler and R. C. Bolles, 'Random Sample Consensus: A Paradigm for Model Fitting With Applications to Image Analysis and Automated Cartography', *Commun. ACM*, vol. 24, no. 6, pp. 381–395, Jun. 1981, doi: 10.1145/358669.358692.

[79] Y. Aoki, H. Goforth, R. A. Srivatsan, and S. Lucey, 'PointNetLK: Robust & Efficient Point Cloud Registration Using PointNet', in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2019, pp. 7156–7165. doi: 10.1109/CVPR.2019.00733.

[80] Y. Wang and J. Solomon, 'Deep Closest Point: Learning Representations for Point Cloud Registration', in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, IEEE, Oct. 2019, pp. 3522–3531. doi: 10.1109/ICCV.2019.00362.

[81] Z. Dang, L. Wang, Y. Guo, and M. Salzmann, 'Match Normalization: Learning-Based Point Cloud Registration for 6D Object Pose Estimation in the Real World', *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 6, pp. 4489–4503, Jun. 2024, doi: 10.1109/TPAMI.2024.3355198.

[82] Z. J. Yew and G. H. Lee, 'RPM-Net: Robust Point Matching Using Learned Features', in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2020, pp. 11821–11830. doi: 10.1109/CVPR42600.2020.01184.

[83] S. Huang, Z. Gojcic, M. Usvyatsov, A. Wieser, and K. Schindler, 'PREDATOR: Registration of 3D Point Clouds with Low Overlap', in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2021, pp. 4265–4274. doi: 10.1109/CVPR46437.2021.00425.

[84] Z. Qin *et al.*, 'GeoTransformer: Fast and Robust Point Cloud Registration With Geometric Transformer', *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 9806–9821, Aug. 2023, doi: 10.1109/TPAMI.2023.3259038.

[85] S. Hinterstoisser *et al.*, 'Model Based Training, Detection and Pose Estimation of Texture-Less 3D Objects in Heavily Cluttered Scenes', 2013, pp. 548–562. doi: 10.1007/978-3-642-37331-2_42.

[86] C. Ferrari and J. Canny, 'Planning optimal grasps', in *Proceedings 1992 IEEE International Conference on Robotics and Automation*, IEEE Comput. Soc. Press, pp. 2290–2295. doi: 10.1109/ROBOT.1992.219918.

[87] A. Bicchi and V. Kumar, 'Robotic grasping and contact: a review', in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, IEEE, pp. 348–353. doi: 10.1109/ROBOT.2000.844081.

[88] J. Bohg, A. Morales, T. Asfour, and D. Kragic, 'Data-Driven Grasp Synthesis—A Survey', *IEEE Transactions on Robotics*, vol. 30, no. 2, pp. 289–309, Apr. 2014, doi: 10.1109/TRO.2013.2289018.

[89] J. Mahler *et al.*, 'Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics', *Robotics: Science and Systems*, 2017.

[90] D. Coleman, L. A. Șucan, S. Chitta, and N. Correll, 'Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study', *ournal of Software Engineering for Robotics*, May 2014.

[91] M. Gorner, R. Haschke, H. Ritter, and J. Zhang, 'MoveIt! Task Constructor for Task-Level Motion Planning', in *2019 International Conference on Robotics and Automation (ICRA)*, IEEE, May 2019, pp. 190–196. doi: 10.1109/ICRA.2019.8793898.

[92] B. Zhang, Y. Xie, J. Zhou, K. Wang, and Z. Zhang, 'State-of-the-art robotic grippers, grasping and control strategies, as well as their applications in agricultural robots: A review', *Comput. Electron. Agric.*, vol. 177, p. 105694, Oct. 2020, doi: 10.1016/j.compag.2020.105694.

[93] N. Boubekri and P. Chakraborty, 'Robotic grasping: gripper designs, control methods and grasp configurations – a review of research', *Integrated Manufacturing Systems*, vol. 13, no. 7, pp. 520–531, Nov. 2002, doi: 10.1108/09576060210442978.

[94] K. Tai, A.-R. El-Sayed, M. Shahriari, M. Biglarbegian, and S. Mahmud, 'State of the Art Robotic Grippers and Applications', *Robotics*, vol. 5, no. 2, p. 11, Jun. 2016, doi: 10.3390/robotics5020011.

[95] M. Guo *et al.*, 'Design of parallel-jaw gripper tip surfaces for robust grasping', in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2017, pp. 2831–2838. doi: 10.1109/ICRA.2017.7989330.

[96] M. Gualtieri, A. ten Pas, K. Saenko, and R. Platt, 'High precision grasp pose detection in dense clutter', in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Oct. 2016, pp. 598–605. doi: 10.1109/IROS.2016.7759114.

[97] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, 'Probabilistic roadmaps for path planning in high-dimensional configuration spaces', *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996, doi: 10.1109/70.508439.

[98] S. M. LaValle, 'Rapidly-Exploring Random Trees: A New Tool for Path Planning', *The annual research report*, 1998.

[99] J. J. Kuffner and S. M. LaValle, 'RRT-connect: An efficient approach to single-query path planning', in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, IEEE, Apr. 2000, pp. 995–1001. doi: 10.1109/ROBOT.2000.844730.

[100] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, 'CHOMP: Gradient optimization techniques for efficient motion planning', in *2009 IEEE International Conference on Robotics and Automation*, IEEE, May 2009, pp. 489–494. doi: 10.1109/ROBOT.2009.5152817.

[101] C. Wu, X. Bi, J. Pfrommer, A. Cebulla, S. Mangold, and J. Beyerer, 'Sim2real Transfer Learning for Point Cloud Segmentation: An Industrial Application Case on Autonomous Disassembly', in *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, IEEE, Jan. 2023, pp. 4520–4529. doi: 10.1109/WACV56688.2023.00451.

[102] J. Tremblay *et al.*, 'Training Deep Networks with Synthetic Data: Bridging the Reality Gap by Domain Randomization', in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, IEEE, Jun. 2018, pp. 1082–10828. doi: 10.1109/CVPRW.2018.00143.

[103] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, 'Domain randomization for transferring deep neural networks from simulation to the real world', in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Sep. 2017, pp. 23–30. doi: 10.1109/IROS.2017.8202133.

[104] E. Todorov, T. Erez, and Y. Tassa, 'MuJoCo: A Physics Engine for Model-Based Control', in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, Oct. 2012, pp. 5026–5033. doi: 10.1109/IROS.2012.6386109.

[105] E. Coumans, 'Bullet physics simulation', in *ACM SIGGRAPH 2015 Courses*, New York, NY, USA: ACM, Jul. 2015, p. 1. doi: 10.1145/2776880.2792704.

[106] N. Koenig and A. Howard, 'Design and use paradigms for gazebo, an open-source multi-robot simulator', in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, IEEE, 2004, pp. 2149–2154. doi: 10.1109/IROS.2004.1389727.

[107] R. Smith, 'Open Dynamics Engine', http://www.ode.org/.

[108] NVIDIA, 'Isaac Sim', *NVIDIA*: 5.1.0.

[109] S. Katz, A. Tal, and R. Basri, 'Direct visibility of point sets', in *ACM SIGGRAPH 2007 papers*, New York, NY, USA: ACM, Jul. 2007, p. 24. doi: 10.1145/1275808.1276407.

[110] NVIDIA, 'Domain Randomization With Replicator', Isaac Sim Documentation.

[111] S. Huch and M. Lienkamp, 'Towards Minimizing the LiDAR Sim-to-Real Domain Shift: Object-Level Local Domain Adaptation for 3D Point Clouds of Autonomous Vehicles', *Sensors*, vol. 23, no. 24, p. 9913, Dec. 2023, doi: 10.3390/s23249913.

[112] M. Quigley *et al.*, 'ROS: an open-source Robot Operating System', *ICRA workshop on open source software*, vol. 3, May 2009.

[113] ROS-Industrial, 'ROS-Industrial', https://rosindustrial.org/.

[114] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, 'Digital Twin in manufacturing: A categorical literature review and classification', *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018, doi: 10.1016/j.ifacol.2018.08.474.

[115] M. Soori, B. Arezoo, and R. Dastres, 'Digital twin for smart manufacturing, A review', *Sustainable Manufacturing and Service Economics*, vol. 2, p. 100017, Apr. 2023, doi: 10.1016/j.smse.2023.100017.

[116] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, 'Digital Twin in Industry: State-of-the-Art', *IEEE Trans. Industr. Inform.*, vol. 15, no. 4, pp. 2405–2415, Apr. 2019, doi: 10.1109/TII.2018.2873186.

[117] M. Singh, E. Fuenmayor, E. Hinchy, Y. Qiao, N. Murray, and D. Devine, 'Digital Twin: Origin to Future', *Applied System Innovation*, vol. 4, no. 2, p. 36, May 2021, doi: 10.3390/asi4020036.

[118] M. Singh *et al.*, 'Applications of Digital Twin across Industries: A Review', *Applied Sciences*, vol. 12, no. 11, p. 5727, Jun. 2022, doi: 10.3390/app12115727.

[119] M. Singh *et al.*, 'Unity and ROS as a Digital and Communication Layer for Digital Twin Application: Case Study of Robotic Arm in a Smart Manufacturing Cell', *Sensors*, vol. 24, no. 17, p. 5680, Aug. 2024, doi: 10.3390/s24175680.

[120] S. Chitta, I. Sucan, and S. Cousins, 'MoveIt! [ROS Topics]', *IEEE Robot. Autom. Mag.*, vol. 19, no. 1, pp. 18–19, Mar. 2012, doi: 10.1109/MRA.2011.2181749.

[121] P. Hoffmann, R. Schumann, T. M. Maksoud, and G. C. Premier, 'Virtual Commissioning of Manufacturing Systems a Review and New Approaches for Simplification', *Proceedings 24th European Conference on Modelling and Simulation*, vol. 1, pp. 175–181, 2010.

[122] R. Raffaeli, P. Bilancia, F. Neri, M. Peruzzini, and M. Pellicciari, 'Engineering Method and Tool for the Complete Virtual Commissioning of Robotic Cells', *Applied Sciences*, vol. 12, no. 6, p. 3164, Mar. 2022, doi: 10.3390/app12063164.

[123] M. Kolodiazhnyi, A. Vorontsova, A. Konushin, and D. Rukhovich, 'OneFormer3D: One Transformer for Unified Point Cloud Segmentation', in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2024, pp. 20943–20953. doi: 10.1109/CVPR52733.2024.01979.

[124] K. S. Khor, C. Liu, and C. C. Cheah, 'Robotic Grasping of Unknown Objects Based on Deep Learning-Based Feature Detection', *Sensors*, vol. 24, no. 15, p. 4861, Jul. 2024, doi: 10.3390/s24154861.

[125] I. A. Sucan, M. Moll, and L. E. Kavraki, 'The Open Motion Planning Library', *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, Dec. 2012, doi: 10.1109/MRA.2012.2205651.

[126] X. Li *et al.*, 'A Sim-to-Real Object Recognition and Localization Framework for Industrial Robotic Bin Picking', *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 3961–3968, Apr. 2022, doi: 10.1109/LRA.2022.3149026.

[127] ROS-Industrial, 'abb_driver', https://github.com/ros-industrial/abb_driver.

[128] ABB Robotics, 'Application Manual - Robot Web Services', https://developercenter.robotstudio.com/api/rwsApi/. [Online]. Available: https://developercenter.robotstudio.com/api/rwsApi/

[129] ROS-Industrial, 'abb_librws', https://github.com/ros-industrial/abb_librws/tree/master.

[130] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, 'STOMP: Stochastic trajectory optimization for motion planning', in *2011 IEEE International Conference on Robotics and Automation*, IEEE, May 2011, pp. 4569–4574. doi: 10.1109/ICRA.2011.5980280.

[131] M. Moll, I. A. Sucan, and L. E. Kavraki, 'Benchmarking Motion Planning Algorithms: An Extensible Infrastructure for Analysis and Visualization', *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 96–102, Sep. 2015, doi: 10.1109/MRA.2015.2448276.

[132] S. Liu and P. Liu, 'Benchmarking and optimization of robot motion planning with motion planning pipeline', *The International Journal of Advanced Manufacturing Technology*, vol. 118, no. 3–4, pp. 949–961, Jan. 2022, doi: 10.1007/s00170-021-07985-5.

[133] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, and others, 'A density-based algorithm for discovering clusters in large spatial databases with noise', in *kdd*, 1996, pp. 226–231.

[134] F. Pedregosa *et al.*, 'Scikit-learn: Machine learning in Python', *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.

[135] S. Wold, K. Esbensen, and P. Geladi, 'Principal component analysis', *Chemometrics and Intelligent Laboratory Systems*, vol. 2, no. 1–3, pp. 37–52, Aug. 1987, doi: 10.1016/0169-7439(87)80084-9.

[136] B. Gerkey, 'Why ROS 2?'

[137] K. Fazzari, 'ROS 2 DDS-Security integration', Jul. 2020.

[138] W. Woodall, 'ROS on DDS', Jul. 2019.

[139] ROS 2 Documentation, 'Quality of Service settings'.

[140] F. Poiesi and D. Boscaini, 'Learning general and distinctive 3D local deep descriptors for point cloud registration', *IEEE Trans. Pattern Anal. Mach. Intell.*, pp. 1–1, 2022, doi: 10.1109/TPAMI.2022.3175371.

[141] J. P. . Beaumont, *Runner and gating design handbook : tools for successful injection molding*. Hanser Publications, 2019.

[142] A. Riley, 'Plastics manufacturing processes for packaging materials', in *Packaging Technology*, Elsevier, 2012, pp. 310–360. doi: 10.1533/9780857095701.2.310.

[143] A. A. Tseng and J. D. Kaplan, 'A computer-aided analysis of automatic degating molds for injection molding of plastic balls', *Polym. Eng. Sci.*, vol. 34, no. 3, pp. 238–248, Feb. 1994, doi: 10.1002/pen.760340310.

[144] X. Bouthillier *et al.*, 'Accounting for Variance in Machine Learning Benchmarks', in *Proceedings of Machine Learning and Systems*, A. Smola, A. Dimakis, and I. Stoica, Eds., 2021, pp. 747–769. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2021/file/0184b0cd3cfb185989f858a1d9f5c1eb-Paper.pdf

[145] C. Summers and M. J. Dinneen, ‘Nondeterminism and Instability in Neural Network Optimization’, in *Proceedings of the 38th International Conference on Machine Learning*, M. Meila and T. Zhang, Eds., in Proceedings of Machine Learning Research, vol. 139. PMLR, Aug. 2021, pp. 9913–9922. [Online]. Available: https://proceedings.mlr.press/v139/summers21a.html

[146] Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, ‘PoseCNN: A Convolutional Neural Network for 6D Object Pose Estimation in Cluttered Scenes’, in *Robotics: Science and Systems XIV*, Robotics: Science and Systems Foundation, Jun. 2018. doi: 10.15607/RSS.2018.XIV.019.

[147] K. Kleeberger, C. Landgraf, and M. F. Huber, ‘Large-scale 6D Object Pose Estimation Dataset for Industrial Bin-Picking’, in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Nov. 2019, pp. 2573–2578. doi: 10.1109/IROS40897.2019.8967594.

[148] A. Cordeiro, L. F. Rocha, J. Boaventura-Cunha, E. J. S. Pires, and J. P. Souza, ‘Object segmentation dataset generation framework for robotic bin-picking: Multi-metric analysis between results trained with real and synthetic data’, *Comput. Ind. Eng.*, vol. 205, p. 111139, Jul. 2025, doi: 10.1016/j.cie.2025.111139.

APPENDICES

A. Photoneo PhoXi Scanner S Calibration Sheet

This appendix presents the calibration sheet and technical procedure used to define the working coordinate frame of the Photoneo PhoXi 3D Scanner S used in this research. The calibration was carried out using the MarkerSpace functionality available in PhoXi Control. This allowed the point cloud to be expressed relative to a physical marker pattern placed in the scanning area, rather than relative to the scanner body.

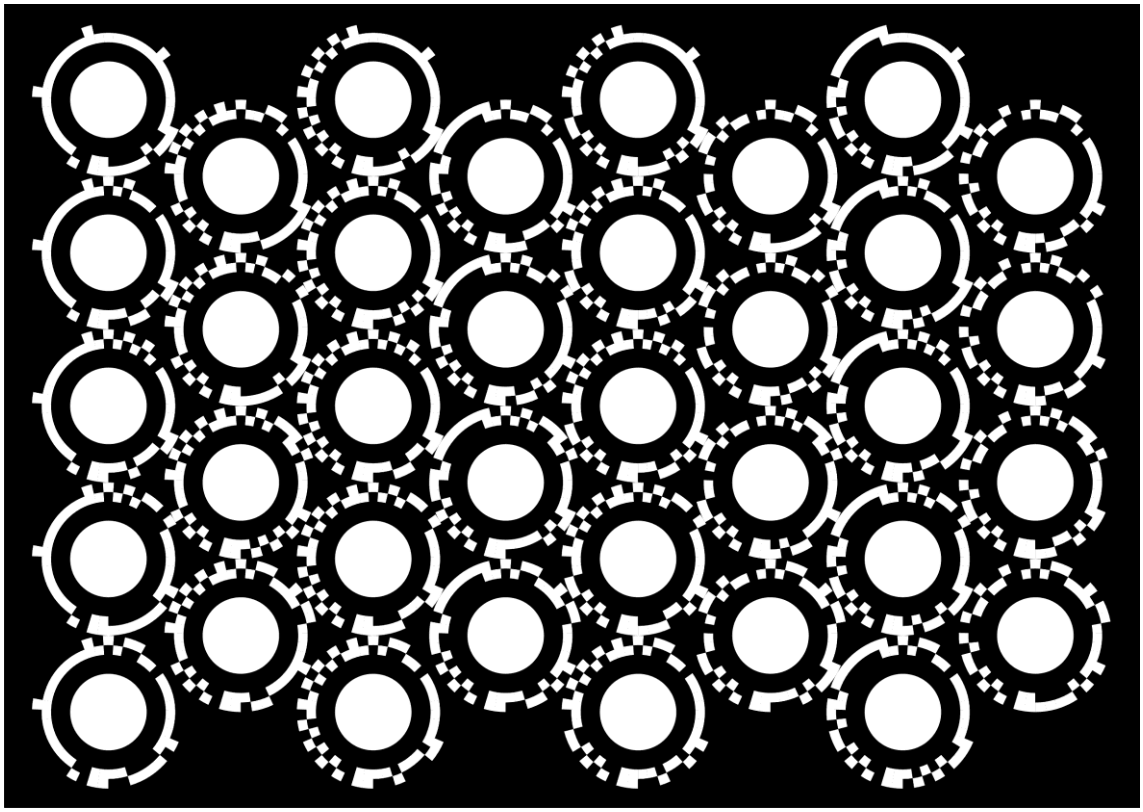


Figure A.1: Photoneo A4 marker calibration sheet used to define MarkerSpace for the PhoXi Scanner S. The marker sheet defines the physical reference plane used to align point-cloud acquisition with the robotic-cell coordinate structure.

The default coordinate space of the PhoXi scanner is CameraSpace. In this coordinate system, the origin is located at the scanner camera sensor, the XY plane is defined by the camera sensor, and the Z-axis represents the depth direction from the sensor to the scene. For the *PhoXi Scanner S*, the camera sensor is mounted at an angle of 15.45 degrees relative to the scanner body. This means that the point cloud axes are not necessarily aligned with the conveyor surface or the physical base of the robotic cell. For this reason, MarkerSpace was used to define a more convenient and repeatable reference frame for the pick-and-place experiments.

MarkerSpace is defined by the physical marker pattern. The origin of the coordinate system is located at the centre of one of the corner circles of the marker sheet. The XY plane is defined by the plane of the marker pattern. The X-axis is aligned through neighbouring circle centres on the shorter side of the pattern, while the Y-axis

is aligned along the longer side of the pattern. The Z-axis is perpendicular to the marker plane and points upwards from the calibration sheet.

The marker sheet used for the *PhoXi Scanner S* was the A4 marker pattern recommended for this scanner model. The A4 sheet has a paper size of 210 × 297 mm. The marker was printed at 100 percent scale, and therefore the Marker Scale parameters in PhoXi Control were set to $\mathrm{X} = 1$ and $Y = 1$. If the marker pattern is printed with scaling, the scale values must be adjusted in PhoXi Control because incorrect scaling can shift the computed origin away from the centre of the marker circle. The calibration procedure used in this work was as follows:

1. The PhoXi Scanner S was connected in PhoXi Control.
2. The marker sheet was placed flat in the scanning area, aligned with the supporting surface used as the reference plane.
3. In the Coordinate Settings menu, the Coordinate Space parameter was set to MarkerSpace.
4. The Recognise Markers option was enabled so that the scanner could detect the marker pattern in the scene.
5. The Save Transformations option was enabled so that the transformation from CameraSpace to MarkerSpace could be stored on the scanner.
6. The Set button was used to apply the calibration settings.
7. The Set and Store button was used to save the transformation in the active scanner profile.
8. The 3D viewer axes were checked to confirm that the point cloud was expressed in MarkerSpace.

For the calibration to be valid, a sufficient part of the marker pattern had to be visible to the scanner. The manufacturer recommends that at least an area of 2 columns × 3 rows of marker circles should be visible, and the visible circles must not be collinear. For the *PhoXi Scanner S* using an A4 marker pattern, the manufacturer reports a recognisable distance range of approximately 380 mm to 580 mm and a maximum marker angle of approximately 80 degrees from the position parallel to the scanner.

After calibration, the marker sheet did not need to remain in the scene for normal scanning. The scanner was used with Coordinate Space set to MarkerSpace, while Recognize Markers and Save Transformations were disabled. This allowed subsequent point clouds to be generated directly in the calibrated marker coordinate frame. The calibration remains valid only while the relative position and orientation between the scanner and the calibrated workspace remain unchanged. If the scanner is moved, or if the geometry between the scanner and the marker plane changes, the MarkerSpace transformation must be recalibrated.

Table A.1: The main calibration parameters used in this work.

Parameter	Value
Scanner model	Photoneo PhoXi 3D Scanner S
Calibration method	PhoXi Control MarkerSpace
Default coordinate space	CameraSpace
Calibrated coordinate space	MarkerSpace
Scanner sensor mounting angle	15.45°
Marker pattern size	A4
A4 sheet size	210 mm × 297 mm
Marker Scale X	1
Marker Scale Y	1
Minimum visible marker area	2 columns × 3 rows of circles
Required circle arrangement	Non-collinear visible circles
Recognisable distance range for PhoXi S with A4 marker	380 mm to 580 mm
Maximum marker angle for PhoXi S with A4 marker	80°
Calibration setting	Coordinate Space set to MarkerSpace
Calibration setting	Recognize Markers enabled
Calibration setting	Save Transformations enabled
Normal-use setting after calibration	Recognize Markers disabled
Normal-use setting after calibration	Save Transformations disabled

B. Two-Finger Electronic Gripper Development

This appendix describes the development of the custom two-finger electronic gripper used in the second experimental stage of this thesis. The gripper was developed to provide a software-controlled end-effector for the manipulation of complex industrial parts within the robotic cell. Its development included mechanical design, additive manufacturing, electronic hardware, embedded firmware, ROS communication, digital description and MoveIt integration.

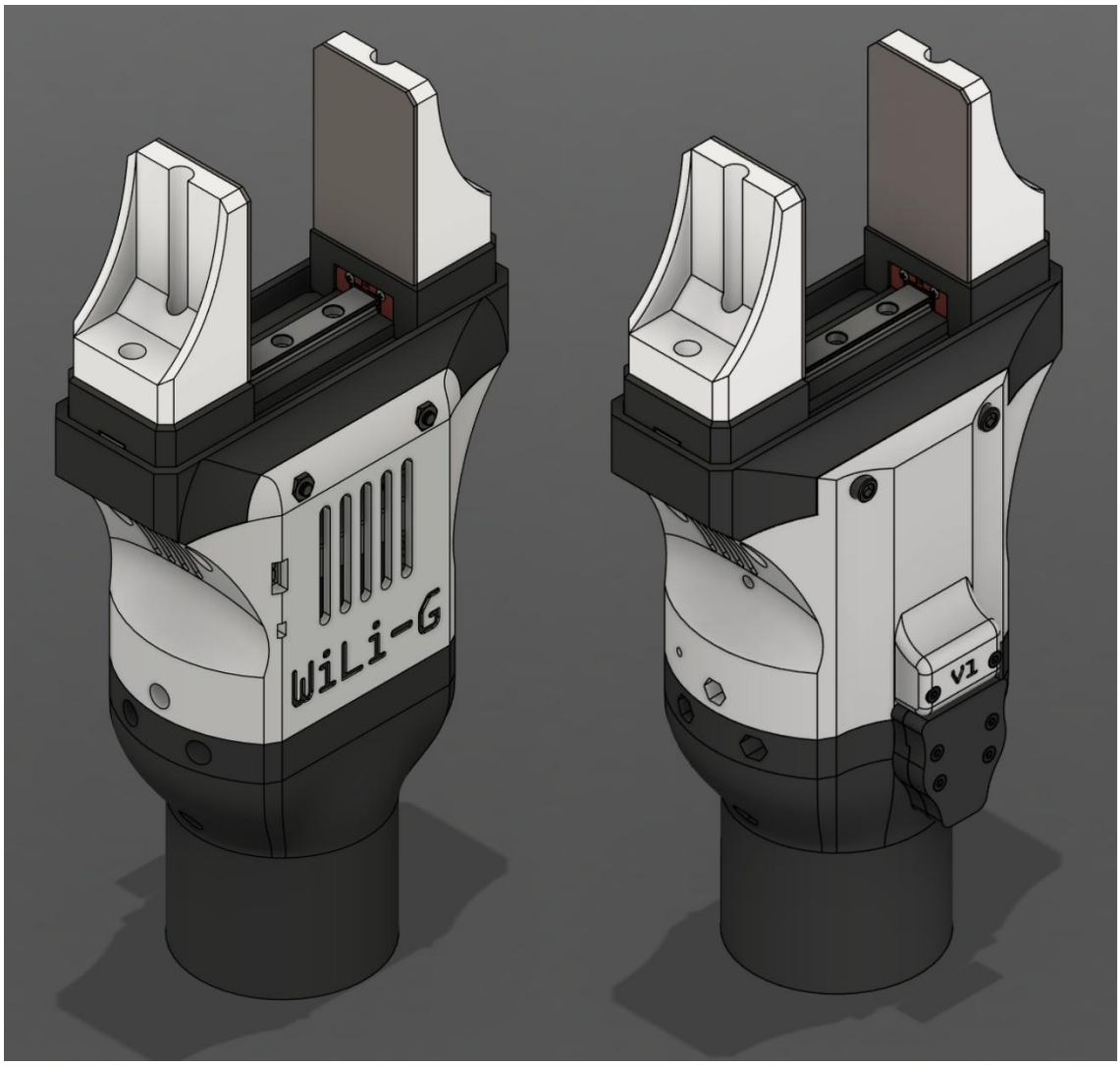


Figure B.1: CAD model of the WiLi-Gripper showing the main body, two-finger structure, flange interface and tool-frame region.

The first experimental stage described in Chapter 3 used a pneumatic gripper to manipulate simple rectangular objects. This was sufficient for validating the foundational robotic-cell architecture because the task was constrained to top-down picking and simple object geometries. However, the second experimental stage required an end-effector that could be controlled directly from the ROS-based software architecture and represented as part of the digital robotic cell. For this reason, a custom electronic gripper was developed.

The gripper was named WiLi-Gripper, referring to Wireless communication, Linear motion and Internet of Things (IoT)-based control. The name reflects three main design aspects of the device. First, the gripper communicates with the ROS computer

through a wireless network interface, although it still requires a wired 24 V power supply for operation. Second, the gripping motion is represented as a linear finger displacement. Third, the gripper operates as an embedded IoT-style mechatronic device controlled by an ESP32 microcontroller.

The gripper was developed as a modular end-effector subsystem. The mechanical structure provides the physical finger motion. The electronic layer controls the motor and reads the sensors. The embedded firmware receives commands and publishes gripper feedback. The ROS interface connects the device to the pick-and-place software pipeline. The digital description and MoveIt configuration allow the gripper to be visualised and commanded in the same environment used by the robotic cell.

B.1 Design Requirements

The design requirements were defined according to the needs of the second experimental stage. The gripper had to be compatible with the ABB IRB 1200 robot and compact enough to operate within the available workspace. It also had to be suitable for grasping the complex industrial parts used in Chapter 4.

The gripper had to be represented in the ROS digital model. This was necessary because the end-effector geometry affects visualisation, tool-frame definition and motion planning. The gripper therefore needed a URDF/Xacro model, visual meshes, collision meshes and MoveIt configuration files.

The gripper was also required to support software-based actuation during the pick-and-place sequence. It was required to open before the robot approached the selected object, close at the picking pose until the configured force-sensitive resistor (FSR) threshold was reached, remain closed while the object was transferred and open again at the placing pose. These actions had to be synchronised with the robot motion sequence so that gripper actuation occurred at the correct stage of the task. For this reason, the gripper was designed to receive high-level commands from ROS and to return feedback describing its estimated opening and contact state.

A further requirement was modularity. The mechanical design, embedded controller, firmware, ROS interface and MoveIt model had to be developed and tested as separate layers. This followed the same modular design principle used throughout this work. The robot controller remains responsible for robot motion, while the gripper controller remains responsible for local end-effector actuation.

The gripper was developed as a research prototype for integration with the robotic-cell framework. It was not intended to be evaluated as a commercial industrial end-effector. Its purpose was to support the second experimental stage by providing a custom, software-controlled, ROS-integrated grasping device. The project was also developed as an open-source system, allowing other researchers to replicate, study and extend the gripper by accessing the repository: https://gitlab.com/tus4/wiligripper.

B.2 Mechanical Design

The mechanical design followed a two-finger parallel-gripper structure. The gripper head contains two opposing jaws that move towards and away from each other. This produces a parallel closing and opening motion suitable for grasping objects whose graspable width is within the maximum total jaw opening of 48 mm of the gripper and whose geometry provides suitable contact surfaces for the two jaws.

The actuation mechanism is based on a rack-and-pinion arrangement. A central gear is connected to the motor shaft. The gear engages with teeth on the two sliding jaws. When the motor rotates the gear in one direction, the jaws move apart and the gripper opens. When the motor rotates in the opposite direction, the jaws move towards each other and the gripper closes. Figure B.2 shows the internal top view of the gripper head, where the central gear, rack teeth, left and right jaws, and limit switch can be observed.

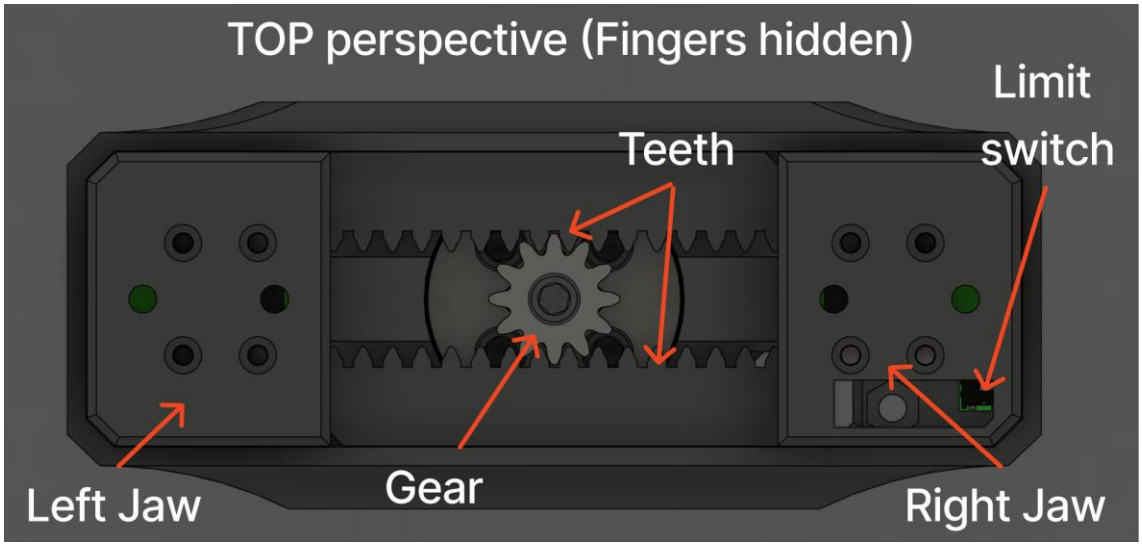


Figure B.2: Internal top view of the gripper head showing the central pinion, integrated jaw racks, linear guide arrangement and fully open limit switch.

This mechanism was selected because it converts rotary motor motion into symmetric linear jaw motion using a compact mechanical arrangement. It also allows the two jaws to be driven by a single actuator. The mechanism is suitable for the digital representation of the gripper because the two jaw motions can be modelled as coupled prismatic movements.

The jaws slide along a linear rail mounted inside the gripper head. The rail constrains the jaw motion and helps maintain alignment during opening and closing. The rack teeth are integrated with the sliding jaw structure, while the central gear transfers the motor rotation to both jaws at the same time. As also shown in Figure B.2, the limit switch is positioned on the right side of the gripper head and is activated when the right jaw reaches the fully open position.

The gripper was mounted to the ABB robot through a custom flange and adapter. The digital model includes the main elements required for this connection, including the gripper flange, main body, left jaw, right jaw and tool centre point. The tool centre point provides the reference used for end-effector positioning in the robotic-cell model. The main body, jaw supports, covers and mounting components were designed as custom parts and manufactured using additive manufacturing.

This approach supported rapid prototyping and allowed the geometry, internal component arrangement and cable-routing features to be modified during development. The materials selected for the manufactured parts and gripping surfaces are described in the next section.

B.3 Materials and Manufactured Components

The WiLi-Gripper was developed using a combination of 3D-printed structural components, standard mechanical components and embedded electronic hardware. The main body, jaw supports, covers, mounting features and electronics housing were designed as custom components and 3D printed, while elements such as bolts, the linear rail and sliders were standard steel components that were purchased and integrated into the design to provide strength and durability.

The complete mechanical and electronic arrangement of the WiLi-Gripper is presented in Figure B.3. The exploded view shows the relationship between the custom printed components, jaw assemblies, linear guide system, central pinion, stepper motor, electronic control boards, electrical connectors and robot-mounting adapter. It also identifies the principal materials used in the manufactured components. The figure illustrates how the mechanical transmission, sensing elements and embedded electronics were integrated within a compact end-effector assembly.

Item	Name	Material
1	Custom fingertips	PLA
2	3M Gripping Material TB641	Thermoplastic
3	FSR	-
4	Jaw bodies with integrated racks	PLA
5	Linear guide carriages	Steel
6	Linear guide rail	Steel
7	Central pinion gear	Nylon
8	Gripper head housing	PLA
9	Pinion mounting hub	PLA
10	ESP32 Board	-
11	Rigid shaft coupling	Alloy steel
12	DRV8825 motor driver module	-
13	DC-DC buck converter module	-
14	Stepper motor	-
15	JST electrical connector	-
16	Custom ABB flange adapter	PETG-CF

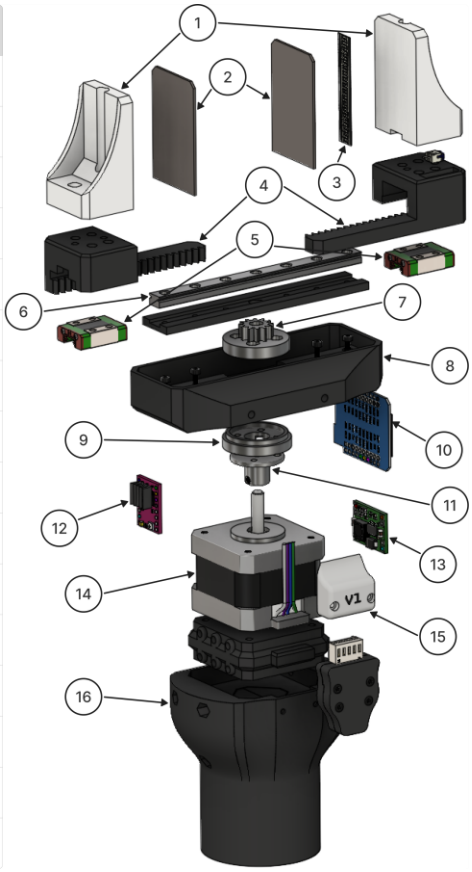


Figure B.3: Exploded view of the WiLi-Gripper showing the principal mechanical, electronic, sensing and gripping components.

Most structural components were manufactured from polylactic acid (PLA). This material was selected because it supported rapid prototyping, easy design iteration and sufficient stiffness for the laboratory application. Its use allowed the dimensions, internal mounting features and external geometry of the gripper to be modified without requiring specialised manufacturing equipment. However, the custom adapter connecting the gripper to the ABB tool flange was manufactured from carbon-fibre-reinforced PETG (PETG-CF) to provide greater stiffness and durability at the robot-to-gripper mounting interface.

Nylon filament was used for the central gear. Unlike the main structural components, the gear is exposed to repeated contact with the rack teeth during opening and closing. Nylon was considered more suitable for this component because of its improved wear behaviour and reduced brittleness compared with PLA under repeated gear contact.

The gripping surfaces were designed to increase friction and reduce the risk of object slippage during lifting and transfer. Different contact materials were considered during development, and 3M Gripping Material TB641 was selected for the final prototype. TB641 is a soft thermoplastic elastomer material with a micro-replicated surface formed by a large number of small gripping features. These features increase the interaction between the gripping surface and the object, while the compliant material accommodates minor variations in object geometry.

Table B.1: Main mechanical, electronic and sensing components used in the WiLi-Gripper.

Component group	Role in the gripper
PLA printed components	Main body, covers, jaw supports and structural parts
Nylon printed gear	Central gear used in the rack-and-pinion mechanism
42-34 BJ12D15 Stepper motor	Actuator used to rotate the central gear
DRV8825 driver	Motor driver used to control the stepper motor
ESP32 board	Embedded controller and network interface
Linear rail and carriage	Mechanical guide for the jaw motion
Flange and couplings	Mechanical connection and motion transmission
JST connectors and circular connectors	Electrical connection and maintenance
Buck converter	Voltage conversion for embedded electronics
Limit switch	Fully open reference for homing
Force-sensitive resistor	Contact detection during grasping
3M gripping material TB641	Contact material used on the gripping surface
Wiring and capacitors	Electrical connection and signal stability

The material has a nominal thickness of 0.97 mm and includes an acrylic pressure-sensitive adhesive, allowing it to be attached directly to the jaw surfaces. A layer was applied to the contact surface of each jaw. In addition to improving friction, the flexible layer transfers contact pressure to the FSR positioned beneath the material

on the right jaw. The sensor arrangement and contact-detection process are described in Section B.4.

B.4 Actuation and Sensing System

The gripper is actuated by a stepper motor coupled to the central pinion described in Section B.2. The motor provides controlled incremental rotation, allowing the jaw displacement and movement speed to be determined from the commanded number of steps and the interval between step pulses.

A DRV8825 stepper motor driver provides the low-level interface between the Wemos D1 Mini ESP32 board and the motor. The ESP32 generates the direction signal required to select opening or closing motion and the step pulses required to control the motor displacement. The DRV8825 converts these control signals into the electrical drive sequence needed by the stepper motor, which then actuates the parallel jaw mechanism.

The sensing system provides two basic feedback functions. The first is homing through a limit switch. The limit switch is located on the right side of the gripper head. When the right jaw reaches the fully open position, it activates the limit switch. This position is used as the reference for the fully open state.

The second feedback function is contact detection through a FSR. The sensor is positioned on the contact surface of the right jaw underneath the 3M Gripping Material TB641 layer. As shown in Figure B.4, the left side of the figure shows the right finger of the gripper with the gripping material applied to the contact surface, while the right side shows the same finger with the gripping material removed, exposing the FSR mounted underneath. When the jaws close around an object, the contact pressure is transferred through the compliant gripping material to the sensor. The firmware monitors the sensor value during closing and compares it with a configurable threshold. When the threshold is reached, the firmware identifies that contact has occurred and stops the closing movement.

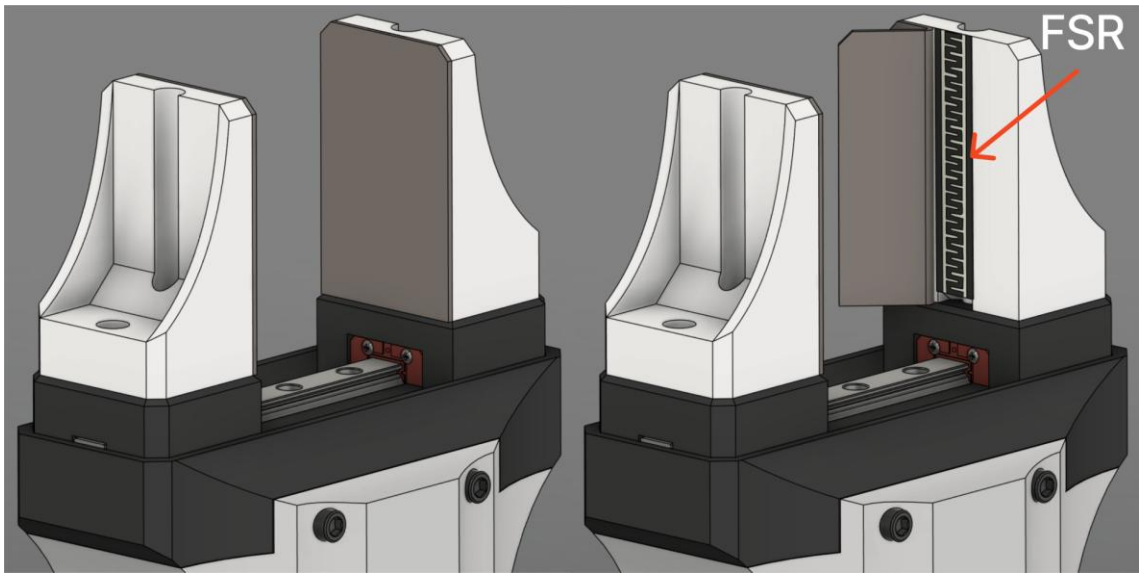


Figure B.4: Right gripper finger showing the 3M gripping layer on the contact surface (left) and the FSR mounted underneath (right).

This sensing layout was selected because it supports the minimum feedback required by the pick-and-place task. The limit switch provides a repeatable reference for the gripper opening. The FSR provides a simple contact-detection signal during grasping.

B.5 Embedded Electronic Hardware

The electronic control system was organised around three principal boards. These were an ESP32 board, a DRV8825 stepper-motor driver and an adjustable direct-current (DC)-to-DC step-down converter module based on the MP1584EN regulator. Together, these boards provided embedded processing, wireless communication, motor control and voltage regulation.

A Wemos D1 Mini ESP32 board provided the main embedded control and communication functions of the gripper. It received commands from the ROS computer, generated the direction and step signals required by the motor driver, monitored the limit switch and FSR, and transmitted gripper-state feedback through the network interface.

The DRV8825 stepper-motor driver provided the electrical interface between the ESP32 board and the stepper motor. Direction and step signals generated by the ESP32 board were received by the driver and converted into the phase currents required to rotate the motor. The driver therefore separated the low-power control signals from the higher-power motor-actuation circuit. The firmware defines the main hardware pins used by the device. These are summarised in Table B.2.

Table B.2: Main ESP32 pin assignments used by the gripper firmware.

Signal	ESP32 pin	Function
Direction signal	25	Sets the motor direction
Step signal	22	Generates motor steps
Sleep control	26	Enables or disables the motor driver
Reset control	27	Resets the motor driver
FSR analogue input	34	Reads contact pressure signal
FSR power	2	Powers the FSR circuit
Limit switch input	4	Detects fully open jaw position

The gripper was supplied from a 24 V DC source. This voltage was suitable for the motor-actuation circuit but could not be connected directly to the ESP32 board. An adjustable step-down converter module based on the MP1584EN regulator was therefore used to reduce the 24 V supply to 5 V. The regulated 5 V output was connected to the power input of the ESP32 board. The converter allowed the low-voltage control electronics and the motor-actuation system to operate from the same external power source while maintaining the voltage required by each part of the system.

The electronic system also included wiring, connectors, capacitors and supporting components required for power distribution, sensor connection and signal stability. These components were mounted within the internal side regions of the gripper main body, where they remained separated from the moving jaws and rack-and-pinion mechanism. Ventilation openings were incorporated around the electronic compartments to support passive heat dissipation from components such as the motor driver and voltage-regulation circuitry during operation. This arrangement protected the electronics while preventing interference with jaw travel, the force-sensing elements and the object-contact surfaces.

B.6 Firmware Architecture

The embedded firmware was developed for the ESP32 using PlatformIO and the Arduino framework. The firmware is implemented in the *GripperInterface* project inside the wilig_*control* package. Its role is to initialise the hardware, manage network communication, execute motor commands, perform homing and publish gripper feedback.

At startup, the firmware initialises the motor driver pins, FSR input, limit switch input and communication interfaces. It then performs a startup homing routine. During this routine, the gripper moves towards the fully open direction until the limit switch is activated. Once the limit switch is reached, the gripper sets the internal stroke value to the fully open state.

The firmware represents the total gripper stroke as a range from 0 mm to 48 mm. A value of 0 mm represents the fully closed state. A value of 48 mm represents the fully open state. In the ROS digital model, one active prismatic joint represents half of this physical opening. For this reason, the 48 mm physical stroke corresponds to a 24 mm active joint displacement in MoveIt.

The firmware uses the mechanical relationship between the central gear and motor steps to estimate jaw displacement. The implementation uses a 12 mm gear pitch diameter and 200 motor steps per revolution. This results in an estimated linear movement of approximately 0.188 mm per motor step, calculated from the gear circumference divided by the number of steps per revolution. Because both jaws move in opposite directions, the total gripper opening changes by approximately 0.376 mm per motor step.

The firmware includes separate functions for closing and opening the gripper. During closing, the motor rotates the gear in the closing direction, and the firmware monitors the force-sensitive resistor value. If the sensor value exceeds the configured threshold, the contact state is set and the closing motion stops. During opening, the contact state is cleared, and the motor rotates in the opening direction. If the limit switch is reached, the stroke is reset to the fully open value.

This firmware structure allows the gripper to execute high-level open, close and partial-position commands while maintaining internal feedback about stroke and contact state.

B.7 Communication Architecture

The gripper communicates with the ROS computer using two communication channels. Hypertext Transfer Protocol (HTTP) is used for command transmission from

ROS to the ESP32, while User Datagram Protocol (UDP) is used for state feedback from the ESP32 to ROS. This design separates command execution from state reporting, and the implemented interfaces are summarised in Table B.3.

HTTP communication is used for explicit requests. The ROS computer sends movement commands to the ESP32 through an HTTP motion endpoint. The command contains the step interval, the number of steps and the FSR threshold. The firmware interprets the sign of the step command to determine whether the gripper should open or close.

UDP communication is used for repeated feedback, where the firmware publishes a state message containing the current stroke and contact state. This feedback is transmitted to the ROS computer at regular intervals, and the lightweight nature of the UDP interface makes it well suited for continuous state broadcasting.

Table B.3: Communication interfaces used by the WiLi-Gripper.

Interface	Direction	Function
HTTP motion endpoint	ROS to ESP32	Sends gripper movement commands
HTTP UDP-target endpoint	ROS to ESP32	Sends the ROS host IP for feedback
HTTP ping endpoint	ROS to ESP32	Checks if the ESP32 is reachable
HTTP reboot endpoint	ROS to ESP32	Restarts the ESP32 controller
UDP state feedback	ESP32 to ROS	Sends current stroke and contact state
UDP reboot confirmation	ESP32 to ROS	Confirms restart completion

The ESP32 uses multicast Domain Name System (mDNS) with the device name *wilig-gripper.local*. This mechanism allows the ROS computer to resolve the gripper's network address dynamically without requiring a fixed IP configuration. As a result, the gripper can be accessed using a consistent hostname even if its IP address changes due to network conditions or Dynamic Host Configuration Protocol (DHCP) assignment. This simplifies system setup, improves robustness during deployment and reduces the need for manual network configuration.

In addition to mDNS-based discovery, the firmware includes a fallback communication mode. If the ESP32 is unable to connect to the configured Wi-Fi network, it automatically switches to access-point mode. In this mode, the gripper creates its own wireless network, allowing a user to connect directly to the device for configuration, diagnostics or firmware updates. This feature ensures that the gripper remains accessible even in the absence of a functioning network connection and supports reliable setup and debugging procedures.

B.8 ROS Software Architecture

The ROS implementation is organised into three packages. These packages separate control, robot description and MoveIt configuration.

The *wilig_control* package contains the communication and control interface between ROS and the physical gripper. It includes the embedded firmware, the Python nodes used for command and feedback handling, the custom ROS messages and the launch files used to start the gripper system.

The *wilig_description* package contains the digital model of the gripper. It defines the links, joints, visual meshes and collision meshes used in ROS and RViz. This package allows the gripper to be displayed as a digital end-effector and included in the robot description.

The *wilig_moveit_config* package contains the MoveIt configuration generated for the gripper. It defines the planning group, named joint states, joint limits and controller interface required to command the gripper through MoveIt.

Table B.4: ROS packages developed for the WiLi-Gripper.

Package	Main function
wilig_control	ROS control nodes, custom messages, launch files, utility scripts and ESP32 firmware
wilig_description	URDF/Xacro description, visual meshes and collision meshes
wilig_moveit_config	MoveIt configuration, planning group, named states, controllers and RViz setup

Two custom ROS messages were developed for communication between the ROS control nodes. The first message is *GripperCommandStructure*. It contains the target stroke, movement speed and FSR threshold. This message is used by ROS nodes to request a gripper movement. The second message is *GripperStateStructure*. It contains the current stroke and a Boolean contact state. The contact state indicates whether the FSR threshold has been reached.

Table B.5: Custom ROS messages used by the gripper interface.

Message	Fields	Function
GripperCommandStructure	targetStroke moveSpeed fsr_threshold	Sends a gripper movement request
GripperStateStructure	currentStroke isGrabbing	Reports stroke and contact state

The command message represents the requested stroke in millimetres. The state message also reports the current stroke in millimetres. This keeps the physical communication layer intuitive because the gripper opening can be understood directly as a stroke value from 0 mm to 48 mm.

The *wilig_control* package contains four main Python nodes that connect the physical gripper and its ESP32 firmware to the ROS environment. Together, these nodes manage motion commands, state feedback, network communication and integration with MoveIt.

Motion commands are handled by *gripper_command_interface.py*. This node receives the requested target stroke, movement speed and FSR threshold through the *gripper_command_topic*. It then converts the target stroke and speed into the corresponding number of motor steps and the time interval between consecutive steps before transmitting the command parameters to the ESP32 through HTTP.

Feedback from the physical gripper is managed by *gripper_state_interface.py*. The node listens for UDP messages sent by the ESP32, decodes the received data and publishes the reported gripper state through *wilig_gripper_state*. The feedback includes the current stroke and the contact-detection state. The node also publishes the estimated jaw position through *joint_states* using the standard *JointState* message from the *sensor_msgs* package. This allows the digital gripper model in RViz to follow the estimated movement of the physical gripper.

Network setup and connection management are provided by *gripper_connection_interface.py*. This node broadcasts the ROS host IP address to the ESP32, sends connection-check requests and processes commands to restart the embedded controller. These functions allow the ESP32 to identify the correct destination for UDP feedback and support communication monitoring between the ROS computer and the gripper.

MoveIt integration is provided by *gripper_action_server.py*, which exposes the *follow_joint_trajectory* action through the *gripper_controller*. The action uses the *FollowJointTrajectoryAction* type from the *control_msgs* package, allowing MoveIt to control the gripper as a trajectory-based end-effector. When a trajectory goal is received, the node extracts the final commanded joint position, converts it into the corresponding target stroke and publishes it as a gripper command for execution by the ESP32.

Table B.6: Main ROS nodes used by the WiLi-Gripper control package.

ROS node	Main function
gripper_command_interface.py	Converts ROS commands into HTTP movement requests
gripper_state_interface.py	Receives UDP feedback and publishes gripper state and joint state
gripper_connection_interface.py	Manages IP setup, ping and reboot communication
gripper_action_server.py	Provides the MoveIt trajectory action interface

The gripper software interface combines ROS topics with a standard trajectory action interface. The topics support direct gripper commands, state feedback and connection management, while the action interface allows the gripper to be controlled through MoveIt as part of the wider robotic task pipeline.

Direct motion commands are sent through *gripper_command_topic* using the GripperCommandStructure message. Each message specifies the target gripper stroke, movement speed and FSR threshold required for the requested movement. After receiving and executing a command, the gripper publishes its state through *wilig_gripper_state* using the GripperStateStructure message. This feedback includes

the current gripper stroke and a Boolean value indicating whether contact was detected during closing.

The estimated gripper position is also published through the *joint_states* topic using the standard *JointState* message from the *sensor_msgs* package. This updates the active prismatic joint in RViz and allows the digital gripper model to follow the estimated position of the physical mechanism. As a result, the digital model remains synchronised with the estimated position of the physical mechanism during opening and closing movements.

A separate group of topics supports communication between the ROS computer and the ESP32 controller. The *broadcast_host_ip_topic* topic sends the ROS host IP address to the ESP32 using the String message from the *std_msgs* package, allowing the embedded controller to identify the destination for UDP feedback. The *ping_topic* topic requests a communication check to determine whether the ESP32 is reachable, while the *reboot_topic* topic requests a restart of the embedded controller. Both interfaces use the same String message type.

For integration with MoveIt, the gripper exposes the *follow_joint_trajectory* action through the *gripper_controller*. The interface uses the *FollowJointTrajectoryAction* type from the *control_msgs* package. Trajectory goals generated by MoveIt are converted into the stroke, speed and force-threshold commands required by the embedded controller.

The interface provides two complementary control modes. The gripper can be operated directly through the command topic for testing and standalone control or through the trajectory action interface during coordinated robotic tasks. The latter enables gripper movements to be incorporated into the MoveIt-based workflow used for robot motion planning and pick-and-place execution.

B.9 Digital Model and MoveIt Integration

The digital gripper model is implemented in the wilig_description package. The model is defined using URDF/Xacro and contains the links and joints required to represent the physical device.

The main links are *gripper_flange*, *gripper_main_body*, *gripper_left_finger*, *gripper_right_finger* and *tool_center_point*. The flange represents the mounting connection to the robot. The main body represents the structural body of the gripper. The left and right finger links represent the moving jaws. The tool centre point is used as the end-effector reference frame.

The digital model includes visual meshes and collision meshes. The visual meshes are used for realistic display in RViz. The collision meshes are simplified representations used for collision checking. This separation follows the standard ROS modelling approach, where detailed visual geometry is kept separate from computationally simpler collision geometry.

The finger motion is represented using prismatic joints. The active joint is *main_body_to_right_finger*. This joint has a displacement range from 0 m to 0.024 m. The left finger is represented using *main_body_to_left_finger*, which mimics the right finger motion with an opposite direction. This creates symmetric jaw movement in the digital model while keeping only one active planning joint.

As described previously, the total physical gripper stroke is 48 mm. Since both jaws move towards each other, the active prismatic joint represents half of the total opening. Therefore, the fully open state corresponds to 0.024 m in the active MoveIt joint, while the fully closed state corresponds to 0 m.

The model also includes a fixed joint from the main body to the tool centre point. This tool frame is used by the robotic cell as the reference for end-effector positioning and grasp execution.

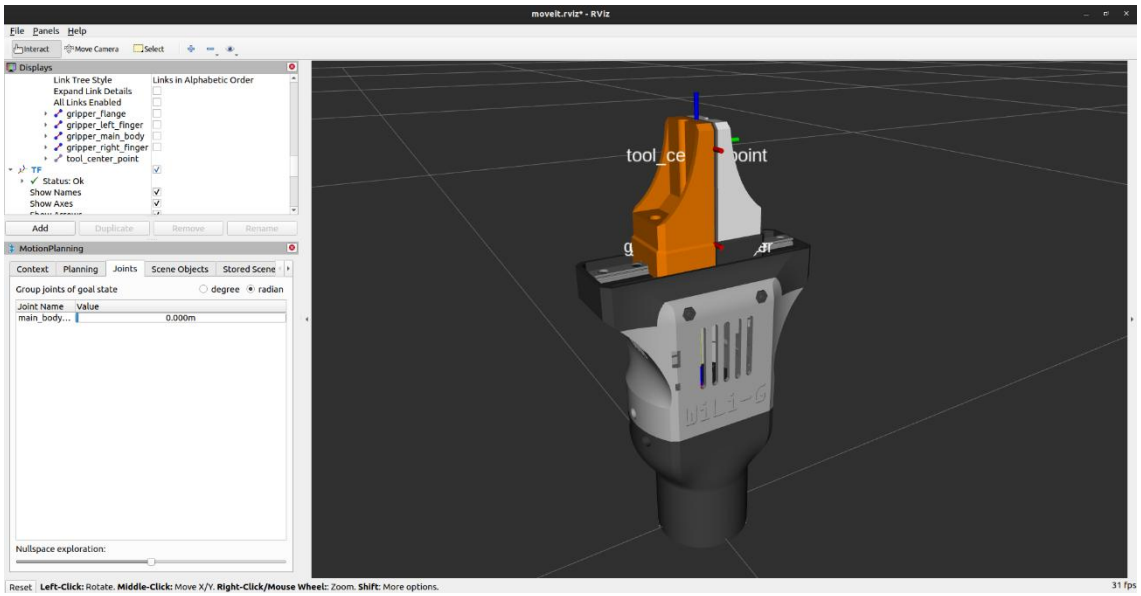


Figure B.5. RViz view of the WiLi-Gripper digital model showing the main body, moving jaws and tool centre point.

Building on the digital gripper description, the *wilig_moveit_config* package provides the MoveIt configuration required to plan and execute gripper movements. It contains the semantic robot description, planning-group definition, joint limits, predefined gripper states, controller configuration and RViz visualisation settings.

Within MoveIt, the gripper is represented by a planning group named gripper. This group contains the active prismatic joint *main_body_to_right_finger*, which describes the linear movement of the right finger relative to the main body. The opposite finger is treated as passive because its displacement is determined by the mimic relationship defined in the URDF model. Consequently, MoveIt only needs to command one active joint, while the digital model produces the corresponding symmetrical movement of the opposite finger.

Two predefined states are provided for standard gripper operation. The *fullyClosed* state sets the active prismatic joint to 0 m, while the *fullyOpen* state sets it to 0.024 m. These named states allow the gripper to be moved conveniently between its standard open and closed configurations during planning, testing and task execution.

Physical execution is enabled through a controller named gripper_controller, which uses the *FollowJointTrajectory* interface. This controller is connected to the trajectory action server implemented in the *wilig_control* package. As a result, trajectory goals generated by MoveIt are received by the action server, converted into target stroke commands and transmitted to the ESP32 for execution by the physical gripper.

The package also includes the standard MoveIt launch and RViz configuration files required for visualisation and testing. These files allow the gripper to be planned and controlled as a standalone mechanism during development or incorporated into the complete robotic-cell model for the online experiment presented in Chapter 4.

The digital model and MoveIt configuration are supported by a set of launch files for complete-cell integration, standalone physical testing and digital-only validation. This structure allows the gripper software to be tested independently before being incorporated into the wider pick-and-place pipeline.

Integration with the complete robotic system is provided through *wilig_control* launch file. This file starts the command interface, state interface, connection interface and trajectory action server required to communicate with the physical gripper. It is used when the gripper model and MoveIt configuration are loaded as part of the larger ABB robotic-cell environment.

For independent testing, *standalone_wilig_control* launch file starts the gripper as a self-contained ROS and MoveIt system. It loads the gripper robot description, starts the required state publisher, launches MoveIt and opens RViz. When simulation is disabled, it also starts the physical interface nodes so that commands generated in MoveIt can be executed by the ESP32-controlled gripper. This mode supports mechanical testing, communication checks and motion verification without launching the complete robotic cell.

Digital-only operation is provided through *launch_digital_wilig* launch file in the description package. This file loads the digital gripper model, starts the required state-publishing components and launches MoveIt with RViz without requiring the physical gripper. It allows the joint relationships, named states, planning configuration and visual behaviour of the gripper to be inspected before hardware execution.

Together, these operating modes support a staged development process in which the digital model, physical gripper interface and complete robotic-cell integration can be tested separately. This reflects the wider thesis architecture, where components are first verified in the digital environment and then incorporated into the physical pick-and-place system.

B.10 Use in the Chapter 4 Online Experiment

The WiLi-Gripper was used as the end-effector in the Chapter 4 online experiment. In this stage, the robotic cell was extended from simple rectangular objects to complex industrial parts. The gripper supported this extension by providing a software-controlled grasping device that could be integrated with the updated perception and motion-planning pipeline.

During the online pick-and-place sequence, the robot moves to a safe pose and then approaches the selected object. Before reaching the picking pose, the gripper is commanded to open. Once the robot reaches the picking pose, the gripper is commanded to close around the object. The FSR provides contact feedback during closing. After grasping, the robot lifts the object, moves through the planned transfer path and approaches the target placement position. At the placing pose, the gripper opens and releases the object.

Therefore, the gripper forms part of the complete perception-to-action chain used in the second experimental stage. The gripper executed the opening, contact-controlled closing, object-transfer and release commands generated by the robotic-cell pipeline. The complete perception, registration and motion-planning sequence is described in Chapter 4.

The gripper was not treated as the main research contribution of the thesis. Instead, it was developed as an enabling subsystem that allowed the proposed adaptive pick-and-place pipeline to be tested with a custom software-controlled end-effector.

B.11 Limitations and Future Improvements

The WiLi-Gripper was developed as a research prototype. It provided the control and integration functions required for the second experimental stage, but several limitations remain.

First, the gripping performance depends on the selected jaw geometry and the contact properties of the target parts. Different object shapes may require alternative fingertips, softer contact surfaces or additional compliance. The current finger geometry was suitable for the laboratory experiment, but future applications may require a family of interchangeable fingertips.

Second, the FSR is used as a contact-threshold sensor rather than as a calibrated force-measurement device. The firmware detects when the sensor value exceeds a predefined threshold. This is sufficient for contact detection, but it does not provide a calibrated gripping force in newtons or kilograms-force. Future work could calibrate the sensor response and use it for adaptive force control.

Third, most structural components were manufactured from PLA. This was suitable for rapid prototyping and laboratory use, but long-term industrial deployment would require stronger materials and more robust manufacturing methods. Future versions of the gripper could use reinforced materials such as carbon-fibre-reinforced PLA (PLA-CF) or PETG-CF for the structural components. PLA-CF combines PLA with carbon fibres to increase stiffness, while PETG-CF combines PETG with carbon fibres to provide improved durability and impact resistance. These materials could improve the long-term mechanical performance of the gripper. However, standard PLA provided adequate performance for the prototype developed and used in this research.

Fourth, the communication architecture uses HTTP for commands and UDP for state feedback. This was suitable for the experimental setup because it provided a simple and modular interface between ROS and the embedded controller. However, future industrial versions could improve timing guarantees, cybersecurity, connection recovery and diagnostic reporting.

Finally, the gripper is currently designed as a compact two-finger device for controlled pick-and-place experiments. Future work could extend it with battery-powered operation, calibrated force feedback, tactile sensing, adaptive grip logic, parameter tuning through a graphical interface and ROS 2 support.

Despite these limitations, the WiLi-Gripper provided an important enabling component for the second experimental stage. It demonstrated that a custom end-effector could be mechanically designed, electronically controlled, represented in ROS,

integrated with MoveIt and synchronised with the robotic pick-and-place pipeline. It also supported the broader thesis argument that adaptable robotic cells require modular integration across physical hardware, embedded control, perception, planning and execution.